You can retrieve the current local time on a specific computer using the **Win32_LocalTime** WMI class.

PowerShell

```
Get-CimInstance -ClassName Win32_LocalTime
```

Output

```
Day            : 23
DayOfWeek      : 1
Hour           : 8
Milliseconds   :
Minute         : 52
Month          : 12
Quarter        : 4
Second         : 55
WeekInMonth    : 4
Year           : 2019
PSComputerName :
```

Displaying service status

To view the status of all services on a specific computer, you can locally use the `Get-Service` cmdlet. For remote systems, you can use the **Win32_Service** WMI class. If you also use `Select-Object` to filter the results to **Status**, **Name**, and **DisplayName**, the output format is almost identical to that from `Get-Service`:

PowerShell

```
Get-CimInstance -ClassName Win32_Service |
    Select-Object -Property Status,Name,DisplayName
```

To allow the complete display of names for services with long names, use the **AutoSize** and **Wrap** parameters of `Format-Table`. These parameters optimize column width and allow long names to wrap instead of being truncated:

PowerShell

```
Get-CimInstance -ClassName Win32_Service |
    Format-Table -Property Status, Name, DisplayName -AutoSize -Wrap
```

Creating Get-WinEvent queries with FilterHashtable

> This sample only applies to Windows platforms.

To read the original June 3, 2014 **Scripting Guy** blog post, see Use FilterHashTable to Filter Event Log with PowerShell.

This article is an excerpt of the original blog post and explains how to use the `Get-WinEvent` cmdlet's **FilterHashtable** parameter to filter event logs. PowerShell's `Get-WinEvent` cmdlet is a powerful method to filter Windows event and diagnostic logs. Performance improves when a `Get-WinEvent` query uses the **FilterHashtable** parameter.

When you work with large event logs, it's not efficient to send objects down the pipeline to a `Where-Object` command. Prior to PowerShell 6, the `Get-EventLog` cmdlet was another option to get log data. For example, the following commands are inefficient to filter the **Microsoft-Windows-Defrag** logs:

PowerShell

```powershell
Get-EventLog -LogName Application | Where-Object Source -Match defrag

Get-WinEvent -LogName Application | Where-Object { $_.ProviderName -match 'defrag'
}
```

The following command uses a hash table that improves the performance:

PowerShell

```powershell
Get-WinEvent -FilterHashtable @{
   LogName='Application'
   ProviderName='*defrag'
}
```

Blog posts about enumeration

This article presents information about how to use enumerated values in a hash table. For more information about enumeration, read these **Scripting Guy** blog posts. To create a function that

returns the enumerated values, see Enumerations and Values. For more information, see the Scripting Guy series of blog posts about enumeration.

Hash table key-value pairs

To build efficient queries, use the `Get-WinEvent` cmdlet with the **FilterHashtable** parameter. **FilterHashtable** accepts a hash table as a filter to get specific information from Windows event logs. A hash table uses **key-value** pairs. For more information about hash tables, see about_Hash_Tables.

If the **key-value** pairs are on the same line, they must be separated by a semicolon. If each **key-value** pair is on a separate line, the semicolon isn't needed. For example, this article places **key-value** pairs on separate lines and doesn't use semicolons.

This sample uses several of the **FilterHashtable** parameter's **key-value** pairs. The completed query includes **LogName**, **ProviderName**, **Keywords**, **Id**, and **Level**.

The accepted **key-value** pairs are shown in the following table and are included in the documentation for the Get-WinEvent **FilterHashtable** parameter.

The following table displays the key names, data types, and whether wildcard characters are accepted for a data value.

Expand table

Key name	Value data type	Accepts wildcard characters?
LogName	`<String[]>`	Yes
ProviderName	`<String[]>`	Yes
Path	`<String[]>`	No
Keywords	`<Long[]>`	No
ID	`<Int32[]>`	No
Level	`<Int32[]>`	No
StartTime	`<DateTime>`	No
EndTime	`<DateTime>`	No
UserID	`<SID>`	No

Key name	Value data type	Accepts wildcard characters?
Data	`<String[]>`	No
`<named-data>`	`<String[]>`	No

The `<named-data>` key represents a named event data field. For example, the Perflib event 1008 can contain the following event data:

XML

```
<EventData>
  <Data Name="Service">BITS</Data>
  <Data Name="Library">C:\Windows\System32\bitsperf.dll</Data>
  <Data Name="Win32Error">2</Data>
</EventData>
```

You can query for these events using the following command:

PowerShell

```
Get-WinEvent -FilterHashtable @{LogName='Application'; 'Service'='Bits'}
```

> ⓘ Note
>
> The ability to query for `<named-data>` was added in PowerShell 6.

Building a query with a hash table

To verify results and troubleshoot problems, it helps to build the hash table one **key-value** pair at a time. The query gets data from the **Application** log. The hash table is equivalent to `Get-WinEvent -LogName Application`.

To begin, create the `Get-WinEvent` query. Use the **FilterHashtable** parameter's **key-value** pair with the key, **LogName**, and the value, **Application**.

PowerShell

```
Get-WinEvent -FilterHashtable @{
   LogName='Application'
}
```

Continue to build the hash table with the **ProviderName** key. Usually, the **ProviderName** is the name that appears in the **Source** field in the **Windows Event Viewer**. For example, `.NET Runtime` in the following screenshot:

Image of Windows Event Viewer sources

Update the hash table and include the **key-value** pair with the key, **ProviderName**, and the value, `.NET Runtime`.

PowerShell

```
Get-WinEvent -FilterHashtable @{
   LogName='Application'
   ProviderName='.NET Runtime'
}
```

> **Note**
>
> For some event providers, the correct **ProviderName** can be obtained by looking on the **Details** tab in **Event Properties**. For example, events where the **Source** field shows `Defrag`, the correct **ProviderName** is `Microsoft-Windows-Defrag`.

If your query needs to get data from archived event logs, use the **Path** key. The **Path** value specifies the full path to the log file. For more information, see the **Scripting Guy** blog post, Use PowerShell to Parse Saved Event Logs for Errors.

Using enumerated values in a hash table

Keywords is the next key in the hash table. The **Keywords** data type is an array of the `[long]` value type that holds a large number. Use the following command to find the maximum value of `[long]`:

PowerShell

```
[long]::MaxValue
```

Output

```
9223372036854775807
```

For the **Keywords** key, PowerShell uses a number, not a string such as **Security**. **Windows Event Viewer** displays the **Keywords** as strings, but they're enumerated values. In the hash table, if you use the **Keywords** key with a string value, an error message is displayed.

Open the **Windows Event Viewer** and from the **Actions** pane, click on **Filter current log**. The **Keywords** drop-down menu displays the available keywords, as shown in the following screenshot:

Image of Windows Event Viewer keywords

Use the following command to display the `StandardEventKeywords` property names.

PowerShell

```
[System.Diagnostics.Eventing.Reader.StandardEventKeywords] |
    Get-Member -Static -MemberType Property
```

Output

```
   TypeName: System.Diagnostics.Eventing.Reader.StandardEventKeywords
Name             MemberType Definition
--               ---------- ----------
AuditFailure     Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
AuditSuccess     Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
CorrelationHint  Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
CorrelationHint2 Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
EventLogClassic  Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
None             Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
ResponseTime     Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
Sqm              Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
WdiContext       Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
WdiDiagnostic    Property   static
System.Diagnostics.Eventing.Reader.StandardEventKey…
```

The enumerated values are documented in the **.NET Framework**. For more information, see StandardEventKeywords Enumeration.

The **Keywords** names and enumerated values are as follows:

Expand table

Name	Value
AuditFailure	4503599627370496
AuditSuccess	9007199254740992
CorrelationHint2	18014398509481984
EventLogClassic	36028797018963968
Sqm	2251799813685248
WdiDiagnostic	1125899906842624
WdiContext	562949953421312
ResponseTime	281474976710656
None	0

Update the hash table and include the **key-value** pair with the key, **Keywords**, and the **EventLogClassic** enumeration value, **36028797018963968**.

PowerShell

```
Get-WinEvent -FilterHashtable @{
   LogName='Application'
   ProviderName='.NET Runtime'
   Keywords=36028797018963968
}
```

Keywords static property value (optional)

The **Keywords** key is enumerated, but you can use a static property name in the hash table query. Rather than using the returned string, the property name must be converted to a value with the **value__** property.

For example, the following script uses the **value__** property.

PowerShell

```
$C = [System.Diagnostics.Eventing.Reader.StandardEventKeywords]::EventLogClassic
Get-WinEvent -FilterHashtable @{
   LogName='Application'
   ProviderName='.NET Runtime'
```

```
    Keywords=$C.value__
}
```

Filtering by Event Id

To get more specific data, the query's results are filtered by **Event Id**. The **Event Id** is referenced in the hash table as the key **Id** and the value is a specific **Event Id**. The **Windows Event Viewer** displays the **Event Id**. This example uses **Event Id 1023**.

Update the hash table and include the **key-value** pair with the key, **Id** and the value, **1023**.

PowerShell

```powershell
Get-WinEvent -FilterHashtable @{
    LogName='Application'
    ProviderName='.NET Runtime'
    Keywords=36028797018963968
    ID=1023
}
```

Filtering by Level

To further refine the results and include only events that are errors, use the **Level** key. **Windows Event Viewer** displays the **Level** as string values, but they're enumerated values. In the hash table, if you use the **Level** key with a string value, an error message is displayed.

Level has values such as **Error**, **Warning**, or **Informational**. Use the following command to display the `StandardEventLevel` property names.

PowerShell

```powershell
[System.Diagnostics.Eventing.Reader.StandardEventLevel] |
    Get-Member -Static -MemberType Property
```

Output

```
   TypeName: System.Diagnostics.Eventing.Reader.StandardEventLevel

Name          MemberType Definition
----          ---------- ----------
Critical      Property   static
System.Diagnostics.Eventing.Reader.StandardEventLevel Critical {get;}
Error         Property   static
System.Diagnostics.Eventing.Reader.StandardEventLevel Error {get;}
Informational Property   static
```

```
System.Diagnostics.Eventing.Reader.StandardEventLevel Informational {get;}
LogAlways     Property   static
System.Diagnostics.Eventing.Reader.StandardEventLevel LogAlways {get;}
Verbose       Property   static
System.Diagnostics.Eventing.Reader.StandardEventLevel Verbose {get;}
Warning       Property   static
System.Diagnostics.Eventing.Reader.StandardEventLevel Warning {get;}
```

The enumerated values are documented in the **.NET Framework**. For more information, see StandardEventLevel Enumeration.

The **Level** key's names and enumerated values are as follows:

Expand table

Name	Value
Verbose	5
Informational	4
Warning	3
Error	2
Critical	1
LogAlways	0

The hash table for the completed query includes the key, **Level**, and the value, **2**.

PowerShell

```powershell
Get-WinEvent -FilterHashtable @{
   LogName='Application'
   ProviderName='.NET Runtime'
   Keywords=36028797018963968
   ID=1023
   Level=2
}
```

Level static property in enumeration (optional)

The **Level** key is enumerated, but you can use a static property name in the hash table query. Rather than using the returned string, the property name must be converted to a value with the **value__** property.

For example, the following script uses the **value__** property.

PowerShell

```powershell
$C = [System.Diagnostics.Eventing.Reader.StandardEventLevel]::Informational
Get-WinEvent -FilterHashtable @{
   LogName='Application'
   ProviderName='.NET Runtime'
   Keywords=36028797018963968
   ID=1023
   Level=$C.value__
}
```

Last updated on 03/24/2025

Managing processes with Process cmdlets

> This sample only applies to Windows PowerShell 5.1.

You can use the Process cmdlets in PowerShell to manage local and remote processes in PowerShell.

Getting processes

To get the processes running on the local computer, run a `Get-Process` with no parameters.

You can get particular processes by specifying their process names or process IDs. The following command gets the Idle process:

PowerShell

```powershell
Get-Process -Id 0
```

Output

```
Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
      0       0        0         16     0               0 Idle
```

Although it's normal for cmdlets to return no data in some situations, when you specify a process by its **ProcessId**, `Get-Process` generates an error if it finds no matches, because the usual intent is to retrieve a known running process. If there is no process with that ID, it's likely that the ID is incorrect or that the process of interest has already exited:

PowerShell

```powershell
Get-Process -Id 99
```

Output

```
Get-Process : No process with process ID 99 was found.
At line:1 char:12
+ Get-Process  <<<< -Id 99
```

You can use the Name parameter of the `Get-Process` cmdlet to specify a subset of processes based on the process name. The Name parameter can take multiple names in a comma-separated list and it supports the use of wildcards, so you can type name patterns.

For example, the following command gets process whose names begin with "ex."

PowerShell

```
Get-Process -Name ex*
```

Output

```
Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    234       7     5572      12484   134     2.98   1684 EXCEL
    555      15    34500      12384   134   105.25    728 explorer
```

Because the .NET **System.Diagnostics.Process** class is the foundation for PowerShell processes, it follows some of the conventions used by **System.Diagnostics.Process**. One of those conventions is that the process name for an executable never includes the `.exe` at the end of the executable name.

`Get-Process` also accepts multiple values for the Name parameter.

PowerShell

```
Get-Process -Name exp*,power*
```

Output

```
Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    540      15    35172      48148   141    88.44    408 explorer
    605       9    30668      29800   155     7.11   3052 powershell
```

You can use the ComputerName parameter of `Get-Process` to get processes on remote computers. For example, the following command gets the PowerShell processes on the local computer (represented by "localhost") and on two remote computers.

PowerShell

```
Get-Process -Name powershell -ComputerName localhost, Server01, Server02
```

Output

```
Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    258       8    29772      38636   130             3700 powershell
    398      24    75988      76800   572             5816 powershell
    605       9    30668      29800   155     7.11    3052 powershell
```

The computer names aren't evident in this display, but they're stored in the **MachineName** property of the process objects that `Get-Process` returns. The following command uses the `Format-Table` cmdlet to display the process **Id**, **ProcessName** and **MachineName** (ComputerName) properties of the process objects.

PowerShell

```
Get-Process -Name powershell -ComputerName localhost, Server01, Server01 |
    Format-Table -Property Id, ProcessName, MachineName
```

Output

```
  Id ProcessName MachineName
  -- ----------- -----------
3700 powershell  Server01
3052 powershell  Server02
5816 powershell  localhost
```

This more complex command adds the **MachineName** property to the standard `Get-Process` display.

PowerShell

```
Get-Process powershell -ComputerName localhost, Server01, Server02 |
    Format-Table -Property Handles,
        @{Label="NPM(K)";Expression={[int]($_.NPM/1024)}},
        @{Label="PM(K)";Expression={[int]($_.PM/1024)}},
        @{Label="WS(K)";Expression={[int]($_.WS/1024)}},
        @{Label="VM(M)";Expression={[int]($_.VM/1MB)}},
        @{Label="CPU(s)";Expression={if ($_.CPU -ne $()){$_.CPU.ToString("N")}}},
        Id, ProcessName, MachineName -Auto
```

Output

```
Handles  NPM(K)  PM(K) WS(K) VM(M) CPU(s)  Id ProcessName  MachineName
-------  ------  ----- ----- ----- ------  -- -----------  -----------
    258       8  29772 38636   130         3700 powershell Server01
```

```
    398      24  75988 76800   572          5816 powershell localhost
    605       9  30668 29800   155 7.11    3052 powershell Server02
```

Stopping processes

PowerShell gives you flexibility for listing processes, but what about stopping a process?

The `Stop-Process` cmdlet takes a **Name** or **Id** to specify a process you want to stop. Your ability to stop processes depends on your permissions. Some processes can't be stopped. For example, if you try to stop the idle process, you get an error:

PowerShell

```powershell
Stop-Process -Name Idle
```

Output

```
Stop-Process : Process 'Idle (0)' cannot be stopped due to the following error:
 Access is denied
At line:1 char:13
+ Stop-Process  <<<< -Name Idle
```

You can also force prompting with the **Confirm** parameter. This parameter is particularly useful if you use a wildcard when specifying the process name, because you may accidentally match some processes you don't want to stop:

PowerShell

```powershell
Stop-Process -Name t*,e* -Confirm
```

Output

```
Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Process" on Target "explorer (408)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "Y"):n
Confirm
Are you sure you want to perform this action?
Performing operation "Stop-Process" on Target "taskmgr (4072)".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "Y"):n
```

Complex process manipulation is possible by using some of the object filtering cmdlets. Because a Process object has a **Responding** property that's true when it's no longer responding, you can stop all nonresponsive applications with the following command:

```
PowerShell

Get-Process | Where-Object -FilterScript {$_.Responding -eq $false} | Stop-Process
```

You can use the same approach in other situations. For example, suppose a secondary notification area application automatically runs when users start another application. You may find that this doesn't work correctly in Terminal Services sessions, but you still want to keep it in sessions that run on the physical computer console. Sessions connected to the physical computer desktop always have a session ID of 0, so you can stop all instances of the process that are in other sessions by using `Where-Object` and the process, **SessionId**:

```
PowerShell

Get-Process -Name BadApp | Where-Object -FilterScript {$_.SessionId -neq 0} | Stop-
Process
```

The `Stop-Process` cmdlet doesn't have a **ComputerName** parameter. Therefore, to run a stop process command on a remote computer, you need to use the `Invoke-Command` cmdlet. For example, to stop the PowerShell process on the Server01 remote computer, type:

```
PowerShell

Invoke-Command -ComputerName Server01 {Stop-Process PowerShell}
```

Stopping All Other PowerShell Sessions

It may occasionally be useful to be able to stop all running PowerShell sessions other than the current session. If a session is using too many resources or is inaccessible (it may be running remotely or in another desktop session), you may not be able to directly stop it. If you try to stop all running sessions, however, the current session may be terminated instead.

Each PowerShell session has an environment variable PID that contains the **Id** of the Windows PowerShell process. You can check the $PID against the Id of each session and terminate only Windows PowerShell sessions that have a different Id. The following pipeline command does this and returns the list of terminated sessions (because of the use of the **PassThru** parameter):

PowerShell

```
Get-Process -Name powershell | Where-Object -FilterScript {$_.Id -ne $PID} |
    Stop-Process -PassThru
```

Output

```
Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    334       9    23348      29136   143     1.03    388 powershell
    304       9    23152      29040   143     1.03    632 powershell
    302       9    20916      26804   143     1.03   1116 powershell
    335       9    25656      31412   143     1.09   3452 powershell
    303       9    23156      29044   143     1.05   3608 powershell
    287       9    21044      26928   143     1.02   3672 powershell
```

Starting, debugging, and waiting for processes

PowerShell also comes with cmdlets to start (or restart), debug a process, and wait for a process to complete before running a command. For information about these cmdlets, see the cmdlet help topic for each cmdlet.

See also

- Get-Process
- Stop-Process
- Start-Process
- Wait-Process
- Debug-Process
- Invoke-Command

Last updated on 03/24/2025

Managing services

> This sample only applies to Windows PowerShell 5.1.

There are eight core **Service** cmdlets, designed for a wide range of service tasks . This article only looks at listing and changing running state for services. You can get a list of service cmdlets using `Get-Command *-Service`. You can find information about each cmdlet by using `Get-Help <Cmdlet-Name>`, such as `Get-Help New-Service`.

Getting services

You can get the services on a local or remote computer by using the `Get-Service` cmdlet. As with `Get-Process`, using the `Get-Service` command without parameters returns all services. You can filter by name, even using an asterisk as a wildcard:

PowerShell

```
PS> Get-Service -Name se*

Status   Name               DisplayName
------   ----               -----------
Running  seclogon           Secondary Logon
Running  SENS               System Event Notification
Stopped  ServiceLayer       ServiceLayer
```

Because it isn't always apparent what the real name for the service is, you may find you need to find services by display name. You can search by specific name, use wildcards, or provide a list of display names:

PowerShell

```
PS> Get-Service -DisplayName se*

Status   Name               DisplayName
------   ----               -----------
Running  lanmanserver       Server
Running  SamSs              Security Accounts Manager
Running  seclogon           Secondary Logon
Stopped  ServiceLayer       ServiceLayer
Running  wscsvc             Security Center

PS> Get-Service -DisplayName ServiceLayer, Server
```

```
Status   Name               DisplayName
------   ----               -----------
Running  lanmanserver       Server
Stopped  ServiceLayer       ServiceLayer
```

Getting remote services

With Windows PowerShell, you can use the **ComputerName** parameter of the `Get-Service` cmdlet to get the services on remote computers. The **ComputerName** parameter accepts multiple values and wildcard characters, so you can get the services on multiple computers with a single command. For example, the following command gets the services on the Server01 remote computer.

PowerShell

```powershell
Get-Service -ComputerName Server01
```

Starting in PowerShell 6.0, the `*-Service` cmdlets don't have the **ComputerName** parameter. You can still get services on remote computers with PowerShell remoting. For example, the following command gets the services on the Server02 remote computer.

PowerShell

```powershell
Invoke-Command -ComputerName Server02 -ScriptBlock { Get-Service }
```

You can also manage services with the other `*-Service` cmdlets. For more information on PowerShell remoting, see about_Remote.

Getting required and dependent services

The Get-Service cmdlet has two parameters that are very useful in service administration. The DependentServices parameter gets services that depend on the service.

The **RequiredServices** parameter gets services upon which the LanmanWorkstation service depends.

PowerShell

```
PS> Get-Service -Name LanmanWorkstation -RequiredServices

Status   Name               DisplayName
------   ----               -----------
Running  MRxSmb20           SMB 2.0 MiniRedirector
```

```
Running  bowser              Bowser
Running  MRxSmb10            SMB 1.x MiniRedirector
Running  NSI                 Network Store Interface Service
```

The **DependentServices** parameter gets that require the LanmanWorkstation service.

PowerShell

```
PS> Get-Service -Name LanmanWorkstation -DependentServices

Status   Name               DisplayName
------   ----               -----------
Running  SessionEnv         Terminal Services Configuration
Running  Netlogon           Netlogon
Stopped  Browser            Computer Browser
Running  BITS               Background Intelligent Transfer Ser...
```

The following command gets all services that have dependencies. The `Format-Table` cmdlet to display the **Status**, **Name**, **RequiredServices**, and **DependentServices** properties of the services.

PowerShell

```
Get-Service -Name * | Where-Object {$_.RequiredServices -or $_.DependentServices} |
  Format-Table -Property Status, Name, RequiredServices, DependentServices -Auto
```

Stopping, starting, suspending, and restarting services

The Service cmdlets all have the same general form. Services can be specified by common name or display name, and take lists and wildcards as values. To stop the print spooler, use:

PowerShell

```
Stop-Service -Name spooler
```

To start the print spooler after it's stopped, use:

PowerShell

```
Start-Service -Name spooler
```

To suspend the print spooler, use:

PowerShell

```
Suspend-Service -Name spooler
```

The `Restart-Service` cmdlet works in the same manner as the other Service cmdlets:

PowerShell

```
PS> Restart-Service -Name spooler

WARNING: Waiting for service 'Print Spooler (Spooler)' to finish starting...
WARNING: Waiting for service 'Print Spooler (Spooler)' to finish starting...
PS>
```

Notice that you get a repeated warning message about the Print Spooler starting up. When you perform a service operation that takes some time, PowerShell notifies you that it's still attempting to perform the task.

If you want to restart multiple services, you can get a list of services, filter them, and then perform the restart:

PowerShell

```
PS> Get-Service | Where-Object -FilterScript {$_.CanStop} | Restart-Service

WARNING: Waiting for service 'Computer Browser (Browser)' to finish stopping...
WARNING: Waiting for service 'Computer Browser (Browser)' to finish stopping...
Restart-Service : can't stop service 'Logical Disk Manager (dmserver)' because
 it has dependent services. It can only be stopped if the Force flag is set.
At line:1 char:57
+ Get-Service | Where-Object -FilterScript {$_.CanStop} | Restart-Service <<<<
WARNING: Waiting for service 'Print Spooler (Spooler)' to finish starting...
WARNING: Waiting for service 'Print Spooler (Spooler)' to finish starting...
```

These Service cmdlets don't have a **ComputerName** parameter, but you can run them on a remote computer by using the `Invoke-Command` cmdlet. For example, the following command restarts the Spooler service on the Server01 remote computer.

PowerShell

```
Invoke-Command -ComputerName Server01 {Restart-Service Spooler}
```

Setting service properties

The `Set-Service` cmdlet changes the properties of a service on a local or remote computer. Because the service status is a property, you can use this cmdlet to start, stop, and suspend a service. The Set-Service cmdlet also has a StartupType parameter that lets you change the service startup type.

To use `Set-Service` on Windows Vista and later versions of Windows, open PowerShell with the **Run as administrator** option.

For more information, see Set-Service

See also

- about_Remote
- Get-Service
- Set-Service
- Restart-Service
- Suspend-Service

Last updated on 03/24/2025

Working with printers in Windows

Article • 12/09/2022

> This sample only applies to Windows platforms.

You can use PowerShell to manage printers using WMI and the **WScript.Network** COM object from WSH.

Listing printer connections

The simplest way to list the printers installed on a computer is to use the WMI **Win32_Printer** class:

PowerShell

```
Get-CimInstance -Class Win32_Printer
```

You can also list the printers using the **WScript.Network** COM object that's typically used in WSH scripts:

PowerShell

```
(New-Object -ComObject WScript.Network).EnumPrinterConnections()
```

Because this command returns a simple string collection of port names and printer device names without any distinguishing labels, it isn't easy to interpret.

Adding a network printer

To add a new network printer, use **WScript.Network**:

PowerShell

```
(New-Object -ComObject
WScript.Network).AddWindowsPrinterConnection("\\Printserver01\Xerox5")
```

Setting a default printer

To use WMI to set the default printer, find the printer in the **Win32_Printer** collection and then invoke the **SetDefaultPrinter** method:

```
PowerShell

$printer = Get-CimInstance -Class Win32_Printer -Filter "Name='HP LaserJet 5Si'"
Invoke-CimMethod -InputObject $printer -MethodName SetDefaultPrinter
```

WScript.Network is a little simpler to use, because it has a **SetDefaultPrinter** method that takes only the printer name as an argument:

```
PowerShell

(New-Object -ComObject WScript.Network).SetDefaultPrinter('HP LaserJet 5Si')
```

Removing a printer connection

To remove a printer connection, use the **WScript.Network RemovePrinterConnection** method:

```
PowerShell

(New-Object -ComObject WScript.Network).RemovePrinterConnection("\\Printserver01\Xerox5")
```

Performing networking tasks

> This sample only applies to Windows platforms.

Because TCP/IP is the most commonly used network protocol, most low-level network protocol administration tasks involve TCP/IP. In this section, we use PowerShell and WMI to do these tasks.

Listing IP addresses for a computer

To get all IP addresses in use on the local computer, use the following command:

PowerShell

```powershell
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter IPEnabled=$true |
    Select-Object -ExpandProperty IPAddress
```

Since the **IPAddress** property of a **Win32_NetworkAdapterConfiguration** object is an array, you must use the **ExpandProperty** parameter of `Select-Object` to see the entire list of addresses.

Output

```
10.0.0.1
fe80::60ea:29a7:a233:7cb7
2601:600:a27f:a470:f532:6451:5630:ec8b
2601:600:a27f:a470:e167:477d:6c5c:342d
2601:600:a27f:a470:b021:7f0d:eab9:6299
2601:600:a27f:a470:a40e:ebce:1a8c:a2f3
2601:600:a27f:a470:613c:12a2:e0e0:bd89
2601:600:a27f:a470:444f:17ec:b463:7edd
2601:600:a27f:a470:10fd:7063:28e9:c9f3
2601:600:a27f:a470:60ea:29a7:a233:7cb7
2601:600:a27f:a470::2ec1
```

Using the `Get-Member` cmdlet, you can see that the **IPAddress** property is an array:

PowerShell

```powershell
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter IPEnabled=$true |
    Get-Member -Name IPAddress
```

Output

```
   TypeName:
Microsoft.Management.Infrastructure.CimInstance#root/cimv2/Win32_NetworkAdapterConf
iguration

Name      MemberType Definition
----      ---------- ----------
IPAddress Property   string[] IPAddress {get;}
```

The IPAddress property for each network adapter is actually an array. The braces in the definition indicate that **IPAddress** isn't a **System.String** value, but an array of **System.String** values.

Listing IP configuration data

To display detailed IP configuration data for each network adapter, use the following command:

PowerShell

```
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter IPEnabled=$true
```

The default display for the network adapter configuration object is a very reduced set of the available information. For in-depth inspection and troubleshooting, use `Select-Object` or a formatting cmdlet, such as `Format-List`, to specify the properties to be displayed.

In modern TCP/IP networks you are probably not interested in IPX or WINS properties. You can use the **ExcludeProperty** parameter of `Select-Object` to hide properties with names that begin with "WINS" or "IPX".

PowerShell

```
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter IPEnabled=$true |
    Select-Object -ExcludeProperty IPX*,WINS*
```

This command returns detailed information about DHCP, DNS, routing, and other minor IP configuration properties.

Pinging computers

You can perform a simple ping against a computer by using **Win32_PingStatus**. The following command performs the ping, but returns lengthy output:

PowerShell

```
Get-CimInstance -Class Win32_PingStatus -Filter "Address='127.0.0.1'"
```

The response from **Win32_PingStatus** contains 29 properties. You can use `Format-Table` to select the properties that are most interesting to you. The **AutoSize** parameter of `Format-Table` resizes the table columns so that they display properly in PowerShell.

PowerShell

```
Get-CimInstance -Class Win32_PingStatus -Filter "Address='127.0.0.1'" |
    Format-Table -Property Address,ResponseTime,StatusCode -AutoSize
```

Output

```
Address   ResponseTime StatusCode
-------   ------------ ----------
127.0.0.1            0          0
```

A StatusCode of 0 indicates a successful ping.

You can use an array to ping multiple computers with a single command. Because there is more than one address, use the `ForEach-Object` to ping each address separately:

PowerShell

```
'127.0.0.1','localhost','bing.com' |
  ForEach-Object -Process {
    Get-CimInstance -Class Win32_PingStatus -Filter ("Address='$_'") |
      Select-Object -Property Address,ResponseTime,StatusCode
  }
```

You can use the same command format to ping all the addresses on a subnet, such as a private network that uses network number 192.168.1.0 and a standard Class C subnet mask (255.255.255.0)., Only addresses in the range of 192.168.1.1 through 192.168.1.254 are legitimate local addresses (0 is always reserved for the network number and 255 is a subnet broadcast address).

To represent an array of the numbers from 1 through 254 in PowerShell, use the expression `1..254`. A complete subnet ping can be performed by adding each value in the range to a partial address in the ping statement:

PowerShell

```
1..254| ForEach-Object -Process {
  Get-CimInstance -Class Win32_PingStatus -Filter ("Address='192.168.1.$_'") } |
    Select-Object -Property Address,ResponseTime,StatusCode
```

Note that this technique for generating a range of addresses can be used elsewhere as well. You can generate a complete set of addresses in this way:

PowerShell

```
$ips = 1..254 | ForEach-Object -Process {'192.168.1.' + $_}
```

Retrieving network adapter properties

Earlier, we mentioned that you could retrieve general configuration properties using the **Win32_NetworkAdapterConfiguration** class. Although not strictly TCP/IP information, network adapter information such as MAC addresses and adapter types can be useful for understanding what's going on with a computer. To get a summary of this information, use the following command:

PowerShell

```
Get-CimInstance -Class Win32_NetworkAdapter -ComputerName .
```

Assigning the DNS domain for a network adapter

To assign the DNS domain for automatic name resolution, use the **SetDNSDomain** method of the **Win32_NetworkAdapterConfiguration**. The **Query** parameter of `Invoke-CimMethod` takes a WQL query string. The cmdlet calls the method specified on each instance returned by the query.

PowerShell

```
$wql = 'SELECT * FROM Win32_NetworkAdapterConfiguration WHERE IPEnabled=True'
$args = @{ DnsDomain = 'fabrikam.com'}
Invoke-CimMethod -MethodName SetDNSDomain -Arguments $args -Query $wql
```

Filtering on `IPEnabled=True` is necessary, because even on a network that uses only TCP/IP, several of the network adapter configurations on a computer aren't true TCP/IP adapters. they're general software elements supporting RAS, VPN, QoS, and other services for all adapters and thus don't have an address of their own.

Performing DHCP configuration tasks

Modifying DHCP details involves working with a set of network adapters, just as the DNS configuration does. There are several distinct actions you can perform using WMI.

Finding DHCP-enabled adapters

To find the DHCP-enabled adapters on a computer, use the following command:

PowerShell

```
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter
"DHCPEnabled=$true"
```

To exclude adapters with IP configuration problems, you can retrieve only IP-enabled adapters:

PowerShell

```
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter "IPEnabled=$true
and DHCPEnabled=$true"
```

Retrieving DHCP properties

Because DHCP-related properties for an adapter generally begin with `DHCP`, you can use the Property parameter of `Format-Table` to display only those properties:

PowerShell

```
Get-CimInstance -Class Win32_NetworkAdapterConfiguration -Filter  "IPEnabled=$true
and DHCPEnabled=$true" |
  Format-Table -Property DHCP*
```

Enabling DHCP on each adapter

To enable DHCP on all adapters, use the following command:

PowerShell

```
$wql = 'SELECT * from Win32_NetworkAdapterConfiguration WHERE IPEnabled=True and
DHCPEnabled=False'
Invoke-CimMethod -MethodName ReleaseDHCPLease -Query $wql
```

Using the filter statement `IPEnabled=True and DHCPEnabled=False` avoids enabling DHCP where it's already enabled.

Releasing and renewing DHCP leases on specific adapters

Instances of the **Win32_NetworkAdapterConfiguration** class has `ReleaseDHCPLease` and `RenewDHCPLease` methods. Both are used in the same way. In general, use these methods if you only need to release or renew addresses for an adapter on a specific subnet. The easiest way to filter adapters on a subnet is to choose only the adapter configurations that use the gateway for that subnet. For example, the following command releases all DHCP leases on adapters on the local computer that are obtaining DHCP leases from 192.168.1.254:

PowerShell

```
$wql = 'SELECT * from Win32_NetworkAdapterConfiguration WHERE
DHCPServer="192.168.1.1"'
Invoke-CimMethod -MethodName ReleaseDHCPLease -Query $wql
```

The only change for renewing a DHCP lease is to use the `RenewDHCPLease` method instead of the `ReleaseDHCPLease` method:

PowerShell

```
$wql = 'SELECT * from Win32_NetworkAdapterConfiguration WHERE
DHCPServer="192.168.1.1"'
Invoke-CimMethod -MethodName RenewDHCPLease -Query $wql
```

> **Note**
>
> When using these methods on a remote computer, be aware that you can lose access to the remote system if you are connected to it through the adapter with the released or renewed lease.

Releasing and renewing DHCP leases on all adapters

You can perform global DHCP address releases or renewals on all adapters by using the **Win32_NetworkAdapterConfiguration** methods, `ReleaseDHCPLeaseAll` and `RenewDHCPLeaseAll`. However, the command must apply to the WMI class, rather than a particular adapter, because releasing and renewing leases globally is performed on the class, not on a specific adapter. The `Invoke-CimMethod` cmdlet can call the methods of a class.

PowerShell

```
Invoke-CimMethod -ClassName Win32_NetworkAdapterConfiguration -MethodName
ReleaseDHCPLeaseAll
```

You can use the same command format to invoke the **RenewDHCPLeaseAll** method:

PowerShell

```
Invoke-CimMethod -ClassName Win32_NetworkAdapterConfiguration -MethodName
RenewDHCPLeaseAll
```

Creating a network share

To create a network share, use the `Create` method of **Win32_Share**:

PowerShell

```
Invoke-CimMethod -ClassName Win32_Share -MethodName Create -Arguments @{
    Path = 'C:\temp'
    Name = 'TempShare'
    Type = [uint32]0 #Disk Drive
    MaximumAllowed = [uint32]25
    Description = 'test share of the temp folder'
}
```

This is equivalent to the following `net share` command on Windows:

PowerShell

```
net share tempshare=C:\temp /users:25 /remark:"test share of the temp folder"
```

To call a method of a WMI class that takes parameters you must know what parameters are available and the types of those parameters. For example, you can list the methods of the **Win32_Class** with the following commands:

PowerShell

```
(Get-CimClass -ClassName Win32_Share).CimClassMethods
```

Output

```
Name          ReturnType Parameters                                    Qualifiers
----          ---------- ----------                                    ----------
Create            UInt32 {Access, Description, MaximumAllowed, Name…} {Constructor,
```

```
Implemented, MappingStrings, Stati…
SetShareInfo      UInt32 {Access, Description, MaximumAllowed}        {Implemented,
MappingStrings}
GetAccessMask     UInt32 {}                                           {Implemented,
MappingStrings}
Delete            UInt32 {}                                           {Destructor,
Implemented, MappingStrings}
```

Use the following command to list the parameters of the `Create` method.

PowerShell

```powershell
(Get-CimClass -ClassName Win32_Share).CimClassMethods['Create'].Parameters
```

Output

```
Name            CimType Qualifiers
ReferenceClassName
----            ------- ----------                                  --------------
---
Access         Instance {EmbeddedInstance, ID, In, MappingStrings…}
Description      String {ID, In, MappingStrings, Optional}
MaximumAllowed   UInt32 {ID, In, MappingStrings, Optional}
Name             String {ID, In, MappingStrings}
Password         String {ID, In, MappingStrings, Optional}
Path             String {ID, In, MappingStrings}
Type             UInt32 {ID, In, MappingStrings}
```

You can also read the documentation for Create method of the **Win32_Share** class.

Removing a network share

You can remove a network share with **Win32_Share**, but the process is slightly different from creating a share, because you need to retrieve the specific instance to be removed, rather than the **Win32_Share** class. The following example deletes the share **TempShare**:

PowerShell

```powershell
$wql = 'SELECT * from Win32_Share WHERE Name="TempShare"'
Invoke-CimMethod -MethodName Delete -Query $wql
```

Connecting a Windows-accessible network drive

The `New-PSDrive` cmdlet can create a PowerShell drive that's mapped to a network share.

PowerShell

```
New-PSDrive -Name "X" -PSProvider "FileSystem" -Root "\\Server01\Public"
```

However, drives created this way are only available to PowerShell session where they're created. To map a drive that's available outside of PowerShell (or to other PowerShell sessions), you must use the **Persist** parameter.

PowerShell

```
New-PSDrive -Persist -Name "X" -PSProvider "FileSystem" -Root "\\Server01\Public"
```

> **Note**
>
> Persistently mapped drives may not be available when running in an elevated context. This is the default behavior of Windows UAC. For more information, see the following article:
>
> - Mapped drives aren't available from an elevated prompt when UAC is configured to Prompt for credentials

---

Last updated on 03/24/2025

Working with software installations

Article • 03/17/2023

Applications installed with the Windows Installer can be found through WMI's queries, but not all applications use the Windows Installer. The specific techniques for find applications installed with other tools depends on the installer software.

For example, applications installed by copying the files to a folder on the computer usually can't be managed using techniques discussed here. You can manage these applications as files and folders using the techniques discussed in Working With Files and Folders.

For software installed using an installer package, the Windows Installer can be found using the **Win32Reg_AddRemovePrograms** or the **Win32_Product** classes. However, both of these have problems. The **Win32Reg_AddRemovePrograms** is only available if you are using System Center Configuration Manager (SCCM). And the **Win32_Product** class can be slow and has side effects.

> **⊗ Caution**
>
> The **Win32_Product** class isn't query optimized. Queries that use wildcard filters cause WMI to use the MSI provider to enumerate all installed products then parse the full list sequentially to handle the filter. This also initiates a consistency check of packages installed, verifying and repairing the install. The validation is a slow process and may result in errors in the event logs. For more information seek **KB article 974524**.

This article provides an alternative method for finding installed software.

Querying the Uninstall registry key to find installed software

Because most standard applications register an uninstaller with Windows, we can work with those locally by finding them in the Windows registry. There is no guaranteed way to find every application on a system. However, it's possible to find all programs with listings displayed in **Add or Remove Programs** in the following registry key:

`HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Uninstall`.

We can find the number of installed applications by counting the number of registry keys:

PowerShell

```
$UninstallPath = 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall'
(Get-ChildItem -Path $UninstallPath).Count
```

Output

```
459
```

We can search this list of applications further using a variety of techniques. To display the values of the registry values in the registry keys under `Uninstall`, use the `GetValue()` method of the registry keys. The value of the method is the name of the registry entry. For example, to find the display names of applications in the `Uninstall` key, use the following command:

PowerShell

```
Get-ChildItem -Path $UninstallPath |
    ForEach-Object -Process { $_.GetValue('DisplayName') } |
    Sort-Object
```

> **Note**
>
> There is no guarantee that the **DisplayName** values are unique.

The following example produces output similar to the **Win32Reg_AddRemovePrograms** class:

PowerShell

```
Get-ChildItem $UninstallPath |
    ForEach-Object {
        $ProdID = ($_.Name -split '\\')[-1]
        Get-ItemProperty -Path "$UninstallPath\$ProdID" -ea SilentlyContinue
|
        Select-Object -Property DisplayName, InstallDate, @{n='ProdID'; e=
{$ProdID}}, Publisher, DisplayVersion
} | Select-Object -First 3
```

For the sake of brevity, this example uses `Select-Object` to limit the number of items returned to three.

Output

```
DisplayName    : 7-Zip 22.01 (x64)
InstallDate    :
ProdID         : 7-Zip
Publisher      : Igor Pavlov
DisplayVersion : 22.01

DisplayName    : AutoHotkey 1.1.33.10
InstallDate    :
ProdID         : AutoHotkey
Publisher      : Lexikos
DisplayVersion : 1.1.33.10

DisplayName    : Beyond Compare 4.4.6
InstallDate    : 20230310
ProdID         : BeyondCompare4_is1
Publisher      : Scooter Software
DisplayVersion : 4.4.6.27483
```

Decode a PowerShell command from a running process

> This sample only runs on Windows platforms.

At times, you may have a PowerShell process running that's taking up a large amount of resources. This process could be running in the context of a Task Scheduler job or a SQL Server Agent job. Where there are multiple PowerShell processes running, it can be difficult to know which process represents the problem. This article shows how to decode a script block that a PowerShell process is currently running.

Create a long running process

To demonstrate this scenario, open a new PowerShell window and run the following code. It executes a PowerShell command that outputs a number every minute for 10 minutes.

PowerShell

```
powershell.exe -Command {
    $i = 1
    while ( $i -le 10 )
    {
        Write-Output -InputObject $i
        Start-Sleep -Seconds 60
        $i++
    }
}
```

View the process

The body of the command that PowerShell is executing is stored in the **CommandLine** property of the Win32_Process class. If the command is an encoded command, the **CommandLine** property contains the string "EncodedCommand". Using this information, the encoded command can be de-obfuscated via the following process.

Start PowerShell as Administrator. It's vital that PowerShell is running as administrator, otherwise no results are returned when querying the running processes.

Execute the following command to get all the PowerShell processes that have an encoded command:

PowerShell

```
$powerShellProcesses = Get-CimInstance -ClassName Win32_Process -Filter
'CommandLine LIKE "%EncodedCommand%"'
```

The following command creates a custom PowerShell object that contains the process ID and the encoded command.

PowerShell

```
$commandDetails = $powerShellProcesses | Select-Object -Property ProcessId,
@{
    Name       = 'EncodedCommand'
    Expression = {
        if ( $_.CommandLine -match 'encodedCommand (.*) -inputFormat' )
        {
            return $Matches[1]
        }
    }
}
```

Now the encoded command can be decoded. The following snippet iterates over the command details object, decodes the encoded command, and adds the decoded command back to the object for further investigation.

PowerShell

```
$commandDetails | ForEach-Object -Process {
    # Get the current process
    $currentProcess = $_

    # Convert the Base 64 string to a Byte Array
    $commandBytes =
[System.Convert]::FromBase64String($currentProcess.EncodedCommand)

    # Convert the Byte Array to a string
    $decodedCommand = [System.Text.Encoding]::Unicode.GetString($commandBytes)

    # Add the decoded command back to the object
    $commandDetails |
        Where-Object -FilterScript { $_.ProcessId -eq $currentProcess.ProcessId } |
        Add-Member -MemberType NoteProperty -Name DecodedCommand -Value
$decodedCommand
}
$commandDetails[0] | Format-List -Property *
```

The decoded command can now be reviewed by selecting the decoded command property.

Output

```
ProcessId      : 8752
EncodedCommand :
IAAKAAoACgAgAAoAIAAgACAAIAAkAGkAIAA9ACAAMQAgAAoACgAKACAACgAgACAAIAAgAHcAaABpAGwAZQA
gACgAIAAkAGkAIAAtAG

wAZQAgADEAMAAgACkAIAAKAAoACgAgAAoAIAAgACAAIAB7ACAACgAKAAoAIAAKACAAIAAgACAAIAAgACAAI
ABXAHIAaQB0AGUALQBP

AHUAdABwAHUAdAAgAC0ASQBuAHAAdQB0AE8AYgBqAGUAYwB0ACAAJABpACAACgAKAAoAIAAKACAAIAAgACA
AIAAgACAAIABTAHQAYQ

ByAHQALQBTAGwAZQBlAHAAIAAtAFMAZQBjAG8AbgBkAHMAIAA2ADAAIAAKAAoACgAgAAoAIAAgACAAIAAgA
CAAIAAgACQAaQArACsA
                 IAAKAAoACgAgAAoAIAAgACAAIAB9ACAACgAKAAoAIAAKAA==
DecodedCommand :
                     $i = 1
                     while ( $i -le 10 )
                     {
                         Write-Output -InputObject $i
                         Start-Sleep -Seconds 60
                         $i++
                     }
```

Last updated on 03/24/2025

Redirecting output

PowerShell provides several cmdlets that let you control data output directly. These cmdlets share two important characteristics.

First, they generally transform data to some form of text. They do this because they output the data to system components that require text input. This means they need to represent the objects as text. Therefore, the text is formatted as you see it in the PowerShell console window.

Second, these cmdlets use the PowerShell verb **Out** because they send information out from PowerShell to somewhere else.

Console output

By default, PowerShell sends data to the host window, which is exactly what the `Out-Host` cmdlet does. The primary use for the `Out-Host` cmdlet is paging. For example, the following command uses `Out-Host` to page the output of the `Get-Command` cmdlet:

PowerShell

```
Get-Command | Out-Host -Paging
```

The host window display is outside of PowerShell. This is important because when data is sent out of PowerShell, it's actually removed. You can see this if you try to create a pipeline that pages data to the host window, and then attempt to format it as a list, as shown here:

PowerShell

```
Get-Process | Out-Host -Paging | Format-List
```

You might expect the command to display pages of process information in list format. Instead, it displays the default tabular list:

Output

```
Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    101       5     1076       3316    32     0.05   2888 alg
...
    618      18    39348      51108   143   211.20    740 explorer
    257       8     9752      16828    79     3.02   2560 explorer
```

```
...
<SPACE> next page; <CR> next line; Q quit
...
```

The `Out-Host` cmdlet sends the data directly to the console, so the `Format-List` command never receives anything to format.

The correct way to structure this command is to put the `Out-Host` cmdlet at the end of the pipeline as shown below. This causes the process data to be formatted in a list before being paged and displayed.

PowerShell

```powershell
Get-Process | Format-List | Out-Host -Paging
```

Output

```
Id      : 2888
Handles : 101
CPU     : 0.046875
Name    : alg
...

Id      : 740
Handles : 612
CPU     : 211.703125
Name    : explorer

Id      : 2560
Handles : 257
CPU     : 3.015625
Name    : explorer
...
<SPACE> next page; <CR> next line; Q quit
...
```

This applies to all of the **Out** cmdlets. An **Out** cmdlet should always appear at the end of the pipeline.

> ⓘ **Note**
>
> All the **Out** cmdlets render output as text, using the formatting in effect for the console window, including line length limits.

Discarding output

The `Out-Null` cmdlet is designed to immediately discard any input it receives. This is useful for discarding unnecessary data that you get as a side-effect of running a command. When type the following command, you don't get anything back from the command:

PowerShell

```
Get-Command | Out-Null
```

The `Out-Null` cmdlet doesn't discard error output. For example, if you enter the following command, a message is displayed informing you that PowerShell doesn't recognize `Is-NotACommand`:

```
PS> Get-Command Is-NotACommand | Out-Null
Get-Command : 'Is-NotACommand' isn't recognized as a cmdlet, function, operable
program, or script file.
At line:1 char:12
+ Get-Command  <<<< Is-NotACommand | Out-Null
```

Printing data

> `Out-Printer` is only available on Windows platforms.

You can print data using the `Out-Printer` cmdlet. The `Out-Printer` cmdlet uses your default printer if you don't provide a printer name. You can use any Windows-based printer by specifying its display name. There is no need for any kind of printer port mapping or even a real physical printer. For example, if you have the Microsoft Office document imaging tools installed, you can send the data to an image file by typing:

PowerShell

```
Get-Command -Name Get-* | Out-Printer -Name 'Microsoft Office Document Image
Writer'
```

Saving data

You can send output to a file instead of the console window using the `Out-File` cmdlet. The following command line sends a list of processes to the file `C:\temp\processlist.txt`:

PowerShell

```
Get-Process | Out-File -FilePath C:\temp\processlist.txt
```

The results of using the `Out-File` cmdlet may not be what you expect if you are used to traditional output redirection. To understand its behavior, you must be aware of the context in which the `Out-File` cmdlet operates.

On Window PowerShell 5.1, the `Out-File` cmdlet creates a Unicode file. Some tools, that expect ASCII files, don't work correctly with the default output format. You can change the default output format to ASCII using the **Encoding** parameter:

PowerShell

```
Get-Process | Out-File -FilePath C:\temp\processlist.txt -Encoding ascii
```

`Out-File` formats file contents to look like console output. This causes the output to be truncated just as it's in a console window in most circumstances. For example, if you run the following command:

PowerShell

```
Get-Command | Out-File -FilePath C:\temp\output.txt
```

The output will look like this:

Output

```
CommandType     Name                               Definition
-----------     ----                               ----------
Cmdlet          Add-Content                        Add-Content [-Path] <String[...
Cmdlet          Add-History                        Add-History [[-InputObject] ...
...
```

To get output that doesn't force line wraps to match the screen width, you can use the **Width** parameter to specify line width. Because **Width** is a 32-bit integer parameter, the maximum value it can have is 2147483647. Type the following to set the line width to this maximum value:

PowerShell

```
Get-Command | Out-File -FilePath C:\temp\output.txt -Width 2147483647
```

The `Out-File` cmdlet is most useful when you want to save output as it would have displayed on the console.

Last updated on 03/24/2025

Using Format commands to change output view

PowerShell has a set of cmdlets that allow you to control how properties are displayed for particular objects. The names of all the cmdlets begin with the verb `Format`. They let you select which properties you want to show.

PowerShell

```
Get-Command -Verb Format -Module Microsoft.PowerShell.Utility
```

Output

```
CommandType     Name               Version    Source
-----------     ----               -------    ------
Cmdlet          Format-Custom      6.1.0.0    Microsoft.PowerShell.Utility
Cmdlet          Format-Hex         6.1.0.0    Microsoft.PowerShell.Utility
Cmdlet          Format-List        6.1.0.0    Microsoft.PowerShell.Utility
Cmdlet          Format-Table       6.1.0.0    Microsoft.PowerShell.Utility
Cmdlet          Format-Wide        6.1.0.0    Microsoft.PowerShell.Utility
```

This article describes the `Format-Wide`, `Format-List`, and `Format-Table` cmdlets.

Each object type in PowerShell has default properties that are used when you don't select the properties to display. Each cmdlet uses the same **Property** parameter to specify which properties you want displayed. Because `Format-Wide` only shows a single property, its **Property** parameter only takes a single value, but the **Property** parameter of `Format-List` and `Format-Table` accepts a list of property names.

In this example, the default output of `Get-Process` cmdlet shows that we've two instances of Internet Explorer running.

PowerShell

```
Get-Process -Name iexplore
```

The default format for **Process** objects displays the properties shown here:

Output

```
 NPM(K)    PM(M)      WS(M)     CPU(s)      Id  SI ProcessName
 ------    -----      -----     ------      --  -- -----------
     32    25.52      10.25      13.11   12808   1 iexplore
     52    11.46      26.46       3.55   21748   1 iexplore
```

Using Format-Wide for single-item output

The `Format-Wide` cmdlet, by default, displays only the default property of an object. The information associated with each object is displayed in a single column:

PowerShell

```powershell
Get-Command -Verb Format | Format-Wide
```

Output

```
Format-Custom          Format-Hex
Format-List            Format-Table
Format-Wide
```

You can also specify a non-default property:

PowerShell

```powershell
Get-Command -Verb Format | Format-Wide -Property Noun
```

Output

```
Custom                 Hex
List                   Table
Wide
```

Controlling Format-Wide display with column

With the `Format-Wide` cmdlet, you can only display a single property at a time. This makes it useful for displaying large lists in multiple columns.

PowerShell

```powershell
Get-Command -Verb Format | Format-Wide -Property Noun -Column 3
```

Output

```
Custom                Hex                List
Table                 Wide
```

Using Format-List for a list view

The `Format-List` cmdlet displays an object in the form of a listing, with each property labeled and displayed on a separate line:

PowerShell

```powershell
Get-Process -Name iexplore | Format-List
```

Output

```
Id      : 12808
Handles : 578
CPU     : 13.140625
SI      : 1
Name    : iexplore

Id      : 21748
Handles : 641
CPU     : 3.59375
SI      : 1
Name    : iexplore
```

You can specify as many properties as you want:

PowerShell

```powershell
Get-Process -Name iexplore | Format-List -Property
ProcessName,FileVersion,StartTime,Id
```

Output

```
ProcessName : iexplore
FileVersion : 11.00.18362.1 (WinBuild.160101.0800)
StartTime   : 10/22/2019 11:23:58 AM
Id          : 12808

ProcessName : iexplore
FileVersion : 11.00.18362.1 (WinBuild.160101.0800)
StartTime   : 10/22/2019 11:23:57 AM
Id          : 21748
```

Getting detailed information using Format-List with wildcards

The `Format-List` cmdlet lets you use a wildcard as the value of its **Property** parameter. This lets you display detailed information. Often, objects include more information than you need, which is why PowerShell doesn't show all property values by default. To show all properties of an object, use the `Format-List -Property *` command. The following command generates more than 60 lines of output for a single process:

PowerShell

```powershell
Get-Process -Name iexplore | Format-List -Property *
```

Although the `Format-List` command is useful for showing detail, if you want an overview of output that includes many items, a simpler tabular view is often more useful.

Using Format-Table for tabular output

If you use the `Format-Table` cmdlet with no property names specified to format the output of the `Get-Process` command, you get exactly the same output as you do without a `Format` cmdlet. By default, PowerShell displays **Process** objects in a tabular format.

PowerShell

```powershell
Get-Service -Name win* | Format-Table
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  WinDefend          Windows Defender Antivirus Service
Running  WinHttpAutoProx... WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt            Windows Management Instrumentation
Running  WinRM              Windows Remote Management (WS-Manag...
```

> ⓘ **Note**
>
> `Get-Service` is only available on Windows platforms.

Improving Format-Table output

Although a tabular view is useful for displaying lots of information, it may be difficult to interpret if the display is too narrow for the data. In the previous example, the output is truncated. If you specify the **AutoSize** parameter when you run the `Format-Table` command, PowerShell calculates column widths based on the actual data displayed. This makes the columns readable.

PowerShell

```powershell
Get-Service -Name win* | Format-Table -AutoSize
```

Output

```
Status  Name               DisplayName
------  ----               -----------
Running WinDefend          Windows Defender Antivirus Service
Running WinHttpAutoProxySvc WinHTTP Web Proxy Auto-Discovery Service
Running Winmgmt            Windows Management Instrumentation
Running WinRM              Windows Remote Management (WS-Management)
```

The `Format-Table` cmdlet might still truncate data, but it only truncates at the end of the screen. Properties, other than the last one displayed, are given as much size as they need for their longest data element to display correctly.

PowerShell

```powershell
Get-Service -Name win* |
    Format-Table -Property Name, Status, StartType, DisplayName, DependentServices
-AutoSize
```

Output

```
Name                Status StartType DisplayName                                    DependentServi
                                                                                                ces
----                ------ --------- -----------                                    ---
-----------
WinDefend           Running Automatic Windows Defender Antivirus Service             {}
WinHttpAutoProxySvc Running    Manual WinHTTP Web Proxy Auto-Discovery Service
{NcaSvc, iphl…
Winmgmt             Running Automatic Windows Management Instrumentation
{vmms, TPHKLO…
WinRM               Running Automatic Windows Remote Management (WS-Management) {}
```

The `Format-Table` command assumes that properties are listed in order of importance. The cmdlet attempts to fully display the properties nearest the beginning. If the `Format-Table`

command can't display all the properties, it removes some columns from the display. You can see this behavior in the **DependentServices** property previous example.

Wrapping Format-Table output in columns

You can force lengthy `Format-Table` data to wrap within its display column using the **Wrap** parameter. Using the **Wrap** parameter may not do what you expect, since it uses default settings if you don't also specify **AutoSize**:

PowerShell

```
Get-Service -Name win* |
    Format-Table -Property Name, Status, StartType, DisplayName, DependentServices
-Wrap
```

Output

```
Name                 Status StartType DisplayName
DependentServi
                                                                                ces
----                 ------ --------- -----------                              ---
-----------
WinDefend            Running Automatic Windows Defender Antivirus Service       {}
WinHttpAutoProxySvc Running    Manual WinHTTP Web Proxy Auto-Discovery Service
{NcaSvc,

iphlpsvc}
Winmgmt              Running Automatic Windows Management Instrumentation
{vmms,

TPHKLOAD,

SUService,

smstsmgr…}
WinRM                Running Automatic Windows Remote Management (WS-Management) {}
```

Using the **Wrap** parameter by itself doesn't slow down processing very much. However, using **AutoSize** to format a recursive file listing of a large directory structure can take a long time and use lots of memory before displaying the first output items.

If you aren't concerned about system load, then **AutoSize** works well with the **Wrap** parameter. The initial columns still use as much width as needed to display items on one line, but the final column is wrapped, if necessary.

> **Note**
>
> Some columns may not be displayed when you specify the widest columns first. For best results, specify the smallest data elements first.

In the following example, we specify the widest properties first.

PowerShell

```
Get-Process -Name iexplore |
    Format-Table -Wrap -AutoSize -Property FileVersion, Path, Name, Id
```

Even with wrapping, the final **Id** column is omitted:

Output

```
FileVersion                         Path
Nam

e
-----------                         ----
---
11.00.18362.1 (WinBuild.160101.0800) C:\Program Files (x86)\Internet
Explorer\IEXPLORE.EXE iex

plo

re
11.00.18362.1 (WinBuild.160101.0800) C:\Program Files\Internet
Explorer\iexplore.exe       iex

plo

re
```

Organizing table output

Another useful parameter for tabular output control is **GroupBy**. Longer tabular listings in particular may be hard to compare. The **GroupBy** parameter groups output based on a property value. For example, we can group services by **StartType** for easier inspection, omitting the **StartType** value from the property listing:

PowerShell

```
Get-Service -Name win* | Sort-Object StartType | Format-Table -GroupBy StartType
```

Output

```
   StartType: Automatic
Status   Name               DisplayName
------   ----               -----------
Running  WinDefend          Windows Defender Antivirus Service
Running  Winmgmt            Windows Management Instrumentation
Running  WinRM              Windows Remote Management (WS-Managem…

   StartType: Manual
Status   Name               DisplayName
------   ----               -----------
Running  WinHttpAutoProxyS… WinHTTP Web Proxy Auto-Discovery Serv…
```

Last updated on 03/24/2025

Managing current location

When navigating folder systems in File Explorer, you usually have a specific working location - namely, the current open folder. Items in the current folder can be manipulated easily by clicking them. For command-line interfaces such as Cmd.exe, when you are in the same folder as a particular file, you can access it by specifying a relatively short name, rather than needing to specify the entire path to the file. The current directory is called the working directory.

PowerShell uses the noun **Location** to refer to the working directory, and implements a family of cmdlets to examine and manipulate your location.

Getting your current location (Get-Location)

To determine the path of your current directory location, enter the `Get-Location` command:

PowerShell

```
Get-Location
```

Output

```
Path
----
C:\Documents and Settings\PowerUser
```

> ⓘ Note
>
> The `Get-Location` cmdlet is similar to the **pwd** command in the BASH shell. The `Set-Location` cmdlet is similar to the **cd** command in Cmd.exe.

Setting your current location (Set-Location)

The `Get-Location` command is used with the `Set-Location` command. The `Set-Location` command allows you to specify your current directory location.

PowerShell

```
Set-Location -Path C:\Windows
```

After you enter the command, notice that you don't receive any direct feedback about the effect of the command. Most PowerShell commands that perform an action produce little or no output because the output isn't always useful. To verify that a successful directory change has occurred when you enter the `Set-Location` command, include the **PassThru** parameter when you enter the `Set-Location` command:

PowerShell

```
Set-Location -Path C:\Windows -PassThru
```

Output

```
Path
----
C:\WINDOWS
```

The **PassThru** parameter can be used with many Set commands in PowerShell to return information about the result for cases in which there is no default output.

You can specify paths relative to your current location in the same way as you would in most Unix and Windows command shells. In standard notation for relative paths, a period (`.`) represents your current folder, and a doubled period (`..`) represents the parent directory of your current location.

For example, if you are in the `C:\Windows` folder, a period (`.`) represents `C:\Windows` and double periods (`..`) represent `C:`. You can change from your current location to the root of the `C:` drive by typing:

PowerShell

```
Set-Location -Path .. -PassThru
```

Output

```
Path
----
C:\
```

The same technique works on PowerShell drives that aren't file system drives, such as `HKLM:`. You can set your location to the `HKLM\Software` key in the registry by typing:

PowerShell

```
Set-Location -Path HKLM:\SOFTWARE -PassThru
```

Output

```
Path
----
HKLM:\SOFTWARE
```

You can then change the directory location to the parent directory, using a relative path:

PowerShell

```
Set-Location -Path .. -PassThru
```

Output

```
Path
----
HKLM:\
```

You can type `Set-Location` or use any of the built-in PowerShell aliases for `Set-Location` (`cd`, `chdir`, `sl`). For example:

PowerShell

```
cd -Path C:\Windows
```

PowerShell

```
chdir -Path .. -PassThru
```

PowerShell

```
sl -Path HKLM:\SOFTWARE -PassThru
```

Saving and recalling recent locations (Push-Location and Pop-Location)

When changing locations, it's helpful to keep track of where you have been and to be able to return to your previous location. The `Push-Location` cmdlet in PowerShell creates an ordered

history (a "stack") of directory paths where you have been, and you can step back through the history of directory paths using the `Pop-Location` cmdlet.

For example, PowerShell typically starts in the user's home directory.

PowerShell

```
Get-Location

Path
----
C:\Documents and Settings\PowerUser
```

> **Note**
>
> The word **stack** has a special meaning in many programming settings, including .NET Framework. Like a physical stack of items, the last item you put onto the stack is the first item that you can pull off the stack. Adding an item to a stack is colloquially known as "pushing" the item onto the stack. Pulling an item off the stack is colloquially known as "popping" the item off the stack.

To push the current location onto the stack, and then move to the Local Settings folder, type:

PowerShell

```
Push-Location -Path "Local Settings"
```

You can then push the Local Settings location onto the stack and move to the Temp folder by typing:

PowerShell

```
Push-Location -Path Temp
```

You can verify that you changed directories by entering the `Get-Location` command:

PowerShell

```
Get-Location
```

Output

```
Path
----
C:\Documents and Settings\PowerUser\Local Settings\Temp
```

You can then pop back into the most recently visited directory by entering the `Pop-Location` command, and verify the change by entering the `Get-Location` command:

PowerShell

```
Pop-Location
Get-Location
```

Output

```
Path
----
C:\Documents and Settings\me\Local Settings
```

Just as with the `Set-Location` cmdlet, you can include the **PassThru** parameter when you enter the `Pop-Location` cmdlet to display the directory that you entered:

PowerShell

```
Pop-Location -PassThru
```

Output

```
Path
----
C:\Documents and Settings\PowerUser
```

You can also use the Location cmdlets with network paths. If you have a server named FS01 with a share named Public, you can change your location by typing

PowerShell

```
Set-Location \\FS01\Public
```

or

PowerShell

```
Push-Location \\FS01\Public
```

You can use the `Push-Location` and `Set-Location` commands to change the location to any available drive. For example, if you have a local CD-ROM drive with drive letter D that contains a data CD, you can change the location to the CD drive by entering the `Set-Location D:` command.

If the drive is empty, you get the following error message:

PowerShell

```
Set-Location D:
```

Output

```
Set-Location : Cannot find path 'D:\' because it does not exist.
```

When you are using a command-line interface, it's not convenient to use File Explorer to examine the available physical drives. Also, File Explorer would not show you the all the PowerShell drives. PowerShell provides a set of commands for manipulating PowerShell drives.

Last updated on 03/24/2025

Managing PowerShell drives

> This sample only applies to Windows platforms.

A *PowerShell drive* is a data store location that you can access like a filesystem drive in PowerShell. The PowerShell providers create some drives for you, such as the file system drives (including `C:` and `D:`), the registry drives (`HKCU:` and `HKLM:`), and the certificate drive (`Cert:`), and you can create your own PowerShell drives. These drives are useful, but they're available only within PowerShell. You can't access them using other Windows tools, such as File Explorer or `Cmd.exe`.

PowerShell uses the noun, **PSDrive**, for commands that work with PowerShell drives. For a list of the PowerShell drives in your PowerShell session, use the `Get-PSDrive` cmdlet.

PowerShell

```
Get-PSDrive
```

Output

```
Name       Provider      Root                                    CurrentLocation
----       --------      ----                                    ---------------
A          FileSystem    A:\
Alias      Alias
C          FileSystem    C:\                                ...And Settings\me
cert       Certificate   \
D          FileSystem    D:\
Env        Environment
Function   Function
HKCU       Registry      HKEY_CURRENT_USER
HKLM       Registry      HKEY_LOCAL_MACHINE
Variable   Variable
```

Although the drives in the display vary with the drives on your system, yours should look similar to the output of the `Get-PSDrive` command shown above.

filesystem drives are a subset of the PowerShell drives. You can identify the filesystem drives by the FileSystem entry in the Provider column. The filesystem drives in PowerShell are supported by the PowerShell FileSystem provider.

To see the syntax of the `Get-PSDrive` cmdlet, type a `Get-Command` command with the **Syntax** parameter:

PowerShell

```
Get-Command -Name Get-PSDrive -Syntax
```

Output

```
Get-PSDrive [[-Name] <String[]>] [-Scope <String>] [-PSProvider <String[]>] [-V
erbose] [-Debug] [-ErrorAction <ActionPreference>] [-ErrorVariable <String>] [-
OutVariable <String>] [-OutBuffer <Int32>]
```

The **PSProvider** parameter lets you display only the PowerShell drives that are supported by a particular provider. For example, to display only the PowerShell drives that are supported by the PowerShell FileSystem provider, type a `Get-PSDrive` command with the **PSProvider** parameter and the **FileSystem** value:

PowerShell

```
Get-PSDrive -PSProvider FileSystem
```

Output

```
Name       Provider      Root                                    CurrentLocation
----       --------      ----                                    ---------------
A          FileSystem    A:\
C          FileSystem    C:\                                ...nd Settings\PowerUser
D          FileSystem    D:\
```

To view the PowerShell drives that represent registry hives, use the **PSProvider** parameter to display only the PowerShell drives that are supported by the PowerShell Registry provider:

PowerShell

```
Get-PSDrive -PSProvider Registry
```

Output

```
Name       Provider      Root                                    CurrentLocation
----       --------      ----                                    ---------------
HKCU       Registry      HKEY_CURRENT_USER
HKLM       Registry      HKEY_LOCAL_MACHINE
```

You can also use the standard Location cmdlets with the PowerShell drives:

PowerShell

```
Set-Location HKLM:\SOFTWARE
Push-Location .\Microsoft
Get-Location
```

Output

```
Path
----
HKLM:\SOFTWARE\Microsoft
```

Adding new PowerShell drives

You can add your own PowerShell drives by using the `New-PSDrive` command. To get the syntax for the `New-PSDrive` command, enter the `Get-Command` command with the **Syntax** parameter:

PowerShell

```
Get-Command -Name New-PSDrive -Syntax
```

Output

```
New-[-Description <String>] [-Scope <String>] [-Credential <PSCredential>] [-
Verbose] [-Debug ]
[-ErrorAction <ActionPreference>] [-ErrorVariable <String>] [-OutVariable <St
ring>]
[-OutBuffer <Int32>] [-WhatIf] [-Confirm]
```

To create a new PowerShell drive, you must supply three parameters:

- A name for the drive (you can use any valid PowerShell name)
- The PSProvider - use `FileSystem` for filesystem locations and `Registry` for registry locations
- The root, that is, the path to the root of the new drive

For example, you can create a drive named `Office` that's mapped to the folder that contains the Microsoft Office applications on your computer, such as `C:\Program Files\MicrosoftOffice\OFFICE11`. To create the drive, type the following command:

PowerShell

```
New-PSDrive -Name Office -PSProvider FileSystem -Root "C:\Program Files\Microsoft
Office\OFFICE11"
```

Output

```
Name       Provider      Root                                    CurrentLocation
----       --------      ----                                    ---------------
Office     FileSystem    C:\Program Files\Microsoft Offic...
```

> **Note**
>
> In general, paths aren't case-sensitive.

A PowerShell drive is accessed using its name followed by a colon (`:`).

A PowerShell drive can make many tasks much simpler. For example, some of the most important keys in the Windows registry have extremely long paths, making them cumbersome to access and difficult to remember. Critical configuration information resides under `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion`. To view and change items in the CurrentVersion registry key, you can create a PowerShell drive that's rooted in that key by typing:

PowerShell

```
New-PSDrive -Name cvkey -PSProvider Registry -Root
HKLM\Software\Microsoft\Windows\CurrentVersion
```

Output

```
Name       Provider      Root                                    CurrentLocation
----       --------      ----                                    ---------------
cvkey      Registry      HKLM\Software\Microsoft\Windows\...
```

You can then change location to the **cvkey:** drive as you would any other drive:

PowerShell

```
cd cvkey:
```

or:

PowerShell

```
Set-Location cvkey: -PassThru
```

Output

```
Path
----
cvkey:\
```

The `New-PSDrive` cmdlet adds the new drive only to the current PowerShell session. If you close the PowerShell window, the new drive is lost. To save a PowerShell drive, use the `Export-Console` cmdlet to export the current PowerShell session, and then use the `powershell.exe` **PSConsoleFile** parameter to import it. Or, add the new drive to your Windows PowerShell profile.

Deleting PowerShell drives

You can delete drives from PowerShell using the `Remove-PSDrive` cmdlet. For example, if you added the `Office:` PowerShell drive, as shown in the `New-PSDrive` topic, you can delete it by typing:

PowerShell

```
Remove-PSDrive -Name Office
```

To delete the `cvkey:` PowerShell drive, use the following command:

PowerShell

```
Remove-PSDrive -Name cvkey
```

However, you can't delete it while you are in the drive. For example:

PowerShell

```
cd office:
Remove-PSDrive -Name Office
```

Output

```
Remove-PSDrive : Cannot remove drive 'Office' because it is in use.
At line:1 char:15
+ Remove-PSDrive  <<<< -Name Office
```

Adding and removing drives outside PowerShell

PowerShell detects filesystem drives that are added or removed in Windows, including:

- network drives that are mapped
- USB drives that are attached
- Drives that are deleted using the `net use` command or from a Windows Script Host (WSH) script

---

Last updated on 03/24/2025

Working with files and folders

Navigating through PowerShell drives and manipulating the items on them is similar to manipulating files and folders on Windows disk drives. This article discusses how to deal with specific file and folder manipulation tasks using PowerShell.

Listing all files and folders within a folder

You can get all items directly within a folder using `Get-ChildItem`. Add the optional **Force** parameter to display hidden or system items. For example, this command displays the direct contents of PowerShell Drive `C:`.

PowerShell

```
Get-ChildItem -Path C:\ -Force
```

The command lists only the directly contained items, much like using the `dir` command in `cmd.exe` or `ls` in a Unix shell. To show items in subfolder, you need to specify the **Recurse** parameter. The following command lists everything on the `C:` drive:

PowerShell

```
Get-ChildItem -Path C:\ -Force -Recurse
```

`Get-ChildItem` can filter items with its **Path**, **Filter**, **Include**, and **Exclude** parameters, but those are typically based only on name. You can perform complex filtering based on other properties of items using `Where-Object`.

The following command finds all executables within the Program Files folder that were last modified after October 1, 2005 and that are neither smaller than 1 megabyte nor larger than 10 megabytes:

PowerShell

```
Get-ChildItem -Path $Env:ProgramFiles -Recurse -Include *.exe |
    Where-Object -FilterScript {
        ($_.LastWriteTime -gt '2005-10-01') -and ($_.Length -ge 1mb) -and
($_.Length -le 10mb)
    }
```

Copying files and folders

Copying is done with `Copy-Item`. The following command backs up your PowerShell profile script:

PowerShell

```
if (Test-Path -Path $PROFILE) {
    Copy-Item -Path $PROFILE -Destination $($PROFILE -replace 'ps1$', 'bak')
}
```

The `Test-Path` command checks whether the profile script exists.

If the destination file already exists, the copy attempt fails. To overwrite a pre-existing destination, use the **Force** parameter:

PowerShell

```
if (Test-Path -Path $PROFILE) {
    Copy-Item -Path $PROFILE -Destination $($PROFILE -replace 'ps1$', 'bak') -Force
}
```

This command works even when the destination is read-only.

Folder copying works the same way. This command copies the folder `C:\temp\test1` to the new folder `C:\temp\DeleteMe` recursively:

PowerShell

```
Copy-Item C:\temp\test1 -Recurse C:\temp\DeleteMe
```

You can also copy a selection of items. The following command copies all `.txt` files contained anywhere in `C:\data` to `C:\temp\text`:

PowerShell

```
Copy-Item -Filter *.txt -Path C:\data -Recurse -Destination C:\temp\text
```

You can still run native commands like `xcopy.exe` and `robocopy.exe` to copy files.

Creating files and folders

Creating new items works the same on all PowerShell providers. If a PowerShell provider has more than one type of item—for example, the FileSystem PowerShell provider distinguishes between directories and files—you need to specify the item type.

This command creates a new folder `C:\temp\New Folder`:

PowerShell

```
New-Item -Path 'C:\temp\New Folder' -ItemType Directory
```

This command creates a new empty file `C:\temp\New Folder\file.txt`

PowerShell

```
New-Item -Path 'C:\temp\New Folder\file.txt' -ItemType File
```

> **ⓘ Important**
>
> When using the **Force** switch with the `New-Item` command to create a folder, and the folder already exists, it *won't* overwrite or replace the folder. It will simply return the existing folder object. However, if you use `New-Item -Force` on a file that already exists, the file is overwritten.

Removing all files and folders within a folder

You can remove contained items using `Remove-Item`, but you will be prompted to confirm the removal if the item contains anything else. For example, if you attempt to delete the folder `C:\temp\DeleteMe` that contains other items, PowerShell prompts you for confirmation before deleting the folder:

PowerShell

```
Remove-Item -Path C:\temp\DeleteMe
```

Output

```
Confirm
The item at C:\temp\DeleteMe has children and the Recurse parameter wasn't
specified. If you continue, all children will be removed with the item. Are you
sure you want to continue?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "Y"):
```

If you don't want to be prompted for each contained item, specify the **Recurse** parameter:

PowerShell

```
Remove-Item -Path C:\temp\DeleteMe -Recurse
```

Mapping a local folder as a drive

You can also map a local folder, using the `New-PSDrive` command. The following command creates a local drive `P:` rooted in the local Program Files directory, visible only from the PowerShell session:

PowerShell

```
New-PSDrive -Name P -Root $Env:ProgramFiles -PSProvider FileSystem
```

Just as with network drives, drives mapped within PowerShell are immediately visible to the PowerShell shell. To create a mapped drive visible from File Explorer, use the **Persist** parameter. However, only remote paths can be used with **Persist**.

Reading a text file into an array

One of the more common storage formats for text data is in a file with separate lines treated as distinct data elements. The `Get-Content` cmdlet can be used to read an entire file in one step, as shown here:

PowerShell

```
Get-Content -Path $PROFILE
# Load modules and change to the PowerShell-Docs repository folder
Import-Module posh-git
Set-Location C:\Git\PowerShell-Docs
```

`Get-Content` treats the data read from the file as an array, with one element per line of file content. You can confirm this by checking the **Length** of the returned content:

PowerShell

```
PS> (Get-Content -Path $PROFILE).Length
3
```

This command is most useful for getting lists of information into PowerShell. For example, you might store a list of computer names or IP addresses in the file `C:\temp\domainMembers.txt`,

with one name on each line of the file. You can use `Get-Content` to retrieve the file contents and put them in the variable `$Computers`:

PowerShell

```
$Computers = Get-Content -Path C:\temp\DomainMembers.txt
```

`$Computers` is now an array containing a computer name in each element.

Last updated on 10/18/2023

Working with files, folders and registry keys

> This sample only applies to Windows platforms.

PowerShell uses the noun **Item** to refer to items found on a PowerShell drive. When dealing with the PowerShell FileSystem provider, an **Item** might be a file, a folder, or the PowerShell drive. Listing and working with these items is a critical basic task in most administrative settings, so we want to discuss these tasks in detail.

Enumerating files, folders, and registry keys

Since getting a collection of items from a particular location is such a common task, the `Get-ChildItem` cmdlet is designed specifically to return all items found within a container such as a folder.

If you want to return all files and folders that are contained directly within the folder `C:\Windows`, type:

```
PS> Get-ChildItem -Path C:\Windows
    Directory: Microsoft.PowerShell.Core\FileSystem::C:\Windows

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
-a---         2006-05-16   8:10 AM          0 0.log
-a---         2005-11-29   3:16 PM         97 acc1.txt
-a---         2005-10-23  11:21 PM       3848 actsetup.log
...
```

The listing looks similar to what you would see when you enter the `dir` command in `cmd.exe`, or the `ls` command in a Unix command shell.

You can perform complex listings using parameters of the `Get-ChildItem` cmdlet. You can see the syntax the `Get-ChildItem` cmdlet by typing:

PowerShell

```
Get-Command -Name Get-ChildItem -Syntax
```

These parameters can be mixed and matched to get highly customized output.

Listing all contained items

To see both the items inside a Windows folder and any items that are contained within the subfolders, use the **Recurse** parameter of `Get-ChildItem`. The listing displays everything within the Windows folder and the items in its subfolders. For example:

```
PS> Get-ChildItem -Path C:\WINDOWS -Recurse

    Directory: Microsoft.PowerShell.Core\FileSystem::C:\WINDOWS
    Directory: Microsoft.PowerShell.Core\FileSystem::C:\WINDOWS\AppPatch
Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
-a---         2004-08-04   8:00 AM    1852416 AcGenral.dll
...
```

Filtering items by name

To display only the names of items, use the **Name** parameter of `Get-ChildItem`:

```
PS> Get-ChildItem -Path C:\WINDOWS -Name
addins
AppPatch
assembly
...
```

Forcibly listing hidden items

Items that are hidden in File Explorer or `cmd.exe` aren't displayed in the output of a `Get-ChildItem` command. To display hidden items, use the **Force** parameter of `Get-ChildItem`. For example:

PowerShell

```
Get-ChildItem -Path C:\Windows -Force
```

This parameter is named **Force** because you can forcibly override the normal behavior of the `Get-ChildItem` command. **Force** is a widely used parameter that forces an action that a cmdlet

wouldn't normally perform, although it can't perform any action that compromises the security of the system.

Matching item names with wildcards

The `Get-ChildItem` command accepts wildcards in the path of the items to list.

Because wildcard matching is handled by the PowerShell engine, all cmdlets that accepts wildcards use the same notation and have the same matching behavior. The PowerShell wildcard notation includes:

- Asterisk (`*`) matches zero or more occurrences of any character.
- Question mark (`?`) matches exactly one character.
- Left bracket (`[`) character and right bracket (`]`) character surround a set of characters to be matched.

Here are some examples of how wildcard specification works.

To find all files in the Windows directory with the suffix `.log` and exactly five characters in the base name, enter the following command:

```
PS> Get-ChildItem -Path C:\Windows\?????.log

    Directory: Microsoft.PowerShell.Core\FileSystem::C:\Windows
Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
...
-a---        2006-05-11   6:31 PM     204276 ocgen.log
-a---        2006-05-11   6:31 PM      22365 ocmsn.log
...
-a---        2005-11-11   4:55 AM         64 setup.log
-a---        2005-12-15   2:24 PM      17719 VxSDM.log
...
```

To find all files that begin with the letter `x` in the Windows directory, type:

PowerShell

```
Get-ChildItem -Path C:\Windows\x*
```

To find all files whose names begin with "x" or "z", type:

PowerShell

```
Get-ChildItem -Path C:\Windows\[xz]*
```

For more information about wildcards, see about_Wildcards.

Excluding items

You can exclude specific items using the **Exclude** parameter of `Get-ChildItem`. This lets you perform complex filtering in a single statement.

For example, suppose you are trying to find the Windows Time Service DLL in the **System32** folder, and all you can remember about the DLL name is that it begins with "W" and has "32" in it.

An expression like `w*32*.dll` will find all DLLs that satisfy the conditions, but you may want to further filter out the files and omit any win32 files. You can omit these files using the **Exclude** parameter with the pattern `win*`:

```
PS> Get-ChildItem -Path C:\WINDOWS\System32\w*32*.dll -Exclude win*

    Directory: C:\WINDOWS\System32

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---           3/18/2019  9:43 PM         495616 w32time.dll
-a---           3/18/2019  9:44 PM          35328 w32topl.dll
-a---           1/24/2020  5:44 PM         401920 Wldap32.dll
-a---          10/10/2019  5:40 PM         442704 ws2_32.dll
-a---           3/18/2019  9:44 PM          66048 wsnmp32.dll
-a---           3/18/2019  9:44 PM          18944 wsock32.dll
-a---           3/18/2019  9:44 PM          64792 wtsapi32.dll
```

Mixing Get-ChildItem parameters

You can use several of the parameters of the `Get-ChildItem` cmdlet in the same command. Before you mix parameters, be sure that you understand wildcard matching. For example, the following command returns no results:

PowerShell

```
Get-ChildItem -Path C:\Windows\*.dll -Recurse -Exclude [a-y]*.dll
```

There are no results, even though there are two DLLs that begin with the letter "z" in the Windows folder.

No results were returned because we specified the wildcard as part of the path. Even though the command was recursive, the `Get-ChildItem` cmdlet restricted the items to those that are in the Windows folder with names ending with `.dll`.

To specify a recursive search for files whose names match a special pattern, use the **Include** parameter.

```
PS> Get-ChildItem -Path C:\Windows -Include *.dll -Recurse -Exclude [a-y]*.dll

    Directory: Microsoft.PowerShell.Core\FileSystem::C:\Windows\System32\Setup

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
-a---         2004-08-04   8:00 AM       8261 zoneoc.dll

    Directory: Microsoft.PowerShell.Core\FileSystem::C:\Windows\System32

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
-a---         2004-08-04   8:00 AM     337920 zipfldr.dll
```

Last updated on 03/24/2025

Working with registry entries

> This sample only applies to Windows platforms.

Because registry entries are properties of keys and, as such, can't be directly browsed, we need to take a slightly different approach when working with them.

Listing registry entries

There are many different ways to examine registry entries. The simplest way is to get the property names associated with a key. For example, to see the names of the entries in the registry key `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion`, use `Get-Item`. Registry keys have a property with the generic name of "Property" that's a list of registry entries in the key. The following command selects the Property property and expands the items so that they're displayed in a list:

PowerShell

```
Get-Item -Path
Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion |
    Select-Object -ExpandProperty Property
```

Output

```
DevicePath
MediaPathUnexpanded
ProgramFilesDir
CommonFilesDir
ProductId
```

To view the registry entries in a more readable form, use `Get-ItemProperty`:

PowerShell

```
Get-ItemProperty -Path
Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion
```

Output

```
ProgramFilesDir          : C:\Program Files
CommonFilesDir           : C:\Program Files\Common Files
```

```
ProgramFilesDir (x86)    : C:\Program Files (x86)
CommonFilesDir (x86)     : C:\Program Files (x86)\Common Files
CommonW6432Dir           : C:\Program Files\Common Files
DevicePath               : C:\WINDOWS\inf
MediaPathUnexpanded      : C:\WINDOWS\Media
ProgramFilesPath         : C:\Program Files
ProgramW6432Dir          : C:\Program Files
SM_ConfigureProgramsName : Set Program Access and Defaults
SM_GamesName             : Games
PSPath                   :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWA
                           RE\Microsoft\Windows\CurrentVersion
PSParentPath             :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWA
                           RE\Microsoft\Windows
PSChildName              : CurrentVersion
PSDrive                  : HKLM
PSProvider               : Microsoft.PowerShell.Core\Registry
```

The Windows PowerShell-related properties for the key are all prefixed with "PS", such as **PSPath**, **PSParentPath**, **PSChildName**, and **PSProvider**.

You can use the `*.*` notation for referring to the current location. You can use `Set-Location` to change to the **CurrentVersion** registry container first:

PowerShell

```powershell
Set-Location -Path
Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion
```

Alternatively, you can use the built-in `HKLM:` PSDrive with `Set-Location`:

PowerShell

```powershell
Set-Location -Path HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion
```

You can then use the `.` notation for the current location to list the properties without specifying a full path:

PowerShell

```powershell
Get-ItemProperty -Path .
```

Output

```
...
DevicePath          : C:\WINDOWS\inf
```

```
MediaPathUnexpanded : C:\WINDOWS\Media
ProgramFilesDir     : C:\Program Files
...
```

Path expansion works the same as it does within the filesystem, so from this location you can get the **ItemProperty** listing for `HKLM:\SOFTWARE\Microsoft\Windows\Help` using `Get-ItemProperty -Path ..\Help`.

Getting a single registry entry

If you want to retrieve a specific entry in a registry key, you can use one of several possible approaches. This example finds the value of **DevicePath** in `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion`.

Using `Get-ItemProperty`, use the **Path** parameter to specify the name of the key, and the **Name** parameter to specify the name of the **DevicePath** entry.

PowerShell

```
Get-ItemProperty -Path HKLM:\Software\Microsoft\Windows\CurrentVersion -Name
DevicePath
```

Output

```
DevicePath   : C:\WINDOWS\inf
PSPath       :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\C
urrentVersion
PSParentPath :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows
PSChildName  : CurrentVersion
PSDrive      : HKLM
PSProvider   : Microsoft.PowerShell.Core\Registry
```

This command returns the standard Windows PowerShell properties as well as the **DevicePath** property.

> ⓘ **Note**
>
> Although `Get-ItemProperty` has **Filter**, **Include**, and **Exclude** parameters, they can't be used to filter by property name. These parameters refer to registry keys, which are item paths and not registry entries, which are item properties.

Another option is to use the `reg.exe` command line tool. For help with `reg.exe`, type `reg.exe /?` at a command prompt. To find the **DevicePath** entry, use `reg.exe` as shown in the following command:

PowerShell

```
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion /v DevicePath
```

Output

```
! REG.EXE VERSION 3.0

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion
    DevicePath  REG_EXPAND_SZ   %SystemRoot%\inf
```

You can also use the **WshShell** COM object to find some registry entries, although this method doesn't work with large binary data or with registry entry names that include characters such as backslash (`\`). Append the property name to the item path with a `\` separator:

PowerShell

```
(New-Object -ComObject
WScript.Shell).RegRead("HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\DevicePath")
```

Output

```
%SystemRoot%\inf
```

Setting a single registry entry

If you want to change a specific entry in a registry key, you can use one of several possible approaches. This example modifies the **Path** entry under `HKEY_CURRENT_USER\Environment`. The **Path** entry specifies where to find executable files.

1. Retrieve the current value of the **Path** entry using `Get-ItemProperty`.
2. Add the new value, separating it with a `;`.
3. Use `Set-ItemProperty` with the specified key, entry name, and value to modify the registry entry.

PowerShell

```
$value = Get-ItemProperty -Path HKCU:\Environment -Name Path
$newpath = $value.Path += ";C:\src\bin\"
Set-ItemProperty -Path HKCU:\Environment -Name Path -Value $newpath
```

> **Note**
>
> Although `Set-ItemProperty` has **Filter**, **Include**, and **Exclude** parameters, they can't be used to filter by property name. These parameters refer to registry keys—which are item paths—and not registry entries—which are item properties.

Another option is to use the Reg.exe command line tool. For help with reg.exe, type `reg.exe /?` at a command prompt.

The following example changes the **Path** entry by removing the path added in the example above. `Get-ItemProperty` is still used to retrieve the current value to avoid having to parse the string returned from `reg query`. The **SubString** and **LastIndexOf** methods are used to retrieve the last path added to the **Path** entry.

PowerShell

```
$value = Get-ItemProperty -Path HKCU:\Environment -Name Path
$newpath = $value.Path.SubString(0, $value.Path.LastIndexOf(';'))
reg add HKCU\Environment /v Path /d $newpath /f
```

Output

```
The operation completed successfully.
```

Creating new registry entries

To add a new entry named "PowerShellPath" to the **CurrentVersion** key, use `New-ItemProperty` with the path to the key, the entry name, and the value of the entry. For this example, we will take the value of the Windows PowerShell variable `$PSHOME`, which stores the path to the installation directory for Windows PowerShell.

You can add the new entry to the key using the following command, and the command also returns information about the new entry:

PowerShell

```powershell
$newItemPropertySplat = @{
    Path = 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion'
    Name = 'PowerShellPath'
    PropertyType = 'String'
    Value = $PSHOME
}
New-ItemProperty @newItemPropertySplat
```

Output

```
PSPath         :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\C
urrentVersion
PSParentPath   :
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows
PSChildName    : CurrentVersion
PSDrive        : HKLM
PSProvider     : Microsoft.PowerShell.Core\Registry
PowerShellPath : C:\Program Files\Windows PowerShell\v1.0
```

The **PropertyType** must be the name of a **Microsoft.Win32.RegistryValueKind** enumeration member from the following table:

- `String` - Used for REG_SZ values. Pass a `[System.String]` object to the **Value** parameter.
- `ExpandString` - Used for REG_EXPAND_SZ values. Pass a `[System.String]` object to the **Value** parameter. The string should contain unexpanded references to environment variables that are expanded when the value is retrieved.
- `Binary` - Used for REG_BINARY values. Pass a `[System.Byte[]]` object to the **Value** parameter.
- `DWord` - Used for REG_DWORD values. Pass a `[System.Int32]` object to the **Value** parameter.
- `MultiString` - Used for REG_MULTI_SZ values. Pass a `[System.String[]]` object to the **Value** parameter.
- `QWord` - Used for REG_QWORD values. Pass a `[System.Int64]` object to the **Value** parameter.

You can add a registry entry to multiple locations by specifying an array of values for the **Path** parameter:

PowerShell

```powershell
$newItemPropertySplat = @{
    Name = 'PowerShellPath'
    PropertyType = 'String'
    Value = $PSHOME
```

```
    Path = 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion',
           'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion'
}
New-ItemProperty @newItemPropertySplat
```

You can also overwrite a pre-existing registry entry value by adding the **Force** parameter to any `New-ItemProperty` command.

The following examples show how to create new registry entries of various types. The registry values are created in a new key named **MySoftwareKey** under `HKEY_CURRENT_USER\Software`. The `$key` variable is used to store the new key object.

PowerShell

```
$key = New-Item -Path HKCU:\Software -Name MySoftwareKey
$newItemPropertySplat = @{
    Path = $key.PSPath
    Name = 'DefaultFolders'
    PropertyType = 'MultiString'
    Value = 'Home', 'Temp', 'Publish'
}
New-ItemProperty @newItemPropertySplat
```

Output

```
DefaultFolders : {Home, Temp, Publish}
PSPath         :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\MySoftwareKey
PSParentPath   : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software
PSChildName    : MySoftwareKey
PSProvider     : Microsoft.PowerShell.Core\Registry
```

You can use the **PSPath** property of the key object in subsequent commands.

PowerShell

```
New-ItemProperty -Path $key.PSPath -Name MaxAllowed -PropertyType QWord -Value 1024
```

Output

```
MaxAllowed   : 1024
PSPath       :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\MySoftwareKey
PSParentPath : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software
PSChildName  : MySoftwareKey
PSProvider   : Microsoft.PowerShell.Core\Registry
```

You can also pipe `$key` to `New-ItemProperty` to add a value to the key.

PowerShell

```powershell
$date = Get-Date -Format 'dd-MMM-yyyy'
$newItemPropertySplat = @{
    Name = 'BinaryDate'
    PropertyType = 'Binary'
    Value = ([System.Text.Encoding]::UTF8.GetBytes($date))
}
$key | New-ItemProperty @newItemPropertySplat
```

Output

```
BinaryDate   : {51, 49, 45, 74…}
PSPath       :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\MySoftwareKey
PSParentPath : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software
PSChildName  : MySoftwareKey
PSProvider   : Microsoft.PowerShell.Core\Registry
```

Displaying the content of `$key` shows the new entries.

PowerShell

```powershell
$key
```

Output

```
    Hive: HKEY_CURRENT_USER\Software

Name                           Property
----                           --------
MySoftwareKey                  DefaultFolders : {Home, Temp, Publish}
                               MaxAllowed     : 1024
                               BinaryDate     : {51, 49, 45, 74…}
```

The following example shows the value type for each kind of registry entry:

PowerShell

```powershell
$key.GetValueNames() | Select-Object @{n='ValueName';e={$_}},
     @{n='ValueKind';e={$key.GetValueKind($_)}},
     @{n='Type';e={$key.GetValue($_).GetType()}},
     @{n='Value';e={$key.GetValue($_)}}
```

Output

```
ValueName        ValueKind Type            Value
---------        --------- ----            -----
DefaultFolders MultiString System.String[] {Home, Temp, Publish}
MaxAllowed           QWord System.Int64    1024
BinaryDate          Binary System.Byte[]   {51, 49, 45, 74…}
```

Renaming registry entries

To rename the **PowerShellPath** entry to "PSHome," use `Rename-ItemProperty`:

PowerShell

```powershell
$renameItemPropertySplat = @{
    Path = 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion'
    Name = 'PowerShellPath'
    NewName = 'PSHome'
}
Rename-ItemProperty @renameItemPropertySplat
```

To display the renamed value, add the **PassThru** parameter to the command.

PowerShell

```powershell
$renameItemPropertySplat = @{
    Path = 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion'
    Name = 'PowerShellPath'
    NewName = 'PSHome'
    PassThru = $true
}
Rename-ItemProperty @renameItemPropertySplat
```

Deleting registry entries

To delete both the PSHome and PowerShellPath registry entries, use `Remove-ItemProperty`:

PowerShell

```powershell
Remove-ItemProperty -Path HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion -Name
PSHome
Remove-ItemProperty -Path HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion -Name
PowerShellPath
```

Last updated on 02/24/2025

Working with registry keys

> This sample only applies to Windows platforms.

Because registry keys are items on PowerShell drives, working with them is very similar to working with files and folders. One critical difference is that every item on a registry-based PowerShell drive is a container, just like a folder on a file system drive. However, registry entries and their associated values are properties of the items, not distinct items.

Listing all subkeys of a registry key

You can show all items directly within a registry key using `Get-ChildItem`. Add the optional **Force** parameter to display hidden or system items. For example, this command displays the items directly within PowerShell drive `HKCU:`, which corresponds to the `HKEY_CURRENT_USER` registry hive:

PowerShell

```
Get-ChildItem -Path HKCU:\ | Select-Object Name
```

Output

```
   Hive: Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER

Name
----
HKEY_CURRENT_USER\AppEvents
HKEY_CURRENT_USER\Console
HKEY_CURRENT_USER\Control Panel
HKEY_CURRENT_USER\DirectShow
HKEY_CURRENT_USER\dummy
HKEY_CURRENT_USER\Environment
HKEY_CURRENT_USER\EUDC
HKEY_CURRENT_USER\Keyboard Layout
HKEY_CURRENT_USER\MediaFoundation
HKEY_CURRENT_USER\Microsoft
HKEY_CURRENT_USER\Network
HKEY_CURRENT_USER\Printers
HKEY_CURRENT_USER\Software
HKEY_CURRENT_USER\System
HKEY_CURRENT_USER\Uninstall
HKEY_CURRENT_USER\WXP
HKEY_CURRENT_USER\Volatile Environment
```

These are the top-level keys visible under `HKEY_CURRENT_USER` in the Registry Editor (`regedit.exe`).

You can also specify this registry path by specifying the Registry provider's name, followed by `::`. The Registry provider's full name is `Microsoft.PowerShell.Core\Registry`, but this can be shortened to just `Registry`. Any of the following commands will list the contents directly under `HKCU:`.

PowerShell

```
Get-ChildItem -Path Registry::HKEY_CURRENT_USER
Get-ChildItem -Path Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER
Get-ChildItem -Path Registry::HKCU
Get-ChildItem -Path Microsoft.PowerShell.Core\Registry::HKCU
Get-ChildItem HKCU:
```

These commands list only the directly contained items, much like using `DIR` in `cmd.exe` or `ls` in a Unix shell. To show contained items, you need to specify the **Recurse** parameter. To list all registry keys in `HKCU:`, use the following command.

PowerShell

```
Get-ChildItem -Path HKCU:\ -Recurse
```

`Get-ChildItem` can perform complex filtering capabilities through its **Path**, **Filter**, **Include**, and **Exclude** parameters, but those parameters are typically based only on name. You can perform complex filtering based on other properties of items using the `Where-Object` cmdlet. The following command finds all keys within `HKCU:\Software` that have no more than one subkey and also have exactly four values:

PowerShell

```
Get-ChildItem -Path HKCU:\Software -Recurse |
    Where-Object {($_.SubKeyCount -le 1) -and ($_.ValueCount -eq 4) }
```

Copying keys

Copying is done with `Copy-Item`. The following example copies the `CurrentVersion` subkey of `HKLM:\SOFTWARE\Microsoft\Windows\` and all of its properties to `HKCU:\`.

PowerShell

```
Copy-Item -Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion' -Destination
HKCU:
```

If you examine this new key in the registry editor or using `Get-ChildItem`, you notice that you don't have copies of the contained subkeys in the new location. In order to copy all of the contents of a container, you need to specify the **Recurse** parameter. To make the preceding copy command recursive, you would use this command:

PowerShell

```
Copy-Item -Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion' -Destination
HKCU: -Recurse
```

You can still use other tools you already have available to perform filesystem copies. Any registry editing tools—including `reg.exe`, `regini.exe`, `regedit.exe`, and COM objects that support registry editing, such as **WScript.Shell** and WMI's **StdRegProv** class can be used from within PowerShell.

Creating keys

Creating new keys in the registry is simpler than creating a new item in a file system. Because all registry keys are containers, you don't need to specify the item type. Just provide an explicit path, such as:

PowerShell

```
New-Item -Path HKCU:\Software_DeleteMe
```

You can also use a provider-based path to specify a key:

PowerShell

```
New-Item -Path Registry::HKCU\Software_DeleteMe
```

Deleting keys

Deleting items is essentially the same for all providers. The following commands silently remove items:

PowerShell

```
Remove-Item -Path HKCU:\Software_DeleteMe
Remove-Item -Path 'HKCU:\key with spaces in the name'
```

Removing all keys under a specific key

You can remove contained items using `Remove-Item`, but you will be prompted to confirm the removal if the item contains anything else. For example, if we attempt to delete the `HKCU:\CurrentVersion` subkey we created, we see this:

PowerShell

```
Remove-Item -Path HKCU:\CurrentVersion
```

Output

```
Confirm
The item at HKCU:\CurrentVersion\AdminDebug has children and the Recurse
parameter was not specified. If you continue, all children will be removed with
the item. Are you sure you want to continue?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"):
```

To delete contained items without prompting, specify the **Recurse** parameter:

PowerShell

```
Remove-Item -Path HKCU:\CurrentVersion -Recurse
```

If you wanted to remove all items within `HKCU:\CurrentVersion` but not `HKCU:\CurrentVersion` itself, you could instead use:

PowerShell

```
Remove-Item -Path HKCU:\CurrentVersion\* -Recurse
```

Last updated on 03/24/2025

Creating a custom input box

> This sample only applies to Windows platforms.

Script a graphical custom input box using Microsoft .NET Framework form-building features in Windows PowerShell 3.0 and later releases.

Create a custom, graphical input box

Copy and then paste the following into Windows PowerShell ISE, and then save it as a PowerShell script (`.ps1`) file.

PowerShell

```
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing

$form = New-Object System.Windows.Forms.Form
$form.Text = 'Data Entry Form'
$form.Size = New-Object System.Drawing.Size(300,200)
$form.StartPosition = 'CenterScreen'

$okButton = New-Object System.Windows.Forms.Button
$okButton.Location = New-Object System.Drawing.Point(75,120)
$okButton.Size = New-Object System.Drawing.Size(75,23)
$okButton.Text = 'OK'
$okButton.DialogResult = [System.Windows.Forms.DialogResult]::OK
$form.AcceptButton = $okButton
$form.Controls.Add($okButton)

$cancelButton = New-Object System.Windows.Forms.Button
$cancelButton.Location = New-Object System.Drawing.Point(150,120)
$cancelButton.Size = New-Object System.Drawing.Size(75,23)
$cancelButton.Text = 'Cancel'
$cancelButton.DialogResult = [System.Windows.Forms.DialogResult]::Cancel
$form.CancelButton = $cancelButton
$form.Controls.Add($cancelButton)

$label = New-Object System.Windows.Forms.Label
$label.Location = New-Object System.Drawing.Point(10,20)
$label.Size = New-Object System.Drawing.Size(280,20)
$label.Text = 'Please enter the information in the space below:'
$form.Controls.Add($label)

$textBox = New-Object System.Windows.Forms.TextBox
$textBox.Location = New-Object System.Drawing.Point(10,40)
$textBox.Size = New-Object System.Drawing.Size(260,20)
```

```
$form.Controls.Add($textBox)

$form.Topmost = $true

$form.Add_Shown({$textBox.Select()})
$result = $form.ShowDialog()

if ($result -eq [System.Windows.Forms.DialogResult]::OK)
{
    $x = $textBox.Text
    $x
}
```

The script begins by loading two .NET Framework classes: **System.Drawing** and **System.Windows.Forms**. You then start a new instance of the .NET Framework class **System.Windows.Forms.Form**. That provides a blank form or window to which you can start adding controls.

PowerShell

```
$form = New-Object System.Windows.Forms.Form
```

After you create an instance of the Form class, assign values to three properties of this class.

- **Text.** This becomes the title of the window.
- **Size.** This is the size of the form, in pixels. The preceding script creates a form that's 300 pixels wide by 200 pixels tall.
- **StartingPosition.** This optional property is set to **CenterScreen** in the preceding script. If you don't add this property, Windows selects a location when the form is opened. By setting the **StartingPosition** to **CenterScreen**, you're automatically displaying the form in the middle of the screen each time it loads.

PowerShell

```
$form.Text = 'Data Entry Form'
$form.Size = New-Object System.Drawing.Size(300,200)
$form.StartPosition = 'CenterScreen'
```

Next, create an **OK** button for your form. Specify the size and behavior of the **OK** button. In this example, the button position is 120 pixels from the form's top edge, and 75 pixels from the left edge. The button height is 23 pixels, while the button length is 75 pixels. The script uses predefined Windows Forms types to determine the button behaviors.

PowerShell

```
$okButton = New-Object System.Windows.Forms.Button
$okButton.Location = New-Object System.Drawing.Point(75,120)
$okButton.Size = New-Object System.Drawing.Size(75,23)
$okButton.Text = 'OK'
$okButton.DialogResult = [System.Windows.Forms.DialogResult]::OK
$form.AcceptButton = $OKButton
$form.Controls.Add($OKButton)
```

Similarly, you create a **Cancel** button. The **Cancel** button is 120 pixels from the top, but 150 pixels from the left edge of the window.

PowerShell

```
$cancelButton = New-Object System.Windows.Forms.Button
$cancelButton.Location = New-Object System.Drawing.Point(150,120)
$cancelButton.Size = New-Object System.Drawing.Size(75,23)
$cancelButton.Text = 'Cancel'
$cancelButton.DialogResult = [System.Windows.Forms.DialogResult]::Cancel
$form.CancelButton = $cancelButton
$form.Controls.Add($cancelButton)
```

Next, provide label text on your window that describes the information you want users to provide.

PowerShell

```
$label = New-Object System.Windows.Forms.Label
$label.Location = New-Object System.Drawing.Point(10,20)
$label.Size = New-Object System.Drawing.Size(280,20)
$label.Text = 'Please enter the information in the space below:'
$form.Controls.Add($label)
```

Add the control (in this case, a text box) that lets users provide the information you've described in your label text. There are many other controls you can apply besides text boxes. For more controls, see System.Windows.Forms Namespace.

PowerShell

```
$textBox = New-Object System.Windows.Forms.TextBox
$textBox.Location = New-Object System.Drawing.Point(10,40)
$textBox.Size = New-Object System.Drawing.Size(260,20)
$form.Controls.Add($textBox)
```

Set the **Topmost** property to **$true** to force the window to open atop other open windows and dialog boxes.

PowerShell

```
$form.Topmost = $true
```

Next, add this line of code to activate the form, and set the focus to the text box that you created.

PowerShell

```
$form.Add_Shown({$textBox.Select()})
```

Add the following line of code to display the form in Windows.

PowerShell

```
$result = $form.ShowDialog()
```

Finally, the code inside the `if` block instructs Windows what to do with the form after users provide text in the text box, and then click the **OK** button or press the **Enter** key.

PowerShell

```
if ($result -eq [System.Windows.Forms.DialogResult]::OK) {
    $x = $textBox.Text
    $x
}
```

See also

- GitHub: Dave Wyatt's WinFormsExampleUpdates)
- Windows PowerShell Tip of the Week: Creating a Custom Input Box

---

Last updated on 03/24/2025

Creating a graphical date picker

Article • 12/09/2022

> This sample only applies to Windows platforms.

Use Windows PowerShell 3.0 and later releases to create a form with a graphical, calendar-style control that lets users select a day of the month.

Create a graphical date-picker control

Copy and then paste the following into Windows PowerShell ISE, and then save it as a PowerShell script (`.ps1`) file.

PowerShell

```
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing

$form = New-Object Windows.Forms.Form -Property @{
    StartPosition = [Windows.Forms.FormStartPosition]::CenterScreen
    Size          = New-Object Drawing.Size 243, 230
    Text          = 'Select a Date'
    Topmost       = $true
}

$calendar = New-Object Windows.Forms.MonthCalendar -Property @{
    ShowTodayCircle   = $false
    MaxSelectionCount = 1
}
$form.Controls.Add($calendar)

$okButton = New-Object Windows.Forms.Button -Property @{
    Location     = New-Object Drawing.Point 38, 165
    Size         = New-Object Drawing.Size 75, 23
    Text         = 'OK'
    DialogResult = [Windows.Forms.DialogResult]::OK
}
$form.AcceptButton = $okButton
$form.Controls.Add($okButton)

$cancelButton = New-Object Windows.Forms.Button -Property @{
    Location     = New-Object Drawing.Point 113, 165
    Size         = New-Object Drawing.Size 75, 23
    Text         = 'Cancel'
    DialogResult = [Windows.Forms.DialogResult]::Cancel
}
$form.CancelButton = $cancelButton
$form.Controls.Add($cancelButton)
```

```
$result = $form.ShowDialog()

if ($result -eq [Windows.Forms.DialogResult]::OK) {
    $date = $calendar.SelectionStart
    Write-Host "Date selected: $($date.ToShortDateString())"
}
```

The script begins by loading two .NET Framework classes: **System.Drawing** and **System.Windows.Forms**. You then start a new instance of the .NET Framework class **Windows.Forms.Form**. That provides a blank form or window to which you can start adding controls.

PowerShell

```
$form = New-Object Windows.Forms.Form -Property @{
    StartPosition = [Windows.Forms.FormStartPosition]::CenterScreen
    Size          = New-Object Drawing.Size 243, 230
    Text          = 'Select a Date'
    Topmost       = $true
}
```

This example assigns values to four properties of this class by using the **Property** property and hashtable.

1. **StartPosition**: If you don't add this property, Windows selects a location when the form is opened. By setting this property to **CenterScreen**, you're automatically displaying the form in the middle of the screen each time it loads.
2. **Size**: This is the size of the form, in pixels. The preceding script creates a form that's 243 pixels wide by 230 pixels tall.
3. **Text**: This becomes the title of the window.
4. **Topmost**: By setting this property to `$true`, you can force the window to open atop other open windows and dialog boxes.

Next, create and then add a calendar control in your form. In this example, the current day isn't highlighted or circled. Users can select only one day on the calendar at one time.

PowerShell

```
$calendar = New-Object Windows.Forms.MonthCalendar -Property @{
    ShowTodayCircle   = $false
    MaxSelectionCount = 1
```

```
}
$form.Controls.Add($calendar)
```

Next, create an **OK** button for your form. Specify the size and behavior of the **OK** button. In this example, the button position is 165 pixels from the form's top edge, and 38 pixels from the left edge. The button height is 23 pixels, while the button length is 75 pixels. The script uses predefined Windows Forms types to determine the button behaviors.

PowerShell

```
$okButton = New-Object Windows.Forms.Button -Property @{
    Location     = New-Object Drawing.Point 38, 165
    Size         = New-Object Drawing.Size 75, 23
    Text         = 'OK'
    DialogResult = [Windows.Forms.DialogResult]::OK
}
$form.AcceptButton = $okButton
$form.Controls.Add($okButton)
```

Similarly, you create a **Cancel** button. The **Cancel** button is 165 pixels from the top, but 113 pixels from the left edge of the window.

PowerShell

```
$cancelButton = New-Object Windows.Forms.Button -Property @{
    Location     = New-Object Drawing.Point 113, 165
    Size         = New-Object Drawing.Size 75, 23
    Text         = 'Cancel'
    DialogResult = [Windows.Forms.DialogResult]::Cancel
}
$form.CancelButton = $cancelButton
$form.Controls.Add($cancelButton)
```

Add the following line of code to display the form in Windows.

PowerShell

```
$result = $form.ShowDialog()
```

Finally, the code inside the `if` block instructs Windows what to do with the form after users select a day on the calendar, and then click the **OK** button or press the **Enter** key. Windows PowerShell displays the selected date to users.

PowerShell

```
if ($result -eq [Windows.Forms.DialogResult]::OK) {
    $date = $calendar.SelectionStart
    Write-Host "Date selected: $($date.ToShortDateString())"
}
```

See also

- GitHub: Dave Wyatt's WinFormsExampleUpdates
- Windows PowerShell Tip of the Week: Creating a Graphical Date Picker)

Multiple-selection list boxes

Article • 12/09/2022

> This sample only applies to Windows platforms.

Use Windows PowerShell 3.0 and later releases to create a multiple-selection list box control in a custom Windows Form.

Create list box controls that allow multiple selections

Copy and then paste the following into Windows PowerShell ISE, and then save it as a PowerShell script (`.ps1`) file.

PowerShell

```
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing

$form = New-Object System.Windows.Forms.Form
$form.Text = 'Data Entry Form'
$form.Size = New-Object System.Drawing.Size(300,200)
$form.StartPosition = 'CenterScreen'

$OKButton = New-Object System.Windows.Forms.Button
$OKButton.Location = New-Object System.Drawing.Point(75,120)
$OKButton.Size = New-Object System.Drawing.Size(75,23)
$OKButton.Text = 'OK'
$OKButton.DialogResult = [System.Windows.Forms.DialogResult]::OK
$form.AcceptButton = $OKButton
$form.Controls.Add($OKButton)

$CancelButton = New-Object System.Windows.Forms.Button
$CancelButton.Location = New-Object System.Drawing.Point(150,120)
$CancelButton.Size = New-Object System.Drawing.Size(75,23)
$CancelButton.Text = 'Cancel'
$CancelButton.DialogResult = [System.Windows.Forms.DialogResult]::Cancel
$form.CancelButton = $CancelButton
$form.Controls.Add($CancelButton)

$label = New-Object System.Windows.Forms.Label
$label.Location = New-Object System.Drawing.Point(10,20)
$label.Size = New-Object System.Drawing.Size(280,20)
$label.Text = 'Please make a selection from the list below:'
$form.Controls.Add($label)

$listBox = New-Object System.Windows.Forms.Listbox
```

```
$listBox.Location = New-Object System.Drawing.Point(10,40)
$listBox.Size = New-Object System.Drawing.Size(260,20)

$listBox.SelectionMode = 'MultiExtended'

[void] $listBox.Items.Add('Item 1')
[void] $listBox.Items.Add('Item 2')
[void] $listBox.Items.Add('Item 3')
[void] $listBox.Items.Add('Item 4')
[void] $listBox.Items.Add('Item 5')

$listBox.Height = 70
$form.Controls.Add($listBox)
$form.Topmost = $true

$result = $form.ShowDialog()

if ($result -eq [System.Windows.Forms.DialogResult]::OK)
{
    $x = $listBox.SelectedItems
    $x
}
```

The script begins by loading two .NET Framework classes: **System.Drawing** and **System.Windows.Forms**. You then start a new instance of the .NET Framework class **System.Windows.Forms.Form**. That provides a blank form or window to which you can start adding controls.

PowerShell

```
$form = New-Object System.Windows.Forms.Form
```

After you create an instance of the Form class, assign values to three properties of this class.

- **Text.** This becomes the title of the window.
- **Size.** This is the size of the form, in pixels. The preceding script creates a form that's 300 pixels wide by 200 pixels tall.
- **StartingPosition.** This optional property is set to **CenterScreen** in the preceding script. If you don't add this property, Windows selects a location when the form is opened. By setting the **StartingPosition** to **CenterScreen**, you're automatically displaying the form in the middle of the screen each time it loads.

PowerShell

```
$form.Text = 'Data Entry Form'
$form.Size = New-Object System.Drawing.Size(300,200)
```

```
$form.StartPosition = 'CenterScreen'
```

Next, create an **OK** button for your form. Specify the size and behavior of the **OK** button. In this example, the button position is 120 pixels from the form's top edge, and 75 pixels from the left edge. The button height is 23 pixels, while the button length is 75 pixels. The script uses predefined Windows Forms types to determine the button behaviors.

PowerShell

```
$OKButton = New-Object System.Windows.Forms.Button
$OKButton.Location = New-Object System.Drawing.Size(75,120)
$OKButton.Size = New-Object System.Drawing.Size(75,23)
$OKButton.Text = 'OK'
$OKButton.DialogResult = [System.Windows.Forms.DialogResult]::OK
$form.AcceptButton = $OKButton
$form.Controls.Add($OKButton)
```

Similarly, you create a **Cancel** button. The **Cancel** button is 120 pixels from the top, but 150 pixels from the left edge of the window.

PowerShell

```
$CancelButton = New-Object System.Windows.Forms.Button
$CancelButton.Location = New-Object System.Drawing.Point(150,120)
$CancelButton.Size = New-Object System.Drawing.Size(75,23)
$CancelButton.Text = 'Cancel'
$CancelButton.DialogResult = [System.Windows.Forms.DialogResult]::Cancel
$form.CancelButton = $CancelButton
$form.Controls.Add($CancelButton)
```

Next, provide label text on your window that describes the information you want users to provide.

PowerShell

```
$label = New-Object System.Windows.Forms.Label
$label.Location = New-Object System.Drawing.Point(10,20)
$label.Size = New-Object System.Drawing.Size(280,20)
$label.Text = 'Please make a selection from the list below:'
$form.Controls.Add($label)
```

Add the control (in this case, a list box) that lets users provide the information you've described in your label text. There are many other controls you can apply besides text boxes; for more controls, see System.Windows.Forms Namespace.

PowerShell

```
$listBox = New-Object System.Windows.Forms.Listbox
$listBox.Location = New-Object System.Drawing.Point(10,40)
$listBox.Size = New-Object System.Drawing.Size(260,20)
```

Here's how you specify that you want to allow users to select multiple values from the list.

PowerShell

```
$listBox.SelectionMode = 'MultiExtended'
```

In the next section, you specify the values you want the list box to display to users.

PowerShell

```
[void] $listBox.Items.Add('Item 1')
[void] $listBox.Items.Add('Item 2')
[void] $listBox.Items.Add('Item 3')
[void] $listBox.Items.Add('Item 4')
[void] $listBox.Items.Add('Item 5')
```

Specify the maximum height of the list box control.

PowerShell

```
$listBox.Height = 70
```

Add the list box control to your form, and instruct Windows to open the form atop other windows and dialog boxes when it's opened.

PowerShell

```
$form.Controls.Add($listBox)
$form.Topmost = $true
```

Add the following line of code to display the form in Windows.

PowerShell

```
$result = $form.ShowDialog()
```

Finally, the code inside the `if` block instructs Windows what to do with the form after users select one or more options from the list box, and then click the **OK** button or press

the **Enter** key.

PowerShell

```
if ($result -eq [System.Windows.Forms.DialogResult]::OK)
{
    $x = $listBox.SelectedItems
    $x
}
```

See also

- Weekend Scripter: Fixing PowerShell GUI Examples
- GitHub: Dave Wyatt's WinFormsExampleUpdates
- Windows PowerShell Tip of the Week: Multi-Select List Boxes - And More!)

Selecting items from a list box

> This sample only applies to Windows platforms.

Use Windows PowerShell 3.0 and later releases to create a dialog box that lets users select items from a list box control.

Create a list box control, and select items from it

Copy and then paste the following into Windows PowerShell ISE, and then save it as a PowerShell script (`.ps1`) file.

PowerShell

```
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing

$form = New-Object System.Windows.Forms.Form
$form.Text = 'Select a Computer'
$form.Size = New-Object System.Drawing.Size(300,200)
$form.StartPosition = 'CenterScreen'

$okButton = New-Object System.Windows.Forms.Button
$okButton.Location = New-Object System.Drawing.Point(75,120)
$okButton.Size = New-Object System.Drawing.Size(75,23)
$okButton.Text = 'OK'
$okButton.DialogResult = [System.Windows.Forms.DialogResult]::OK
$form.AcceptButton = $okButton
$form.Controls.Add($okButton)

$cancelButton = New-Object System.Windows.Forms.Button
$cancelButton.Location = New-Object System.Drawing.Point(150,120)
$cancelButton.Size = New-Object System.Drawing.Size(75,23)
$cancelButton.Text = 'Cancel'
$cancelButton.DialogResult = [System.Windows.Forms.DialogResult]::Cancel
$form.CancelButton = $cancelButton
$form.Controls.Add($cancelButton)

$label = New-Object System.Windows.Forms.Label
$label.Location = New-Object System.Drawing.Point(10,20)
$label.Size = New-Object System.Drawing.Size(280,20)
$label.Text = 'Please select a computer:'
$form.Controls.Add($label)

$listBox = New-Object System.Windows.Forms.ListBox
$listBox.Location = New-Object System.Drawing.Point(10,40)
$listBox.Size = New-Object System.Drawing.Size(260,20)
```

```
$listBox.Height = 80

[void] $listBox.Items.Add('atl-dc-001')
[void] $listBox.Items.Add('atl-dc-002')
[void] $listBox.Items.Add('atl-dc-003')
[void] $listBox.Items.Add('atl-dc-004')
[void] $listBox.Items.Add('atl-dc-005')
[void] $listBox.Items.Add('atl-dc-006')
[void] $listBox.Items.Add('atl-dc-007')

$form.Controls.Add($listBox)

$form.Topmost = $true

$result = $form.ShowDialog()

if ($result -eq [System.Windows.Forms.DialogResult]::OK)
{
    $x = $listBox.SelectedItem
    $x
}
```

The script begins by loading two .NET Framework classes: **System.Drawing** and **System.Windows.Forms**. You then start a new instance of the .NET Framework class **System.Windows.Forms.Form**. That provides a blank form or window to which you can start adding controls.

PowerShell

```
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing
```

After you create an instance of the **Form** class, assign values to three properties of this class.

- **Text.** This becomes the title of the window.
- **Size.** This is the size of the form, in pixels. The preceding script creates a form that's 300 pixels wide by 200 pixels tall.
- **StartingPosition.** This optional property is set to **CenterScreen** in the preceding script. If you don't add this property, Windows selects a location when the form is opened. By setting the **StartingPosition** to **CenterScreen**, you're automatically displaying the form in the middle of the screen each time it loads.

PowerShell

```
$form.Text = 'Select a Computer'
$form.Size = New-Object System.Drawing.Size(300,200)
```

```
$form.StartPosition = 'CenterScreen'
```

Next, create an **OK** button for your form. Specify the size and behavior of the **OK** button. In this example, the button position is 120 pixels from the form's top edge, and 75 pixels from the left edge. The button height is 23 pixels, while the button length is 75 pixels. The script uses predefined Windows Forms types to determine the button behaviors.

PowerShell

```
$okButton = New-Object System.Windows.Forms.Button
$okButton.Location = New-Object System.Drawing.Point(75,120)
$okButton.Size = New-Object System.Drawing.Size(75,23)
$okButton.Text = 'OK'
$okButton.DialogResult = [System.Windows.Forms.DialogResult]::OK
$form.AcceptButton = $okButton
$form.Controls.Add($okButton)
```

Similarly, you create a **Cancel** button. The **Cancel** button is 120 pixels from the top, but 150 pixels from the left edge of the window.

PowerShell

```
$cancelButton = New-Object System.Windows.Forms.Button
$cancelButton.Location = New-Object System.Drawing.Point(150,120)
$cancelButton.Size = New-Object System.Drawing.Size(75,23)
$cancelButton.Text = 'Cancel'
$cancelButton.DialogResult = [System.Windows.Forms.DialogResult]::Cancel
$form.CancelButton = $cancelButton
$form.Controls.Add($cancelButton)
```

Next, provide label text on your window that describes the information you want users to provide. In this case, you want users to select a computer.

PowerShell

```
$label = New-Object System.Windows.Forms.Label
$label.Location = New-Object System.Drawing.Point(10,20)
$label.Size = New-Object System.Drawing.Size(280,20)
$label.Text = 'Please select a computer:'
$form.Controls.Add($label)
```

Add the control (in this case, a list box) that lets users provide the information you've described in your label text. There are many other controls you can apply besides list boxes; for more controls, see System.Windows.Forms Namespace.

PowerShell

```
$listBox = New-Object System.Windows.Forms.ListBox
$listBox.Location = New-Object System.Drawing.Point(10,40)
$listBox.Size = New-Object System.Drawing.Size(260,20)
$listBox.Height = 80
```

In the next section, you specify the values you want the list box to display to users.

> **Note**
>
> The list box created by this script allows only one selection. To create a list box control that allows multiple selections, specify a value for the **SelectionMode** property, similarly to the following: `$listBox.SelectionMode = 'MultiExtended'`. For more information, see Multiple-selection List Boxes.

PowerShell

```
[void] $listBox.Items.Add('atl-dc-001')
[void] $listBox.Items.Add('atl-dc-002')
[void] $listBox.Items.Add('atl-dc-003')
[void] $listBox.Items.Add('atl-dc-004')
[void] $listBox.Items.Add('atl-dc-005')
[void] $listBox.Items.Add('atl-dc-006')
[void] $listBox.Items.Add('atl-dc-007')
```

Add the list box control to your form, and instruct Windows to open the form atop other windows and dialog boxes when it's opened.

PowerShell

```
$form.Controls.Add($listBox)
$form.Topmost = $true
```

Add the following line of code to display the form in Windows.

PowerShell

```
$result = $form.ShowDialog()
```

Finally, the code inside the `if` block instructs Windows what to do with the form after users select an option from the list box, and then click the **OK** button or press the **Enter** key.

PowerShell

```
if ($result -eq [System.Windows.Forms.DialogResult]::OK) {
    $x = $listBox.SelectedItem
    $x
}
```

See also

- GitHub: Dave Wyatt's WinFormsExampleUpdates
- Windows PowerShell Tip of the Week: Selecting Items from a List Box

---

Last updated on 03/24/2025

Using Experimental Features in PowerShell

The Experimental Features support in PowerShell provides a mechanism for experimental features to coexist with existing stable features in PowerShell or PowerShell modules.

An experimental feature is one where the design isn't finalized. The feature is available for users to test and provide feedback. Once an experimental feature is finalized, the design changes become breaking changes.

> **⊗ Caution**
>
> Experimental features aren't intended to be used in production since the changes are allowed to be breaking. Experimental features aren't officially supported. However, we appreciate any feedback and bug reports. You can file issues in the GitHub source repository.

For more information about enabling or disabling these features, see about_Experimental_Features.

Experimental feature lifecycle

The Get-ExperimentalFeature cmdlet returns all experimental features available to PowerShell. Experimental features can come from modules or the PowerShell engine. Module-based experimental features are only available after you import the module. In the following example, the **PSDesiredStateConfiguration** isn't loaded, so the `PSDesiredStateConfiguration.InvokeDscResource` feature isn't available.

PowerShell

```
Get-ExperimentalFeature
```

Output

```
Name                            Enabled Source   Description
----                            ------- ------   -----------
PSFeedbackProvider                 True PSEngine Replace the hard-coded suggestion
framework with the extensible feedb…
PSLoadAssemblyFromNativeCode      False PSEngine Expose an API to allow assembly
loading from native code
```

```
PSNativeWindowsTildeExpansion      True PSEngine On windows, expand unquoted tilde
(`~`) with the user's current home …
PSRedirectToVariable               True PSEngine Add support for redirecting to the
variable drive
PSSerializeJSONLongEnumAsNumber    True PSEngine Serialize enums based on long or
ulong as an numeric value rather tha…
PSSubsystemPluginModel             True PSEngine A plugin model for registering and
un-registering PowerShell subsyste…
```

Use the Enable-ExperimentalFeature and Disable-ExperimentalFeature cmdlets to enable or disable a feature. You must start a new PowerShell session for this change to be in effect. Run the following command to enable the `PSCommandNotFoundSuggestion` feature:

PowerShell

```
Enable-ExperimentalFeature PSFeedbackProvider
```

Output

```
WARNING: Enabling and disabling experimental features do not take effect until next
start
of PowerShell.
```

When an experimental feature becomes *mainstream*, it's no longer available as an experimental feature because the functionality is now part of the PowerShell engine or module. For example, the `PSAnsiRenderingFileInfo` feature became mainstream in PowerShell 7.3. You get the functionality of the feature automatically.

> **Note**
>
> Some features have configuration requirements, such as preference variables, that must be set to get the desired results from the feature.

When an experimental feature is *discontinued*, that feature is no longer available in the PowerShell. For example, the `PSNativePSPathResolution` feature was discontinued in PowerShell 7.3.

Available features

This article describes the experimental features that are available and how to use the feature.

Legend

- The 🚧 icon indicates that the experimental feature is available in the version of PowerShell
- The ✅ icon indicates the version of PowerShell where the experimental feature became mainstream
- The ❌ icon indicates the version of PowerShell where the experimental feature was removed

Expand table

Name	7.4	7.5	7.6	7.7
PSCommandNotFoundSuggestion	🚧	✅	✅	✅
PSCommandWithArgs	🚧	✅	✅	✅
PSFeedbackProvider	🚧	🚧	✅	✅
PSLoadAssemblyFromNativeCode	🚧	🚧	🚧	🚧
PSModuleAutoLoadSkipOfflineFiles	🚧	✅	✅	✅
PSNativeWindowsTildeExpansion		🚧	✅	✅
PSProfileDSCResource			🚧	🚧
PSRedirectToVariable		🚧	✅	✅
PSSerializeJSONLongEnumAsNumber		🚧	🚧	🚧
PSSubsystemPluginModel	🚧	🚧	✅	✅

PSDesiredStateConfiguration module v2 and higher

- PSDesiredStateConfiguration.InvokeDscResource

PSCommandNotFoundSuggestion

> **Note**
>
> This feature became mainstream in PowerShell 7.5-preview.5.

Recommends potential commands based on fuzzy matching search after a **CommandNotFoundException**.

PowerShell

```
PS> get
```

Output

```
get: The term 'get' isn't recognized as the name of a cmdlet, function, script file,
or operable program. Check the spelling of the name, or if a path was included, verify
that the path is correct and try again.

Suggestion [4,General]: The most similar commands are: set, del, ft, gal, gbp, gc,
gci,
gcm, gdr, gcs.
```

PSCommandWithArgs

> **Note**
>
> This feature became mainstream in PowerShell 7.5-preview.5.

This feature enables the `-CommandWithArgs` parameter for `pwsh`. This parameter allows you to execute a PowerShell command with arguments. Unlike `-Command`, this parameter populates the `$args` built-in variable that can be used by the command.

The first string is the command and subsequent strings delimited by whitespace are the arguments.

For example:

PowerShell

```
pwsh -CommandWithArgs '$args | % { "arg: $_" }' arg1 arg2
```

This example produces the following output:

Output

```
arg: arg1
arg: arg2
```

This feature was added in PowerShell 7.4-preview.2.

PSDesiredStateConfiguration.InvokeDscResource

Enables compilation to MOF on non-Windows systems and enables the use of `Invoke-DSCResource` without an LCM.

Beginning with PowerShell 7.2, the **PSDesiredStateConfiguration** module was removed and this feature is disabled by default. To enable this feature you must install the **PSDesiredStateConfiguration** v2.0.5 module from the PowerShell Gallery and enable the feature.

DSC v3 doesn't have this experimental feature. DSC v3 only supports `Invoke-DSCResource` and doesn't use or support MOF compilation. For more information, see PowerShell Desired State Configuration v3.

PSFeedbackProvider

> **Note**
>
> This experimental feature was added in PowerShell 7.4-preview.3. This feature became mainstream in PowerShell 7.6-preview.6.

When you enable this feature, PowerShell uses a new feedback provider to give you feedback when a command can't be found. The feedback provider is extensible, and can be implemented by third-party modules. The feedback provider can be used by other subsystems, such as the predictor subsystem, to provide predictive IntelliSense results.

This feature includes two built-in feedback providers:

- **GeneralCommandErrorFeedback** serves the same suggestion functionality existing today
- **UnixCommandNotFound**, available on Linux, provides feedback similar to bash.

  The **UnixCommandNotFound** serves as both a feedback provider and a predictor. The suggestion from command-not-found command is used both for providing the feedback when command can't be found in an interactive run, and for providing predictive IntelliSense results for the next command line.

PSLoadAssemblyFromNativeCode

Exposes an API to allow assembly loading from native code.

PSModuleAutoLoadSkipOfflineFiles

> ⓘ Note
>
> This feature became mainstream in PowerShell 7.5-preview.5.

With this feature enabled, if a user's **PSModulePath** contains a folder from a cloud provider, such as OneDrive, PowerShell no longer triggers the download of all files contained within that folder. Any file marked as not downloaded are skipped. Users who use cloud providers to sync their modules between machines should mark the module folder as **Pinned** or the equivalent status for providers other than OneDrive. Marking the module folder as **Pinned** ensures that the files are always kept on disk.

This feature was added in PowerShell 7.4-preview.1.

PSNativeWindowsTildeExpansion

> ⓘ Note
>
> This experimental feature was added in PowerShell 7.5-preview.2. This feature became mainstream in PowerShell 7.6-preview.6.

When this feature is enabled, PowerShell expands unquoted tilde (`~`) to the user's current home folder before invoking native commands. The following examples show how the feature works.

With the feature disabled, the tilde is passed to the native command as a literal string.

PowerShell

```
PS> cmd.exe /c echo ~
~
```

With the feature enabled, PowerShell expands the tilde before it's passed to the native command.

PowerShell

```
PS> cmd.exe /c echo ~
C:\Users\username
```

This feature only applies to Windows. On non-Windows platforms, tilde expansion is handled natively.

PSProfileDSCResource

> ⓘ Note
>
> This experimental feature was added in PowerShell 7.6-preview.6.

The PowerShell 7.6-preview.6 release added this feature as an advertisement for a new DSCv3 resource. The experimental feature flag doesn't do anything. You can use the new DSC v3 resource regardless of whether this feature is enabled or disabled.

The `Microsoft.PowerShell/Profile` resource enables you to manage PowerShell profiles using Desired State Configuration (DSC) v3. This release includes two new files in the `$PSHOME` folder:

- `pwsh.profile.dsc.resource.json` - DSC v3 resource manifest file
- `pwsh.profile.resource.ps1` - DSC v3 resource implementation file

The resource supports operations to get, set, and export profile content for different profile types. For more information, see the notes in the PR PowerShell/PowerShell#26157.

PSRedirectToVariable

> ⓘ Note
>
> This experimental feature was added in PowerShell 7.5-preview.4. This feature became mainstream in PowerShell 7.6-preview.6.

When enabled, this feature adds support for redirecting to the Variable: drive. This feature allows you to redirect data to a variable using the `Variable:name` syntax. PowerShell inspects the target of the redirection and if it uses the Variable provider it calls `Set-Variable` rather than `Out-File`.

The following example shows how to redirect the output of a command to a Variable:

PowerShell

```
. {
    "Output 1"
    Write-Warning "Warning, Warning!"
    "Output 2"
} 3> Variable:warnings
$warnings
```

Output

```
Output 1
Output 2
WARNING: Warning, Warning!
```

PSSerializeJSONLongEnumAsNumber

> ⓘ Note
>
> This experimental feature was added in PowerShell 7.5-preview.5.

This feature enables the cmdlet ConvertTo-Json to serialize any enum values based on `Int64/long` or `UInt64/ulong` as a numeric value rather than the string representation of that enum value. This aligns the behavior of enum serialization with other enum base types where the cmdlet serializes enums as their numeric value. Use the **EnumsAsStrings** parameter to serialize as the string representation.

For example:

PowerShell

```powershell
# PSSerializeJSONLongEnumAsNumber disabled
@{
    Key = [System.Management.Automation.Tracing.PowerShellTraceKeywords]::Cmdlets
} | ConvertTo-Json
# { "Key": "Cmdlets" }

# PSSerializeJSONLongEnumAsNumber enabled
@{
    Key = [System.Management.Automation.Tracing.PowerShellTraceKeywords]::Cmdlets
} | ConvertTo-Json
# { "Key": 32 }

# -EnumsAsStrings to revert back to the old behaviour
@{
    Key = [System.Management.Automation.Tracing.PowerShellTraceKeywords]::Cmdlets
} | ConvertTo-Json -EnumsAsStrings
# { "Key": "Cmdlets" }
```

PSSubsystemPluginModel

> ⓘ Note

This feature became mainstream in PowerShell 7.6-preview.6.

This feature enables the subsystem plugin model in PowerShell. The feature makes it possible to separate components of `System.Management.Automation.dll` into individual subsystems that reside in their own assembly. This separation reduces the disk footprint of the core PowerShell engine and allows these components to become optional features for a minimal PowerShell installation.

Currently, only the **CommandPredictor** subsystem is supported. This subsystem is used along with the PSReadLine module to provide custom prediction plugins. In future, **Job**, **CommandCompleter**, **Remoting** and other components could be separated into subsystem assemblies outside of `System.Management.Automation.dll`.

The experimental feature includes a new cmdlet, Get-PSSubsystem. This cmdlet is only available when the feature is enabled. This cmdlet returns information about the subsystems that are available on the system.

Last updated on 04/20/2026

Using aliases

An alias is an alternate name or shorthand name for a cmdlet or for a command element, such as a function, script, file, or executable file. You can run the command using the alias instead of the executable name.

Managing command aliases

PowerShell provides cmdlets for managing command aliases. The following command shows the cmdlets that manage aliases.

PowerShell

```
Get-Command -Noun Alias
```

Output

```
CommandType Name         Version Source
----------- ----         ------- ------
Cmdlet      Export-Alias 7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Get-Alias    7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Import-Alias 7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      New-Alias    7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Remove-Alias 7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Set-Alias    7.0.0.0 Microsoft.PowerShell.Utility
```

For more information, see about_Aliases.

Use the Get-Alias cmdlet to list the aliases available in your environment. To list the aliases for a single cmdlet, use the **Definition** parameter and specify the executable name.

PowerShell

```
Get-Alias -Definition Get-ChildItem
```

Output

```
CommandType     Name
-----------     ----
Alias           dir -> Get-ChildItem
Alias           gci -> Get-ChildItem
Alias           ls -> Get-ChildItem
```

To get the definition of a single alias, use the **Name** parameter.

PowerShell

```
Get-Alias -Name gci
```

Output

```
CommandType     Name
-----------     ----
Alias           gci -> Get-ChildItem
```

To create an alias, use the `Set-Alias` command. You can create aliases for cmdlets, functions, scripts, and native executables files.

PowerShell

```
Set-Alias -Name np -Value Notepad.exe
Set-Alias -Name cmpo  -Value Compare-Object
```

Compatibility aliases in Windows

PowerShell has several aliases that allow **Unix** and `cmd.exe` users to use familiar commands in Windows. The following table shows common commands, the related PowerShell cmdlet, and the PowerShell alias:

Expand table

Windows Command Shell	Unix command	PowerShell cmdlet	PowerShell alias
`cd`, `chdir`	`cd`	`Set-Location`	`sl`, `cd`, `chdir`
`cls`	`clear`	`Clear-Host`	`cls` `clear`
`copy`	`cp`	`Copy-Item`	`cpi`, `cp`, `copy`
`del`, `erase`, `rd`, `rmdir`	`rm`	`Remove-Item`	`ri`, `del`, `erase`, `rd`, `rm`, `rmdir`
`dir`	`ls`	`Get-ChildItem`	`gci`, `dir`, `ls`
`echo`	`echo`	`Write-Output`	`write` `echo`
`md`	`mkdir`	`New-Item`	`ni`
`move`	`mv`	`Move-Item`	`mi`, `move`, `mv`
`popd`	`popd`	`Pop-Location`	`popd`
	`pwd`	`Get-Location`	`gl`, `pwd`, `$PWD`

Windows Command Shell	Unix command	PowerShell cmdlet	PowerShell alias
`pushd`	`pushd`	`Push-Location`	`pushd`
`ren`	`mv`	`Rename-Item`	`rni`, `ren`
`type`	`cat`	`Get-Content`	`gc`, `cat`, `type`

> **Note**
>
> The aliases in this table are Windows-specific. Some aliases aren't available on other platforms to allow an existing native command to work in a PowerShell session. For example, `ls` isn't defined as a PowerShell alias on macOS or Linux so that PowerShell runs the native command instead of `Get-ChildItem`.

Creating alternate names for commands with parameters

You can assign an alias to a cmdlet, script, function, or executable file. Unlike some Unix shells, the definition of the alias can't include parameters. For example, you can assign an alias to the `Get-Eventlog` cmdlet, but you can't assign an alias to the `Get-Eventlog -LogName System` command. You must create a function that contains the command with parameters.

For more information, see about_Aliases.

Parameter aliases and shorthand names

PowerShell also provides ways to create shorthand names for parameters. Parameter aliases are defined using the `Alias` attribute when you declare the parameter. Parameter aliases can't be defined using the `*-Alias` cmdlets.

For more information, see the Alias attribute documentation.

In addition to parameter aliases, PowerShell lets you specify the parameter name using the fewest characters needed to uniquely identify the parameter. For example, the `Get-ChildItem` cmdlet has the **Filter** and **Force** parameters. Using `-F` is ambiguous because both parameters start with the letter `F`. If you try to use `-F`, PowerShell returns an error:

PowerShell

```
PS> Get-ChildItem -f
```

Output

```
Get-ChildItem : Parameter cannot be processed because the parameter name 'f' is
ambiguous. Possible
matches include: -Filter -Force.
```

To uniquely identify the **Filter** parameter, you need to use `-Fi`.

> **Note**
>
> The `Get-ChildItem` cmdlet also has a dynamic parameter, `-File`. Using `-f`, `-fi` and `-fil` doesn't create ambiguity with `-File` because `-File` is a dynamic `FileSystem` provider parameter. PowerShell binds dynamic parameters after binding static parameters. The only way to specify the parameter is to use the full name, `-File`.

Don't use aliases in scripts

Aliases are a convenience feature to be used interactively in the shell. You should always use the full command and parameter names in your scripts.

- You can delete or redefine aliases in a profile script
- Aliases you define in your profile aren't be available to other users
- Aliases make your code harder to read and maintain

Last updated on 01/13/2026

PowerShell learning resources

Additional resources for learning about PowerShell.

Learn training modules

Microsoft Learn is a free, online training platform that provides interactive learning for Microsoft products and more. Our goal is to help you become proficient on our technologies and learn more skills with fun, guided, hands-on, interactive content that's specific to your role and goals.

- PowerShell training

Blogs and community

In addition to the Help available at the command line, the following resources provide more information for users who want to run PowerShell.

- PowerShell Team Blog. The best resource for learning directly from the PowerShell product team.
- PowerShell Community Blog articles are scenario-driven. Written by the community, for the community.
- Have questions about using PowerShell? Connect with hundreds of other people who have similar interests in one of the many community forums listed on the PowerShell Community page.

Microsoft Virtual Academy

The Microsoft Virtual Academy videos have been moved to Channel 9.

- Getting Started with Microsoft PowerShell
- Advanced Tools & Scripting with PowerShell 3.0 Jump Start
- Testing PowerShell with Pester
- Getting Started with PowerShell Desired State Configuration (DSC)
- Advanced PowerShell DSC and Custom Resources
- SharePoint Automation with DSC

Resources for PowerShell Developers

The following resources provide resources to help developers create their own PowerShell modules, functions, cmdlets, providers, and hosting applications.

- PowerShell SDK
- PowerShell SDK API Browser

Last updated on 11/21/2025

PowerShell Glossary

This article lists common terms used to talk about PowerShell.

B

binary module

A PowerShell module whose root module is a binary (`.dll`) file. A binary module may or may not include a module manifest.

C

CommonParameter

A parameter that's added to all cmdlets, advanced functions, and workflows by the PowerShell engine.

D

dot source

In PowerShell, to start a command by typing a dot and a space before the command. Commands that are dot sourced run in the current scope instead of in a new scope. Any variables, aliases, functions, or drives that command creates are created in the current scope and are available to users when the command is completed.

dynamic module

A module that exists only in memory. The `New-Module` and `Import-PSSession` cmdlets create dynamic modules.

dynamic parameter

A parameter that's added to a PowerShell cmdlet, function, or script under certain conditions. Cmdlets, functions, providers, and scripts can add dynamic parameters.

F

format file

A PowerShell XML file that has the `.format.ps1xml` extension and that defines how PowerShell displays an object based on its .NET Framework type.

G

global session state

The session state that contains the data that's accessible to the user of a PowerShell session.

H

Host

The interface that the PowerShell engine uses to communicate with the user. For example, the host specifies how prompts are handled between PowerShell and the user.

host application

A program that loads the PowerShell engine into its process and uses it to perform operations.

I

input processing method

A method that a cmdlet can use to process the records it receives as input. The input processing methods include the `BeginProcessing` method, the `ProcessRecord` method, the `EndProcessing` method, and the `StopProcessing` method.

M

manifest module

A PowerShell module that has a manifest and whose **RootModule** key is empty.

member-access enumeration

A PowerShell convenience feature to automatically enumerate items in a collection when using the member-access operator (`.`).

module

A self-contained reusable unit that allows you to partition, organize, and abstract your PowerShell code. A module can contain cmdlets, providers, functions, variables, and other types of resources that can be imported as a single unit.

module manifest

A PowerShell data file (`.psd1`) that describes the contents of a module and that controls how a module is processed.

module session state

The session state that contains the public and private data of a PowerShell module. The private data in this session state isn't available to the user of a PowerShell session.

N

non-terminating error

An error that doesn't stop PowerShell from continuing to process the command. See also, terminating error.

noun

The word that follows the hyphen in a PowerShell cmdlet name. The noun describes the resources upon which the cmdlet acts.

P

parameter set

A group of parameters that can be used in the same command to perform a specific action.

pipe

In PowerShell, to send the results of the preceding command as input to the next command in the pipeline.

pipeline

A series of commands connected by pipeline operators (`|`). Each pipeline operator sends the results of the preceding command as input to the next command.

PowerShell cmdlet

A single command that participates in the pipeline semantics of PowerShell. This includes binary (C#) cmdlets, advanced script functions, CDXML, and Workflows.

PowerShell command

The elements in a pipeline that cause an action to be carried out. PowerShell commands are either typed at the keyboard or invoked programmatically.

PowerShell data file

A text file that has the `.psd1` file extension. PowerShell uses data files for various purposes such as storing module manifest data and storing translated strings for script internationalization.

PowerShell drive

A virtual drive that provides direct access to a data store. It can be defined by a PowerShell provider or created at the command line. Drives created at the command line are session-specific drives and are lost when the session is closed.

provider

A Microsoft .NET Framework-based program that makes the data in a specialized data store available in PowerShell so that you can view and manage it.

PSSession

A type of PowerShell session that's created, managed, and closed by the user.

R

root module

The module specified in the **RootModule** key in a module manifest.

runspace

In PowerShell, the operating environment in which each command in a pipeline is executed.

S

scalar value

In PowerShell, a scalar value is any value type that is not enumerable. This includes the .NET primitive types, such as booleans and numbers, and other value types such as **String**, **DateTime** and **Guid**.

For a list of .NET primitive types, see the *Remarks* section of System.Type.IsPrimitive Property.

script block

In the PowerShell programming language, a collection of statements or expressions that can be used as a single unit. A script block can accept arguments and return values.

script file

A file that has the `.ps1` extension and contains a script written in the PowerShell language.

script module

A PowerShell module whose root module is a script module (`.psm1`) file. A script module may include a module manifest. The script defines the members that the script module exports.

shell

The command interpreter that's used to pass commands to the operating system.

`[switch]` parameter

A parameter that doesn't take an argument. The value of a `[switch]` parameter defaults to `$false`. When a `[switch]` parameter is used, its value becomes `$true`.

T

terminating error

An error that stops PowerShell from processing the command. See also, non-terminating error.

transaction

An atomic unit of work. The work in a transaction must be completed as a whole. If any part of the transaction fails, the entire transaction fails.

type file

A PowerShell XML file that has the `.types.ps1xml` extension and that extends the properties of Microsoft .NET Framework types in PowerShell.

V

verb

The word that precedes the hyphen in a PowerShell cmdlet name. The verb describes the action that the cmdlet performs.

W

Windows PowerShell ISE

The Integrated Scripting Environment (ISE) - A Windows PowerShell host application that enables you to run commands and to write, test, and debug scripts in a friendly, syntax-colored, Unicode-compliant environment.

Windows PowerShell snap-in

A resource that defines a set of cmdlets, providers, and Microsoft .NET Framework types that can be added to the Windows PowerShell environment. PowerShell snap-ins have been replaced by modules.

Windows PowerShell Workflow

A workflow is a sequence of programmed, connected steps that perform long-running tasks or require the coordination of multiple steps across multiple devices or managed nodes. Windows PowerShell Workflow lets IT pros and developers author sequences of multi-device management activities, or single tasks within a workflow, as workflows. Windows PowerShell Workflow lets you adapt and run both PowerShell scripts and XAML files as workflows.

Windows PowerShell Workflow is built on the Windows Workflow Foundation, which has been deprecated.

Last updated on 04/08/2026

Overview of what's new in PowerShell

A collection of release notes and documentation about the new features available in new versions of PowerShell.

What's new in PowerShell 7

WHAT'S NEW

What's new in PowerShell 7.6 (LTS)

What's new in PowerShell 7.5

What's new in PowerShell 7.4 (LTS)

Differences between Windows PowerShell 5.1 and PowerShell 7

PowerShell differences on non-Windows platforms

What's new in PowerShell 5.1

WHAT'S NEW

What is Windows PowerShell?

Differences between Windows PowerShell 5.1 and PowerShell 7

Migrating from Windows PowerShell 5.1 to PowerShell 7

History & Compatibility

WHAT'S NEW

Release history of modules and cmdlets

Module compatibility

Previous versions of PowerShell

What's New in PowerShell 7.7

PowerShell 7.7.0-preview.5 includes the following features, updates, and breaking changes. PowerShell 7.7.0-preview.5 is built on the .NET 11.0.0-rc.1 runtime.

For a complete list of changes, see the CHANGELOG in the GitHub repository.

Updated modules

PowerShell 7.7 includes the following updated modules:

- **Microsoft.PowerShell.PSResourceGet** v1.3.0-preview2
- **PSReadLine** v2.4.5

Breaking Changes

- `New-Guid` now generates Version 7 (time-sortable) UUIDs by default instead of Version 4 (random). The output format is unchanged. Scripts that depend on fully random GUIDs should use `[guid]::NewGuid()` directly. (#27033)
- Add `ValidateNotNullOrEmpty` attribute to the `-Property` of `Format-Table`, `Format-List`, and `Format-Custom` (#26552)
- Use ArgumentException.ThrowIfNullOrEmpty for not-null-not-empty argument validation. (#26668)
  - The exception thrown changes to `System.ArgumentException` from `System.Management.Automation.PSArgumentNullException`.
- Correct handling of explicit `-[<Operator>]:$false` parameter values in `Where-Object` (#26485) (Thanks @yotsuda!)

Tab completion improvements

- Add tab completion for `$PSBoundParameters.Keys` switch cases and access patterns (#26483) (Thanks @yotsuda!)
- Fix the regression in tab completing positional parameters (#27857)

Cmdlet improvements

- Allow user to force the default implicit formatting to use `AutoSize` (#28018)
- Prevent localization of WPF key gesture strings (#28020)

- Improve pointer lifecycle handling in PdhHelper (#28008)
- Improve validation for `Copy-Item -FromSession` (#28006)
- Add `GetImportModuleCommand` method back and check for session restriction in `ShowCommand` (#25557)
- Improve output file path determination for `Invoke-WebRequest` (#27872)
- Improve `Authorization` header default and redirect behavior (#27873)
- Add `New-TemporaryDirectory` cmdlet (#27549) (Thanks @KirtiRamchandani!)
- Change `New-Guid` to generate UUID v7 by default (#27033) (Thanks @SufficientDaikon!)
- Handle empty-string and null-value results returned from custom argument completer more properly (#27398)
- Add missing resource strings for `Get-WinEvent` (#27397) (Thanks @MartinGC94!)
- Improve `Get-WinEvent -ListLog` exception handling (#27395) (Thanks @MartinGC94!)
- Update `MaxVisitCount` and `MaxHashtableKeyCount` if `VisitorSafeValueContext` indicates `SkipLimitCheck` is true for `Import-PowerShellDataFile`
- Correct handling of explicit `-<SwitchParameter>:$false` parameter values for the following cmdlets:
  - `ConvertFrom-Csv`, `ConvertTo-Csv`, `Export-Csv`, and `Import-Csv` (#26719) (Thanks @yotsuda!)
  - `Get-Random` (#26457) (Thanks @yotsuda!)
  - `Get-SecureRandom` (#26460) (Thanks @yotsuda!)
  - `Get-TimeZone` (#26463) (Thanks @yotsuda!)
  - `Get-Uptime` (#26141) (Thanks @logiclrd!)
  - `New-Guid` (#26140) (Thanks @logiclrd!)
  - `New-PSSession` (#26469) (Thanks @yotsuda!)
  - `Split-Path` (#26474) (Thanks @yotsuda!)
  - `Test-Connection` (#26479) (Thanks @yotsuda!)
  - `Where-Object` (#26485) (Thanks @yotsuda!)
- Add `-ExcludeProperty` parameter to `Format-*` cmdlets (#26514) (Thanks @yotsuda!)
- Add `-Extension` parameter to `Join-Path` cmdlet (#26482) (Thanks @yotsuda!)
- Make `Export-Csv` `-Append` and `-NoHeader` mutually exclusive (#26472) (Thanks @yotsuda!)
- Mark `-NoTypeInformation` as obsolete no-op and evaluate `-IncludeTypeInformation` on by value on Csv cmdlets (#26719) (Thanks @yotsuda!)
- Add `SubjectAlternativeName` property to the `Signature` object returned from `Get-AuthenticodeSignature` (#26252)
- Add property and event for debug attach (#25788) (Thanks @jborean93!)
- Add `$PSApplicationOutputEncoding` variable (#21219) (Thanks @jborean93!)

- Add `ToRegex` method to `WildcardPattern` class (#26515) (Thanks @yotsuda!)
- Update PowerShell Profile DSC resource manifests to allow null for content (#26929)
- DSC v3 resource for PowerShell Profile (#26157)
- Dynamically evaluate width of `LastWriteTime` for formatting output on Unix (#24624) (Thanks @MathiasMagnus!)
- Fix formatting to properly handle the `Reset` VT sequences that appear in the middle of a string (#26424)
- Fix `Invoke-RestMethod` to support read-only files in multipart form data (#26454) (Thanks @yotsuda!)
- Fix memory leak in `GetFileShares` (#25896) (Thanks @xtqqczze!)
- Fix **NOTES** section formatting in comment-based help (#26512) (Thanks @yotsuda!)
- Fix `Test-Json` false positive errors when using `oneOf` or `anyOf` in schema (#26618) (Thanks @yotsuda!)
- Fix the CLR internal error and null ref exception when running `Show-Command` with PowerShell API (#26669)
- Handle null reference exception in `CsvCommands.cs`: `ConvertPSObjectToCSV` (#26144) (Thanks @mikkas456!)
- Improve `ValidateLength` error message consistency and refactor validation tests (#25806) (Thanks @jorgeasaurus!)
- Support **TargetObject** position in `ParserErrors` (#26649) (Thanks @jborean93!)
- Add verbose message to `Get-Service` when properties cannot be returned (#27109) (Thanks @reabr!)
- Fix `Remove-Item` confirmation message to use provider path instead (#27123) (Thanks @scuzqy!)
- PSStyle: validate background index against `BackgroundColorMap` (#27106) (Thanks @cuiweixie!)

Engine improvements

- Tolerate IO/JSON errors reading the per-user `powershell.config.json` (#27373)
- Improve remote prompt handling for `PSCredential` and `SecureString` arrays (#28007)
- Improve CAB extraction path handling (#28009)
- Handle MSIX installation specially when prepending to PATH (#27782)
- Call `CodeGeneration.EscapeSingleQuotedStringContent` API in SyncCurrentLocationHandler (#27874)
- Improve PowerShell Remoting Argument Validation (#27870)
- Improve validation for data fragments for PSRP frame headers (#27871)

- Add the xsd validation back for CIM cmdlets (#27869)
- Fix PATH caching in CommandDiscovery code (#27809)
- Fix `FileOnlyEntry` crashing on older versions of Windows (#27880)
- Add support for new WLDP setting `EnableFileOnlyEntry` (#26752)
- Fix the dot-sourcing behavior of `pwsh -file` for advanced-function scripts (#27727)
- Fix progress bar rendering with double-width Unicode characters (#26185) (Thanks @yotsuda!)
- Update PowerShell telemetry to respect the diagnostics and feedback setting on Windows (#27328)
- Fix up default value for parameters with the `in` modifier (#26785) (Thanks @jborean93!)
- Fix `WSManInstance` COM interface with `ResourceURI` (#26692) (Thanks @jborean93!)
- Refactor the module path construction code to make it more robust and easier to maintain (#26565)
- Fix checks for local user config file paths (#26269)
- Delay update notification for one week to ensure all packages become available (#27095)
- Disable AMSI content logging in release (#26235) (Thanks @xtqqczze!)
- Add property and event for debug attach (#25788) (Thanks @jborean93!)
- Enable usage in AppContainers (#27266)

Experimental features

PowerShell 7.7 includes the following experimental features.

- PSLoadAssemblyFromNativeCode - Load assemblies from native code
- PSSerializeJSONLongEnumAsNumber - `ConvertTo-Json` now treats large enums as numbers
- PSProfileDSCResource - Add DSC v3 resource for PowerShell Profiles

---

Last updated on 09/22/2026

What's New in PowerShell 7.6

PowerShell 7.6.6 includes the following features, updates, and breaking changes. PowerShell 7.6.6 is built on the .NET 10.0.12 runtime.

For a complete list of changes, see the CHANGELOG in the GitHub repository.

Installer updates

The macOS PKG package is now notarized and signed by Microsoft. For more information, see Install PowerShell 7 on macOS.

Updated modules

PowerShell 7.6.6 includes the following updated modules:

- **Microsoft.PowerShell.PSResourceGet** v1.2.0
- **PSReadLine** v2.4.5
- **Microsoft.PowerShell.ThreadJob** v2.2.0

Breaking Changes

- The **Microsoft.PowerShell.ThreadJob** replaces the **ThreadJob** module. The `Start-ThreadJob` cmdlet hasn't changed, so there shouldn't be an impact unless you have scripts that use the module qualified name. If you are using the module qualified name, update the name to `Microsoft.PowerShell.ThreadJob\Start-ThreadJob`.
- Fix `WildcardPattern.Escape` to escape lone backticks correctly (#25211) (Thanks @ArmaanMcleod!)
- Convert `-ChildPath` parameter to `string[]` for `Join-Path` cmdlet (#24677) (Thanks @ArmaanMcleod!)
- Remove trailing space from event source name (#24192) (Thanks @MartinGC94!)

Tab completion improvements

- Properly Expand Aliases to their actual ResolvedCommand (#26571) (Thanks @kilasuit!)
- Use parameter `HelpMessage` for tool tip in parameter completion (#25108) (Thanks @jborean93!)
- Remove duplicate modules from completion results (#25538) (Thanks @MartinGC94!)

- Add completion for variables assigned in `ArrayLiteralAst` and `ParenExpressionAst` (#25303) (Thanks @MartinGC94!)
- Fix tab completion for env/function variables (#25346) (Thanks @jborean93!)
- Update Named and Statement block type inference to not consider **AssignmentStatements** and Increment/decrement operators as part of their output (#21137) (Thanks @MartinGC94!)
- Add `-PropertyType` argument completer for `New-ItemProperty` (#21117) (Thanks @ArmaanMcleod!)
- Add completion single/double quote support for `-Noun` parameter for `Get-Command` (#24977) (Thanks @ArmaanMcleod!)
- Add completion single/double quote support for `-PSEdition` parameter for `Get-Module` (#24971) (Thanks @ArmaanMcleod!)
- Convert **InvalidCommandNameCharacters** in AnalysisCache to `SearchValues<char>` for more efficient char searching (#24880) (Thanks @ArmaanMcleod!)
- Convert **s_charactersRequiringQuotes** in Completion Completers to `SearchValues<char>` for more efficient char searching (#24879) (Thanks @ArmaanMcleod!)
- Update `IndexOfAny()` calls with invalid path/filename to `SearchValues<char>` for more efficient char searching ([#24896][24896]) (Thanks @ArmaanMcleod!)
- Replace `char[]` array in `CompletionRequiresQuotes` with cached `SearchValues<char>` (#24907) (Thanks @ArmaanMcleod!)
- Add quote handling in `Verb`, `StrictModeVersion`, `Scope` and `PropertyType` Argument Completers with single helper method (#24839) (Thanks @ArmaanMcleod!)
- Fix share completion with provider and spaces (#19440) (Thanks @MartinGC94!)
- Improve variable type inference (#19830) (Thanks @MartinGC94!)
- Add tooltips for hashtable key completions (#17864) (Thanks @MartinGC94!)
- Fix type inference of parameters in classic functions (#25172) (Thanks @MartinGC94!)
- Improve assignment type inference (#21143) (Thanks @MartinGC94!)
- Exclude **OutVariable** assignments within the same `CommandAst` when inferring variables (#25224) (Thanks @MartinGC94!)
- Fix parameter completion when script requirements fail (#17687) (Thanks @MartinGC94!)
- Improve the completion for attribute arguments (#25129) (Thanks @MartinGC94!)
- Fix completion that relies on pseudobinding in script blocks (#25122) (Thanks @MartinGC94!)
- Don't complete duplicate command names (#21113) (Thanks @MartinGC94!)
- Add completion for variables assigned by command redirection (#25104) (Thanks @MartinGC94!)

- Fix `TypeName.GetReflectionType()` to work when the `TypeName` instance represents a generic type definition within a `GenericTypeName` (#24985)
- Update variable/property assignment completion so it can fallback to type inference (#21134) (Thanks @MartinGC94!)
- Handle type inference for redirected commands (#21131) (Thanks @MartinGC94!)
- Use `Get-Help` approach to find `about_*.help.txt` files with correct locale for completions (#24194) (Thanks @MartinGC94!)
- Fix completion of variables assigned inside Do loops (#25076) (Thanks @MartinGC94!)
- Fix completion of provider paths when a path returns itself instead of its children (#24755) (Thanks @MartinGC94!)
- Enable completion of scoped variables without specifying scope (#20340) (Thanks @MartinGC94!)
- Fix issue with incomplete results when completing paths with wildcards in non-filesystem providers (#24757) (Thanks @MartinGC94!)

Cmdlet improvements

- Update `MaxVisitCount` and `MaxHashtableKeyCount` if `VisitorSafeValueContext` indicates `SkipLimitCheck` is true for `Import-PowerShellDataFile`
- Add implicit localization fallback to `Import-LocalizedData` (#19896) (Thanks @chrisdent-de!)
- Add `-Delimiter` parameter to `Get-Clipboard` (#26572) (Thanks @MartinGC94!)
- Fix Out-GridView by replacing use of obsolete BinaryFormatter with custom implementation (#25497) (Thanks @mawosoft!)
- Improve verbose and debug logging level messaging in web cmdlets (#25510) (Thanks @JustinGrote!)
- Improve debug logging of Web cmdlet request and response (#25479) (Thanks @JustinGrote!)
- Add the parameter `Register-ArgumentCompleter -NativeFallback` to support registering a cover-all completer for native commands (#25230)
- Treat `-Target` as literal in `New-Item` (#25186) (Thanks @GameMicrowave!)
- Update PATH environment variable for package manager executable on Windows (#25847)
- Update `Get-Service` to ignore common errors when retrieving non-critical properties for a service (#24245) (Thanks @jborean93!)
- Add single/double quote support for `Join-String` Argument Completer (#25283) (Thanks @ArmaanMcleod!)

- Remove `IsScreenReaderActive()` check from `ConsoleHost` (#26118)
- Improve the `$using` expression support in `Invoke-Command` (#24025) (Thanks @jborean93!)
- Change the default feedback provider timeout from 300ms to 1000ms (#25910)
- Add support for thousands separators in `[bigint]` casting (#25396) (Thanks @AbishekPonmudi!)
- Add `MethodInvocation` trace for overload tracing (#21320) (Thanks @jborean93!)
- Fix `ConvertFrom-Json` to ignore comments inside array literals (#14553) (#26050) (Thanks @MatejKafka!)
- Fix `-Debug` to not trigger the `ShouldProcess` prompt (#26081)
- Fix `Write-Host` to respect `OutputRendering = PlainText` (#21188)
- Fix debug tracing error with magic extents (#25726) (Thanks @jborean93!)
- Fix quoting in completion if the path includes a double quote character (#25631) (Thanks @MartinGC94!)
- Fix the common parameter `-ProgressAction` for advanced functions (#24591) (Thanks @cmkb3!)
- Fix the `NullReferenceException` when writing progress records to console from multiple threads (#25440) (Thanks @kborowinski!)
- Use absolute path in `FileSystemProvider.CreateDirectory` (#24615) (Thanks @Tadas!)
- Make inherited protected internal instance members accessible in PowerShell class scope (#25245) (Thanks @mawosoft!)
- Add internal methods to check Preferences (#25514) (Thanks @iSazonov!)
- Add `-ExcludeModule` parameter to `Get-Command` (#18955) (Thanks @MartinGC94!)
- Return correct **FileName** property for `Get-Item` when listing alternate data streams (#18019) (Thanks @kilasuit!)
- Fix `Get-ItemProperty` to report non-terminating error for cast exception (#21115) (Thanks @ArmaanMcleod!)
- Fix a bug in how q handles XmlNode object (#24669) (Thanks @brendandburns!)
- Error when `New-Item -Force` is passed an invalid directory name (#24936) (Thanks @kborowinski!)
- Allow `Start-Transcript` to use `$Transcript` which is a `PSObject` wrapped string to specify the transcript path (#24963) (Thanks @kborowinski!)
- Improve `Start-Process -Wait` polling efficiency (#24711) (Thanks @jborean93!)
- Add completion of modules by their shortname (#20330) (Thanks @MartinGC94!)

Engine improvements

- Add support for new WLDP setting `EnableFileOnlyEntry` (#27922)
- Fix the dot-sourcing behavior of `pwsh -file` for advanced-function scripts (#27762)
- Fix checks for local user config file paths (#27432)
- Update PowerShell telemetry to respect the diagnostics and feedback setting on Windows (#27438)
- Enable usage in AppContainers (#27423)
- Delay update notification for one week to ensure all packages become available (#27220)
- Fix a regression in the API `CompletionCompleters.CompleteFilename()` that causes null reference exception (#26487)
- Close pipe client handles after creating the child ssh process (#26564)
- Update the **PSDiagnostics** module to manage the PowerShellCore provider in PowerShell 7 (#25590)
- Allow opt-out of the named-pipe listener using the environment variable `POWERSHELL_DIAGNOSTICS_OPTOUT` (#26086)
- Ensure that socket timeouts are set only during the token validation (#26066)
- Fix `stderr` output of console host to respect `NO_COLOR` (#24391)
- Update PSRP protocol to deprecate session key exchange between newer client and server (#25774)
- Fix the `ssh` PATH check in `SSHConnectionInfo` when the default Runspace is not available (#25780) (Thanks @jborean93!)
- Adding hex format for native command exit codes (#21067) (Thanks @sba923!)
- Fix infinite loop crash in variable type inference (#25696) (Thanks @MartinGC94!)
- Add `PSForEach` and `PSWhere` as aliases for the PowerShell intrinsic methods `Where` and `Foreach` (#25511) (Thanks @powercode!)
- Added the AIShell module to telemetry collection list (#24747)
- Added helper in `EnumSingleTypeConverter` to get enum names as array (#17785) (Thanks @fflaten!)
- Update **DnsNameList** for **X509Certificate2** to use `X509SubjectAlternativeNameExtension.EnumerateDnsNames()` Method (#24714) (Thanks @ArmaanMcleod!)
- Stringify **ErrorRecord** with empty exception message to empty string (#24949) (Thanks @MatejKafka!)
- Add `PipelineStopToken` to `Cmdlet` which will be signaled when the pipeline is stopping (#24620) (Thanks @jborean93!)
- Fallback to AppLocker after `WldpCanExecuteFile` (#24912)
- Move .NET method invocation logging to after the needed type conversion is done for method arguments (#25022)

- Fix infinite loop in variable type inference (#25206) (Thanks @MartinGC94!)
- Remove the old fuzzy suggestion and fix the local script file name suggestion (#25177)
- Make `SystemPolicy` public APIs visible but non-op on Unix platforms so that they can be included in `PowerShellStandard.Library` (#25051)
- Set standard handles explicitly when starting a process with `-NoNewWindow` (#25061)
- Fix tooltip for variable expansion and include desc (#25112) (Thanks @jborean93!)
- Use script filepath when completing relative paths for using statements (#20017) (Thanks @MartinGC94!)
- Allow DSC parsing through OS architecture translation layers (#24852) (Thanks @bdeb1337!)

Experimental features

PowerShell 7.6 includes the following changes to experimental features.

The following features have been converted to mainstream features:

- PSFeedbackProvider
- PSNativeWindowsTildeExpansion
- PSRedirectToVariable
- PSSubsystemPluginModel

This release includes the following experimental features:

- PSSerializeJSONLongEnumAsNumber - `ConvertTo-Json` now treats large enums as numbers
- PSProfileDSCResource - Add DSC v3 resource for PowerShell Profiles

---

Last updated on 09/08/2026

What's New in PowerShell 7.5

PowerShell 7.5.11 includes the following features, updates, and breaking changes. PowerShell 7.5.11 is built on the .NET 9.0.20 runtime.

For a complete list of changes, see the CHANGELOG in the GitHub repository. For more information about .NET 9, see What's new in .NET 9.

Installer updates

The macOS PKG package is now notarized and signed by Microsoft. For more information, see Install PowerShell 7 on macOS.

Breaking Changes

- Fix `-OlderThan` and `-NewerThan` parameters for `Test-Path` when using `PathType` and date range (#20942) (Thanks @ArmaanMcleod!)
  - Previously `-OlderThan` would be ignored if specified together
- Change `New-FileCatalog -CatalogVersion` default to 2 (#20428) (Thanks @ThomasNieto!)
- Block getting help from network locations in restricted remoting sessions (#20593)
- The Windows installer now remembers installation options used and uses them to initialize options for the next installation (#20420) (Thanks @reduckted!)
- `ConvertTo-Json` now serializes `BigInteger` as a number (#21000) (Thanks @jborean93!)

Updated modules

PowerShell 7.5.10 includes the following updated modules:

- **Microsoft.PowerShell.PSResourceGet** v1.1.1
- **PSReadLine** v2.3.6

Tab completion improvements

Many thanks to **@ArmaanMcleod** and others for all their work to improve tab completion.

- Fall back to type inference when hashtable key-value cannot be retrieved from safe expression (#21184) (Thanks @MartinGC94!)
- Fix the regression when doing type inference for `$_` (#21223) (Thanks @MartinGC94!)

- Expand `~` to `$HOME` on Windows with tab completion (#21529)
- Don't complete when declaring parameter name and class member (#21182) (Thanks @MartinGC94!)
- Prevent fallback to file completion when tab completing type names (#20084) (Thanks @MartinGC94)
- Add argument completer to `-Version` for `Set-StrictMode` (#20554) (Thanks @ArmaanMcleod!)
- Add `-Verb` argument completer for `Get-Verb`/ `Get-Command` and refactor `Get-Verb` (#20286) (Thanks @ArmaanMcleod)
- Add `-Verb` argument completer for `Start-Process` (#20415) (Thanks @ArmaanMcleod)
- Add `-Scope` argument completer for `*-Variable`, `*-Alias` & `*-PSDrive` commands (#20451) (Thanks @ArmaanMcleod)
- Add `-Module` completion for `Save-Help`/`Update-Help` commands (#20678) (Thanks @ArmaanMcleod)

New cmdlets

- Add `ConvertTo-CliXml` and `ConvertFrom-CliXml` cmdlets (#21063) (Thanks @ArmaanMcleod!)

Web cmdlets improvements

- Fix to allow `-PassThru` and `-Outfile` work together (#24086)
- Add `OutFile` property in `WebResponseObject` (#24047)
- Show filename in `Invoke-WebRequest -OutFile -Verbose` (#24041)
- Fix WebCmdlets when `-Body` is specified but `ContentType` is not (#23952) (Thanks @CarloToso!)
- Fix `Invoke-WebRequest` to report correct size when `-Resume` is specified (#20207) (Thanks @LNKLEO!)
- Fix Web Cmdlets to allow `WinForm` apps to work correctly (#20606)

Other cmdlet improvements

- Update `MaxVisitCount` and `MaxHashtableKeyCount` if `VisitorSafeValueContext` indicates `SkipLimitCheck` is true for `Import-PowerShellDataFile`
- Close pipe client handles after creating the child ssh process (#26822)
- Fix the progress preference variable in script cmdlets (#26791) (Thanks @cmkb3!)

- Fix `Out-GridView` by replacing the use of obsolete `BinaryFormatter` with custom implementation (#25559)
- Enable `-NoRestart` to work with `Register-PSSessionConfiguration` (#23891)
- Add `IgnoreComments` and `AllowTrailingCommas` options to `Test-Json` cmdlet (#23817) (Thanks @ArmaanMcleod!)
- Get-Help may report parameters with `ValueFromRemainingArguments` attribute as pipeline-able (#23871)
- Change type of `LineNumber` to `ulong` in `Select-String` (#24075) (Thanks @Snowman-25!)
- `Get-Process`: Remove admin requirement for `-IncludeUserName` (#21302) (Thanks @jborean93!)
- Fix `Test-Path -IsValid` to check for invalid path and filename characters (#21358)
- Add `RecommendedAction` to `ConciseView` of the error reporting (#20826) (Thanks @JustinGrote!)
- Added progress bar for `Remove-Item` cmdlet (#20778) (Thanks @ArmaanMcleod!)
- Fix `Test-Connection` due to .NET 8 changes (#20369)
- Fix `Get-Service` non-terminating error message to include category (#20276)
- Add `-Empty` and `-InputObject` parameters to `New-Guid` (#20014) (Thanks @CarloToso!)
- Add the alias `r` to the parameter `-Recurse` for the `Get-ChildItem` command (#20100) (Thanks @kilasuit!)
- Add `LP` to `LiteralPath` aliases for functions still missing it (#20820)
- Add implicit localization fallback to `Import-LocalizedData` (#19896) (Thanks @chrisdent-de!)
- Add `Aliases` to the properties shown up when formatting the help content of the parameter returned by `Get-Help` (#20994)
- Add `HelpUri` to `Remove-Service` (#20476)
- Fix completion crash for the SCCM provider (#20815, #20919, #20915) (Thanks @MartinGC94!)
- Fix regression in `Get-Content` when `-Tail 0` and `-Wait` are used together (#20734) (Thanks @CarloToso!)
- Fix `Start-Process -PassThru` to make sure the `ExitCode` property is accessible for the returned `Process` object (#20749) (Thanks @CodeCyclone!)
- Fix `Group-Object` to use current culture for its output (#20608)
- Fix `Group-Object` output using interpolated strings (#20745) (Thanks @mawosoft!)
- Fix rendering of `DisplayRoot` for network `PSDrive` (#20793)
- Fix `Copy-Item` progress to only show completed when all files are copied (#20517)
- Fix UNC path completion regression (#20419) (Thanks @MartinGC94!)
- Report error if invalid `-ExecutionPolicy` is passed to `pwsh` (#20460)

- Add **WinGetCommandNotFound** and **CompletionPredictor** modules to track usage (#21040)
- Add **DateKind** parameter to `ConvertFrom-Json` (#20925) (Thanks @jborean93!)
- Add **DirectoryInfo** to the OutputType for New-Item (#21126) (Thanks @MartinGC94!)
- Fix `Get-Error` serialization of array values (#21085) (Thanks @jborean93!)
- Fix `Test-ModuleManifest` so it can use a UNC path (#24115)
- Fix `Get-TypeData` to write to the pipeline immediately instead of collecting data first (#24236) (Thanks @MartinGC94)
- Add `-Force` parameter to `Resolve-Path` and `Convert-Path` cmdlets to support wildcard hidden files #20981 (Thanks @ArmaanMcleod!)
- Set standard handles explicitly when starting a process with `-NoNewWindow` (#25324)
- Make inherited protected internal instance members accessible in class scope. (#25547) (Thanks @mawosoft!)
- Remove the old fuzzy suggestion and fix the local script file name suggestion (#25330)
- Fix `PSMethodInvocationConstraints.GetHashCode` method (#25306) (Thanks @crazyjncsu!)

Engine improvements

- Fix the dot-sourcing behavior of `pwsh -file` for advanced-function scripts (#27761)
- Fix checks for local user config file paths (#27479)
- Update PowerShell telemetry to respect the diagnostics and feedback setting on Windows (#27472)
- Fix the logic for finding the ssh executable in the PATH (#26165) (Thanks @jborean93!)
- Move .NET method invocation logging to after the needed type conversion is done for method arguments (#25357)
- Fallback to AppLocker after `WldpCanExecuteFile` (#25305)
- Explicitly start and stop ANSI Error Color (#24065) (Thanks @JustinGrote!)
- Improve .NET overload definition of generic methods (#21326) (Thanks @jborean93!)
- Optimize the `+=` operation for a collection when it's an object array (#23901) (Thanks @jborean93!)
- Add telemetry to check for specific tags when importing a module (#20371)
- Add `PSAdapter` and `ConsoleGuiTools` to module load telemetry allowlist (#20641)
- Add WinGet module to track usage (#21040)
- Ensure the filename is not null when logging WDAC ETW events (#20910) (Thanks @jborean93!)
- Fix four regressions introduced by the WDAC logging feature (#20913)
- Leave the input, output, and error handles unset when they are not redirected (#20853)

- Fix implicit remoting proxy cmdlets to act on common parameters (#20367)
- Include the module version in error messages when module is not found (#20144) (Thanks @ArmaanMcleod!)
- Fix `unixmode` to handle `setuid` and `sticky` when file is not an executable (#20366)
- Fix using assembly to use Path.Combine when constructing assembly paths (#21169)
- Validate the value for using namespace during semantic checks to prevent declaring invalid namespaces (#21162)
- Handle global tool specially when prepending `$PSHOME` to PATH (#24228)
- Delay update notification for one week to ensure all packages become available (#27220)

Experimental features

The following experimental features were converted to mainstream features in PowerShell 7.5-rc.1:

- PSCommandNotFoundSuggestion
- PSCommandWithArgs
- PSModuleAutoLoadSkipOfflineFiles

The following experimental features are included in PowerShell 7.5-rc.1:

- PSRedirectToVariable - Allow redirecting to a variable (#20381)
- PSNativeWindowsTildeExpansion - Add tilde expansion for Windows-native executables (#20402) (Thanks @domsleee!)
- PSSerializeJSONLongEnumAsNumber - `ConvertTo-Json` now treats large enums as numbers (#20999) (Thanks @jborean93!)

Performance improvements

PowerShell 7.5-rc.1 included PR#23901 from @jborean93 that improves the performance of the `+=` operation for an array of objects.

The following example measures the performance for different methods of adding elements to an array.

PowerShell

```
$tests = @{
    'Direct Assignment' = {
        param($count)

    $result = foreach($i in 1..$count) {
```

```
            $i
        }
    }
    'List<T>.Add(T)' = {
        param($count)

        $result = [Collections.Generic.List[int]]::new()
        foreach($i in 1..$count) {
            $result.Add($i)
        }
    }
    'Array+= Operator' = {
        param($count)

        $result = @()
        foreach($i in 1..$count) {
            $result += $i
        }
    }
}

5kb, 10kb | ForEach-Object {
    $groupResult = foreach($test in $tests.GetEnumerator()) {
        $ms = (Measure-Command { & $test.Value -Count $_ }).TotalMilliseconds

        [pscustomobject]@{
            CollectionSize    = $_
            Test              = $test.Key
            TotalMilliseconds = [Math]::Round($ms, 2)
        }

        [GC]::Collect()
            [GC]::WaitForPendingFinalizers()
    }

    $groupResult = $groupResult | Sort-Object TotalMilliseconds
        $groupResult | Select-Object *, @{
            Name       = 'RelativeSpeed'
            Expression = {
                $relativeSpeed = $_.TotalMilliseconds /
$groupResult[0].TotalMilliseconds
                $speed = [Math]::Round($relativeSpeed, 2).ToString() + 'x'
                if ($speed -eq '1x') { $speed } else { $speed + ' slower' }
            }
        } | Format-Table -AutoSize
}
```

When you run the script in PowerShell 7.4.6, you see that using the `+=` operator is the slowest method.

Output

```
CollectionSize Test                TotalMilliseconds RelativeSpeed
-------------- ----                ----------------- -------------
          5120 Direct Assignment                4.17 1x
          5120 List<T>.Add(T)                  90.79 21.77x slower
          5120 Array+= Operator               342.58 82.15x slower


CollectionSize Test                TotalMilliseconds RelativeSpeed
-------------- ----                ----------------- -------------
         10240 Direct Assignment                0.64 1x
         10240 List<T>.Add(T)                 184.10 287.66x slower
         10240 Array+= Operator              1668.13 2606.45x slower
```

When you run the script in PowerShell 7.5-rc.1, you see that using the `+=` operator is much faster than PowerShell 7.4.6. Now, it's also faster than using the `List<T>.Add(T)` method.

Output

```
CollectionSize Test                TotalMilliseconds RelativeSpeed
-------------- ----                ----------------- -------------
          5120 Direct Assignment                4.71 1x
          5120 Array+= Operator                40.42 8.58x slower
          5120 List<T>.Add(T)                  92.17 19.57x slower


CollectionSize Test                TotalMilliseconds RelativeSpeed
-------------- ----                ----------------- -------------
         10240 Direct Assignment                1.76 1x
         10240 Array+= Operator               104.73 59.51x slower
         10240 List<T>.Add(T)                 173.00 98.3x slower
```

Last updated on 09/08/2026

What's New in PowerShell 7.4

PowerShell 7.4.20 includes the following features, updates, and breaking changes. PowerShell 7.4.20 is built on the .NET 8.0.31 runtime.

For a complete list of changes, see the CHANGELOG in the GitHub repository.

Breaking changes

- Nano server docker images aren't available for this release
- Added the **ProgressAction** parameter to the Common Parameters
- Update some PowerShell APIs to throw **ArgumentException** instead of **ArgumentNullException** when the argument is an empty string (#19215) (Thanks @xtqqczze!)
- Remove code related to `#Requires -PSSnapin` (#19320)
- `Test-Json` now uses JsonSchema.NET instead of Newtonsoft.Json.Schema.
  - With this change, `Test-Json` no longer supports the older Draft 4 schemas. (#18141) (Thanks @gregsdennis!). For more information about JSON schemas, see JSON Schema documentation. This also breaks `Test-Json` for JSON and JSONC files with comments.
  - `ConvertFrom-Json` support still uses Newtonsoft.Json.Schema so it can convert JSON files with comments.
- Output from `Test-Connection` now includes more detailed information about TCP connection tests
- .NET introduced changes that affected `Test-Connection`. The cmdlet now returns an error about the need to use `sudo` on Linux platforms when using a custom buffer size (#20369)
- Experimental feature PSNativeCommandPreserveBytePipe is now mainstream. PowerShell now preserves the byte-stream data when redirecting the **stdout** stream of a native command to a file or when piping byte-stream data to the stdin stream of a native command.
- Change how relative paths in `Resolve-Path` are handled when using the **RelativeBasePath** parameter (#19755) (Thanks @MartinGC94!)
- Remove unused PSv2 code - removes TabExpansion function (#18337)

Installer updates

The macOS PKG package is now notarized and signed by Microsoft. For more information, see Install PowerShell 7 on macOS.

The Windows MSI package provides an option to disable PowerShell telemetry during installation. For more information, see Install the msi package from the command line.

Updated versions of PSResourceGet and PSReadLine

PowerShell 7.4 includes **Microsoft.PowerShell.PSResourceGet** v1.1.1. This module is installed side-by-side with **PowerShellGet** v2.2.5 and **PackageManagement** v1.4.8.1. For more information, see the documentation for Microsoft.PowerShell.PSResourceGet.

PowerShell 7.4 now includes **PSReadLine** v2.3.6. For more information, see the documentation for PSReadLine.

Tab completion improvements

Many thanks to **@MartinGC94** and others for all their work to improve tab completion.

- Fix issue when completing the first command in a script with an empty array expression (#18355)
- Fix positional argument completion (#17796)
- Prioritize the default parameter set when completing positional arguments (#18755)
- Improve pseudo binding for dynamic parameters (#18030)
- Improve type inference of hashtable keys (#17907)
- Fix type inference error for empty return statements (#18351)
- Improve type inference for Get-Random (#18972)
- Fix type inference for all scope variables (#18758)
- Improve enumeration of inferred types in pipeline (#17799)
- Add completion for values in comparisons when comparing Enums (#17654)
- Add property assignment completion for enums (#19178)
- Fix completion for PSCustomObject variable properties (#18682)
- Fix member completion in attribute argument (#17902)
- Exclude redundant parameter aliases from completion results (#19382)
- Fix class member completion for classes with base types (#19179)
- Add completion for the `using` keyword (#16514)
- Fix TabExpansion2 variable leak when completing variables (#18763)
- Enable completion of variables across ScriptBlock scopes (#19819)
- Fix completion of the foreach statement variable (#19814)

- Fix variable type inference precedence (#18691)
- Fix member completion for PowerShell Enum class (#19740)
- Fix parsing for array literals in index expressions in method calls (#19224)
- Improve path completion (#19489)
- Fix an indexing out of bound error in CompleteInput for empty script input (#19501)
- Improve variable completion performance (#19595)
- Improve Hashtable key completion for type constrained variable assignments, nested Hashtables and more (#17660)
- Infer external application output as strings (#19193)
- Update parameter completion for enums to exclude values not allowed by `ValidateRange` attributes (#17750) (Thanks @fflaten!).
- Fix dynamic parameter completion (#19510)
- Add completion for variables assigned by the `data` statement (#19831)
- Fix expanding tilde (`~`) on Windows systems to `$HOME` to prevent breaking use cases with native commands (#21529)

Web cmdlet improvements

Many thanks to **@CarloToso** and others for all the work on improving web cmdlets.

- Fix decompression in web cmdlets to include Brotli (#17955) (Thanks @iSazonov!)
- Webcmdlets add 308 to redirect codes and small cleanup (#18536)
- Complete the progress bar rendering in Invoke-WebRequest when downloading is complete or cancelled (#18130)
- Web cmdlets get **Retry-After** interval from response headers if the status code is 429 (#18717)
- Web cmdlets set default charset encoding to UTF8 (#18219)
- Preserve WebSession.MaximumRedirection from changes (#19190)
- WebCmdlets parse XML declaration to get encoding value, if present. (#18748)
- Fix using xml -Body in webcmdlets without an encoding (#19281)
- Adjust PUT method behavior to POST one for default content type in WebCmdlets (#19152)
- Take into account ContentType from Headers in WebCmdlets (#19227)
- Allow to preserve the original HTTP method by adding **-PreserveHttpMethodOnRedirect** to Web cmdlets (#18894)
- Webcmdlets display an error on https to http redirect (#18595)
- Add **AllowInsecureRedirect** switch to Web cmdlets (#18546)
- Improve verbose message in web cmdlets when content length is unknown (#19252)

- Build the relative URI for links from the response in `Invoke-WebRequest` (#19092)
- Fix redirection for `-CustomMethod POST` in WebCmdlets (#19111)
- Dispose previous response in Webcmdlets (#19117)
- Improve `Invoke-WebRequest` xml and json errors format (#18837)
- Add ValidateNotNullOrEmpty to **OutFile** and **InFile** parameters of WebCmdlets (#19044)
- HttpKnownHeaderNames update headers list (#18947)
- `Invoke-RestMethod -FollowRelLink` fix links containing commas (#18829)
- Fix bug with managing redirection and KeepAuthorization in Web cmdlets (#18902)
- Add **StatusCode** to **HttpResponseException** (#18842)
- Support HTTP persistent connections in Web Cmdlets (#19249) (Thanks @stevenebutler!)
- Small cleanup `Invoke-RestMethod` (#19490)
- Improve the verbose message of WebCmdlets to show correct HTTP version (#19616)
- Add **FileNameStar** to **MultipartFileContent** in WebCmdlets (#19467)
- Fix HTTP status from 409 to 429 for WebCmdlets to get retry interval from Retry-After header. (#19622) (Thanks @mkht!)
- Change `-TimeoutSec` to `-ConnectionTimeoutSeconds` and add `-OperationTimeoutSeconds` to web cmdlets (#19558) (Thanks @stevenebutler!) Other cmdlets
- Support Ctrl+c when connection hangs while reading data in WebCmdlets (#19330) (Thanks @stevenebutler!)
- Support Unix domain socket in WebCmdlets (#19343)

Other cmdlet improvements

- Fix `Out-GridView` by replacing use of obsolete BinaryFormatter with custom implementation (#27426)
- Update `MaxVisitCount` and `MaxHashtableKeyCount` if `VisitorSafeValueContext` indicates `SkipLimitCheck` is true for `Import-PowerShellDataFile`
- `Test-Connection` now returns error about the need to use `sudo` on Linux platforms when using a custom buffer size (#20369)
- Add output types to Format commands (#18746) (Thanks @MartinGC94!)
- Add output type attributes for `Get-WinEvent` (#17948) (Thanks @MartinGC94!)
- Add **Path** and **LiteralPath** parameters to `Test-Json` cmdlet (#19042) (Thanks @ArmaanMcleod!)
- Add **NoHeader** parameter to `ConvertTo-Csv` and `Export-Csv` cmdlets (#19108) (Thanks @ArmaanMcleod!)
- Add **Confirm** and **WhatIf** parameters to `Stop-Transcript` (#18731) (Thanks @JohnLBevan!)

- Add **FuzzyMinimumDistance** parameter to `Get-Command` (#18261)
- Make **Encoding** parameter able to take `ANSI` encoding in PowerShell (#19298) (Thanks @CarloToso!)
- Add progress to `Copy-Item` (#18735)
- `Update-Help` now reports an error when using implicit culture on non-US systems. (#17780) (Thanks @dkaszews!)
- Don't require activity when creating a completed progress record (#18474) (Thanks @MartinGC94!)
- Disallow negative values for `Get-Content` cmdlet parameters `-Head` and `-Tail` (#19715) (Thanks @CarloToso!)
- Make `Update-Help` throw proper error when current culture isn't associated with a language (#19765) (Thanks @josea!)
- Allow combining of `-Skip` and `-SkipLast` parameters in `Select-Object` cmdlet. (#18849) (Thanks @ArmaanMcleod!)
- Add `Get-SecureRandom` cmdlet (#19587)
- `Set-Clipboard -AsOSC52` for remote usage (#18222) (Thanks @dkaszews!)
- Speed up `Resolve-Path` relative path resolution (#19171) (Thanks @MartinGC94!)
- Added the `[switch]` parameter `-CaseInsensitive` to `Select-Object` and `Get-Unique` cmdlets (#19683) (Thanks @ArmaanMcleod!)
- `Restart-Computer` and `Stop-Computer` should fail with error when not running via sudo on Unix (#19824)

Engine improvements

Updates to `$PSStyle`

- Adds **Dim** and **DimOff** properties (#18653)
- Added static methods to the **PSStyle** class that map foreground and background **ConsoleColor** values to ANSI escape sequences (#17938)
- Table headers for calculated fields are formatted in italics by default
- Add support of respecting `$PSStyle.OutputRendering` on the remote host (#19601)
- Updated telemetry data to include use of `CrescendoBuilt` modules (#20371)

Other Engine updates

- Fix the dot-sourcing behavior of `pwsh -file` for advanced-function scripts (#27760)
- Fix checks for local user config file paths (#27454)
- Update PowerShell telemetry to respect the diagnostics and feedback setting on Windows (#27430)

- Delay update notification for one week to ensure all packages become available (#27229)
- Close pipe client handles after creating the child ssh process (#27139)
- Move .NET method invocation logging to after the needed type conversion is done for method arguments (#25568)
- Fallback to AppLocker after WldpCanExecuteFile (#25229)
- Make PowerShell class not affiliate with Runspace when declaring the `NoRunspaceAffinity` attribute (#18138)
- Add the `ValidateNotNullOrWhiteSpace` attribute (#17191) (Thanks @wmentha!)
- Add `sqlcmd` to the list for legacy argument passing (#18559)
- Add the function `cd~` (#18308) (Thanks @GigaScratch!)
- Fix array type parsing in generic types (#19205) (Thanks @MartinGC94!)
- Fix wildcard globbing in root of device paths (#19442) (Thanks @MartinGC94!)
- Add a public API for getting locations of PSModulePath elements (#19422)
- Fix incorrect string to type conversion (#19560) (Thanks @MartinGC94!)
- Fix slow execution when many breakpoints are used (#14953) (Thanks @nohwnd!)
- Remove code related to `#Requires -PSSnapin` (#19320)

Experimental Features

PowerShell 7.4 introduces the following experimental features:

- PSFeedbackProvider - Replaces the hard-coded suggestion framework with an extensible feedback provider.
  - This feature also adds the **FeedbackName**, **FeedbackText**, and **FeedbackAction** properties to `$PSStyle.Formatting` that allow you to change the formatting of feedback messages.
- PSModuleAutoLoadSkipOfflineFiles - Module discovery now skips over files that are marked by cloud providers as not fully on disk.
- PSCommandWithArgs - Add support for passing arguments to commands as a single string

The following experimental features became mainstream:

- PSConstrainedAuditLogging
- PSCustomTableHeaderLabelDecoration
- PSNativeCommandErrorActionPreference
- PSNativeCommandPreserveBytePipe
- PSWindowsNativeCommandArgPassing

PowerShell 7.4 changed the following experimental features:

- PSCommandNotFoundSuggestion - This feature now uses an extensible feedback provider rather than hard-coded suggestions (#18726)

For more information about the Experimental Features, see Using Experimental Features.

Last updated on 09/08/2026

What's New in PowerShell 7.3

PowerShell 7.3 is the next stable release, built on .NET 7.0.

PowerShell 7.3 includes the following features, updates, and breaking changes.

Breaking Changes and Improvements

- In this release, Windows APIs were updated or removed for compliance, which means that PowerShell 7.3 doesn't run on Windows 7. While Windows 7 is no longer supported, previous builds could run on Windows 7.
- PowerShell Direct for Hyper-V is only supported on Windows 10, version 1809 and higher.
- `Test-Connection` is broken due to an intentional breaking change in .NET 7. It's tracked by #17018
- Add `clean` block to script block as a peer to `begin`, `process`, and `end` to allow easy resource cleanup (#15177)
- Change default for `$PSStyle.OutputRendering` to **Host**
- Make `Out-String` and `Out-File` keep string input unchanged (#17455)
- Move the type data definition of System.Security.AccessControl.ObjectSecurity to the Microsoft.PowerShell.Security module (#16355) (Thanks @iSazonov!)
  - Before this change, a user doesn't need to explicitly import the **Microsoft.PowerShell.Security** module to use the code properties defined for an instance of **System.Security.AccessControl.ObjectSecurity**.
  - After this change, a user needs to explicitly import **Microsoft.PowerShell.Security** module in order to use those code properties and code methods.

Tab completion improvements

- PowerShell 7.3 includes PSReadLine 2.2.6, which enables Predictive IntelliSense by default. For more information, see about_PSReadLine.
- Fix tab completion within the script block specified for the `ValidateScriptAttribute`. (#14550) (Thanks @MartinGC94!)
- Added tab completion for loop labels after `break`/`continue` (#16438) (Thanks @MartinGC94!)
- Improve Hashtable completion in multiple scenarios (#16498) (Thanks @MartinGC94!)
  - Parameter splatting

    - **Arguments** parameter for `Invoke-CimMethod`
    - **FilterHashtable** parameter for `Get-WinEvent`
    - **Property** parameter for the CIM cmdlets
    - Removes duplicates from member completion scenarios
- Support forward slashes in network share (UNC path) completion (#17111) (Thanks @sba923!)
- Improve member autocompletion (#16504) (Thanks @MartinGC94!)
- Prioritize ValidateSet completions over Enums for parameters (#15257) (Thanks @MartinGC94!)
- Add type inference support for generic methods with type parameters (#16951) (Thanks @MartinGC94!)
- Improve type inference and completions (#16963) (Thanks @MartinGC94!)
    - Allows methods to be shown in completion results for `ForEach-Object -MemberName`
    - Prevents completion on expressions that return void like `([void](""))`
    - Allows non-default Class constructors to show up when class completion is based on the AST
- Improve type inference for `$_` (#17716) (Thanks @MartinGC94!)
- Fix type inference for **ICollection** (#17752) (Thanks @MartinGC94!)
- Prevent braces from being removed when completing variables (#17751) (Thanks @MartinGC94!)
- Add completion for index expressions for dictionaries (#17619) (Thanks @MartinGC94!)
- Fix type completion for attribute tokens (#17484) (Thanks @MartinGC94!)
- Improve dynamic parameter tab completion (#17661) (Thanks @MartinGC94!)
- Avoid binding positional parameters when completing parameter in front of value (#17693) (Thanks @MartinGC94!)

Improved error handling

- Set `$?` correctly for command expression with redirections (#16046)
- Fix a casting error when using `$PSNativeCommandUseErrorActionPreference` (#15993)
- Make the native command error handling optionally honor `ErrorActionPreference` (#15897)
- Specify the executable path as `TargetObject` for non-zero exit code ErrorRecord (#16108) (Thanks @rkeithhill!)

Session and remoting improvements

- Add `-Options` to the PSRP over SSH commands to allow passing OpenSSH options directly (#12802) (Thanks @BrannenGH!)
- Add `-ConfigurationFile` parameter to `pwsh` to allow starting a new process with the session configuration defined in a `.pssc` file (#17447)
- Add support for using `New-PSSessionConfigurationFile` on non-Windows platforms (#17447)

Updated cmdlets

- Add `-HttpVersion` parameter to web cmdlets (#15853) (Thanks @hayhay27!)
- Add support to web cmdlets for open-ended input tags (#16193) (Thanks @farmerau!)
- Fix `ConvertTo-Json -Depth` to allow 100 at maximum (#16197) (Thanks @KevRitchie!)
- Improve variable handling when calling `Invoke-Command` with the `$Using:` expression (#16113) (Thanks @dwtaber!)
- Add `-StrictMode` to `Invoke-Command` to allow specifying strict mode when invoking command locally (#16545) (Thanks @Thomas-Yu!)
- Add `clean` block to script block as a peer to `begin`, `process`, and `end` to allow easy resource cleanup (#15177)
- Add `-Amended` switch to `Get-CimClass` cmdlet (#17477) (Thanks @iSazonov)
- Changed `ConvertFrom-Json -AsHashtable` to use ordered hashtable (#17405)
- Removed ANSI escape sequences in strings before sending to `Out-GridView` (#17664)
- Added the **Milliseconds** parameter to `New-TimeSpan` (#17621) (Thanks @NoMoreFood!)
- Show optional parameters when displaying method definitions and overloads (#13799) (Thanks @eugenesmlv!)
- Allow commands to still be executed even if the current working directory no longer exists (#17579)
- Add support for HTTPS with `Set-AuthenticodeSignature -TimeStampServer` (#16134) (Thanks @Ryan-Hutchison-USAF!)
- Render decimal numbers in a table using current culture (#17650)
- Add type accelerator ordered for **OrderedDictionary** (#17804) (Thanks @fflaten!)
- Add `find.exe` to legacy argument binding behavior for Windows (#17715)
- Add `-NoProfileLoadTime` switch to pwsh (#17535) (Thanks @rkeithhill!)

For a complete list of changes, see the Change Log in the GitHub repository.

Experimental Features

In PowerShell 7.3, following experimental features became mainstream:

- `PSAnsiRenderingFileInfo` - This feature adds the `$PSStyle.FileInfo` member and enables coloring of specific file types.
- `PSCleanBlock` - Adds `clean` block to script block as a peer to `begin`, `process`, and `end` to allow easy resource cleanup.
- `PSAMSIMethodInvocationLogging` - Extends the data sent to AMSI for inspection to include all invocations of .NET method members.
- PSNativeCommandArgumentPassing - PowerShell now uses the **ArgumentList** property of the **StartProcessInfo** object rather than the old mechanism of reconstructing a string when invoking a native executable.

  PowerShell 7.3.1 adds `sqlcmd.exe` to the list of native commands in Windows that use the `Legacy` style of argument passing.
- `PSExec` - Adds the new `Switch-Process` cmdlet (alias `exec`) to provide `exec` compatibility for non-Windows systems.

  PowerShell 7.3.1 changed the `exec` alias to a function that wraps `Switch-Process`. The function allows you to pass parameters to the native command that might have erroneously bound to the **WithCommand** parameter.

PowerShell 7.3 introduces the following experimental features:

- PSNativeCommandErrorActionPreference - Adds the `$PSNativeCommandUseErrorActionPreference` variable to enable errors produced by native commands to be PowerShell errors.

PowerShell 7.3 removed the following experimental features:

- `PSNativePSPathResolution` experimental feature is no longer supported.
- `PSStrictModeAssignment` experimental feature is no longer supported.

For more information about the Experimental Features, see Using Experimental Features.

Last updated on 03/24/2025

What's New in PowerShell 7.2

Article • 01/23/2025

PowerShell 7.2 is the next Long Term Servicing (LTS) release is built on .NET 6.0.

PowerShell 7.2 includes the following features, updates, and breaking changes.

- New universal installer packages for most supported Linux distributions
- Microsoft Update support on Windows
- 2 new experimental features
  - Improved native command argument passing support
  - ANSI FileInfo color support
- Improved Tab Completions
- PSReadLine 2.1 with Predictive IntelliSense
- 7 experimental features promoted to mainstream and 1 removed
- Separating DSC from PowerShell 7 to enable future improvements
- Several breaking changes to improve usability

For a complete list of changes, see the Change Log in the GitHub repository.

Installation updates

Check the installation instructions for your preferred operating system:

- Windows
- macOS
- Linux

Additionally, PowerShell 7.2 supports ARM64 versions of Windows and macOS and ARM32 and ARM64 versions of Debian and Ubuntu.

For up-to-date information about supported operating systems and support lifecycle, see the PowerShell Support Lifecycle.

New universal install packages for Linux distributions

Previously, we created separate installer packages for each supported version of CentOS, RHEL, Debian, and Ubuntu. The universal installer package combines eight different packages into one, making installation on Linux simpler. The universal package installs the necessary dependencies for the target distribution and creates the platform-specific changes to make PowerShell work.

Microsoft Update support for Windows

PowerShell 7.2 add support for Microsoft Update. When you enable this feature, you'll get the latest PowerShell 7 updates in your traditional Windows Update (WU) management flow, whether that's with Windows Update for Business, WSUS, SCCM, or the interactive WU dialog in Settings.

The PowerShell 7.2 MSI package includes following command-line options:

- `USE_MU` - This property has two possible values:
  - `1` (default) - Opts into updating through Microsoft Update or WSUS
  - `0` - don't opt into updating through Microsoft Update or WSUS
- `ENABLE_MU`
  - `1` (default) - Opts into using Microsoft Update the Automatic Updates or Windows Update
  - `0` - don't opt into using Microsoft Update the Automatic Updates or Windows Update

Experimental Features

The following experimental features are now mainstream features in this release:

- `Microsoft.PowerShell.Utility.PSImportPSDataFileSkipLimitCheck` - see Import-PowerShellDataFile
- `Microsoft.PowerShell.Utility.PSManageBreakpointsInRunspace`
- `PSAnsiRendering` - see about_ANSI_Terminals
- `PSAnsiProgress` - see about_ANSI_Terminals
- `PSCultureInvariantReplaceOperator`
- `PSNotApplyErrorActionToStderr`
- `PSUnixFileStat`

The following experimental feature was added in this release:

- PSNativeCommandArgumentPassing - When this experimental feature is enabled PowerShell uses the **ArgumentList** property of the **StartProcessInfo** object rather than our current mechanism of reconstructing a string when invoking a native executable. This feature adds a new automatic variable `$PSNativeCommandArgumentPassing` that allows you to select the behavior at runtime.

- PSAnsiRenderingFileInfo - Allow ANSI color customization of file information.

- PSLoadAssemblyFromNativeCode - Exposes an API to allow assembly loading from native code.

For more information about the Experimental Features, see Using Experimental Features.

Improved Tab Completions

PowerShell 7.2 includes several improvements to Tab Completion. These changes include bugfixes and improve usability.

- Fix tab completion for unlocalized about* topics (#15265) (Thanks @MartinGC94)
- Fix splatting being treated as positional parameter in completions (#14623) (Thanks @MartinGC94)
- Add completions for comment-based help keywords (#15337) (Thanks @MartinGC94)
- Add completion for Requires statements (#14596) (Thanks @MartinGC94)
- Added tab completion for View parameter of Format-* cmdlets (#14513) (Thanks @iSazonov)

PSReadLine 2.1 Predictive IntelliSense

PSReadLine 2.1 introduced `CommandPrediction` APIs that establish a framework for providing predictions for command-line completion. The API enables users to discover, edit, and execute full commands based on matching predictions from the user's history.

Predictive IntelliSense is disabled by default. To enable predictions, run the following command:

PowerShell

```
Set-PSReadLineOption -PredictionSource History
```

Separating DSC from PowerShell 7 to enable future improvements

The PSDesiredStateConfiguration module was removed from the PowerShell 7.2 package and is now published to the PowerShell Gallery. This allows the PSDesiredStateConfiguration module to be developed independently of PowerShell and users can mix and match versions of PowerShell and PSDesiredStateConfiguration for

their environment. To install PSDesiredStateConfiguration 2.0.5 from the PowerShell Gallery:

```powershell
Install-Module -Name PSDesiredStateConfiguration -Repository PSGallery -
MaximumVersion 2.99
```

> **ⓘ Important**
>
> Be sure to include the parameter MaximumVersion or you could install version 3 (or higher) of PSDesireStateConfiguration that contains significant differences.

Engine updates

- Add `LoadAssemblyFromNativeMemory` function to load assemblies in memory from a native PowerShell host by awakecoding · Pull Request #14652

Breaking Changes and Improvements

- The PSDesiredStateConfiguration was removed from the PowerShell 7.2 package
- Make PowerShell Linux deb and RPM packages universal (#15109)
- Experimental feature `PSNativeCommandArgumentPassing`: Use ArgumentList for native executable invocation (#14692)
- Ensure `-PipelineVariable` is set for all output from script cmdlets (#12766)
- Emit warning if `ConvertTo-Json` exceeds -Depth value (#13692)
- Remove alias D of -Directory switch CL-General #15171
- Improve detection of mutable value types (#12495)
- Restrict `New-Object` in **NoLanguage** mode under lock down (#14140)
- Enforce AppLocker Deny configuration before Execution Policy Bypass configuration (#15035)
- Change `FileSystemInfo.Target` from a **CodeProperty** to an **AliasProperty** that points to `FileSystemInfo.LinkTarget` (#16165)

Migrating from Windows PowerShell 5.1 to PowerShell 7

Designed for cloud, on-premises, and hybrid environments, PowerShell 7 is packed with enhancements and new features.

- Installs and runs side-by-side with Windows PowerShell
- Improved compatibility with existing Windows PowerShell modules
- New language features, like ternary operators and `ForEach-Object -Parallel`
- Improved performance
- SSH-based remoting
- Cross-platform interoperability
- Support for Docker containers

PowerShell 7 works side-by-side with Windows PowerShell letting you easily test and compare between editions before deployment. Migration is simple, quick, and safe.

PowerShell 7 is supported on the following Windows operating systems:

- Windows 10, and 11
- Windows Server 2016, 2019, and 2022

PowerShell 7 also runs on macOS and several Linux distributions. For a list of supported operating systems and information about the support lifecycle, see the PowerShell Support Lifecycle.

Installing PowerShell 7

For flexibility and to support the needs of IT, DevOps engineers, and developers, there are several options available to install PowerShell 7. In most cases, the installation options can be reduced to the following methods:

- Deploy PowerShell using the MSI package
- Deploy PowerShell using the ZIP package

> **Note**
>
> The MSI package can be deployed and updated with management products such as Microsoft Configuration Manager. Download the packages from GitHub Release page.

Deploying the MSI package requires Administrator permission. The ZIP package can be deployed by any user. The ZIP package is the easiest way to install PowerShell 7 for testing, before committing to a full installation.

You may also install PowerShell 7 via the Windows Store or `winget`. For more information about both of these methods, see the detailed instructions in Installing PowerShell on Windows.

Using PowerShell 7 side-by-side with Windows PowerShell 5.1

PowerShell 7 is designed to coexist with Windows PowerShell 5.1. The following features ensure that your investment in PowerShell is protected and your migration to PowerShell 7 is simple.

- Separate installation path and executable name
- Separate PSModulePath
- Separate profiles for each version
- Improved module compatibility
- New remoting endpoints
- Group policy support
- Separate Event logs

Differences in .NET versions

PowerShell 7.4 is built on .NET 8.0. Windows PowerShell 5.1 is built on .NET Framework 4.x. The differences between the .NET versions might affect the behavior of your scripts, especially if you are calling .NET method directly. For more information, Differences between Windows PowerShell 5.1 and PowerShell 7.x.

Separate installation path and executable name

PowerShell 7 installs to a new directory, enabling side-by-side execution with Windows PowerShell 5.1.

Install locations by version:

- Windows PowerShell 5.1: `$Env:windir\System32\WindowsPowerShell\v1.0`
- PowerShell 6.x: `$Env:ProgramFiles\PowerShell\6`
- PowerShell 7: `$Env:ProgramFiles\PowerShell\7`

The new location is added to your PATH allowing you to run both Windows PowerShell 5.1 and PowerShell 7. If you're migrating from PowerShell 6.x to PowerShell 7, PowerShell 6 is removed

and the PATH replaced.

In Windows PowerShell, the PowerShell executable is named `powershell.exe`. In version 6 and above, the executable is named `pwsh.exe`. The new name makes it easy to support side-by-side execution of both versions.

Separate PSModulePath

By default, Windows PowerShell and PowerShell 7 store modules in different locations. PowerShell 7 combines those locations in the `$Env:PSModulePath` environment variable. When importing a module by name, PowerShell checks the location specified by `$Env:PSModulePath`. This allows PowerShell 7 to load both Core and Desktop modules.

Expand table

Install Scope	Windows PowerShell 5.1	PowerShell 7.0
PowerShell modules	`$Env:windir\system32\WindowsPowerShell\v1.0\Modules`	`$Env:ProgramFiles\PowerShell\7\Modules`
User installed AllUsers scope	`$Env:ProgramFiles\WindowsPowerShell\Modules`	`$Env:ProgramFiles\PowerShell\Modules`
User installed CurrentUser scope	`$HOME\Documents\WindowsPowerShell\Modules`	`$HOME\Documents\PowerShell\Modules`

The following examples show the default values of `$Env:PSModulePath` for each version.

- For Windows PowerShell 5.1:

  PowerShell

  ```powershell
  $Env:PSModulePath -split (';')
  ```

  Output

  ```
  C:\Users\<user>\Documents\WindowsPowerShell\Modules
  C:\Program Files\WindowsPowerShell\Modules
  C:\WINDOWS\System32\WindowsPowerShell\v1.0\Modules
  ```

- For PowerShell 7:

PowerShell

```
$Env:PSModulePath -split (';')
```

Output

```
C:\Users\<user>\Documents\PowerShell\Modules
C:\Program Files\PowerShell\Modules
C:\Program Files\PowerShell\7\Modules
C:\Program Files\WindowsPowerShell\Modules
C:\WINDOWS\System32\WindowsPowerShell\v1.0\Modules
```

Notice that PowerShell 7 includes the Windows PowerShell paths and the PowerShell 7 paths to provide autoloading of modules.

> **Note**
>
> Additional paths may exist if you have changed the PSModulePath environment variable or installed custom modules or applications.

For more information, see about_PSModulePath.

For more information about Modules, see about_Modules.

Separate profiles

A PowerShell profile is a script that executes when PowerShell starts. This script customizes your environment by adding commands, aliases, functions, variables, modules, and PowerShell drives. The profile script makes these customizations available in every session without having to manually recreate them.

The path to the location of the profile has changed in PowerShell 7.

- In Windows PowerShell 5.1, the location of the profile is `$HOME\Documents\WindowsPowerShell`.
- In PowerShell 7, the location of the profile is `$HOME\Documents\PowerShell`.

The profile filenames have also changed:

PowerShell

```
$PROFILE | Select-Object *Host* | Format-List
```

Output

```
 AllUsersAllHosts       : C:\Program Files\PowerShell\7\profile.ps1
 AllUsersCurrentHost    : C:\Program
Files\PowerShell\7\Microsoft.PowerShell_profile.ps1
 CurrentUserAllHosts    : C:\Users\<user>\Documents\PowerShell\profile.ps1
 CurrentUserCurrentHost : C:\Users\
<user>\Documents\PowerShell\Microsoft.PowerShell_profile.ps1
```

For more information about_Profiles.

PowerShell 7 compatibility with Windows PowerShell 5.1 modules

Most of the modules you use in Windows PowerShell 5.1 already work with PowerShell 7, including Azure PowerShell and Active Directory. We're continuing to work with other teams to add native PowerShell 7 support for more modules including Microsoft Graph, Office 365, and others. For the current list of supported modules, see PowerShell 7 module compatibility.

> **Note**
>
> On Windows, we've also added a **UseWindowsPowerShell** switch to `Import-Module` to ease the transition to PowerShell 7 for those using incompatible modules. For more information on this functionality, see about_Windows_PowerShell_Compatibility.

PowerShell Remoting

PowerShell remoting lets you run any PowerShell command on one or more remote computers. You can establish persistent connections, start interactive sessions, and run scripts on remote computers.

WS-Management remoting

Windows PowerShell 5.1 and below use the WS-Management (WSMAN) protocol for connection negotiation and data transport. Windows Remote Management (WinRM) uses the WSMAN protocol. If WinRM has been enabled, PowerShell 7 uses the existing Windows PowerShell 5.1 endpoint named `Microsoft.PowerShell` for remoting connections. To update PowerShell 7 to include its own endpoint, run the `Enable-PSRemoting` cmdlet. For information about connecting to specific endpoints, see WS-Management Remoting in PowerShell

To use Windows PowerShell remoting, the remote computer must be configured for remote management. For more information, including instructions, see About Remote Requirements.

For more information about working with remoting, see About Remote

SSH-based remoting

SSH-based remoting was added in PowerShell 6.x to support other operating systems that can't use Windows native components like **WinRM**. SSH remoting creates a PowerShell host process on the target computer as an SSH subsystem. For details and examples on setting up SSH-based remoting on Windows or Linux, see: PowerShell remoting over SSH.

> **Note**
>
> The PowerShell Gallery (PSGallery) contains a module and cmdlet that automatically configures SSH-based remoting. Install the `Microsoft.PowerShell.RemotingTools` module from the PSGallery and run the `Enable-SSH` cmdlet.

The `New-PSSession`, `Enter-PSSession`, and `Invoke-Command` cmdlets have new parameter sets to support SSH connections.

PowerShell

```
[-HostName <string>]  [-UserName <string>]  [-KeyFilePath <string>]
```

To create a remote session, specify the target computer with the **HostName** parameter and provide the user name with **UserName**. When running the cmdlets interactively, you're prompted for a password.

PowerShell

```
Enter-PSSession -HostName <Computer> -UserName <Username>
```

Alternatively, when using the **HostName** parameter, provide the username information followed by the at sign (`@`), followed by the computer name.

PowerShell

```
Enter-PSSession -HostName <Username>@<Computer>
```

You may set up SSH key authentication using a private key file with the **KeyFilePath** parameter. For more information, see OpenSSH Key Management.

Group Policy supported

PowerShell includes Group Policy settings to help you define consistent option values for servers in an enterprise environment. These settings include:

- Console session configuration: Sets a configuration endpoint in which PowerShell is run.
- Turn on Module Logging: Sets the LogPipelineExecutionDetails property of modules.
- Turn on PowerShell Script Block Logging: Enables detailed logging of all PowerShell scripts.
- Turn on Script Execution: Sets the PowerShell execution policy.
- Turn on PowerShell Transcription: enables capturing of input and output of PowerShell commands into text-based transcripts.
- Set the default source path for Update-Help: Sets the source for Updatable Help to a directory, not the Internet.

For more information, see about_Group_Policy_Settings.

PowerShell 7 includes Group Policy templates and an installation script in `$PSHOME`.

Group Policy tools use administrative template files (`.admx`, `.adml`) to populate policy settings in the user interface. This allows administrators to manage registry-based policy settings. The `InstallPSCorePolicyDefinitions.ps1` script installs PowerShell Administrative Templates on the local machine.

PowerShell

```powershell
Get-ChildItem -Path $PSHOME -Filter *Core*Policy*
```

Output

```
    Directory: C:\Program Files\PowerShell\7

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---           2/27/2020 12:38 AM          15861 InstallPSCorePolicyDefinitions.ps1
-a---           2/27/2020 12:28 AM           9675 PowerShellCoreExecutionPolicy.adml
-a---           2/27/2020 12:28 AM           6201 PowerShellCoreExecutionPolicy.admx
```

Separate Event Logs

Windows PowerShell and PowerShell 7 log events to separate event logs. Use the following command to get a list of the PowerShell logs.

PowerShell

```powershell
Get-WinEvent -ListLog *PowerShell*
```

For more information, see about_Logging_Windows.

Improved editing experience with Visual Studio Code

Visual Studio Code (VS Code) with the PowerShell Extension is the supported scripting environment for PowerShell 7. The Windows PowerShell Integrated Scripting Environment (ISE) only supports Windows PowerShell.

The updated PowerShell extension includes:

- New ISE compatibility mode
- PSReadLine in the Integrated Console, including syntax highlighting, multi-line editing, and back search
- Stability and performance improvements
- New CodeLens integration
- Improved path autocompletion

To make the transition to Visual Studio Code easier, use the **Enable ISE Mode** function available in the **Command Palette**. This function switches VS Code into an ISE-style layout. The ISE-style layout gives you all the new features and capabilities of PowerShell in a familiar user experience.

To switch to the new ISE layout, press `Ctrl` + `Shift` + `P` to open the **Command Palette**, type `PowerShell` and select **PowerShell: Enable ISE Mode**.

To set the layout to the original layout, open the **Command Palette**, select **PowerShell: Disable ISE Mode (restore to defaults)**.

For details about customizing the VS Code layout to ISE, see How to Replicate the ISE Experience in Visual Studio Code

> **Note**
>
> There are no plans to update the ISE with new features. In the latest versions of Windows 10 or Windows Server 2019 and higher, the ISE is now a user-uninstallable feature. There are no plans to permanently remove the ISE. The PowerShell Team and its partners are focused on improving the scripting experience in the PowerShell extension for Visual Studio Code.

Next Steps

Armed with the knowledge to effectively migrate, install PowerShell 7 now!

Last updated on 04/16/2025

Differences between Windows PowerShell 5.1 and PowerShell 7.x

Windows PowerShell 5.1 is built on top of the .NET Framework v4.5. With the release of PowerShell 6.0, PowerShell became an open source project built on .NET Core 2.0. Moving from the .NET Framework to .NET Core allowed PowerShell to become a cross-platform solution. PowerShell runs on Windows, macOS, and Linux.

There are few differences in the PowerShell language between Windows PowerShell and PowerShell. The most notable differences are in the availability and behavior of PowerShell cmdlets between Windows and non-Windows platforms and the changes that stem from the differences between the .NET Framework and .NET Core.

This article summarizes the significant differences and breaking changes between Windows PowerShell and the current version of PowerShell. This summary doesn't include new features or cmdlets that have been added. Nor does this article discuss what changed between versions. The goal of this article is to present the current state of PowerShell and how that's different from Windows PowerShell. For a detailed discussion of changes between versions and the addition of new features, see the **What's New** articles for each version.

- What's new in PowerShell 7.5
- What's new in PowerShell 7.4
- What's new in PowerShell 7.3
- What's new in PowerShell 7.2
- What's new in PowerShell 7.1
- What's new in PowerShell 7.0
- What's new in PowerShell 6.x

.NET Framework vs .NET Core

PowerShell on Linux and macOS uses .NET core, which is a subset of the full .NET Framework on Microsoft Windows. This is significant because PowerShell provides direct access to the underlying framework types and methods. As a result, scripts that run on Windows may not run on non-Windows platforms because of the differences in the frameworks. For more information about changes in .NET Core, see Breaking changes for migration from .NET Framework to .NET Core.

Each new release of PowerShell is built on a newer version of .NET. There can be breaking changes in .NET that affect PowerShell.

- PowerShell 7.6 - Built on .NET 10.0 (LTS)
- PowerShell 7.5 - Built on .NET 9.0
- PowerShell 7.4 - Built on .NET 8.0 (LTS)
- PowerShell 7.3 - Built on .NET 7.0
- PowerShell 7.2 - Built on .NET 6.0 (LTS)
- PowerShell 7.1 - Built on .NET 5.0
- PowerShell 7.0 - Built on .NET Core 3.1 (LTS)
- PowerShell 6.2 - Built on .NET Core 2.1
- PowerShell 6.1 - Built on .NET Core 2.1
- PowerShell 6.0 - Built on .NET Core 2.0

With the advent of .NET Standard 2.0, PowerShell can load many traditional Windows PowerShell modules without modification. Additionally, PowerShell 7 includes a Windows PowerShell Compatibility feature that allows you to use Windows PowerShell modules that still require the full framework.

For more information see:

- about_Windows_PowerShell_Compatibility
- PowerShell 7 module compatibility

Be aware of .NET method changes

While .NET method changes aren't specific to PowerShell, they can affect your scripts, especially if you are calling .NET methods directly. Also, there might be new overloads for constructors. This can have an impact on how you create objects using `New-Object` or the `[type]::new()` method.

For example, .NET added overloads to the `[System.String]::Split()` method that aren't available in .NET Framework 4.5. The following list shows the overloads for the `Split()` method available in Windows PowerShell 5.1:

PowerShell

```
PS> "".Split

OverloadDefinitions
-------------------
string[] Split(Params char[] separator)
```

```
string[] Split(char[] separator, int count)
string[] Split(char[] separator, System.StringSplitOptions options)
string[] Split(char[] separator, int count, System.StringSplitOptions options)
string[] Split(string[] separator, System.StringSplitOptions options)
string[] Split(string[] separator, int count, System.StringSplitOptions options)
```

The following list shows the overloads for the `Split()` method available in PowerShell 7:

PowerShell

```
"".Split

OverloadDefinitions
-------------------
string[] Split(char separator, System.StringSplitOptions options)
string[] Split(char separator, int count, System.StringSplitOptions options)
string[] Split(Params char[] separator)
string[] Split(char[] separator, int count)
string[] Split(char[] separator, System.StringSplitOptions options)
string[] Split(char[] separator, int count, System.StringSplitOptions options)
string[] Split(string separator, System.StringSplitOptions options)
string[] Split(string separator, int count, System.StringSplitOptions options)
string[] Split(string[] separator, System.StringSplitOptions options)
string[] Split(string[] separator, int count, System.StringSplitOptions options)
```

In Windows PowerShell 5.1, you could pass a character array (`char[]`) to the `Split()` method as a `string`. The method splits the target string at any occurrence of a character in the array. The following command splits the target string in Windows PowerShell 5.1, but not in PowerShell 7:

PowerShell

```
# PowerShell 7 example
"1111p2222q3333".Split('pq')
```

Output

```
1111p2222q3333
```

To bind to the correct overload, you must typecast the string to a character array:

PowerShell

```
# PowerShell 7 example
"1111p2222q3333".Split([char[]]'pq')
```

Output

```
1111
2222
3333
```

Modules no longer shipped with PowerShell

For various compatibility reasons, the following modules are no longer included in PowerShell.

- ISE
- Microsoft.PowerShell.LocalAccounts
- Microsoft.PowerShell.ODataUtils
- Microsoft.PowerShell.Operation.Validation
- PSScheduledJob
- PSWorkflow
- PSWorkflowUtility

PowerShell Workflow

PowerShell Workflow is a feature in Windows PowerShell that builds on top of Windows Workflow Foundation (WF) that enables the creation of robust runbooks for long-running or parallelized tasks.

Due to the lack of support for Windows Workflow Foundation in .NET Core, we removed PowerShell Workflow from PowerShell.

In the future, we would like to enable native parallelism/concurrency in the PowerShell language without the need for PowerShell Workflow.

If there is a need to use checkpoints to resume a script after the OS restarts, we recommend using Task Scheduler to run a script on OS startup, but the script would need to maintain its own state (like persisting it to a file).

Cmdlets removed from PowerShell

For the modules that are included in PowerShell, the following cmdlets were removed from PowerShell for various compatibility reasons or the use of unsupported APIs.

CimCmdlets

- `Export-BinaryMiLog`

Microsoft.PowerShell.Core

- `Add-PSSnapin`
- `Export-Console`
- `Get-PSSnapin`
- `Remove-PSSnapin`
- `Resume-Job`
- `Suspend-Job`

Microsoft.PowerShell.Diagnostics

- `Export-Counter`
- `Import-Counter`

Microsoft.PowerShell.Management

- `Add-Computer`
- `Checkpoint-Computer`
- `Clear-EventLog`
- `Complete-Transaction`
- `Disable-ComputerRestore`
- `Enable-ComputerRestore`
- `Get-ComputerRestorePoint`
- `Get-ControlPanelItem`
- `Get-EventLog`
- `Get-Transaction`
- `Get-WmiObject`
- `Invoke-WmiMethod`
- `Limit-EventLog`
- `New-EventLog`
- `New-WebServiceProxy`
- `Register-WmiEvent`
- `Remove-Computer`
- `Remove-EventLog`
- `Remove-WmiObject`
- `Reset-ComputerMachinePassword`
- `Restore-Computer`
- `Set-WmiInstance`
- `Show-ControlPanelItem`

- `Show-EventLog`
- `Start-Transaction`
- `Test-ComputerSecureChannel`
- `Undo-Transaction`
- `Use-Transaction`
- `Write-EventLog`

Microsoft.PowerShell.Utility

- `Convert-String`
- `ConvertFrom-String`

PSDesiredStateConfiguration

- `Disable-DscDebug`
- `Enable-DscDebug`
- `Get-DscConfiguration`
- `Get-DscConfigurationStatus`
- `Get-DscLocalConfigurationManager`
- `Publish-DscConfiguration`
- `Remove-DscConfigurationDocument`
- `Restore-DscConfiguration`
- `Set-DscLocalConfigurationManager`
- `Start-DscConfiguration`
- `Stop-DscConfiguration`
- `Test-DscConfiguration`
- `Update-DscConfiguration`

WMI v1 cmdlets

The following WMI v1 cmdlets were removed from PowerShell:

- `Register-WmiEvent`
- `Set-WmiInstance`
- `Invoke-WmiMethod`
- `Get-WmiObject`
- `Remove-WmiObject`

The CimCmdlets module (aka WMI v2) cmdlets perform the same function and provide new functionality and a redesigned syntax.

`New-WebServiceProxy` cmdlet removed

.NET Core doesn't support the Windows Communication Framework, which provides services for using the SOAP protocol. This cmdlet was removed because it requires SOAP.

`*-Transaction` cmdlets removed

These cmdlets had very limited usage. The decision was made to discontinue support for them.

- `Complete-Transaction`
- `Get-Transaction`
- `Start-Transaction`
- `Undo-Transaction`
- `Use-Transaction`

`*-EventLog` cmdlets

Due to the use of unsupported APIs, the `*-EventLog` cmdlets have been removed from PowerShell. `Get-WinEvent` and `New-WinEvent` are available to get and create events on Windows.

Cmdlets that use the Windows Presentation Framework (WPF)

.NET Core 3.1 added support for WPF, so the release of PowerShell 7.0 restored the following Windows-specific features:

- The `Show-Command` cmdlet
- The `Out-GridView` cmdlet
- The **ShowWindow** parameter of `Get-Help`

PowerShell Desired State Configuration (DSC) changes

`Invoke-DscResource` was restored as an experimental feature in PowerShell 7.0.

Beginning with PowerShell 7.2, the PSDesiredStateConfiguration module has been removed from PowerShell and has been published to the PowerShell Gallery. For more information, see the announcement in the PowerShell Team blog.

PowerShell executable changes

Renamed `powershell.exe` to `pwsh.exe`

The binary name for PowerShell has been changed from `powershell(.exe)` to `pwsh(.exe)`. This change provides a deterministic way for users to run PowerShell on machines and support side-by-side installations of Windows PowerShell and PowerShell.

Additional changes to `pwsh(.exe)` from `powershell.exe`:

- Changed the first positional parameter from `-Command` to `-File`. This change fixes the usage of `#!` (aka as a shebang) in PowerShell scripts that are being executed from non-PowerShell shells on non-Windows platforms. It also means that you can run commands like `pwsh foo.ps1` or `pwsh fooScript` without specifying `-File`. However, this change requires that you explicitly specify `-c` or `-Command` when trying to run commands like `pwsh.exe -Command Get-Command`.
- `pwsh` accepts the `-i` (or `-Interactive`) switch to indicate an interactive shell. This allows PowerShell to be used as a default shell on Unix platforms.
- Removed parameters `-ImportSystemModules` and `-PSConsoleFile` from `pwsh.exe`.
- Changed `pwsh -Version` and built-in help for `pwsh.exe` to align with other native tools.
- Invalid argument error messages for `-File` and `-Command` and exit codes consistent with Unix standards
- Added `-WindowStyle` parameter on Windows. Similarly, package-based installations updates on non-Windows platforms are in-place updates.

The shortened name is also consistent with naming of shells on non-Windows platforms.

Support running a PowerShell script with bool parameter

Previously, using `pwsh.exe` to execute a PowerShell script using `-File` provided no way to pass `$true`/`$false` as parameter values. Support for `$true`/`$false` as parsed values to parameters was added. Switch values are also supported.

Improved backwards compatibility with Windows PowerShell

For Windows, a new `[switch]` parameter **UseWindowsPowerShell** is added to `Import-Module`. This parameter creates a proxy module in PowerShell 7 that uses a local Windows PowerShell process to implicitly run any cmdlets contained in that module. For more information, see Import-Module.

For more information on which Microsoft modules work with PowerShell 7.0, see the Module Compatibility Table.

Microsoft Update support for Windows

PowerShell 7.2 added support for Microsoft Update. When you enable this feature, you'll get the latest PowerShell 7 updates in your traditional Windows Update (WU) management flow, whether that's with Windows Update for Business, WSUS, SCCM, or the interactive WU dialog in Settings.

The PowerShell 7.2 MSI package includes following command-line options:

- `USE_MU` - This property has two possible values:
  - `1` (default) - Opts into updating through Microsoft Update or WSUS
  - `0` - Don't opt into updating through Microsoft Update or WSUS
- `ENABLE_MU`
  - `1` (default) - Opts into using Microsoft Update the Automatic Updates or Windows Update
  - `0` - Don't opt into using Microsoft Update the Automatic Updates or Windows Update

Engine changes

Support PowerShell as a default Unix shell

On Unix, it's a convention for shells to accept `-i` for an interactive shell and many tools expect this behavior (`script` for example, and when setting PowerShell as the default shell) and calls the shell with the `-i` switch. This change is breaking in that `-i` previously could be used as short hand to match `-InputFormat`, which now needs to be `-in`.

Custom snap-ins

PowerShell snap-ins are a predecessor to PowerShell modules that don't have widespread adoption in the PowerShell community.

Due to the complexity of supporting snap-ins and their lack of usage in the community, we no longer support custom snap-ins in PowerShell.

Experimental feature flags

PowerShell 6.2 enabled support for Experimental Features. This allows PowerShell developers to deliver new features and get feedback before the design is complete. This way we avoid making breaking changes as the design evolves.

Use `Get-ExperimentalFeature` to get a list of available experimental features. You can enable or disable these features with `Enable-ExperimentalFeature` and `Disable-ExperimentalFeature`.

Load assembly from module base path before trying to load from the GAC

Previously, when a binary module has the module assembly in GAC, we loaded the assembly from GAC before trying to load it from module base path.

Skip null-element check for collections with a value-type element type

For the `Mandatory` parameter and `ValidateNotNull` and `ValidateNotNullOrEmpty` attributes, skip the null-element check if the collection's element type is value type.

Preserve `$?` for ParenExpression, SubExpression and ArrayExpression

This PR alters the way we compile subpipelines `(...)`, subexpressions `$(...)` and array expressions `@()` so that `$?` isn't automatically **true**. Instead the value of `$?` depends on the result of the pipeline or statements executed.

Fix `$?` to not be `$false` when native command writes to `stderr`

`$?` isn't set to `$false` when native command writes to `stderr`. It's common for native commands to write to `stderr` without intending to indicate a failure. `$?` is set to `$false` only when the native command has a non-zero exit code.

Make `$ErrorActionPreference` not affect `stderr` output of native commands

It's common for native commands to write to `stderr` without intending to indicate a failure. With this change, `stderr` output is still captured in **ErrorRecord** objects, but the runtime no longer applies `$ErrorActionPreference` if the **ErrorRecord** comes from a native command.

Change `$OutputEncoding` to use `UTF-8 NoBOM` encoding rather than ASCII

The previous encoding, ASCII (7-bit), would result in incorrect alteration of the output in some cases. Making `UTF-8 NoBOM` the default preserves Unicode output with an encoding supported by most tools and operating systems.

Unify cmdlets with parameter `-Encoding` to be of type `System.Text.Encoding`

The `-Encoding` value `Byte` has been removed from the FileSystem provider cmdlets. A new parameter, `-AsByteStream`, is now used to specify that a byte stream is required as input or that the output is a stream of bytes.

Change `New-ModuleManifest` encoding to `UTF8NoBOM` on non-Windows platforms

Previously, `New-ModuleManifest` creates `psd1` manifests in UTF-16 with BOM, creating a problem for Linux tools. This breaking change changes the encoding of `New-ModuleManifest` to be UTF (no BOM) in non-Windows platforms.

Remove `AllScope` from most default aliases

To speed up scope creation, `AllScope` was removed from most default aliases. `AllScope` was left for a few frequently used aliases where the lookup was faster.

`-Verbose` and `-Debug` no longer overrides `$ErrorActionPreference`

Previously, if `-Verbose` or `-Debug` were specified, it overrode the behavior of `$ErrorActionPreference`. With this change, `-Verbose` and `-Debug` no longer affect the behavior of `$ErrorActionPreference`.

Also, the `-Debug` parameter sets `$DebugPreference` to **Continue** instead of **Inquire**.

Make `$PSCulture` consistently reflect in-session culture changes

In Windows PowerShell, the current culture value is cached, which can allow the value to get out of sync with the culture is change after session-startup. This caching behavior is fixed in PowerShell core.

Allow explicitly specified named parameter to supersede the same one from hashtable splatting

With this change, the named parameters from splatting are moved to the end of the parameter list so that they're bound after all explicitly specified named parameters are bound. Parameter binding for simple functions doesn't throw an error when a specified named parameter can't be found. Unknown named parameters are bound to the `$args` parameter of the simple function. Moving splatting to the end of the argument list changes the order the parameters appears in `$args`.

For example:

PowerShell

```
function SimpleTest {
    param(
        $Name,
        $Path
    )
    "Name: $Name; Path: $Path; Args: $args"
}
```

In the previous behavior, **MyPath** isn't bound to `-Path` because it's the third argument in the argument list. ## So it ends up being stuffed into '$args' along with `Blah = "World"`

PowerShell

```
PS> $hash = @{ Name = "Hello"; Blah = "World" }
PS> SimpleTest @hash "MyPath"
Name: Hello; Path: ; Args: -Blah: World MyPath
```

With this change, the arguments from `@hash` are moved to the end of the argument list. **MyPath** becomes the first argument in the list, so it's bound to `-Path`.

PowerShell

```
PS> SimpleTest @hash "MyPath"
Name: Hello; Path: MyPath; Args: -Blah: World
```

Language changes

Null-coalescing operator `??`

The null-coalescing operator `??` returns the value of its left-hand operand if it isn't null. Otherwise, it evaluates the right-hand operand and returns its result. The `??` operator doesn't evaluate its right-hand operand if the left-hand operand evaluates to non-null.

PowerShell

```
$x = $null
$x ?? 100
```

Output

```
100
```

In the following example, the right-hand operand won't be evaluated.

PowerShell

```
[string] $todaysDate = '1/10/2020'
$todaysDate ?? (Get-Date).ToShortDateString()
```

Output

```
1/10/2020
```

Null-coalescing assignment operator `??=`

The null-coalescing assignment operator `??=` assigns the value of its right-hand operand to its left-hand operand only if the left-hand operand evaluates to null. The `??=` operator doesn't evaluate its right-hand operand if the left-hand operand evaluates to non-null.

PowerShell

```
$x = $null
$x ??= 100
$x
```

Output

```
100
```

In the following example, the right-hand operand won't be evaluated.

PowerShell

```powershell
[string] $todaysDate = '1/10/2020'
$todaysDate ??= (Get-Date).ToShortDateString()
```

Output

```
1/10/2020
```

Null-conditional operators

> **Note**
>
> This feature was moved from experimental to mainstream in PowerShell 7.1.

A null-conditional operator applies a member access, `?.`, or element access, `?[]`, operation to its operand only if that operand evaluates to non-null; otherwise, it returns null.

Since PowerShell allows `?` to be part of the variable name, formal specification of the variable name is required for using these operators. So it's required to use `{}` around the variable names like `${a}` or when `?` is part of the variable name `${a?}`.

In the following example, the value of **PropName** is returned.

PowerShell

```powershell
$a = @{ PropName = 100 }
${a}?.PropName
```

Output

```
100
```

The following example will return null, without trying to access the member name **PropName**.

PowerShell

```
$a = $null
${a}?.PropName
```

Similarly, the value of the element will be returned.

PowerShell

```
$a = 1..10
${a}?[0]
```

Output

```
1
```

And when the operand is null, the element isn't accessed and null is returned.

PowerShell

```
$a = $null
${a}?[0]
```

> ⓘ Note
>
> The variable name syntax of `${<name>}` shouldn't be confused with the `$()` subexpression operator. For more information, see Variable name section of about_Variables.

Added `&` operator for job control

Putting `&` at the end of a pipeline causes the pipeline to be run as a PowerShell job. When a pipeline is backgrounded, a job object is returned. Once the pipeline is running as a job, all of the standard `*-Job` cmdlets can be used to manage the job. Variables (ignoring process-specific variables) used in the pipeline are automatically copied to the job so `Copy-Item $foo $bar &` just works. The job is also run in the current directory instead of the user's home directory.

New methods/properties on `PSCustomObject`

We've added new methods and properties to `PSCustomObject`. `PSCustomObject` now includes a `Count`/`Length` property like other objects.

PowerShell

```
$PSCustomObject = [pscustomobject]@{foo = 1}

$PSCustomObject.Length
```

Output

```
1
```

PowerShell

```
$PSCustomObject.Count
```

Output

```
1
```

This work also includes `ForEach` and `Where` methods that allow you to operate and filter on `PSCustomObject` items:

PowerShell

```
$PSCustomObject.ForEach({$_.foo + 1})
```

Output

```
2
```

PowerShell

```
$PSCustomObject.Where({$_.foo -gt 0})
```

Output

```
foo
---
  1
```

Conversions from PSMethod to Delegate

You can convert a `PSMethod` into a delegate. This allows you to do things like passing `PSMethod` `[M]::DoubleStrLen` as a delegate value into `[M]::AggregateString`:

PowerShell

```
class M {
    static [int] DoubleStrLen([string] $value) { return 2 * $value.Length }

    static [long] AggregateString([string[]] $values, [Func[string, int]]
$selector) {
        [long] $res = 0
        foreach($s in $values){
            $res += $selector.Invoke($s)
        }
        return $res
    }
}

[M]::AggregateString((gci).Name, [M]::DoubleStrLen)
```

String comparison behavior changed in PowerShell 7.1

PowerShell 7.1 is built on .NET 5.0, which introduced the following breaking change:

- Behavior changes when comparing strings on .NET 5+

As of .NET 5.0, culture invariant string comparisons ignore non-printing control characters.

For example, the following two strings are considered to be identical:

PowerShell

```
# Escape sequence "`a" is Ctrl-G or [char]7
'Food' -eq "Foo`ad"
```

Output

```
True
```

New cmdlets

New `Get-Uptime` cmdlet

The Get-Uptime cmdlet returns the time elapsed since the last boot of the operating system. The cmdlet was introduced in PowerShell 6.0.

New `Remove-Alias` cmdlet

The Remove-Alias cmdlet removes an alias from the current PowerShell session. The cmdlet was introduced in PowerShell 6.0.

New `Remove-Service` cmdlet

The Remove-Service cmdlet removes a Windows service in the registry and in the service database. The `Remove-Service` cmdlet was introduced in PowerShell 6.0.

New Markdown cmdlets

Markdown is a standard for creating readable plaintext documents with basic formatting that can be rendered into HTML.

The following cmdlets were added in PowerShell 6.1:

- ConvertFrom-Markdown - Convert the contents of a string or a file to a **MarkdownInfo** object.
- Get-MarkdownOption - Returns the current colors and styles used for rendering Markdown content in the console.
- Set-MarkdownOption - Sets the colors and styles used for rendering Markdown content in the console.
- Show-Markdown - Displays Markdown content in the console or as HTML

New `Test-Json` cmdlet

The Test-Json cmdlet tests whether a string is a valid JavaScript Object Notation (JSON) document and can optionally verify that JSON document against a provided schema.

This cmdlet was introduced in PowerShell 6.1

New cmdlets to support Experimental Features

The following cmdlets were added in PowerShell 6.2 to support Experimental Features.

- Disable-ExperimentalFeature
- Enable-ExperimentalFeature
- Get-ExperimentalFeature

New `Join-String` cmdlet

The Join-String cmdlet combines objects from the pipeline into a single string. This cmdlet was added in PowerShell 6.2.

New view ConciseView and cmdlet `Get-Error`

PowerShell 7.0 enhances the display of error messages to improve the readability of interactive and script errors with a new default view, **ConciseView**. The views are user-selectable through the preference variable `$ErrorView`.

With **ConciseView**, if an error isn't from a script or parser error, then it's a single line error message:

PowerShell

```powershell
Get-ChildItem -Path C:\NotReal
```

Output

```
Get-ChildItem: Can't find path 'C:\NotReal' because it doesn't exist
```

If the error occurs during script execution or is a parsing error, PowerShell returns a multiline error message that contains the error, a pointer, and an error message showing where the error is in that line. If the terminal doesn't support ANSI color escape sequences (VT100), then colors aren't displayed.

The default view in PowerShell 7 is **ConciseView**. The previous default view was **NormalView** and you can select this by setting the preference variable `$ErrorView`.

PowerShell

```powershell
$ErrorView = 'NormalView' # Sets the error view to NormalView
$ErrorView = 'ConciseView' # Sets the error view to ConciseView
```

> ⓘ **Note**
>
> A new property **ErrorAccentColor** is added to `$Host.PrivateData` to support changing the accent color of the error message.

The new `Get-Error` cmdlet provides a complete detailed view of the fully qualified error when desired. By default the cmdlet displays the full details, including inner exceptions, of the last

error that occurred.

The `Get-Error` cmdlet supports input from the pipeline using the built-in variable `$Error`. `Get-Error` displays all piped errors.

PowerShell

```
$Error | Get-Error
```

The `Get-Error` cmdlet supports the **Newest** parameter, allowing you to specify how many errors from the current session you wish displayed.

PowerShell

```
Get-Error -Newest 3 # Displays the lst three errors that occurred in the session
```

For more information, see Get-Error.

Cmdlet changes

Parallel execution added to `ForEach-Object`

Beginning in PowerShell 7.0, the `ForEach-Object` cmdlet, which iterates items in a collection, now has built-in parallelism with the new **Parallel** parameter.

By default, parallel script blocks use the current working directory of the caller that started the parallel tasks.

This example retrieves 50,000 log entries from 5 system logs on a local Windows machine:

PowerShell

```
$logNames = 'Security','Application','System','Windows PowerShell','Microsoft-
Windows-Store/Operational'

$logEntries = $logNames | ForEach-Object -Parallel {
    Get-WinEvent -LogName $_ -MaxEvents 10000
} -ThrottleLimit 5

$logEntries.Count

50000
```

The **Parallel** parameter specifies the script block that's run in parallel for each input log name.

The new **ThrottleLimit** parameter limits the number of script blocks running in parallel at a given time. The default is 5.

Use the `$_` variable to represent the current input object in the script block. Use the `Using:` scope modifier to pass variable references to the running script block.

For more information, see ForEach-Object.

Check `system32` for compatible built-in modules on Windows

In the Windows 10 1809 update and Windows Server 2019, we updated a number of built-in PowerShell modules to mark them as compatible with PowerShell.

When PowerShell starts up, it automatically includes `$windir\System32` as part of the `PSModulePath` environment variable. However, it only exposes modules to `Get-Module` and `Import-Module` if its `CompatiblePSEdition` is marked as compatible with `Core`.

You can override this behavior to show all modules using the `-SkipEditionCheck` `[switch]` parameter. We've also added a `PSEdition` property to the table output.

`-lp` alias for all `-LiteralPath` parameters

We created a standard parameter alias `-lp` for all the built-in PowerShell cmdlets that have a `-LiteralPath` parameter.

Fix `Get-Item -LiteralPath a*b` if `a*b` doesn't actually exist to return error

Previously, `-LiteralPath` given a wildcard would treat it the same as `-Path` and if the wildcard found no files, it would silently exit. Correct behavior should be that `-LiteralPath` is literal so if the file doesn't exist, it should error. Change is to treat wildcards used with `-Literal` as literal.

Set working directory to current directory in `Start-Job`

The `Start-Job` cmdlet now uses the current directory as the working directory for the new job.

Remove `-Protocol` from `*-Computer` cmdlets

The `-Protocol` parameter was removed from following cmdlets:

- `Rename-Computer`

- `Restart-Computer`
- `Stop-Computer`

DCOM is no longer supported for remoting. The cmdlets only support WSMAN remoting.

Remove `-ComputerName` from `*-Service` cmdlets

In order to encourage the consistent use of PSRP, the `-ComputerName` parameter was removed from `*-Service` cmdlets. Use `Invoke-Command` to run the cmdlets on remote computers instead.

Fix `Get-Content -Delimiter` to not include the delimiter in the returned lines

Previously, the output while using `Get-Content -Delimiter` was inconsistent and inconvenient as it required further processing of the data to remove the delimiter. This change removes the delimiter in returned lines.

Changes to `Format-Hex`

The `-Raw` parameter now does nothing. The `Format-Hex` cmdlet displays a true representation of numbers that includes all the bytes for its type. This is what the `-Raw` parameter did prior to this change.

Typo fix in `Get-ComputerInfo` property name

`BiosSerialNumber` was misspelled as `BiosSeralNumber` and has been changed to the correct spelling.

Changes to available hash algorithms

The following hash algorithms have been removed from .NET:

- `MACTripleDES`
- `RIPEMD160`

This change affects the `Get-FileHash` cmdlet.

Add validation on `Get-*` cmdlets where passing $null returns all objects instead of error

Passing `$null` to any of the following now throws an error:

- `Get-Credential -UserName`
- `Get-Event -SourceIdentifier`
- `Get-EventSubscriber -SourceIdentifier`
- `Get-Help -Name`
- `Get-PSBreakpoint -Script`
- `Get-PSProvider -PSProvider`
- `Get-PSSessionConfiguration -Name`
- `Get-Runspace -Name`
- `Get-RunspaceDebug -RunspaceName`
- `Get-Service -Name`
- `Get-TraceSource -Name`
- `Get-Variable -Name`

Add support for the W3C Extended Log File Format in `Import-Csv`

Previously, the `Import-Csv` cmdlet can't be used to directly import the log files in W3C extended log format and additional action would be required. With this change, W3C extended log format is supported.

`Import-Csv` applies `pstypenames` upon import when type information is present in the CSV

Previously, objects exported using `Export-Csv` with `TypeInformation` imported with `ConvertFrom-Csv` weren't retaining the type information. This change adds the type information to `pstypenames` member if available from the CSV file.

`-NoTypeInformation` is the default on `Export-Csv`

Previously, the `Export-Csv` cmdlet would output a comment as the first line containing the type name of the object. The change excludes the type information by default because it's not understood by most CSV tools. This change was made to address customer feedback.

Use `-IncludeTypeInformation` to retain the previous behavior.

Allow `*` to be used in registry path for `Remove-Item`

Previously, `-LiteralPath` given a wildcard would treat it the same as `-Path` and if the wildcard found no files, it would silently exit. Correct behavior should be that `-LiteralPath` is literal so if

the file doesn't exist, it should error. Change is to treat wildcards used with `-Literal` as literal.

`Group-Object` now sorts the groups

As part of the performance improvement, `Group-Object` now returns a sorted listing of the groups. Although you shouldn't rely on the order, you could be broken by this change if you wanted the first group. We decided that this performance improvement was worth the change since the impact of being dependent on previous behavior is low.

Standard deviation in `Measure-Object`

The output from `Measure-Object` now includes a `StandardDeviation` property.

PowerShell

```
Get-Process | Measure-Object -Property CPU -AllStats
```

Output

```
Count             : 308
Average           : 31.3720576298701
Sum               : 9662.59375
Maximum           : 4416.046875
Minimum           :
StandardDeviation : 264.389544720926
Property          : CPU
```

`Get-PfxCertificate -Password`

`Get-PfxCertificate` now has the `Password` parameter, which takes a `SecureString`. This allows you to use it non-interactively:

PowerShell

```
$certFile = '\\server\share\pwd-protected.pfx'
$certPass = Read-Host -AsSecureString -Prompt 'Enter the password for certificate:
'

$certThumbPrint = (Get-PfxCertificate -FilePath $certFile -Password $certPass
).ThumbPrint
```

Removal of the `more` function

In the past, PowerShell shipped a function on Windows called `more` that wrapped `more.com`. That function has now been removed.

Also, the `help` function changed to use `more.com` on Windows, or the system's default pager specified by `$Env:PAGER` on non-Windows platforms.

`cd DriveName:` now returns users to the current working directory in that drive

Previously, using `Set-Location` or `cd` to return to a PSDrive sent users to the default location for that drive. Users are now sent to the last known current working directory for that session.

`cd -` returns to previous directory

PowerShell

```
C:\Windows\System32> cd C:\
C:\> cd -
C:\Windows\System32>
```

Or on Linux:

ShellSession

```
PS /etc> cd /usr/bin
PS /usr/bin> cd -
PS /etc>
```

Also, `cd` and `cd --` change to `$HOME`.

`Update-Help` as non-admin

By popular demand, `Update-Help` no longer needs to be run as an administrator. `Update-Help` now defaults to saving help to a user-scoped folder.

`Where-Object -Not`

With the addition of `-Not` parameter to `Where-Object`, can filter an object at the pipeline for the non-existence of a property, or a null/empty property value.

For example, this command returns all services that don't have any dependent services defined:

```
```

PowerShell

```
Get-Service | Where-Object -Not DependentServices
```

Changes to Web Cmdlets

The underlying .NET API of the Web Cmdlets has been changed to `System.Net.Http.HttpClient`. This change provides many benefits. However, this change along with a lack of interoperability with Internet Explorer have resulted in several breaking changes within `Invoke-WebRequest` and `Invoke-RestMethod`.

- `Invoke-WebRequest` now supports basic HTML Parsing only. `Invoke-WebRequest` always returns a `BasicHtmlWebResponseObject` object. The `ParsedHtml` and `Forms` properties have been removed.
- `BasicHtmlWebResponseObject.Headers` values are now `String[]` instead of `String`.
- `BasicHtmlWebResponseObject.BaseResponse` is now a `System.Net.Http.HttpResponseMessage` object.
- The `Response` property on Web Cmdlet exceptions is now a `System.Net.Http.HttpResponseMessage` object.
- Strict RFC header parsing is now default for the `-Headers` and `-UserAgent` parameter. This can be bypassed with `-SkipHeaderValidation`.
- `file://` and `ftp://` URI schemes are no longer supported.
- `System.Net.ServicePointManager` settings are no longer honored.
- There is currently no certificate based authentication available on macOS.
- Use of `-Credential` over an `http://` URI will result in an error. Use an `https://` URI or supply the `-AllowUnencryptedAuthentication` parameter to suppress the error.
- `-MaximumRedirection` now produces a terminating error when redirection attempts exceed the provided limit instead of returning the results of the last redirection.
- In PowerShell 6.2, a change was made to default to UTF-8 encoding for JSON responses. When a charset isn't supplied for a JSON response, the default encoding should be UTF-8 per RFC 8259.
- Default encoding set to UTF-8 for `application-json` responses
- Added `-SkipHeaderValidation` parameter to allow `Content-Type` headers that aren't standards-compliant
- Added `-Form` parameter to support simplified `multipart/form-data` support
- Compliant, case-insensitive handling of relation keys
- Added `-Resume` parameter for web cmdlets

`Invoke-RestMethod` returns useful info when no data is returned

When an API returns just `null`, `Invoke-RestMethod` was serializing this as the string `"null"` instead of `$null`. This change fixes the logic in `Invoke-RestMethod` to properly serialize a valid single value JSON `null` literal as `$null`.

Web Cmdlets warn when `-Credential` is sent over unencrypted connections

When using HTTP, content including passwords are sent as clear-text. This change is to not allow this by default and return an error if credentials are being passed insecurely. Users can bypass this by using the `-AllowUnencryptedAuthentication` switch.

Make `-OutFile` parameter in web cmdlets to work like `-LiteralPath`

Beginning in PowerShell 7.1, the **OutFile** parameter of the web cmdlets works like **LiteralPath** and doesn't process wildcards.

API changes

Remove `AddTypeCommandBase` class

The `AddTypeCommandBase` class was removed from `Add-Type` to improve performance. This class is only used by the `Add-Type` cmdlet and shouldn't impact users.

Removed `VisualBasic` as a supported language in Add-Type

In the past, you could compile Visual Basic code using the `Add-Type` cmdlet. Visual Basic was rarely used with `Add-Type`. We removed this feature to reduce the size of PowerShell.

Removed `RunspaceConfiguration` support

Previously, when creating a PowerShell runspace programmatically using the API, you could use the legacy RunspaceConfiguration or the newer InitialSessionState classes. This change removed support for `RunspaceConfiguration` and only supports `InitialSessionState`.

`CommandInvocationIntrinsics.InvokeScript` bind arguments to `$input` instead of `$args`

An incorrect position of a parameter resulted in the args passed as input instead of as args.

Remove `ClrVersion` and `BuildVersion` properties from `$PSVersionTable`

The `ClrVersion` property of `$PSVersionTable` isn't useful with CoreCLR. End users shouldn't be using that value to determine compatibility.

The `BuildVersion` property was tied to the Windows build version, which isn't available on non-Windows platforms. Use the `GitCommitId` property to retrieve the exact build version of PowerShell.

Implement Unicode escape parsing

`` `u#### `` or `` `u{####} `` is converted to the corresponding Unicode character. To output a literal `` `u ``, escape the backtick: `` ``u ``.

Parameter binding problem with `ValueFromRemainingArguments` in PS functions

`ValueFromRemainingArguments` now returns the values as an array instead of a single value which itself is an array.

Cleaned up uses of `CommandTypes.Workflow` and `WorkflowInfoCleaned`

Clean up code related to the uses of `CommandTypes.Workflow` and `WorkflowInfo` in **System.Management.Automation**.

These minor breaking changes mainly affect help provider code.

- Change the public constructors of `WorkflowInfo` to internal. We don't support workflow anymore, so it makes sense to not allow people to create `Workflow` instances.
- Remove the type **System.Management.Automation.DebugSource** since it's only used for workflow debugging.
- Remove the overload of `SetParent` from the abstract class **Debugger** that's only used for workflow debugging.

- Remove the same overload of `SetParent` from the derived class **RemotingJobDebugger**.

Don't wrap return result in `PSObject` when converting a `ScriptBlock` to a delegate

When a `ScriptBlock` is converted to a delegate type to be used in C# context, wrapping the result in a `PSObject` brings unneeded troubles:

- When the value is converted to the delegate return type, the `PSObject` essentially gets unwrapped. So the `PSObject` is unneeded.
- When the delegate return type is `object`, it gets wrapped in a `PSObject` making it hard to work with in C# code.

After this change, the returned object is the underlying object.

Remoting Support

PowerShell Remoting (PSRP) using WinRM isn't supported for non-Windows platforms. You can use PowerShell Remoting (PSRP) over WinRM from Windows to connect to other Windows machines. PowerShell also supports remoting over SSH on all platforms (Windows, macOS, and Linux). For more information, see SSH remoting in PowerShell.

PowerShell Direct for Containers tries to use `pwsh` first

PowerShell Direct is a feature of PowerShell and Hyper-V that allows you to connect to a Hyper-V VM or Container without network connectivity or other remote management services.

In the past, PowerShell Direct connected using the built-in Windows PowerShell instance on the Container. Now, PowerShell Direct first attempts to connect using any available `pwsh.exe` on the `PATH` environment variable. If `pwsh.exe` isn't available, PowerShell Direct falls back to use `powershell.exe`.

`Enable-PSRemoting` now creates separate remoting endpoints for preview versions

`Enable-PSRemoting` now creates two remoting session configurations:

- One for the major version of PowerShell. For example, `PowerShell.6`. This endpoint that can be relied upon across minor version updates as the "system-wide" PowerShell 6 session configuration

- One version-specific session configuration, for example: `PowerShell.6.1.0`

This behavior is useful if you want to have multiple PowerShell 6 versions installed and accessible on the same machine.

Additionally, preview versions of PowerShell now get their own remoting session configurations after running the `Enable-PSRemoting` cmdlet:

PowerShell

```
C:\WINDOWS\system32> Enable-PSRemoting
```

Your output may be different if you haven't set up WinRM before.

Output

```
WinRM is already set up to receive requests on this computer.
WinRM is already set up for remote management on this computer.
```

Then you can see separate PowerShell session configurations for the preview and stable builds of PowerShell 6, and for each specific version.

PowerShell

```
Get-PSSessionConfiguration
```

Output

```
Name          : PowerShell.6.2-preview.1
PSVersion     : 6.2
StartupScript :
RunAsUser     :
Permission    : NT AUTHORITY\INTERACTIVE AccessAllowed, BUILTIN\Administrators
AccessAllowed, BUILTIN\Remote Management Users AccessAllowed

Name          : PowerShell.6-preview
PSVersion     : 6.2
StartupScript :
RunAsUser     :
Permission    : NT AUTHORITY\INTERACTIVE AccessAllowed, BUILTIN\Administrators
AccessAllowed, BUILTIN\Remote Management Users AccessAllowed

Name          : powershell.6
PSVersion     : 6.1
StartupScript :
RunAsUser     :
Permission    : NT AUTHORITY\INTERACTIVE AccessAllowed, BUILTIN\Administrators
```

```
AccessAllowed, BUILTIN\Remote Management Users AccessAllowed

Name          : powershell.6.1.0
PSVersion     : 6.1
StartupScript :
RunAsUser     :
Permission    : NT AUTHORITY\INTERACTIVE AccessAllowed, BUILTIN\Administrators
AccessAllowed, BUILTIN\Remote Management Users AccessAllowed
```

`user@host:port` syntax supported for SSH

SSH clients typically support a connection string in the format `user@host:port`. With the addition of SSH as a protocol for PowerShell Remoting, we've added support for this format of connection string:

```
Enter-PSSession -HostName fooUser@ssh.contoso.com:2222
```

Telemetry can only be disabled with an environment variable

PowerShell sends basic telemetry data to Microsoft when it's launched. The data includes the OS name, OS version, and PowerShell version. This data allows us to better understand the environments where PowerShell is used and enables us to prioritize new features and fixes.

To opt-out of this telemetry, set the environment variable `POWERSHELL_TELEMETRY_OPTOUT` to `true`, `yes`, or `1`. We no longer support deletion of the file `DELETE_ME_TO_DISABLE_CONSOLEHOST_TELEMETRY` to disable telemetry.

Last updated on 04/08/2026

PowerShell differences on non-Windows platforms

PowerShell strives to provide feature parity across all supported platforms. However, some features behave differently or aren't available due to differences in .NET Core and platform-specific differences. Other changes were made to improve the interoperability of PowerShell on non-Windows platforms.

.NET Framework vs .NET Core

PowerShell on Linux and macOS uses .NET Core, a subset of the full .NET Framework on Microsoft Windows. As a result, scripts that run on Windows might not run on non-Windows platforms because of the differences in the frameworks.

For more information about changes in .NET Core, see Breaking changes for migration from .NET Framework to .NET Core.

General Unix interoperability changes

- Added support for native command globbing on Unix platforms. This means you can use wildcards with native commands like `ls *.txt`.
- The `more` functionality respects the Linux `$PAGER` and defaults to `less`.
- Trailing backslash is automatically escaped when dealing with native command arguments.
- Fixed ConsoleHost to honor `NoEcho` on Unix platforms.
- Don't add `PATHEXT` environment variable on Unix.
- A `powershell` man-page is included in the package.

Execution policy

PowerShell ignores execution policies when running on non-Windows platforms. `Get-ExecutionPolicy` returns **Unrestricted** on Linux and macOS. `Set-ExecutionPolicy` does nothing on Linux and macOS.

Case-sensitivity in PowerShell

Historically, PowerShell has been uniformly case-insensitive, with few exceptions. On Unix-like operating systems, the file system is predominantly case-sensitive, and PowerShell adheres to the standard of the file system.

- You must use the correct case when a filename in specified in PowerShell.
- If a script tries to load a module and the module name isn't cased correctly, then the module load fails. This behavior might cause a problem with existing scripts if the name referenced by the module doesn't match the proper case of the actual filename.
- While names in the filesystem are case-sensitive, tab-completion of filenames isn't case-sensitive. Tab-completion cycles through the list of names using case-insensitive matching.
- `Get-Help` supports case-insensitive pattern matching on Unix platforms.
- `Import-Module` is case insensitive when used with a filename to determine the module name.

Filesystem support for Linux and macOS

- Paths given to cmdlets are now slash-agnostic (both `/` and `\` work as directory separators)
- XDG Base Directory Specification is now respected and used by default:
  - The Linux/macOS profile path is located at `~/.config/powershell/profile.ps1`
  - The history save path is located at `~/.local/share/powershell/PSReadLine/ConsoleHost_history.txt`
  - The user module path is located at `~/.local/share/powershell/Modules`
- Support for file and folder names containing the colon character on Unix.
- Support for script names or full paths that have commas.
- Detect when the **LiteralPath** parameter is used to suppress wildcard expansion for navigation cmdlets.
- Updated `Get-ChildItem` to work more like the *nix `ls -R` and the Windows `DIR /S` native commands. `Get-ChildItem` now returns the symbolic links encountered during a recursive search and doesn't search the directories that those links target.

.PS1 File Extensions

PowerShell scripts must end in `.ps1` for the interpreter to understand how to load and run them in the current process. Running scripts in the current process is the expected usual behavior for PowerShell. You can add the `#!` magic number to a script that doesn't have a `.ps1` extension, but this causes the script to be run in a new PowerShell instance, preventing

the script from working correctly when interchanging objects. This behavior might be desirable when executing a PowerShell script from Bash or another shell.

Convenience aliases removed

PowerShell provides a set of aliases on Windows that map to Linux command names for user convenience. On Linux and macOS, the "convenience aliases" for the basic commands `ls`, `cp`, `mv`, `rm`, `cat`, `man`, `mount`, and `ps` were removed to allow the native executable to run without specifying a path.

Logging

On macOS, PowerShell uses the native `os_log` APIs to log to Apple's unified logging system. On Linux, PowerShell uses Syslog, a ubiquitous logging solution.

Job Control

There's no Unix-style job-control support in PowerShell on Linux or macOS. The `fg` and `bg` commands aren't available. However, you can use PowerShell jobs that work on all platforms.

Putting `&` at the end of a pipeline causes the pipeline to be run as a PowerShell job. When a pipeline is backgrounded, a job object is returned. Once the pipeline is running as a job, all `*-Job` cmdlets can be used to manage the job. Variables (ignoring process-specific variables) used in the pipeline are automatically copied to the job so `Copy-Item $foo $bar &` just works. The job runs in the current directory instead of the user's home directory.

Remoting Support

PowerShell Remoting (PSRP) using WinRM on Unix platforms requires NTLM/Negotiate or Basic Auth over HTTPS. PSRP on macOS only supports Basic Auth over HTTPS. Kerberos-based authentication isn't supported.

PowerShell supports PowerShell Remoting (PSRP) over SSH on all platforms (Windows, Linux, and macOS). For more information, see SSH remoting in PowerShell.

Just-Enough-Administration (JEA) Support

PowerShell on Linux or macOS doesn't allow you to create constrained administration (JEA) remoting endpoints.

`sudo`, `exec`, and PowerShell

Because PowerShell runs most commands in memory (like Python or Ruby), you can't use `sudo` directly with PowerShell built-ins. You can run `pwsh` from `sudo`. If it's necessary to run a PowerShell cmdlet from within PowerShell with `sudo`, for example, `sudo Set-Date 8/18/2016`, then you would use `sudo pwsh Set-Date 8/18/2016`.

Modules included on non-Windows platforms

For non-Windows platforms, PowerShell includes the following modules:

- Microsoft.PowerShell.Archive
- Microsoft.PowerShell.Core
- Microsoft.PowerShell.Host
- Microsoft.PowerShell.Management
- Microsoft.PowerShell.Security
- Microsoft.PowerShell.Utility
- PackageManagement
- PowerShellGet
- PSReadLine
- ThreadJob

A large number of the commands (cmdlets) commonly available in PowerShell aren't available on Linux or macOS. Often, these commands don't apply to these platforms. For example, commands for Windows-specific features like the registry or services aren't available. Other commands, like `Set-ExecutionPolicy`, are present but not functional.

For a comprehensive list of modules and cmdlets and the platforms they support, see Release history of modules and cmdlets.

Modules no longer shipped with PowerShell

For various compatibility reasons, the following modules are no longer included in PowerShell.

- ISE
- Microsoft.PowerShell.LocalAccounts
- Microsoft.PowerShell.ODataUtils
- Microsoft.PowerShell.Operation.Validation
- PSScheduledJob

- PSWorkflow
- PSWorkflowUtility

The following Windows-specific modules aren't included in PowerShell for Linux or macOS.

- CimCmdlets
- Microsoft.PowerShell.Diagnostics
- Microsoft.WSMan.Management
- PSDiagnostics

Cmdlets not available on non-Windows platforms

Some cmdlets were removed from PowerShell. Others aren't available or might work differently on non-Windows platforms. For a comprehensive list of cmdlets removed from PowerShell, see Cmdlets removed from PowerShell.

Microsoft.PowerShell.Core

The following cmdlets aren't available on Linux or macOS:

- `Disable-PSRemoting`
- `Enable-PSRemoting`
- `Connect-PSSession`
- `Disconnect-PSSession`
- `Receive-PSSession`
- `Get-PSSessionCapability`
- `Disable-PSSessionConfiguration`
- `Enable-PSSessionConfiguration`
- `Get-PSSessionConfiguration`
- `Register-PSSessionConfiguration`
- `Set-PSSessionConfiguration`
- `Unregister-PSSessionConfiguration`
- `Test-PSSessionConfigurationFile`

The **ShowWindow** parameter of `Get-Help` isn't available for non-Windows platforms. PowerShell 7.3 added the `Switch-Process` cmdlet and the `exec` function for Linux and macOS. These commands aren't available on Windows.

Microsoft.PowerShell.Security cmdlets

The following cmdlets aren't available on Linux or macOS:

- `Get-Acl`
- `Set-Acl`
- `Get-AuthenticodeSignature`
- `Set-AuthenticodeSignature`
- `New-FileCatalog`
- `Test-FileCatalog`

These cmdlets are only available beginning in PowerShell 7.1.

- `Get-CmsMessage`
- `Protect-CmsMessage`
- `Unprotect-CmsMessage`

Microsoft.PowerShell.Management cmdlets

The following cmdlets aren't available on Linux and macOS:

- `Rename-Computer`
- `Get-ComputerInfo`
- `Get-HotFix`
- `Clear-RecycleBin`
- `Get-Service`
- `New-Service`
- `Remove-Service`
- `Restart-Service`
- `Resume-Service`
- `Set-Service`
- `Start-Service`
- `Stop-Service`
- `Suspend-Service`
- `Set-TimeZone`

The following cmdlets are available with limitations:

- `Get-Clipboard` - available in PowerShell 7.0+
- `Set-Clipboard` - available in PowerShell 7.0+
- `Restart-Computer` - available for Linux and macOS in PowerShell 7.1+
- `Stop-Computer` - available for Linux and macOS in PowerShell 7.1+

Microsoft.PowerShell.Utility cmdlets

The following cmdlets aren't available on Linux and macOS:

- `Convert-String`
- `ConvertFrom-String`
- `ConvertFrom-SddlString`
- `Out-GridView`
- `Out-Printer`
- `Show-Command`

Aliases not available on Linux or macOS

The following table lists the aliases available for Windows that aren't available on non-Windows platforms. These aliases aren't available because the alias conflicts with a native command on those platforms.

Expand table

Alias	Cmdlet
`ac`	`Add-Content`
`cat`	`Get-Content`
`clear`	`Clear-Host`
`compare`	`Compare-Object`
`cp`	`Copy-Item`
`cpp`	`Copy-ItemProperty`
`diff`	`Compare-Object`
`kill`	`Stop-Process`
`ls`	`Get-ChildItem`
`man`	`help`
`mount`	`New-PSDrive`
`mv`	`Move-Item`
`ps`	`Get-Process`

Alias	Cmdlet
`rm`	`Remove-Item`
`rmdir`	`Remove-Item`
`sleep`	`Start-Sleep`
`sort`	`Sort-Object`
`start`	`Start-Process`
`tee`	`Tee-Object`
`write`	`Write-Output`

The table doesn't include aliases unavailable for cmdlets that don't exist on non-Windows platforms.

PowerShell Desired State Configuration (DSC)

Beginning with PowerShell 7.2, the **PSDesiredStateConfiguration** module was removed from PowerShell and is published in the PowerShell Gallery. For more information, see the announcement on the PowerShell Team blog. For more information about using DSC on Linux, see Get started with DSC for Linux. DSC v1.1 and v2.x aren't supported on macOS. DSC v3 is supported on Windows, Linux, and macOS, but it's still in early development.

Last updated on 03/24/2025

Release history of modules and cmdlets

This article lists the modules and cmdlets that are included in various versions of PowerShell. This is a summary of information found in the release notes. More detailed information can be found in the release notes:

- What's new in PowerShell 7.5
- What's new in PowerShell 7.4
- What's new in PowerShell 7.3
- What's new in PowerShell 7.2
- What's new in PowerShell 7.1
- What's new in PowerShell 7.0

This is a work in progress. Please help us keep this information fresh.

Module release history

Expand table

ModuleName / PSVersion	5.1	7.4 and higher	Note
CimCmdlets	✅	✅	Windows only
ISE (introduced in 2.0)	✅		Windows only
Microsoft.PowerShell.Archive	✅	✅	
Microsoft.PowerShell.Core	✅	✅	
Microsoft.PowerShell.Diagnostics	✅	✅	Windows only
Microsoft.PowerShell.Host	✅	✅	
Microsoft.PowerShell.LocalAccounts	✅		Windows only (64-bit only)
Microsoft.PowerShell.Management	✅	✅	
Microsoft.PowerShell.ODataUtils	✅		Windows only
Microsoft.PowerShell.Operation.Validation	✅		Windows only
Microsoft.PowerShell.PSResourceGet		✅	New versions available from the Gallery
Microsoft.PowerShell.Security	✅	✅	

ModuleName / PSVersion	5.1	7.4 and higher	Note
Microsoft.PowerShell.ThreadJob			Installable in PowerShell 5.1 - Replaced ThreadJob in PowerShell 7.6
Microsoft.PowerShell.Utility	✅	✅	
Microsoft.WsMan.Management	✅	✅	Windows only
PackageManagement	✅	✅	
PowerShellGet 1.1	✅		Must upgrade to v2.x
PowerShellGet 2.x		✅	New versions available from the Gallery
PSDesiredStateConfiguration 1.1	✅		Removed in 7.2 - available from the Gallery
PSDesiredStateConfiguration 2.x			Removed in 7.2 - available from the Gallery
PSDesiredStateConfiguration 3.x			Preview available from the Gallery
PSDiagnostics	✅	✅	Windows only
PSReadLine	✅	✅	New versions available from the Gallery
PSScheduledJob	✅		Windows only
PSWorkflow	✅		Windows only
PSWorkflowUtility	✅		Windows only
ThreadJob		✅	Replaced by Microsoft.PowerShell.ThreadJob in PowerShell 7.6

Cmdlet release history

CimCmdlets

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Export-BinaryMiLog	✅		Windows only
Get-CimAssociatedInstance	✅	✅	Windows only
Get-CimClass	✅	✅	Windows only
Get-CimInstance	✅	✅	Windows only

Cmdlet name	5.1	7.4 and higher	Note
Get-CimSession	✅	✅	Windows only
Import-BinaryMiLog	✅		Windows only
Invoke-CimMethod	✅	✅	Windows only
New-CimInstance	✅	✅	Windows only
New-CimSession	✅	✅	Windows only
New-CimSessionOption	✅	✅	Windows only
Register-CimIndicationEvent	✅	✅	Windows only
Remove-CimInstance	✅	✅	Windows only
Remove-CimSession	✅	✅	Windows only
Set-CimInstance	✅	✅	Windows only

ISE (introduced in 2.0)

This modules is only available in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
Get-IseSnippet	✅	
Import-IseSnippet	✅	
New-IseSnippet	✅	

Microsoft.PowerShell.Archive

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Compress-Archive	✅	✅	
Expand-Archive	✅	✅	

Microsoft.PowerShell.Core

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Add-History	✓	✓	
Add-PSSnapin	✓		Windows only
Clear-History	✓	✓	
Clear-Host	✓	✓	
Connect-PSSession	✓	✓	Windows only
Debug-Job	✓	✓	
Disable-ExperimentalFeature		✓	Added in 6.2
Disable-PSRemoting	✓	✓	Windows only
Disable-PSSessionConfiguration	✓	✓	Windows only
Disconnect-PSSession	✓	✓	Windows only
Enable-ExperimentalFeature		✓	Added in 6.2
Enable-PSRemoting	✓	✓	Windows only
Enable-PSSessionConfiguration	✓	✓	Windows only
Enter-PSHostProcess	✓	✓	Added Linux support in 6.2
Enter-PSSession	✓	✓	
Exit-PSHostProcess	✓	✓	Added Linux support in 6.2
Exit-PSSession	✓	✓	
Export-Console	✓		Windows only
Export-ModuleMember	✓	✓	
ForEach-Object	✓	✓	
Get-Command	✓	✓	
Get-ExperimentalFeature		✓	Added in 6.2
Get-Help	✓	✓	
Get-History	✓	✓	
Get-Job	✓	✓	

Cmdlet name	5.1	7.4 and higher	Note
Get-Module	✅	✅	
Get-PSHostProcessInfo	✅	✅	Added Linux support in 6.2
Get-PSSession	✅	✅	
Get-PSSessionCapability	✅	✅	
Get-PSSessionConfiguration	✅	✅	
Get-PSSnapin	✅		Windows only
Get-Verb	✅		Moved to Microsoft.PowerShell.Utility 6.0+
Import-Module	✅	✅	
Invoke-Command	✅	✅	
Invoke-History	✅	✅	
New-Module	✅	✅	
New-ModuleManifest	✅	✅	
New-PSRoleCapabilityFile	✅	✅	
New-PSSession	✅	✅	
New-PSSessionConfigurationFile	✅	✅	Added Linux support in 7.3
New-PSSessionOption	✅	✅	
New-PSTransportOption	✅	✅	
Out-Default	✅	✅	
Out-Host	✅	✅	
Out-Null	✅	✅	
Receive-Job	✅	✅	
Receive-PSSession	✅	✅	Windows only
Register-ArgumentCompleter	✅	✅	
Register-PSSessionConfiguration	✅	✅	Windows only
Remove-Job	✅	✅	
Remove-Module	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Remove-PSSession	✅	✅	
Remove-PSSnapin	✅		Windows only
Resume-Job	✅		
Save-Help	✅	✅	
Set-PSDebug	✅	✅	
Set-PSSessionConfiguration	✅	✅	Windows only
Set-StrictMode	✅	✅	
Start-Job	✅	✅	
Stop-Job	✅	✅	
Switch-Process			Linux and macOS only
Suspend-Job	✅		Windows only
Test-ModuleManifest	✅	✅	
Test-PSSessionConfigurationFile	✅	✅	Windows only
Unregister-PSSessionConfiguration	✅	✅	Windows only
Update-Help	✅	✅	
Wait-Job	✅	✅	
Where-Object	✅	✅	

Microsoft.PowerShell.Diagnostics

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Export-Counter	✅		Windows only
Get-Counter	✅	✅	Windows only
Get-WinEvent	✅	✅	Windows only
Import-Counter	✅		Windows only
New-WinEvent	✅	✅	Windows only

Microsoft.PowerShell.Host

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Start-Transcript	✅	✅	
Stop-Transcript	✅	✅	

Microsoft.PowerShell.LocalAccounts (64-bit only)

This modules is only available in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
Add-LocalGroupMember	✅	
Disable-LocalUser	✅	
Enable-LocalUser	✅	
Get-LocalGroup	✅	
Get-LocalGroupMember	✅	
Get-LocalUser	✅	
New-LocalGroup	✅	
New-LocalUser	✅	
Remove-LocalGroup	✅	
Remove-LocalGroupMember	✅	
Remove-LocalUser	✅	
Rename-LocalGroup	✅	
Rename-LocalUser	✅	
Set-LocalGroup	✅	
Set-LocalUser	✅	

Microsoft.PowerShell.Management

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Add-Computer	✓		Windows only
Add-Content	✓	✓	
Checkpoint-Computer	✓		Windows only
Clear-Content	✓	✓	
Clear-EventLog	✓		Windows only
Clear-Item	✓	✓	
Clear-ItemProperty	✓	✓	
Clear-RecycleBin	✓	✓	Windows only
Complete-Transaction	✓		Windows only
Convert-Path	✓	✓	
Copy-Item	✓	✓	
Copy-ItemProperty	✓	✓	
Debug-Process	✓	✓	
Disable-ComputerRestore	✓		Windows only
Enable-ComputerRestore	✓		Windows only
Get-ChildItem	✓	✓	
Get-Clipboard	✓	✓	
Get-ComputerInfo	✓	✓	Windows only
Get-ComputerRestorePoint	✓		Windows only
Get-Content	✓	✓	
Get-ControlPanelItem	✓		Windows only
Get-EventLog	✓		Windows only
Get-HotFix	✓	✓	Windows only
Get-Item	✓	✓	
Get-ItemProperty	✓	✓	

Cmdlet name	5.1	7.4 and higher	Note
Get-ItemPropertyValue	✅	✅	
Get-Location	✅	✅	
Get-Process	✅	✅	
Get-PSDrive	✅	✅	
Get-PSProvider	✅	✅	
Get-Service	✅	✅	Windows only
Get-TimeZone	✅	✅	Windows only
Get-Transaction	✅		Windows only
Get-WmiObject	✅		Windows only
Invoke-Item	✅	✅	
Invoke-WmiMethod	✅		Windows only
Join-Path	✅	✅	
Limit-EventLog	✅		Windows only
Move-Item	✅	✅	
Move-ItemProperty	✅	✅	
New-EventLog	✅		Windows only
New-Item	✅	✅	
New-ItemProperty	✅	✅	
New-PSDrive	✅	✅	
New-Service	✅	✅	Windows only
New-WebServiceProxy	✅		Windows only
Pop-Location	✅	✅	
Push-Location	✅	✅	
Register-WmiEvent	✅		Windows only
Remove-Computer	✅		Windows only
Remove-EventLog	✅		Windows only

Cmdlet name	5.1	7.4 and higher	Note
Remove-Item	✅	✅	
Remove-ItemProperty	✅	✅	
Remove-PSDrive	✅	✅	
Remove-Service		✅	Windows only
Remove-WmiObject	✅		Windows only
Rename-Computer	✅	✅	Windows only
Rename-Item	✅	✅	
Rename-ItemProperty	✅	✅	
Reset-ComputerMachinePassword	✅		Windows only
Resolve-Path	✅	✅	
Restart-Computer	✅	✅	Added Linux/macOS support in 7.1
Restart-Service	✅	✅	Windows only
Restore-Computer	✅		Windows only
Resume-Service	✅	✅	Windows only
Set-Clipboard	✅	✅	
Set-Content	✅	✅	
Set-Item	✅	✅	
Set-ItemProperty	✅	✅	
Set-Location	✅	✅	
Set-Service	✅	✅	Windows only
Set-TimeZone	✅	✅	Windows only
Set-WmiInstance	✅		Windows only
Show-ControlPanelItem	✅		Windows only
Show-EventLog	✅		Windows only
Split-Path	✅	✅	
Start-Process	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Start-Service	✅	✅	Windows only
Start-Transaction	✅		Windows only
Stop-Computer	✅	✅	Added Linux/macOS support in 7.1
Stop-Process	✅	✅	
Stop-Service	✅	✅	Windows only
Suspend-Service	✅	✅	Windows only
Test-ComputerSecureChannel	✅		Windows only
Test-Connection	✅	✅	
Test-Path	✅	✅	
Undo-Transaction	✅		Windows only
Use-Transaction	✅		Windows only
Wait-Process	✅	✅	
Write-EventLog	✅		Windows only

Microsoft.PowerShell.ODataUtils

This modules is only available in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
Export-ODataEndpointProxy	✅	

Microsoft.PowerShell.Operation.Validation

This modules is only available in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
Get-OperationValidation	✅	
Invoke-OperationValidation	✅	

Microsoft.PowerShell.PSResourceGet

Expand table

Cmdlet name	7.4 and higher	Note
Compress-PSResource		Added in v1.1.0 of the module
Find-PSResource	✅	
Get-InstalledPSResource	✅	
Get-PSResource	✅	
Get-PSResourceRepository	✅	
Get-PSScriptFileInfo	✅	
Import-PSGetRepository	✅	
Install-PSResource	✅	
New-PSScriptFileInfo	✅	
Publish-PSResource	✅	
Register-PSResourceRepository	✅	
Save-PSResource	✅	
Set-PSResourceRepository	✅	
Test-PSScriptFileInfo	✅	
Uninstall-PSResource	✅	
Unregister-PSResourceRepository	✅	
Update-PSModuleManifest	✅	
Update-PSResource	✅	
Update-PSScriptFileInfo	✅	

Microsoft.PowerShell.Security

Expand table

Cmdlet name	5.1	7.4 and higher	Note
ConvertFrom-SecureString	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
ConvertTo-SecureString	✅	✅	
Get-Acl	✅	✅	Windows only
Get-AuthenticodeSignature	✅	✅	Windows only
Get-CmsMessage	✅	✅	Support for Linux/macOS added in 7.1
Get-Credential	✅	✅	
Get-ExecutionPolicy	✅	✅	Returns **Unrestricted** on Linux/macOS
Get-PfxCertificate	✅	✅	
New-FileCatalog	✅	✅	Windows only
Protect-CmsMessage	✅	✅	Support for Linux/macOS added in 7.1
Set-Acl	✅	✅	Windows only
Set-AuthenticodeSignature	✅	✅	Windows only
Set-ExecutionPolicy	✅	✅	Does nothing on Linux/macOS
Test-FileCatalog	✅	✅	Windows only
Unprotect-CmsMessage	✅	✅	Support for Linux/macOS added in 7.1

Microsoft.PowerShell.Utility

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Add-Member	✅	✅	
Add-Type	✅	✅	
Clear-Variable	✅	✅	
Compare-Object	✅	✅	
Convert-String	✅		
ConvertFrom-CliXml			Added in 7.5
ConvertFrom-Csv	✅	✅	
ConvertFrom-Json	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
ConvertFrom-Markdown		✅	Added in 6.1
ConvertFrom-SddlString	✅	✅	Windows only
ConvertFrom-String	✅		
ConvertFrom-StringData	✅	✅	
ConvertTo-CliXml			Added in 7.5
ConvertTo-Csv	✅	✅	
ConvertTo-Html	✅	✅	
ConvertTo-Json	✅	✅	
ConvertTo-Xml	✅	✅	
Debug-Runspace	✅	✅	
Disable-PSBreakpoint	✅	✅	
Disable-RunspaceDebug	✅	✅	
Enable-PSBreakpoint	✅	✅	
Enable-RunspaceDebug	✅	✅	
Export-Alias	✅	✅	
Export-Clixml	✅	✅	
Export-Csv	✅	✅	
Export-FormatData	✅	✅	
Export-PSSession	✅	✅	
Format-Custom	✅	✅	
Format-Hex	✅	✅	
Format-List	✅	✅	
Format-Table	✅	✅	
Format-Wide	✅	✅	
Get-Alias	✅	✅	
Get-Culture	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Get-Date	✅	✅	
Get-Error		✅	
Get-Event	✅	✅	No event sources available on Linux/macOS
Get-EventSubscriber	✅	✅	
Get-FileHash	✅	✅	
Get-FormatData	✅	✅	
Get-Host	✅	✅	
Get-MarkdownOption		✅	Added in 6.1
Get-Member	✅	✅	
Get-PSBreakpoint	✅	✅	
Get-PSCallStack	✅	✅	
Get-Random	✅	✅	
Get-Runspace	✅	✅	
Get-RunspaceDebug	✅	✅	
Get-SecureRandom		✅	Added in 7.4
Get-TraceSource	✅	✅	
Get-TypeData	✅	✅	
Get-UICulture	✅	✅	
Get-Unique	✅	✅	
Get-Uptime		✅	
Get-Variable	✅	✅	
Get-Verb		✅	Moved from Microsoft.PowerShell.Core
Group-Object	✅	✅	
Import-Alias	✅	✅	
Import-Clixml	✅	✅	
Import-Csv	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Import-LocalizedData	✅	✅	
Import-PowerShellDataFile	✅	✅	
Import-PSSession	✅	✅	
Invoke-Expression	✅	✅	
Invoke-RestMethod	✅	✅	
Invoke-WebRequest	✅	✅	
Join-String		✅	
Measure-Command	✅	✅	
Measure-Object	✅	✅	
New-Alias	✅	✅	
New-Event	✅	✅	No event sources available on Linux/macOS
New-Guid	✅	✅	
New-Object	✅	✅	
New-TemporaryFile	✅	✅	
New-TimeSpan	✅	✅	
New-Variable	✅	✅	
Out-File	✅	✅	
Out-GridView	✅	✅	Windows only
Out-Printer	✅	✅	Windows only
Out-String	✅	✅	
Read-Host	✅	✅	
Register-EngineEvent	✅	✅	No event sources available on Linux/macOS
Register-ObjectEvent	✅	✅	
Remove-Alias		✅	
Remove-Event	✅	✅	No event sources available on Linux/macOS
Remove-PSBreakpoint	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Remove-TypeData	✅	✅	
Remove-Variable	✅	✅	
Select-Object	✅	✅	
Select-String	✅	✅	
Select-Xml	✅	✅	
Send-MailMessage	✅	✅	
Set-Alias	✅	✅	
Set-Date	✅	✅	
Set-MarkdownOption		✅	Added in 6.1
Set-PSBreakpoint	✅	✅	
Set-TraceSource	✅	✅	
Set-Variable	✅	✅	
Show-Command	✅	✅	Windows only
Show-Markdown		✅	Added in 6.1
Sort-Object	✅	✅	
Start-Sleep	✅	✅	
Tee-Object	✅	✅	
Test-Json		✅	
Trace-Command	✅	✅	
Unblock-File	✅	✅	Added support for macOS in 7.0
Unregister-Event	✅	✅	No event sources available on Linux/macOS
Update-FormatData	✅	✅	
Update-List	✅	✅	
Update-TypeData	✅	✅	
Wait-Debugger	✅	✅	
Wait-Event	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Write-Debug	✅	✅	
Write-Error	✅	✅	
Write-Host	✅	✅	
Write-Information	✅	✅	
Write-Output	✅	✅	
Write-Progress	✅	✅	
Write-Verbose	✅	✅	
Write-Warning	✅	✅	

Microsoft.WsMan.Management

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Connect-WSMan	✅	✅	Windows only
Disable-WSManCredSSP	✅	✅	Windows only
Disconnect-WSMan	✅	✅	Windows only
Enable-WSManCredSSP	✅	✅	Windows only
Get-WSManCredSSP	✅	✅	Windows only
Get-WSManInstance	✅	✅	Windows only
Invoke-WSManAction	✅	✅	Windows only
New-WSManInstance	✅	✅	Windows only
New-WSManSessionOption	✅	✅	Windows only
Remove-WSManInstance	✅	✅	Windows only
Set-WSManInstance	✅	✅	Windows only
Set-WSManQuickConfig	✅	✅	Windows only
Test-WSMan	✅	✅	Windows only

PackageManagement

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Find-Package	✅	✅	
Find-PackageProvider	✅	✅	
Get-Package	✅	✅	
Get-PackageProvider	✅	✅	
Get-PackageSource	✅	✅	
Import-PackageProvider	✅	✅	
Install-Package	✅	✅	
Install-PackageProvider	✅	✅	
Register-PackageSource	✅	✅	
Save-Package	✅	✅	
Set-PackageSource	✅	✅	
Uninstall-Package	✅	✅	
Unregister-PackageSource	✅	✅	

PowerShellGet 2.x

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Find-Command	✅	✅	
Find-DscResource	✅	✅	
Find-Module	✅	✅	
Find-RoleCapability	✅	✅	
Find-Script	✅	✅	
Get-CredsFromCredentialProvider		✅	
Get-InstalledModule	✅	✅	
Get-InstalledScript	✅	✅	

Cmdlet name	5.1	7.4 and higher	Note
Get-PSRepository	✅	✅	
Install-Module	✅	✅	
Install-Script	✅	✅	
New-ScriptFileInfo	✅	✅	
Publish-Module	✅	✅	
Publish-Script	✅	✅	
Register-PSRepository	✅	✅	
Save-Module	✅	✅	
Save-Script	✅	✅	
Set-PSRepository	✅	✅	
Test-ScriptFileInfo	✅	✅	
Uninstall-Module	✅	✅	
Uninstall-Script	✅	✅	
Unregister-PSRepository	✅	✅	
Update-Module	✅	✅	
Update-ModuleManifest	✅	✅	
Update-Script	✅	✅	
Update-ScriptFileInfo	✅	✅	

PSDesiredStateConfiguration v1.1

This modules is only available from in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
Configuration	✅	
Disable-DscDebug	✅	
Enable-DscDebug	✅	

Cmdlet name	5.1	Note
Get-DscConfiguration	✅	
Get-DscConfigurationStatus	✅	
Get-DscLocalConfigurationManager	✅	
Get-DscResource	✅	
Invoke-DscResource	✅	
New-DSCCheckSum	✅	
Publish-DscConfiguration	✅	
Remove-DscConfigurationDocument	✅	
Restore-DscConfiguration	✅	
Set-DscLocalConfigurationManager	✅	
Start-DscConfiguration	✅	
Stop-DscConfiguration	✅	
Test-DscConfiguration	✅	
Update-DscConfiguration	✅	

PSDesiredStateConfiguration v2.0.5

This modules is only available from the PowerShell Gallery.

Expand table

Cmdlet name	2.0.5	Note
Configuration	✅	
Get-DscResource	✅	
Invoke-DscResource	✅	Experimental
New-DSCCheckSum	✅	

PSDesiredStateConfiguration v3.x - Preview

This modules is only available from the PowerShell Gallery.

Expand table

Cmdlet name	3.0 (preview)	Note
Configuration	✓	
ConvertTo-DscJsonSchema	✓	
Get-DscResource	✓	
Invoke-DscResource	✓	
New-DscChecksum	✓	

PSDiagnostics

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Disable-PSTrace	✓	✓	Windows only
Disable-PSWSManCombinedTrace	✓	✓	Windows only
Disable-WSManTrace	✓	✓	Windows only
Enable-PSTrace	✓	✓	Windows only
Enable-PSWSManCombinedTrace	✓	✓	Windows only
Enable-WSManTrace	✓	✓	Windows only
Get-LogProperties	✓	✓	Windows only
Set-LogProperties	✓	✓	Windows only
Start-Trace	✓	✓	Windows only
Stop-Trace	✓	✓	Windows only

PSReadLine

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Get-PSReadLineKeyHandler	✓	✓	
Get-PSReadLineOption	✓	✓	

Cmdlet name	5.1	7.4 and higher	Note
PSConsoleHostReadLine	✅	✅	
Remove-PSReadLineKeyHandler	✅	✅	
Set-PSReadLineKeyHandler	✅	✅	
Set-PSReadLineOption	✅	✅	

PSScheduledJob

This modules is only available in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
Add-JobTrigger	✅	
Disable-JobTrigger	✅	
Disable-ScheduledJob	✅	
Enable-JobTrigger	✅	
Enable-ScheduledJob	✅	
Get-JobTrigger	✅	
Get-ScheduledJob	✅	
Get-ScheduledJobOption	✅	
New-JobTrigger	✅	
New-ScheduledJobOption	✅	
Register-ScheduledJob	✅	
Remove-JobTrigger	✅	
Set-JobTrigger	✅	
Set-ScheduledJob	✅	
Set-ScheduledJobOption	✅	
Unregister-ScheduledJob	✅	

PSWorkflow & PSWorkflowUtility

This modules is only available in Windows PowerShell.

Expand table

Cmdlet name	5.1	Note
New-PSWorkflowExecutionOption	✅	
New-PSWorkflowSession	✅	
Invoke-AsWorkflow	✅	

Microsoft.PowerShell.ThreadJob (formerly ThreadJob)

This module can be installed from the PowerShell Gallery on any supported version of PowerShell. The ThreadJob module was initially included in PowerShell 6.0. The ThreadJob module was renamed to Microsoft.PowerShell.ThreadJob in PowerShell 7.6.

Expand table

Cmdlet name	5.1	7.4 and higher	Note
Start-ThreadJob		✅	

Last updated on 04/28/2026

PowerShell 7 module compatibility

08/06/2025

This article contains a partial list of PowerShell modules published by Microsoft.

The PowerShell team is working with the various feature teams that create PowerShell modules to help them produce modules that work in PowerShell 7. These modules are not owned by the PowerShell team.

The following modules are known to support PowerShell 7.

Azure PowerShell

The Az PowerShell module is a set of cmdlets for managing Azure resources directly from PowerShell. PowerShell 7.0.6 LTS or higher is the recommended version of PowerShell for use with the Azure Az PowerShell module on all platforms.

For more information, see Introducing the Azure Az PowerShell module.

MSGraph PowerShell SDK

The Microsoft Graph SDKs are designed to simplify building high-quality, efficient, and resilient applications that access Microsoft Graph. PowerShell 7 and later is the recommended PowerShell version for use with the Microsoft Graph PowerShell SDK.

For more information, see Install the Microsoft Graph PowerShell SDK.

Windows management modules

The Windows management modules provide management and support for various Windows features and services. Most of these modules have been updated to work natively with PowerShell 7, or tested for compatibility with PowerShell 7.

These modules are installed in different ways depending on the Edition of Windows, and how the module is packaged for that Edition.

For more information about installation and compatibility, see PowerShell 7 module compatibility in the Windows documentation.

Exchange Online Management 2.0

The Exchange Online PowerShell V2 module (EXO V2) connects to all Exchange-related PowerShell environments in Microsoft 365: Exchange Online PowerShell, Security & Compliance PowerShell, and standalone Exchange Online Protection (EOP) PowerShell.

EXO v2.0.4 or later is supported in PowerShell 7.0.3 or later.

For more information, see About the Exchange Online PowerShell V2 module.

PowerShell modules for SQL Server

There are two SQL Server PowerShell modules:

- **SqlServer**: This module includes new cmdlets to support the latest SQL features, including updated versions of the cmdlets in SQLPS.
- **SQLPS**: The SQLPS is the module used by SQL Agent to run agent jobs in agent job steps using the PowerShell subsystem.

The SqlServer modules require PowerShell version 5.0 or greater.

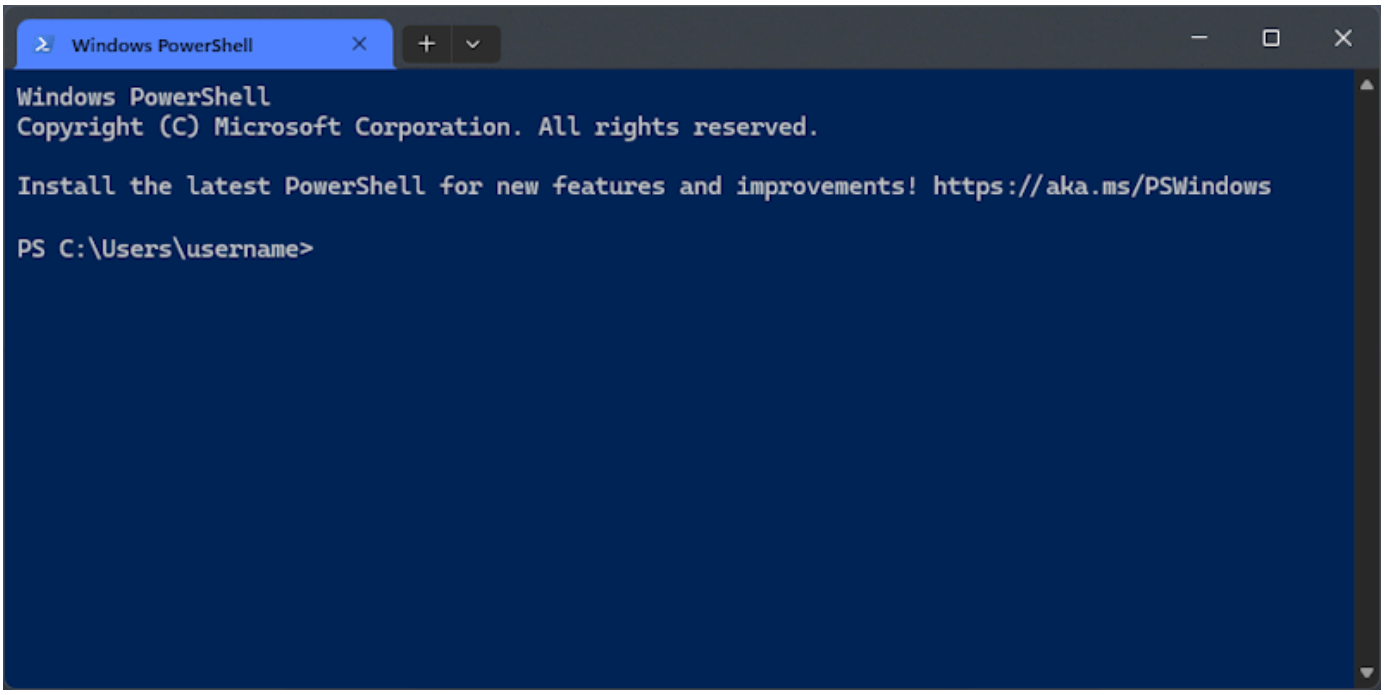
For more information, see Install the SQL Server PowerShell module.

Finding the status of other modules

You can find a complete list of modules using the PowerShell Module Browser. Using the Module Browser, you can find documentation for other PowerShell modules to determine their PowerShell version requirements.

Windows PowerShell update message FAQ

When you start Windows PowerShell you see a message that tells you to install the latest version of PowerShell along with a URL.



This article explains the intent of this message and provides answers to frequently asked questions.

Why am I seeing this message?

Windows PowerShell 5.1 displays this message at startup to make you aware that there is a newer version of PowerShell available. You will see this message every time you start Windows PowerShell (`powershell.exe`). You won't see this message when you start PowerShell 7 (`pwsh.exe`).

I opened the URL but it didn't install an update. How do I install PowerShell 7?

The `aka.ms` link shown in the Windows PowerShell console takes you to this page. To install the latest version of PowerShell 7, follow the instructions in the **Install PowerShell on Windows** article.

But I installed PowerShell 7, why am I still seeing this message?

Because you ran Windows PowerShell 5.1 (`powershell.exe`) instead of PowerShell 7 (`pwsh.exe`). PowerShell 7 and Windows PowerShell 5.1 are separate products. PowerShell 7 doesn't replace Windows PowerShell 5.1. PowerShell 7 installs side-by-side. You can run either version.

How do I run PowerShell 7?

For instructions, see the *Start PowerShell 7* section of the Install PowerShell on Windows article.

Why would I want to install PowerShell 7?

PowerShell 7 is faster, more secure, and is available for Windows, Linux and macOS. PowerShell 7 is actively developed and supported. Windows PowerShell 5.1 is no longer being developed. For more information, see the Differences between Windows PowerShell 5.1 and PowerShell 7 article.

Do I need to install PowerShell 7?

No, you don't need to install PowerShell 7 unless you have a specific requirement for the features it provides.

My install of Windows is up-to-date. Why am I still getting this message?

This message appears every time you start Windows PowerShell 5.1. It doesn't mean that Windows PowerShell 5.1 is not up-to-date.

How does Windows PowerShell 5.1 get updated?

Microsoft only publishes security updates for Windows PowerShell 5.1. It's important to understand that Windows PowerShell 5.1 is no longer being developed. Windows PowerShell 5.1 is a built-in component of Windows. All updates are managed through standard Windows update channels.

Can I disable the message?

No. There is no way to remove the message. However, you can avoid all startup messages by running `powershell.exe -nologo`.

Last updated on 08/18/2026

Starting Windows PowerShell

Article • 03/27/2025

Windows PowerShell is a scripting engine embedded into multiple hosts. The most common hosts are the interactive command-line `powershell.exe` and the Interactive Scripting Environment `powershell_ise.exe`.

PowerShell binary name

PowerShell version 6 and higher uses .NET (Core). Supported versions are available on Windows, macOS, and Linux.

Beginning in PowerShell 6, the PowerShell binary named `pwsh.exe` for Windows and `pwsh` for macOS and Linux. You can start PowerShell preview versions using `pwsh-preview`. For more information, see About pwsh.

To find cmdlet reference and installation documentation for PowerShell 7, use the following links:

Expand table

Document	Link
Cmdlet reference	PowerShell Module Browser
Windows installation	Installing PowerShell on Windows
macOS installation	Installing PowerShell on macOS
Linux installation	Installing PowerShell on Linux

To view content for other PowerShell versions, see How to use the PowerShell documentation.

Run from the Start Menu

- Open the **Start** menu, type **Windows PowerShell**, select **Windows PowerShell**, then select **Open**.

Run from the Command Prompt

In Windows Command shell, Windows PowerShell, or Windows PowerShell ISE, to start Windows PowerShell, type: `PowerShell`.

You can also use the parameters of the `powershell.exe` program to customize the session. For more information, see about_PowerShell_exe.

Run with administrative privileges

Open the **Start** menu, type **Windows PowerShell**, select **Windows PowerShell**, and then select **Run as administrator**.

How to Start Windows PowerShell ISE on Earlier Releases of Windows

Use any of the following methods to start Windows PowerShell ISE.

Run from the Start Menu

- Open the **Start** menu, type **ISE**, select **Windows PowerShell ISE**, then select **Open**.

At the Command Prompt

In Windows Command shell, Windows PowerShell, or Windows PowerShell ISE, to start Windows PowerShell, type: `PowerShell_ISE`. In Windows PowerShell, you can use the alias `ise`.

Run with administrative privileges

Select **Start**, type **ISE**, right-click **Windows PowerShell ISE**, and then click **Run as administrator**.

Starting the 32-Bit Version of Windows PowerShell

64-bit versions of Windows include a 32-bit version of Windows PowerShell, **Windows PowerShell (x86)**, in addition to the 64-bit version. The 64-bit version runs by default.

However, you might occasionally need to run **Windows PowerShell (x86)**, such as when you're using a module that requires the 32-bit version or when you're connecting

remotely to a 32-bit computer.

To start a 32-bit version of Windows PowerShell, use any of the following procedures.

- Select **Start**, type **Windows PowerShell**, select **Windows PowerShell (x86)**, then select **Open**.

The Windows PowerShell ISE

The Windows PowerShell Integrated Scripting Environment (ISE) is a host application for Windows PowerShell. In the ISE, you can run commands and write, test, and debug scripts in a single Windows-based graphic user interface. The ISE provides multiline editing, tab completion, syntax coloring, selective execution, context-sensitive help, and support for right-to-left languages. Menu items and keyboard shortcuts are mapped to many of the same tasks that you would do in the Windows PowerShell console. For example, when you debug a script in the ISE, you can right-click on a line of code in the edit pane to set a breakpoint.

Support

The ISE was first introduced with Windows PowerShell V2 and was re-designed with PowerShell V3. The ISE is supported in all supported versions of Windows PowerShell up to and including Windows PowerShell V5.1.

> **Note**
>
> The PowerShell ISE is no longer in active feature development. As a shipping component of Windows, it continues to be officially supported for security and high-priority servicing fixes. We currently have no plans to remove the ISE from Windows.
>
> There is no support for the ISE in PowerShell v6 and beyond. Users looking for replacement for the ISE should use Visual Studio Code with the PowerShell Extension.

Key Features

Key features in Windows PowerShell ISE include:

- Multiline editing: To insert a blank line under the current line in the Command pane, press `SHIFT` + `ENTER`.
- Selective execution: To run part of a script, select the text you want to run, and then click the **Run Script** button. Or, press `F5`.
- Context-sensitive help: Type `Invoke-Item`, and then press `F1`. The Help file opens to the article for the `Invoke-Item` cmdlet.

The Windows PowerShell ISE lets you customize some aspects of its appearance. It also has its own Windows PowerShell profile script.

To start the Windows PowerShell ISE

Click **Start**, select **Windows PowerShell**, and then click **Windows PowerShell ISE**. Alternately, you can type `powershell_ise.exe` in any command shell or in the Run box.

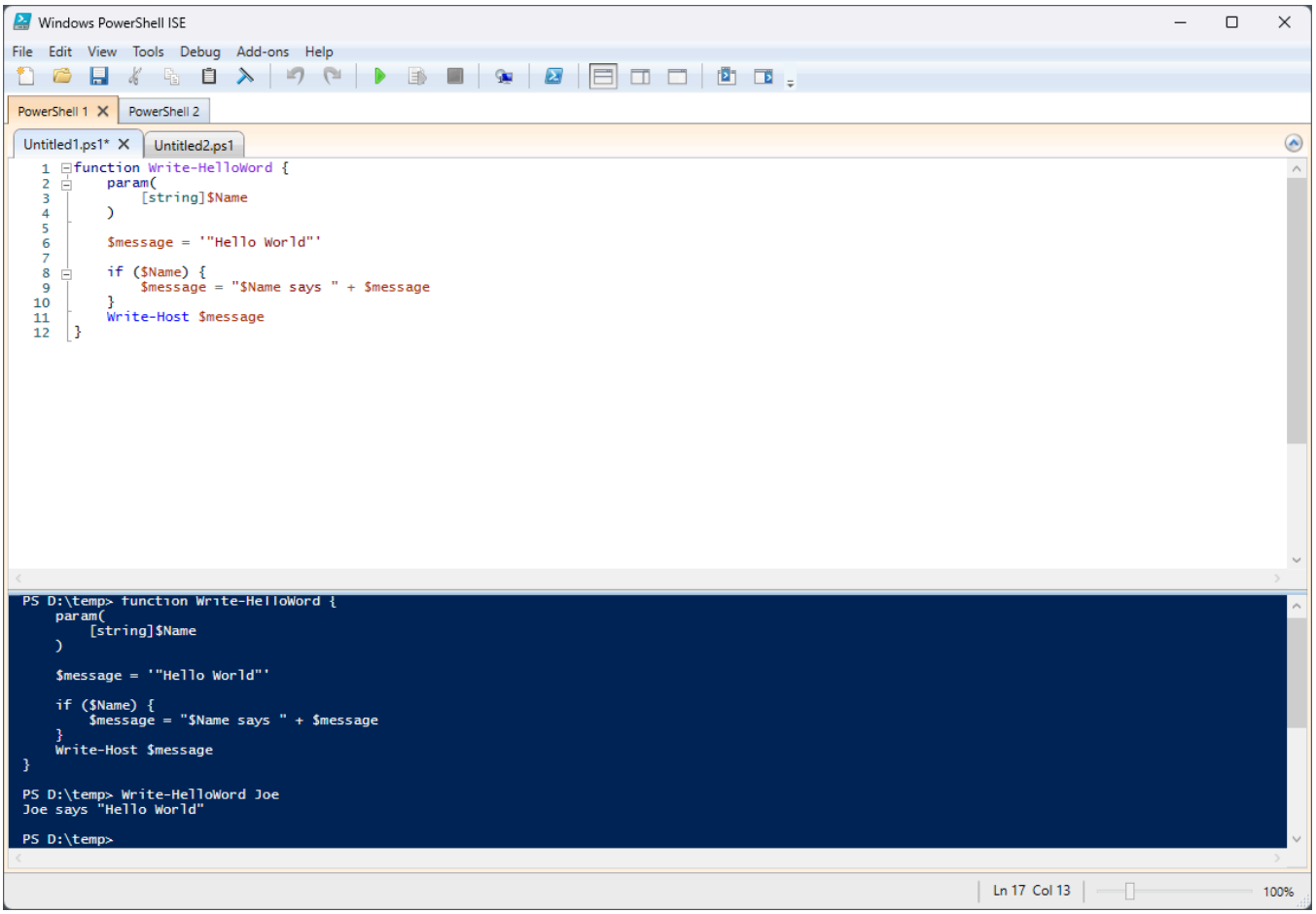
To get Help in the Windows PowerShell ISE

On the **Help** menu, click **Windows PowerShell Help**. Or, press `F1`. The file that opens describes Windows PowerShell ISE and Windows PowerShell, including all the help available from the `Get-Help` cmdlet.

Last updated on 11/20/2025

Exploring the Windows PowerShell ISE

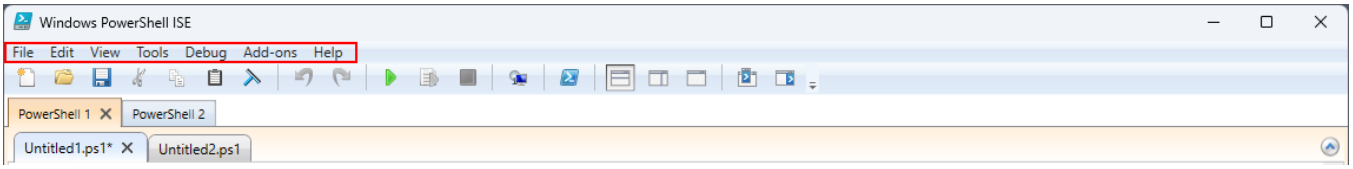
You can use the Windows PowerShell Integrated Scripting Environment (ISE) to create, run, and debug commands and scripts.



The Windows PowerShell ISE consists of the menu bar, Windows PowerShell tabs, the toolbar, script tabs, a Script Pane, a Console Pane, a status bar, a text-size slider and context-sensitive Help.

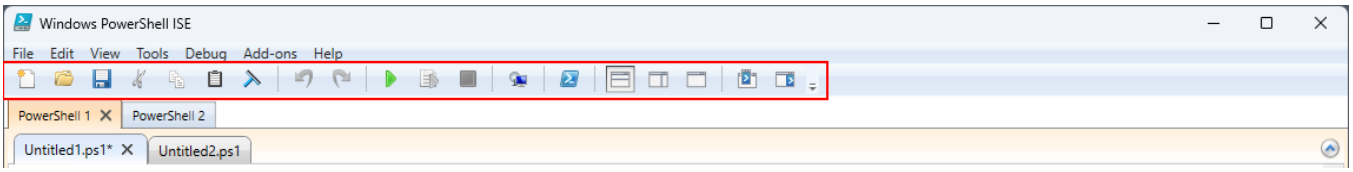
Menu Bar

The menu bar contains the **File**, **Edit**, **View**, **Tools**, **Debug**, **Add-ons**, and **Help** menus.



The buttons on the menus allow you to perform tasks related to writing and running scripts and running commands in the Windows PowerShell ISE.

Toolbar



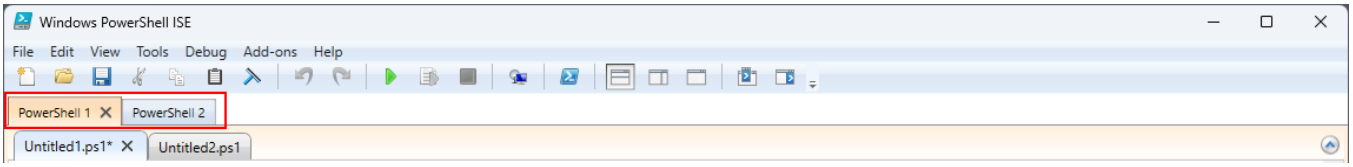
The following buttons are located on the toolbar.

Expand table

Button	Function
New	Opens a new script.
Open	Opens an existing script or file.
Save	Saves a script or file.
Cut	Cuts the selected text and copies it to the clipboard.
Copy	Copies the selected text to the clipboard.
Paste	Pastes the contents of the clipboard at the cursor location.
Clear Console Pane	Clears all content in the Console Pane.
Undo	Reverses the action that was just performed.
Redo	Performs the action that was just undone.
Run Script	Runs a script.
Run Selection	Runs a selected portion of a script.
Stop Operation	Stops a script that's running.
New Remote PowerShell Tab	Creates a new PowerShell Tab that establishes a session on a remote computer. A dialog box appears and prompts you to enter details required to establish the remote connection.
Start powershell.exe	Opens a PowerShell Console.
Show Script Pane Top	Moves the Script Pane to the top in the display.
Show Script Pane Right	Moves the Script Pane to the right in the display.
Show Script Pane Maximized	Maximizes the Script Pane.
Show Command Window	Shows the Commands Pane for installed Modules, as a separate Window.

Button	Function
Show Command Add-on	Shows the Commands Pane for installed Modules, as a sidebar Add-on.

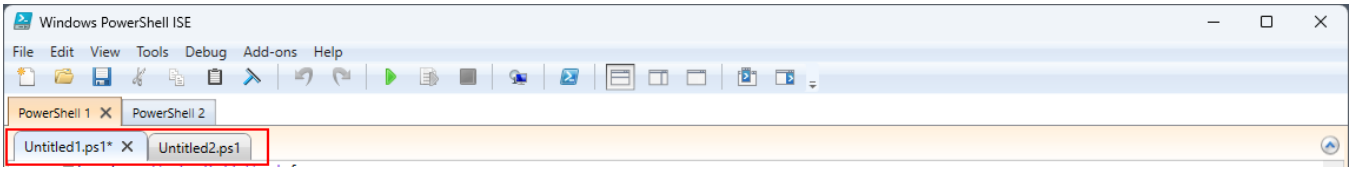
Windows PowerShell Tabs



A Windows PowerShell tab is the environment in which a Windows PowerShell script runs. You can open new Windows PowerShell tabs in the Windows PowerShell ISE to create separate environments on your local computer or on remote computers. You may have a maximum of eight PowerShell tabs simultaneously open.

For more information, see How to Create a PowerShell Tab in Windows PowerShell ISE.

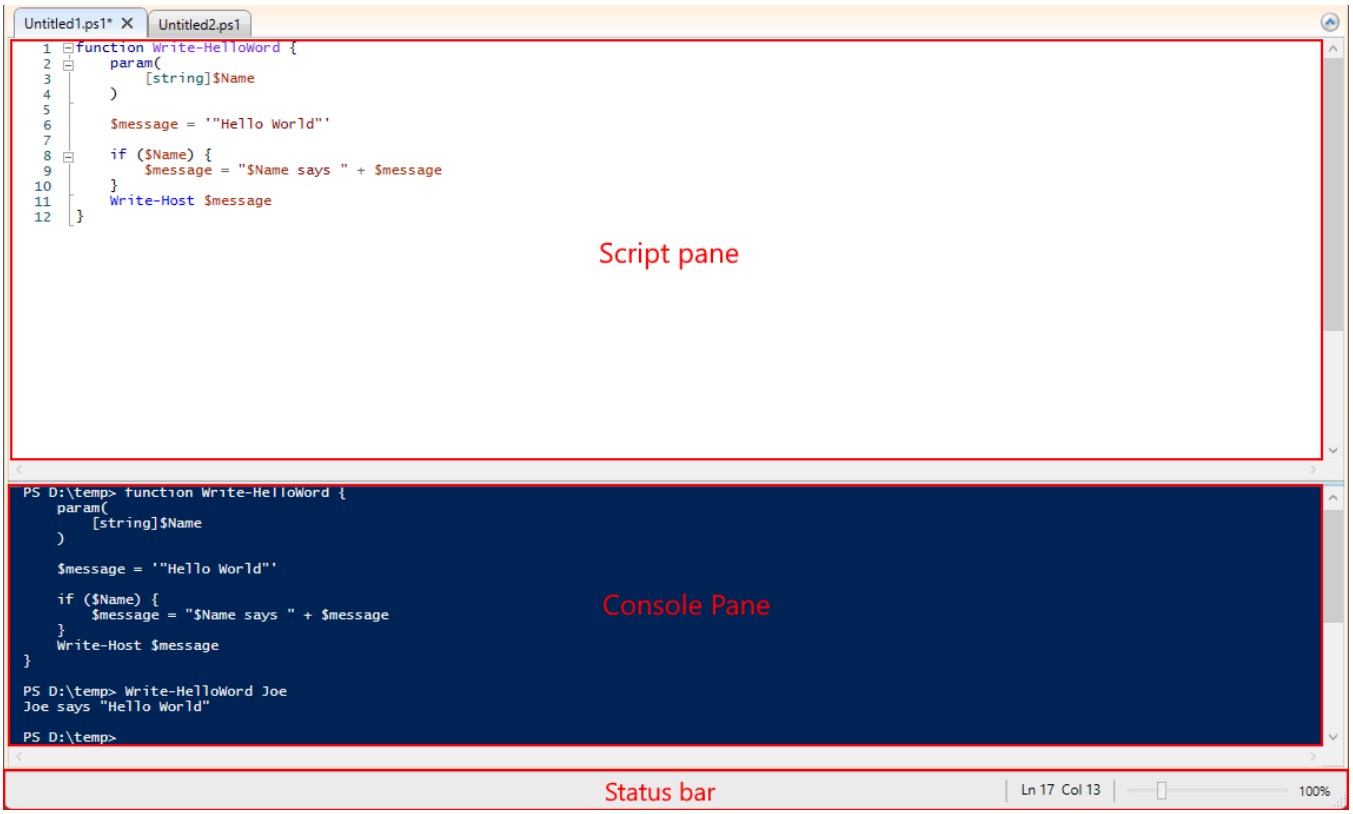
Script Tab



Displays the name of the script you are editing. You can click a script tab to select the script you want to edit.

When you point to the script tab, the fully qualified path to the script file appears in a tooltip.

Script Pane



Allows you to create and run scripts. You can open, edit and run existing scripts in the Script Pane. For more information, see How to Write and Run Scripts in the Windows PowerShell ISE.

Console Pane

Displays the results of the commands and scripts you have run. You can run commands in the Console pane. You can also copy and clear the contents in the Console Pane.

For more information, see the following articles:

- How to Use the Console Pane in the Windows PowerShell ISE
- How to Debug Scripts in Windows PowerShell ISE
- How to Use Tab Completion in the Script Pane and Console Pane

Status Bar

Allows you to see whether the commands and scripts that you run are complete. The status bar is at the bottom of the window. Selected portions of error messages are displayed on the status bar.

Text-Size Slider

Increases or decreases the size of the text on the screen.

Help

Help for Windows PowerShell ISE is available on Microsoft Learn. You can open the Help by clicking **Windows PowerShell ISE Help** on the **Help** menu or by pressing the `F1` key anywhere except when the cursor is on a cmdlet name in either the Script Pane or the Console Pane. From the **Help** menu you can also run the `Update-Help` cmdlet, and display the Command Window, which assists you in constructing commands by showing you all the parameters for a cmdlet and enabling you to fill in the parameters in an easy-to-use form.

See Also

- Introducing the Windows PowerShell ISE
- How to Use Profiles in Windows PowerShell ISE
- Accessibility in Windows PowerShell ISE
- Keyboard Shortcuts for the Windows PowerShell ISE

Last updated on 11/20/2025

How to Create a PowerShell Tab in Windows PowerShell ISE

Tabs in the Windows PowerShell Integrated Scripting Environment (ISE) allow you to simultaneously create and use several execution environments within the same application. Each PowerShell tab corresponds to a separate execution environment or session.

> ⓘ **Note**
>
> Variables, functions, and aliases that you create in one tab don't carry over to another. They are different Windows PowerShell sessions.

Use the following steps to open or close a tab in Windows PowerShell. To rename a tab, set the **DisplayName** property on the Windows PowerShell Tab scripting object.

To create and use a new PowerShell Tab

On the **File** menu, click **New PowerShell Tab**. The new PowerShell tab always opens as the active window. PowerShell tabs are incrementally numbered in the order that they're opened. Each tab is associated with its own Windows PowerShell console window. You can have up to 32 PowerShell tabs with their own session open at a time (this is limited to 8 on Windows PowerShell ISE 2.0.)

Note that clicking the **New** or **Open** icons on the toolbar doesn't create a new tab with a separate session. Instead, those buttons open a new or existing script file on the currently active tab with a session. You can have multiple script files open with each tab and session. The script tabs for a session only appear below the session tabs when the associated session is active.

To make a PowerShell tab active, click the tab. To select from all PowerShell tabs that are open, on the **View** menu, click the PowerShell tab you want to use.

To create and use a new Remote PowerShell tab

On the **File** menu, click **New Remote PowerShell Tab** to establish a session on a remote computer. A dialog box appears and prompts you to enter details required to establish the remote connection. The remote tab functions just like a local PowerShell tab, but the commands and scripts are run on the remote computer.

To close a PowerShell Tab

To close a tab, you can use any of the following techniques:

- Click the tab that you want to close.
- On the **File** menu, click **Close PowerShell Tab**, or click the Close button (**X**) on an active tab to close the tab.

If you have unsaved files open in the PowerShell tab that you are closing, you are prompted to save or discard them. For more information about how to save a script, see How to Save a Script.

See Also

- Introducing the Windows PowerShell ISE
- How to Use the Console Pane in the Windows PowerShell ISE

Last updated on 11/20/2025

How to Debug Scripts in Windows PowerShell ISE

This article describes how to debug scripts on a local computer by using the Windows PowerShell Integrated Scripting Environment (ISE) visual debugging features.

How to manage breakpoints

A breakpoint is a designated spot in a script where you would like operation to pause so that you can examine the current state of the variables and the environment in which your script is running. Once your script is paused by a breakpoint, you can run commands in the Console Pane to examine the state of your script. You can output variables or run other commands. You can even modify the value of any variables that are visible to the context of the currently running script. After you have examined what you want to see, you can resume operation of the script.

You can set three types of breakpoints in the Windows PowerShell debugging environment:

1. **Line breakpoint**. The script pauses when the designated line is reached during the operation of the script

2. **Variable breakpoint.** The script pauses whenever the designated variable's value changes.

3. **Command breakpoint.** The script pauses whenever the designated command is about to be run during the operation of the script. It can include parameters to further filter the breakpoint to only the operation you want. The command can also be a function you created.

Of these, in the Windows PowerShell ISE debugging environment, only line breakpoints can be set by using the menu or the keyboard shortcuts. The other two types of breakpoints can be set, but they are set from the Console Pane by using the Set-PSBreakpoint cmdlet. This section describes how you can perform debugging tasks in Windows PowerShell ISE by using the menus where available, and perform a wider range of commands from the Console Pane by using scripting.

To set a breakpoint

A breakpoint can be set in a script only after it has been saved. Right-click the line where you want to set a line breakpoint, and then click **Toggle Breakpoint**. Or, click the line where you want to set a line breakpoint, and press F9 or, on the **Debug** menu, click **Toggle Breakpoint**.

The following script is an example of how you can set a variable breakpoint from the Console Pane by using the Set-PSBreakpoint cmdlet.

```PowerShell
# This command sets a breakpoint on the Server variable in the Sample.ps1 script.
Set-PSBreakpoint -Script sample.ps1 -Variable Server
```

List all breakpoints

Displays all breakpoints in the current Windows PowerShell session.

On the **Debug** menu, click **List Breakpoints**. The following script is an example of how you can list all breakpoints from the Console Pane by using the Get-PSBreakpoint cmdlet.

```PowerShell
# This command lists all breakpoints in the current session.
Get-PSBreakpoint
```

Remove a breakpoint

Removing a breakpoint deletes it.

If you think you might want to use it again later, consider Disable a Breakpoint it instead. Right-click the line where you want to remove a breakpoint, and then click **ToggleBreakpoint**. Or, click the line where you want to remove a breakpoint, and on the **Debug** menu, click **Toggle Breakpoint**. The following script is an example of how to remove a breakpoint with a specified ID from the Console Pane by using the Remove-PSBreakpoint cmdlet.

```PowerShell
# This command deletes the breakpoint with breakpoint ID 2.
Remove-PSBreakpoint -Id 2
```

Remove All Breakpoints

To remove all breakpoints defined in the current session, on the **Debug** menu, click **Remove All Breakpoints**.

The following script is an example of how to remove all breakpoints from the Console Pane by using the Remove-PSBreakpoint cmdlet.

PowerShell

```
# This command deletes all of the breakpoints in the current session.
Get-PSBreakpoint | Remove-PSBreakpoint
```

Disable a Breakpoint

Disabling a breakpoint doesn't remove it. It turns it off until it's enabled. To disable a specific line breakpoint, right-click the line where you want to disable a breakpoint, and then click **Disable Breakpoint**.

Or, click the line where you want to disable a breakpoint, and press `F9` or, on the **Debug** menu, click **Disable Breakpoint**. The following script is an example of how you can remove a breakpoint with a specified ID from the Console Pane using the Disable-PSBreakpoint cmdlet.

PowerShell

```
# This command disables the breakpoint with breakpoint ID 0.
Disable-PSBreakpoint -Id 0
```

Disable All Breakpoints

Disabling a breakpoint doesn't remove it; it turns it off until it's enabled. To disable all breakpoints in the current session, on the **Debug** menu, click **Disable all Breakpoints**. The following script is an example of how you can disable all breakpoints from the Console Pane by using the Disable-PSBreakpoint cmdlet.

PowerShell

```
# This command disables all breakpoints in the current session.
# You can abbreviate this command as: "gbp | dbp".
Get-PSBreakpoint | Disable-PSBreakpoint
```

Enable a Breakpoint

To enable a specific breakpoint, right-click the line where you want to enable a breakpoint, and then click **Enable Breakpoint**. Or, click the line where you want to enable a breakpoint, and then press `F9` or, on the **Debug** menu, click **Enable Breakpoint**. The following script is an example of how you can enable specific breakpoints from the Console Pane by using the Enable-PSBreakpoint cmdlet.

PowerShell

```
# This command enables breakpoints with breakpoint IDs 0, 1, and 5.
Enable-PSBreakpoint -Id 0, 1, 5
```

Enable All Breakpoints

To enable all breakpoints defined in the current session, on the **Debug** menu, click **Enable all Breakpoints**. The following script is an example of how you can enable all breakpoints from the Console Pane by using the Enable-PSBreakpoint cmdlet.

PowerShell

```
# This command enables all breakpoints in the current session.
# You can abbreviate the command by using their aliases: "gbp | ebp".
Get-PSBreakpoint | Enable-PSBreakpoint
```

How to manage a debugging session

Before you start debugging, you must set one or more breakpoints. You can't set a breakpoint unless the script that you want to debug is saved. For directions on of how to set a breakpoint, see How to manage breakpoints or Set-PSBreakpoint. After you start debugging, you can't edit a script until you stop debugging. A script that has one or more breakpoints set is automatically saved before it's run.

To start debugging

Press `F5` or, on the toolbar, click the **Run Script** icon, or on the **Debug** menu click **Run/Continue**. The script runs until it encounters the first breakpoint. It pauses operation there and highlights the line on which it paused.

To continue debugging

Press `F5` or, on the toolbar, click the **Run Script** icon, or on the **Debug** menu, click **Run/Continue** or, in the Console Pane, type `c` and then press `ENTER`. This causes the script to continue running to the next breakpoint or to the end of the script if no further breakpoints are encountered.

To view the call stack

The call stack displays the current run location in the script. If the script is running in a function that was called by a different function, then that's represented in the display by additional rows

in the output. The bottom-most row displays the original script and the line in it in which a function was called. The next line up shows that function and the line in it in which another function might have been called. The top-most row shows the current context of the current line on which the breakpoint is set.

While paused, to see the current call stack, press `CTRL` + `SHIFT` + `D` or, on the **Debug** menu, click **Display Call Stack** or, in the Console Pane, type `K` and then press `ENTER`.

To stop debugging

Press `SHIFT` + `F5` or, on the **Debug** menu, click **Stop Debugger**, or, in the Console Pane, type `Q` and then press `ENTER`.

How to step over, step into, and step out while debugging

Stepping is the process of running one statement at a time. You can stop on a line of code, and examine the values of variables and the state of the system. The following table describes common debugging tasks such as stepping over, stepping into, and stepping out.

Expand table

Debugging Task	Description	How to accomplish it in PowerShell ISE
Step Into	Executes the current statement and then stops at the next statement. If the current statement is a function or script call, then the debugger steps into that function or script, otherwise it stops at the next statement.	Press `F11` or, on the **Debug** menu, click **Step Into**, or in the Console Pane, type `S` and press `ENTER`.
Step Over	Executes the current statement and then stops at the next statement. If the current statement is a function or script call, then the debugger executes the whole function or script, and it stops at the next statement after the function call.	Press `F10` or, on the **Debug** menu, click **Step Over**, or in the Console Pane, type `V` and press `ENTER`.
Step Out	Steps out of the current function and up one level if the function is nested. If in the main body, the script is executed to the end, or to the next breakpoint. The skipped statements are executed, but not stepped through.	Press `SHIFT` + `F11`, or on the **Debug** menu, click **Step Out**, or in the Console Pane, type `O` and press `ENTER`.
Continue	Continues execution to the end, or to the next breakpoint. The skipped functions and invocations are	Press `F5` or, on the **Debug** menu, click **Run/Continue**, or in

Debugging Task	Description	How to accomplish it in PowerShell ISE
	executed, but not stepped through.	the Console Pane, type `C` and press `ENTER`.

How to display the values of variables while debugging

You can display the current values of variables in the script as you step through the code.

To display the values of standard variables

Use one of the following methods:

- In the Script Pane, hover over the variable to display its value as a tool tip.
- In the Console Pane, type the variable name and press `ENTER`.

All panes in ISE are always in the same scope. Therefore, while you are debugging a script, the commands that you type in the Console Pane run in script scope. This allows you to use the Console Pane to find the values of variables and call functions that are defined only in the script.

To display the values of automatic variables

You can use the preceding method to display the value of almost all variables while you are debugging a script. However, these methods don't work for the following automatic variables.

- `$_`
- `$input`
- `$MyInvocation`
- `$PSBoundParameters`
- `$args`

If you try to display the value of any of these variables, you get the value of that variable for in an internal pipeline the debugger uses, not the value of the variable in the script. You can work around this for a few variables (`$_`, `$input`, `$MyInvocation`, `$PSBoundParameters`, and `$args`) by using the following method:

1. In the script, assign the value of the automatic variable to a new variable.

2. Display the value of the new variable, either by hovering over the new variable in the Script Pane, or by typing the new variable in the Console Pane.

For example, to display the value of the `$MyInvocation` variable, in the script, assign the value to a new variable, such as `$scriptName`, and then hover over or type the `$scriptName` variable to display its value.

PowerShell

```
# In C:\ps-test\MyScript.ps1
$scriptName = $MyInvocation.PSCommandPath
```

PowerShell

```
# In the Console Pane:
.\MyScript.ps1
$scriptName
```

Output

```
C:\ps-test\MyScript.ps1
```

See Also

Exploring the Windows PowerShell ISE

Last updated on 11/20/2025

How to Use Profiles in Windows PowerShell ISE

This article explains how to use Profiles in Windows PowerShell® Integrated Scripting Environment (ISE). We recommend that before performing the tasks in this section, you review about_Profiles, or in the Console Pane, type, `Get-Help about_Profiles` and press `ENTER`.

A profile is a Windows PowerShell ISE script that runs automatically when you start a new session. You can create one or more Windows PowerShell profiles for Windows PowerShell ISE and use them to add the configure the Windows PowerShell or Windows PowerShell ISE environment, preparing it for your use, with variables, aliases, functions, and color and font preferences that you want available. A profile affects every Windows PowerShell ISE session that you start.

> **Note**
>
> The Windows PowerShell execution policy determines whether you can run scripts and load a profile. The default execution policy, "Restricted," prevents all scripts from running, including profiles. If you use the "Restricted" policy, the profile can't load. For more information about execution policy, see about_Execution_Policies.

Selecting a profile to use in the Windows PowerShell ISE

Windows PowerShell ISE supports profiles for the current user and all users. It also supports the Windows PowerShell profiles that apply to all hosts.

The profile that you use is determined by how you use Windows PowerShell and Windows PowerShell ISE.

- If you use only Windows PowerShell ISE to run Windows PowerShell, then save all your items in one of the ISE-specific profiles, such as the **CurrentUserCurrentHost** profile for Windows PowerShell ISE or the **AllUsersCurrentHost** profile for Windows PowerShell ISE.
- If you use multiple host programs to run Windows PowerShell, save your functions, aliases, variables, and commands in a profile that affects all host programs, such as the CurrentUserAllHosts or the **AllUsersAllHosts** profile, and save ISE-specific features, like color and font customization in the **CurrentUserCurrentHost** profile for Windows PowerShell ISE profile or the **AllUsersCurrentHost** profile for Windows PowerShell ISE.

The following are profiles that can be created and used in Windows PowerShell ISE. Each profile is saved to its own specific path.

Expand table

Profile Type	Profile Path
Current user, PowerShell ISE	`$PROFILE.CurrentUserCurrentHost`, or `$PROFILE`
All users, PowerShell ISE	`$PROFILE.AllUsersCurrentHost`
Current user, All hosts	`$PROFILE.CurrentUserAllHosts`
All users, All hosts	`$PROFILE.AllUsersAllHosts`

To create a new profile

To create a new "Current user, Windows PowerShell ISE" profile, run this command:

PowerShell

```
if (!(Test-Path -Path $PROFILE )) {
    New-Item -Type File -Path $PROFILE -Force
}
```

To create a new "All users, Windows PowerShell ISE" profile, run this command:

PowerShell

```
if (!(Test-Path -Path $PROFILE.AllUsersCurrentHost)) {
    New-Item -Type File -Path $PROFILE.AllUsersCurrentHost -Force
}
```

To create a new "Current user, All Hosts" profile, run this command:

PowerShell

```
if (!(Test-Path -Path $PROFILE.CurrentUserAllHosts)) {
    New-Item -Type File -Path $PROFILE.CurrentUserAllHosts -Force
}
```

To create a new "All users, All Hosts" profile, type:

PowerShell

```
if (!(Test-Path -Path $PROFILE.AllUsersAllHosts)) {
    New-Item -Type File -Path $PROFILE.AllUsersAllHosts -Force
```

```
}
```

To edit a profile

1. To open the profile, run the command `psEdit` with the variable that specifies the profile you want to edit. For example, to open the "Current user, Windows PowerShell ISE" profile, type: `psEdit $PROFILE`

2. Add some items to your profile. The following are a few examples to get you started:

   - To change the default background color of the Console Pane to blue, in the profile file type: `$psISE.Options.OutputPaneBackground = 'blue'` .

   - To change font size to 20, in the profile file type: `$psISE.Options.FontSize =20`

3. To save your profile file, on the **File** menu, click **Save**. Next time you open the Windows PowerShell ISE, your customizations are applied.

See Also

- about_Profiles
- Introducing the Windows PowerShell ISE

Last updated on 11/20/2025

How to Use Tab Completion in the Script Pane and Console Pane

Tab completion provides automatic help when you are typing in the Script Pane or in the Command Pane. Use the following steps to take advantage of this feature:

To automatically complete a command entry

In the Command Pane or Script Pane, type a few characters of a command and then press `TAB` to select the desired completion text. If multiple items begin with the text that you initially typed, then continue pressing `TAB` until the item you want appears. Tab completion can help with typing a cmdlet name, parameter name, variable name, object property name, or a file path.

> **Note**
>
> In the Script Pane, pressing `TAB` will automatically complete a command only when you are editing `.ps1`, `.psd1`, or `.psm1` files. Tab completion works any time when you are typing in the Command Pane.

To automatically complete a cmdlet parameter entry

In the Command Pane or Script pane, type a cmdlet followed by a dash and then press `TAB`.

For example, type `Get-Process -` and then press `TAB` multiple times to display each of the parameters for the cmdlet in turn.

See Also

- Introducing the Windows PowerShell ISE
- How to Create a PowerShell Tab

Last updated on 11/20/2025

How to Use the Console Pane in the Windows PowerShell ISE

The Console pane in the Windows PowerShell Integrated Scripting Environment (ISE) operates exactly like the stand-alone Windows PowerShell ISE console window.

To run a command in the Console Pane, type a command, and then press `ENTER`. To enter multiple commands that you want to execute in sequence, type `SHIFT` + `ENTER` between commands. See How to Use Tab Completion in the Script Pane and Console Pane for help in typing commands.

To stop a command, on the toolbar, click **Stop Operation**, or press `CTRL` + `BREAK`. You can also use `CTRL` + `C` to stop a command if the context is unambiguous. For example, if some text has been selected in the current Pane, then `CTRL` + `C` maps to the copy operation.

Beginning in Windows PowerShell v3, the Output pane was combined with the Console pane. This has the benefit of behaving like the stand-alone Windows PowerShell console and eliminates the differences in procedures that were needed when they were separate. You can:

- Select and copy text from the Console pane to the Clipboard for pasting in any other window. To select text, click and hold the mouse in the output pane while dragging the mouse over the text you want to capture. You can also use the cursor arrow keys while holding `SHIFT` to select text. Then press `CTRL` + `C` or click the **Copy** icon in the toolbar.
- Paste the selected text at a current cursor position. Click the **Paste** icon on the toolbar.
- Clear all the text in the Console pane. To clear the Console pane, you can click the **Clear Console Pane** icon on the toolbar, or run the command `Clear-Host` or its alias, `cls`.

See Also

- Introducing the Windows PowerShell ISE

Last updated on 11/20/2025

How to Write and Run Scripts in the Windows PowerShell ISE

This article describes how to create, edit, run, and save scripts in the Script Pane.

How to create and run scripts

You can open and edit Windows PowerShell files in the Script Pane. Specific file types of interest in Windows PowerShell are script files (`.ps1`), script data files (`.psd1`), and script module files (`.psm1`). These file types are syntax colored in the Script Pane editor. Other common file types you may open in the Script Pane are configuration files (`.ps1xml`), XML files, and text files.

> **Note**
>
> The Windows PowerShell execution policy determines whether you can run scripts and load Windows PowerShell profiles and configuration files. The default execution policy, Restricted, prevents all scripts from running, and prevents loading profiles. To change the execution policy to allow profiles to load and be used, see Set-ExecutionPolicy and about_Signing.

To create a new script file

On the toolbar, click **New**, or on the **File** menu, click **New**. The created file appears in a new file tab under the current PowerShell tab. Remember that the PowerShell tabs are only visible when there are more than one. By default a file of type script (`.ps1`) is created, but it can be saved with a new name and extension. Multiple script files can be created in the same PowerShell tab.

To open an existing script

On the toolbar, click **Open**, or on the **File** menu, click **Open**. In the **Open** dialog box, select the file you want to open. The opened file appears in a new tab.

To close a script tab

Click the **Close** icon (**X**) of the file tab you want to close or select the **File** menu and click **Close**.

If the file has been altered since it was last saved, you're prompted to save or discard it.

To display the file path

On the file tab, point to the file name. The fully qualified path to the script file appears in a tooltip.

To run a script

On the toolbar, click **Run Script**, or on the **File** menu, click **Run**.

To run a portion of a script

1. In the Script Pane, select a portion of a script.
2. On the **File** menu, click **Run Selection**, or on the toolbar, click **Run Selection**.

To stop a running script

There are several ways to stop a running script.

- Click **Stop Operation** on the toolbar
- Press `CTRL` + `BREAK`
- Select the **File** menu and click **Stop Operation**.

Pressing `CTRL` + `C` also works unless some text is currently selected, in which case `CTRL` + `C` maps to the copy function for the selected text.

How to write and edit text in the Script Pane

You can copy, cut, paste, find, and replace text in the Script Pane. You can also undo and redo the last action you just performed. The keyboard shortcuts for these actions are the same shortcuts used for all Windows applications.

To enter text in the Script Pane

1. Move the cursor to the Script Pane by clicking anywhere in the Script Pane, or by clicking **Go to Script Pane** in the **View** menu.
2. Create a script. Syntax coloring and tab completion provide a richer editing experience in Windows PowerShell ISE.
3. See How to Use Tab Completion in the Script Pane and Console Pane for details about using the tab completion feature to help in typing.

To find text in the Script Pane

1. To find text anywhere, press `CTRL` + `F` or, on the **Edit** menu, click **Find in Script**.
2. To find text after the cursor, press `F3` or, on the **Edit** menu, click **Find Next in Script**.
3. To find text before the cursor, press `SHIFT` + `F3` or, on the **Edit** menu, click **Find Previous in Script**.

To find and replace text in the Script Pane

Press `CTRL` + `H` or, on the **Edit** menu, click **Replace in Script**. Enter the text you want to find and the replacement text, then press `ENTER`.

To go to a particular line of text in the Script Pane

1. In the Script Pane, press `CTRL` + `G` or, on the **Edit** menu, click **Go to Line**.
2. Enter a line number.

To copy text in the Script Pane

1. In the Script Pane, select the text that you want to copy.
2. Press `CTRL` + `C` or, on the toolbar, click the **Copy** icon, or on the **Edit** menu, click **Copy**.

To cut text in the Script Pane

1. In the Script Pane, select the text that you want to cut.
2. Press `CTRL` + `X` or, on the toolbar, click the **Cut** icon, or on the **Edit** menu, click **Cut**.

To paste text into the Script Pane

Press `CTRL` + `V` or, on the toolbar, click the **Paste** icon, or on the **Edit** menu, click **Paste**.

To undo an action in the Script Pane

Press `CTRL` + `Z` or, on the toolbar, click the **Undo** icon, or on the **Edit** menu, click **Undo**.

To redo an action in the Script Pane

Press `CTRL` + `Y` or, on the toolbar, click the **Redo** icon, or on the **Edit** menu, click **Redo**.

How to save a script

An asterisk appears next to the script name to mark a file that hasn't been saved since it was changed. The asterisk disappears when the file is saved.

To save a script

Press `CTRL` + `S` or, on the toolbar, click the **Save** icon, or on the **File** menu, click **Save**.

To save and name a script

1. On the **File** menu, click **Save As**. The **Save As** dialog box will appear.
2. In the **File name** box, enter a name for the file.
3. In the **Save as type** box, select a file type. For example, in the **Save as type** box, select 'PowerShell Scripts (`*.ps1`)'.
4. Click **Save**.

To save a script in ASCII encoding

By default, Windows PowerShell ISE saves new script files (`.ps1`), script data files (`.psd1`), and script module files (`.psm1`) as Unicode (BigEndianUnicode). To save a script in another encoding, such as ASCII (ANSI), use the **Save** or **SaveAs** methods on the `$psISE.CurrentFile` object.

The following command saves a new script as MyScript.ps1 with ASCII encoding.

PowerShell

```
$psISE.CurrentFile.SaveAs("MyScript.ps1", [System.Text.Encoding]::ASCII)
```

The following command replaces the current script file with a file with the same name, but with ASCII encoding.

PowerShell

```
$psISE.CurrentFile.Save([System.Text.Encoding]::ASCII)
```

The following command gets the encoding of the current file.

PowerShell

```
$psISE.CurrentFile.encoding
```

Windows PowerShell ISE supports the following encoding options: ASCII, BigEndianUnicode, Unicode, UTF32, UTF7, UTF8, and Default. The value of the Default option varies with the system.

Windows PowerShell ISE doesn't change the encoding of script files when you use the Save or Save As commands.

See Also

- Exploring the Windows PowerShell ISE

Last updated on 11/20/2025

Keyboard Shortcuts for the Windows PowerShell ISE

Use the following keyboard shortcuts to perform actions in Windows PowerShell Integrated Scripting Environment (ISE). Windows PowerShell ISE is available as part of the Windows Server and Windows client operating systems.

Keyboard shortcuts for editing text

You can use the following keyboard shortcuts when you edit text.

Expand table

Action	Keyboard Shortcuts	Use in
Help	`F1`	Script Pane **Important:** You can specify that `F1` help comes from Microsoft Learn or downloaded Help (see `Update-Help`). To select, click **Tools**, **Options**, then on the **General Settings** tab, set or clear **Use local help content instead of online content.**
Select All	`CTRL` + `A`	Script Pane
Copy	`CTRL` + `C`	Script Pane, Command Pane, Output Pane
Cut	`CTRL` + `X`	Script Pane, Command Pane
Expand or Collapse Outlining	`CTRL` + `M`	Script Pane
Find in Script	`CTRL` + `F`	Script Pane
Find Next in Script	`F3`	Script Pane
Find Previous in Script	`SHIFT` + `F3`	Script Pane
Find Matching Brace	`CTRL` + `]`	Script Pane
Paste	`CTRL` + `V`	Script Pane, Command Pane
Make Lowercase	`CTRL` + `U`	Script Pane, Command Pane

Action	Keyboard Shortcuts	Use in
Make Uppercase	`CTRL` + `SHIFT` + `U`	Script Pane, Command Pane
Redo	`CTRL` + `Y`	Script Pane, Command Pane
Replace in Script	`CTRL` + `H`	Script Pane
Save	`CTRL` + `S`	Script Pane
Select All	`CTRL` + `A`	Script Pane, Command Pane, Output Pane
Show Snippets	`CTRL` + `J`	Script Pane, Command Pane
Undo	`CTRL` + `Z`	Script Pane, Command Pane
Show Intellisense Help	`CTRL` + `Space`	Script Pane
Delete word to left	`CTRL` + `Backspace`	Script Pane
Delete word to right	`CTRL` + `Delete`	Script Pane

Keyboard shortcuts for running scripts

You can use the following keyboard shortcuts when you run scripts in the Script Pane.

Expand table

Action	Keyboard Shortcut
New	`CTRL` + `N`
Open	`CTRL` + `O`
Run	`F5`
Run Selection	`F8`
Stop Execution	`CTRL` + `BREAK`. `CTRL` + `C` can be used when the context is unambiguous (when there is no text selected).

Action	Keyboard Shortcut
Tab (to next script)	`CTRL` + `TAB` **Note:** Tab to next script works only when you have a single Windows PowerShell tab open, or when you have more than one Windows PowerShell tab open, but the focus is in the Script Pane.
Tab (to previous script)	`CTRL` + `SHIFT` + `TAB` **Note:** Tab to previous script works when you have only one Windows PowerShell tab open, or if you have more than one Windows PowerShell tab open, and the focus is in the Script Pane.

Keyboard shortcuts for customizing the view

You can use the following keyboard shortcuts to customize the view in Windows PowerShell ISE. They are accessible from all the panes in the application.

Expand table

Action	Keyboard Shortcut
Go to Command (v2) or Console (v3 and later) Pane	`CTRL` + `D`
Go to Output Pane (v2 only)	`CTRL` + `SHIFT` + `O`
Go to Script Pane	`CTRL` + `I`
Show Script Pane	`CTRL` + `R`
Hide Script Pane	`CTRL` + `R`
Move Script Pane Up	`CTRL` + `1`
Move Script Pane Right	`CTRL` + `2`
Maximize Script Pane	`CTRL` + `3`
Zoom In	`CTRL` + `+`
Zoom Out	`CTRL` + `-`

Keyboard shortcuts for debugging scripts

You can use the following keyboard shortcuts when you debug scripts.

Expand table

Action	Keyboard Shortcut	Use in
Run/Continue	`F5`	Script Pane, when debugging a script
Step Into	`F11`	Script Pane, when debugging a script
Step Over	`F10`	Script Pane, when debugging a script
Step Out	`SHIFT` + `F11`	Script Pane, when debugging a script
Display Call Stack	`CTRL` + `SHIFT` + `D`	Script Pane, when debugging a script
List Breakpoints	`CTRL` + `SHIFT` + `L`	Script Pane, when debugging a script
Toggle Breakpoint	`F9`	Script Pane, when debugging a script
Remove All Breakpoints	`CTRL` + `SHIFT` + `F9`	Script Pane, when debugging a script
Stop Debugger	`SHIFT` + `F5`	Script Pane, when debugging a script

> **Note**
>
> You can also use the keyboard shortcuts designed for the Windows PowerShell console when you debug scripts in Windows PowerShell ISE. To use these shortcuts, you must type the shortcut in the Command Pane and press `ENTER`.

Expand table

Action	Keyboard Shortcut	Use in
Continue	`C`	Console Pane, when debugging a script
Step Into	`S`	Console Pane, when debugging a script
Step Over	`V`	Console Pane, when debugging a script
Step Out	`O`	Console Pane, when debugging a script
Repeat Last Command (for Step Into or Step Over)	`ENTER`	Console Pane, when debugging a script
Display Call Stack	`K`	Console Pane, when debugging a script

Action	Keyboard Shortcut	Use in
Stop Debugging	`Q`	Console Pane, when debugging a script
List the Script	`L`	Console Pane, when debugging a script
Display Console Debugging Commands	`H` or `?`	Console Pane, when debugging a script

Keyboard shortcuts for Windows PowerShell tabs

You can use the following keyboard shortcuts when you use Windows PowerShell tabs.

Expand table

Action	Keyboard Shortcut
Close PowerShell Tab	`CTRL` + `W`
New PowerShell Tab	`CTRL` + `T`
Previous PowerShell tab	`CTRL` + `SHIFT` + `TAB`. This shortcut works only when no files are open on any Windows PowerShell tab.
Next Windows PowerShell tab	`CTRL` + `TAB`. This shortcut works only when no files are open on any Windows PowerShell tab.

Keyboard shortcuts for starting and exiting

You can use the following keyboard shortcuts to exit the Windows PowerShell ISE or to start a new Windows PowerShell session outside of the ISE.

Expand table

Action	Keyboard Shortcut
Exit	`ALT` + `F4` closes the ISE.
Start powershell.exe	`CTRL` + `SHIFT` + `P` opens a new Windows PowerShell session outside of the ISE.

See Also

- PowerShell Magazine: The Complete List of Windows PowerShell ISE Keyboard Shortcuts

---

Last updated on 11/20/2025

Accessibility in Windows PowerShell ISE

This topic describes the accessibility features of Windows PowerShell Integrated Scripting Environment (ISE) that you might find helpful.

- How to change the size and location of the Console and Script Panes
- Keyboard shortcuts for editing text
- Keyboard shortcuts for running scripts
- Keyboard shortcuts for customizing the view
- Keyboard shortcuts for debugging scripts
- Keyboard shortcuts for Windows PowerShell tabs
- Keyboard shortcuts for starting and exiting
- Breakpoint management with cmdlets

Microsoft is committed to making its products and services easier for everyone to use. The following topics provide information about the features, products, and services that make Windows PowerShell ISE more accessible for people with disabilities.

In addition to accessibility features and utilities in Microsoft Windows, the following features make Windows PowerShell ISE more accessible for people with disabilities:

- Keyboard Shortcuts
- Syntax Coloring Table and the ability to modify several other color settings using the `$psISE.Options` scripting object.
- Text Size Change

How to change the size and location of the Console and Script Panes

You can use the following steps to change the size and location of the Console Pane and the Script Pane. When you open the Windows PowerShell ISE again, the size and location changes you made will be retained.

To resize the Script Pane and Console Pane

1. Pause the pointer on the split line between the Script Pane and Console Pane.
2. When the mouse pointer changes to a two-headed arrow, drag the border to change the size of the pane.

To move the Script Pane and Console Pane

Do one of the following:

- To move the Script Pane above the Console Pane, press `CTRL` + `1` or, on the toolbar, click the **Show Script Pane Top** icon, or in the **View** menu, click **Show Script Pane Top**.
- To move the Script Pane to the right of the Console Pane, press `CTRL` + `2` or, on the toolbar, click the **Show Script Pane Right** icon, or in the **View** menu, click **Show Script Pane Right**.
- To maximize the Script Pane, press `CTRL` + `3` or, on the toolbar, click the **Show Script Pane Maximized** icon, or in the **View** menu, click **Show Script Pane Maximized**.
- To maximize the Console Pane and hide the Script Pane, on the far right edge of the row of tabs, click the **Hide Script Pane** icon, in the **View** menu, click to deselect the **Show Script Pane** menu option.
- To display the Script Pane when the Console Pane is maximized, on the far right edge of the row of tabs, click the **Show Script Pane** icon, or in the **View** menu, click to select the **Show Script Pane** menu option.

Keyboard shortcuts for editing text

You can use the following keyboard shortcuts when you edit text.

Expand table

Action	Keyboard Shortcuts	Use in
Copy	`CTRL` + `C`	Script Pane, Console Pane
Cut	`CTRL` + `X`	Script Pane, Console Pane
Find in Script	`CTRL` + `F`	Script Pane
Find Next in Script	`F3`	Script Pane
Find Previous in Script	`SHIFT` + `F3`	Script Pane
Paste	`CTRL` + `V`	Script Pane, Console Pane
Redo	`CTRL` + `Y`	Script Pane, Console Pane
Replace in Script	`CTRL` + `H`	Script Pane
Save	`CTRL` + `S`	Script Pane
Select All	`CTRL` + `A`	Script Pane, Console Pane

Action	Keyboard Shortcuts	Use in
Undo	`CTRL` + `Z`	Script Pane, Console Pane

Keyboard shortcuts for running scripts

You can use the following keyboard shortcuts when you run scripts in the Script Pane.

Expand table

Action	Keyboard Shortcut
New	`CTRL` + `N`
Open	`CTRL` + `O`
Run	`F5`
Run Selection	`F8`
Stop Execution	`CTRL` + `BREAK`. `CTRL` + `C` can be used when the context is unambiguous (when there is no text selected).
Tab (to next script)	`CTRL` + `TAB` **Note:** Tab to next script works only when you have a single PowerShell tab open, or when you have more than one PowerShell tab open, but the focus is in the Script Pane.
Tab (to previous script)	`CTRL` + `SHIFT` + `TAB` **Note:** Tab to previous script works when you have only one PowerShell tab open, or if you have more than one PowerShell tab open, and the focus is in the Script Pane.

Keyboard shortcuts for customizing the view

You can use the following keyboard shortcuts to customize the view in Windows PowerShell ISE. They are accessible from all the panes in the application.

Expand table

Action	Keyboard Shortcut
Go to Console Pane	`CTRL` + `D`
Go to Script Pane	`CTRL` + `I`
Show Script Pane	`CTRL` + `R`

Action	Keyboard Shortcut
Hide Script Pane	`CTRL` + `R`
Move Script Pane Up	`CTRL` + `1`
Move Script Pane Right	`CTRL` + `2`
Maximize Script Pane	`CTRL` + `3`
Zoom In	`CTRL` + `PLUS`
Zoom Out	`CTRL` + `MINUS`

Keyboard shortcuts for debugging scripts

You can use the following keyboard shortcuts when you debug scripts.

Expand table

Action	Keyboard Shortcut	Use in
Run/Continue	`F5`	Script Pane, when debugging a script
Step Into	`F11`	Script Pane, when debugging a script
Step Over	`F10`	Script Pane, when debugging a script
Step Out	`SHIFT` + `F11`	Script Pane, when debugging a script
Display Call Stack	`CTRL` + `SHIFT` + `D`	Script Pane, when debugging a script
List Breakpoints	`CTRL` + `SHIFT` + `L`	Script Pane, when debugging a script
Toggle Breakpoint	`F9`	Script Pane, when debugging a script
Remove All Breakpoints	`CTRL` + `SHIFT` + `F9`	Script Pane, when debugging a script
Stop Debugger	`SHIFT` + `F5`	Script Pane, when debugging a script

> **Note**
>
> You can also use the keyboard shortcuts designed for the Windows PowerShell console when you debug scripts in Windows PowerShell ISE. To use these shortcuts, you must type the shortcut in the Console Pane and press `ENTER`.

Expand table

Action	Keyboard Shortcut	Use in
Continue	`C`	Console Pane, when debugging a script
Step Into	`S`	Console Pane, when debugging a script
Step Over	`V`	Console Pane, when debugging a script
Step Out	`O`	Console Pane, when debugging a script
Repeat Last Command (Step Into/Over)	`ENTER`	Console Pane, when debugging a script
Display Call Stack	`K`	Console Pane, when debugging a script
Stop Debugging	`Q`	Console Pane, when debugging a script
List the Script	`L`	Console Pane, when debugging a script
Display Console Debugging Commands	`H` or `?`	Console Pane, when debugging a script

Keyboard shortcuts for Windows PowerShell tabs

You can use the following keyboard shortcuts when you use Windows PowerShell tabs.

Expand table

Action	Keyboard Shortcut
Close PowerShell Tab	`CTRL` + `W`
New PowerShell Tab	`CTRL` + `T`
Previous PowerShell tab	`CTRL` + `SHIFT` + `TAB` (Only when no files are open on any PowerShell tab)
Next Windows PowerShell tab	`CTRL` + `TAB` (Only when no files are open on any PowerShell tab)

Keyboard shortcuts for starting and exiting

You can use the following keyboard shortcuts to start the Windows PowerShell console (`powershell.exe`) or to exit Windows PowerShell ISE.

Expand table

Action	Keyboard Shortcut
Exit	`ALT` + `F4`
Start powershell.exe (Windows PowerShell console)	`CTRL` + `SHIFT` + `P`

Breakpoint Management

For the visually impaired, breakpoint information is available through the cmdlets for managing breakpoints, such as Get-PSBreakpoint and Set-PSBreakpoint. For more information please see 'How to manage breakpoints' in How to Debug Scripts in the Windows PowerShell ISE.

See Also

Introducing the Windows PowerShell ISE

Last updated on 11/20/2025

Windows Management Framework

08/29/2025

Windows Management Framework (WMF) provides a consistent management interface for Windows. WMF provides a seamless way to manage various versions of Windows client and Windows Server. WMF installer packages contain updates to management functionality and are available for older versions of Windows.

> **Note**
>
> WMF 5.1 is the only supported version of WMF and is included in all currently supported versions of Windows. This information in this article provides a history of WMF versions.

WMF installation adds and/or updates the following features:

- Windows PowerShell
- Windows PowerShell Desired State Configuration (DSC)
- Windows PowerShell Integrated Script Environment (ISE)
- Windows Remote Management (WinRM)
- Windows Management Instrumentation (WMI)
- Windows PowerShell Web Services (Management OData IIS Extension)
- Software Inventory Logging (SIL)
- Server Manager CIM Provider

WMF Release Notes

To learn about the enhancements in Windows PowerShell and other components, see the release notes for each version of WMF:

- WMF 5.1
- WMF 5.0

WMF availability across Windows operating systems

Expand table

OS Version	End of Support	WMF 5.1	WMF 5.0	WMF 4.0	WMF 3.0	WMF 2.0
Windows Server 2022	2031-10-14	Included				
Windows Server 2019	2029-01-09	Included				
Windows Server 2016	2027-01-11	Included				
Windows 11	2025-10-14	Included				
Windows 10	2025-10-14	Included in 1607+	Included			
Windows Server 2012 R2	*Out of support*	Yes	Yes	Included		
Windows 8.1	*Out of support*	Yes	Yes	Included		
Windows Server 2012	*Out of support*	Yes	Yes	Yes	Included	
Windows 8	*Out of support*				Included	
Windows Server 2008 R2 SP1	*Out of support*	Yes	Yes	Yes	Yes	Included
Windows 7 SP1	*Out of support*	Yes	Yes	Yes	Yes	Included
Windows Server 2008 SP2	*Out of support*				Yes	Yes
Windows Vista	*Out of support*					Yes
Windows Server 2003	*Out of support*					Yes
Windows XP	*Out of support*					Yes

- **Included**: The features of the specified version of WMF were shipped in the indicated version of Windows client or Windows Server.
- **Out of support**: Microsoft no longer supports these products. You must upgrade to a supported version. For more information, see the Microsoft Lifecycle Policy page.

> **Note**

The version of WMF that shipped in an operating system is supported for the lifetime of support for that version of the operating system. The standalone installers for WMF 5.0 and older are no longer available or supported.

PowerShell Security

Learn about PowerShell's security features and best practices.

Security features

OVERVIEW

PowerShell security features

Using App Control for Business

HOW-TO GUIDE

Preventing script injection attacks

Securing a restricted PowerShell remoting session

PowerShell remoting

CONCEPT

Running remote commands

Using WS-Management (WSMan) Remoting in PowerShell

Security Considerations for PowerShell Remoting using WinRM

PowerShell Remoting FAQ

HOW-TO GUIDE

Making the second hop in PowerShell Remoting

PowerShell remoting over SSH

Just Enough Administration (JEA)

CONCEPT

Overview

Prerequisites

JEA Role Capabilities

Session configurations

Security considerations

HOW-TO GUIDE

Registering JEA Configurations

Using JEA

Auditing and Reporting on JEA

Using App Control

OVERVIEW

Using App Control for Business

How App Control works with PowerShell

HOW-TO GUIDE

How to use App Control to secure PowerShell

Managing secrets

CONCEPT

Overview of the SecretManagement and SecretStore modules

Understanding the security features of SecretManagement and SecretStore

HOW-TO GUIDE

Managing a SecretStore vault

Use the SecretStore in automation

Use Azure Key Vault in automation

REFERENCE

Microsoft.PowerShell.SecretManagement module

Microsoft.PowerShell.SecretStore module

PowerShell security features

PowerShell has several features designed to improve the security of your scripting environment.

Execution policy

PowerShell's execution policy is a safety feature that controls the conditions under which PowerShell loads configuration files and runs scripts. This feature helps prevent the execution of malicious scripts. You can use a Group Policy setting to set execution policies for computers and users. Execution policies only apply to the Windows platform.

For more information, see about_Execution_Policies.

Use of the SecureString class

PowerShell has several cmdlets that support the use of the `System.Security.SecureString` class. And, as with any .NET class, you can use **SecureString** in your own scripts. However, Microsoft doesn't recommend using **SecureString** for new development. Microsoft recommends that you avoid using passwords and rely on other means to authenticate, such as certificates or Windows authentication.

PowerShell continues to support the **SecureString** class for backward compatibility. Using a **SecureString** is still more secure than using a plain text string. PowerShell still relies on the **SecureString** type to avoid accidentally exposing the contents to the console or in logs. Use **SecureString** carefully, because it can be easily converted to a plain text string. For a full discussion about using **SecureString**, see the System.Security.SecureString class documentation.

Module and script block logging

Module Logging allows you to enable logging for selected PowerShell modules. This setting is effective in all sessions on the computer. PowerShell records pipeline execution events for the specified modules in the Windows PowerShell event log.

Script Block Logging enables logging for the processing of commands, script blocks, functions, and scripts - whether invoked interactively, or through automation. PowerShell logs this information to the **Microsoft-Windows-PowerShell/Operational** event log.

For more information, see the following articles:

- about_Group_Policy_Settings
- about_Logging_Windows
- about_Logging_Non-Windows

AMSI Support

The Windows Antimalware Scan Interface (AMSI) is an API that allows applications to pass actions to an antimalware scanner, such as Windows Defender, to scan for malicious payloads. Beginning with PowerShell 5.1, PowerShell running on Windows 10 (and higher) passes all script blocks to AMSI.

PowerShell 7.3 extends the data it sends to AMSI for inspection. It now includes all .NET method invocations.

For more information about AMSI, see How AMSI helps.

Constrained language mode

ConstrainedLanguage mode protects your system by limiting the cmdlets and .NET types allowed in a PowerShell session. For a full description, see about_Language_Modes.

Application Control

Windows 10 includes two technologies, App Control for Business and AppLocker that you can use to control applications. PowerShell detects if a system wide application control policy is being enforced. The policy applies certain behaviors when running script blocks, script files, or loading module files to prevent arbitrary code execution on the system.

App Control for Business is designed as a security feature under the servicing criteria defined by the Microsoft Security Response Center (MSRC). App Control for Business is the preferred application control system for Windows. For more information about how PowerShell supports AppLocker and App Control for Business, see Use App Control to secure PowerShell.

AppLocker is a legacy application control system that's still supported in and Windows 11. AppLocker isn't a security feature under the servicing criteria defined by MSRC. For more information about servicing criteria, see Microsoft Security Servicing Criteria for Windows.

System Lockdown mode

In PowerShell, System Lockdown mode is an abstraction of the system-wide application control policy enforced by Windows through App Control for Business or AppLocker. When an application control policy is active, PowerShell enters System Lockdown mode. In System Lockdown mode, the application control policy determines the language mode for each runspace.

> **Important**
>
> Without System Lockdown mode, language mode doesn't propagate between runspaces. Each runspace independently queries the Windows application control policy to determine its language mode. Setting the language mode on one runspace doesn't affect other runspaces. Without an active application control policy, new runspaces default to `FullLanguage` mode.

Software Bill of Materials (SBOM)

Beginning with PowerShell 7.2, all install packages contain a Software Bill of Materials (SBOM). The PowerShell team also produces SBOMs for modules that they own but ship independently from PowerShell.

You can find SBOM files in the following locations:

- In PowerShell, find the SBOM at `$PSHOME/_manifest/spdx_2.2/manifest.spdx.json`.
- For modules, find the SBOM in the module's folder under `_manifest/spdx_2.2/manifest.spdx.json`.

The creation and publishing of the SBOM is the first step to modernize Federal Government cybersecurity and enhance software supply chain security. For more information about this initiative, see the blog post Generating SBOMs with SPDX at Microsoft.

Secure data transfer in PowerShell remoting

Prior to PowerShell v7.6-preview5, a `Session_Key` is used to encrypt a **SecureString** before sending it a PowerShell remote session. The PowerShell Remoting Protocol (PSRP) performs a key exchange between client and server when a `SecureString` object needs to be transferred. The exchange involves the following steps:

1. The client side generates a public/private key pair and sends the public key to the server.
2. The server generates a session key for symmetric encryption.

3. The server uses the public key to encrypt the session key and sends it to the client.
4. Both the client and server use the new session key to encrypt a **SecureString** object.

The PowerShell Remoting Protocol (PSRP) uses the `RSAEncryptionPadding.Pkcs1` algorithm during the key exchange. The algorithm is **NOT** secure, so the key exchange doesn't provide any extra security.

> **ⓘ Important**
>
> You must use a secure transport layer to ensure secure data transfer over PSRP.

Beginning in PowerShell v7.6-preview.5, the key exchange was deprecated. The version of PSRP was incremented to v2.4 and includes the following changes:

- The following PSRP messages are deprecated when both client and server are v2.4 or higher:
  - PUBLIC_KEY
  - PUBLIC_KEY_REQUEST
  - ENCRYPTED_SESSION_KEY
- The encryption and decryption steps for `SecureString` are skipped when both client and server are v2.4 or higher.

This change is backward compatible.

- For old clients or servers (v2.3 or lower), the key exchange is still used when needed.
- PSRP can use a named pipe remote sessions when both client and server are on the same machine. Since it's possible for a remote client to connect to named pipe and the data is no longer encrypted with a session key, the named pipe (used for `Enter-PSHostProcess`) rejects the remote client.

Security Servicing Criteria

A security boundary provides a logical separation between the code and data of security domains with different levels of trust. Security features build upon security boundaries to provide robust protection against specific threats. For security features in this category, Microsoft intends to address reported vulnerabilities through servicing.

Security features of PowerShell

- System Lockdown with App Control for Business
- Constrained language mode with App Control for Business

For more information, see the Microsoft Security Servicing Criteria for Windows documentation.

In some cases, a security feature may provide protection against a threat without being able to provide a robust defense. These security features are typically referred to as *defense-in-depth* features or mitigations because they provide additional security but may have by-design limitations that prevent them from fully mitigating a threat. A bypass for a defense-in-depth security feature by itself does not pose a direct risk because an attacker must also have found a vulnerability that affects a security boundary, or they must rely on additional techniques, such as social engineering to achieve the initial stage of a device compromise.

Defense-in-depth features of PowerShell

- Constrained language mode with AppLocker or configured through session configuration or by manually setting `$ExecutionContext.SessionState.LanguageMode`
- System Lockdown with AppLocker
- Execution Policy

Last updated on 07/17/2026

Use App Control to secure PowerShell

Windows 10 includes two technologies, App Control for Business and AppLocker, that you can use to control applications. They allow you to create a lockdown experience to help secure your PowerShell environment.

AppLocker builds on the application control features of Software Restriction Policies. AppLocker allows you to create rules to allow or deny apps for specific users or groups. You identify the apps based on unique properties of the files.

Application Control for Business, introduced in Windows 10 as Windows Defender Application Control (WDAC), allows you to control which drivers and applications are allowed to run on Windows.

Lockdown policy detection

PowerShell detects both AppLocker and App Control for Business system wide policies. AppLocker doesn't have way to query the policy enforcement status. To detect that AppLocker is enforcing a policy, PowerShell creates two temporary files and tries to run them. The filenames use the following name format:

- `$Env:TEMP/__PSScriptPolicyTest_<random-8dot3-name>.ps1`
- `$Env:TEMP/__PSScriptPolicyTest_<random-8dot3-name>.psm1`

App Control for Business is the preferred application control system for Windows. App Control puts the system into System Lockdown mode. System Lockdown mode is the feature that detects the policies and determines if a context needs to be initialized or changed to a specific language mode.

App Control is designed as a security feature under the servicing criteria defined by the Microsoft Security Response Center (MSRC). For more information, see Application Controls for Windows and App Control and AppLocker feature availability.

> ⓘ **Note**
>
> When choosing between App Control or AppLocker, we recommend that you implement application control using App Control for Business rather than AppLocker. Microsoft is no longer investing in AppLocker. AppLocker will only receive security fixes.

App Control policy enforcement

When PowerShell runs under an App Control policy, its behavior changes based on the defined security policy. Under an App Control policy, PowerShell runs trusted scripts and modules allowed by the policy in `FullLanguage` mode. All other scripts and script blocks are untrusted and run in `ConstrainedLanguage` mode. PowerShell throws errors when the untrusted scripts attempt to perform actions that aren't allowed in `ConstrainedLanguage` mode. It can be difficult to know why a script failed to run correctly in `ConstrainedLanguage` mode.

App Control policy auditing

PowerShell 7.4 added a new feature to support App Control policies in **Audit** mode. In audit mode, PowerShell runs the untrusted scripts in `ConstrainedLanguage` mode without errors, but logs messages to the event log instead. The log messages describe what restrictions would apply if the policy were in **Enforce** mode.

History of changes

Windows PowerShell 5.1 was the first version of PowerShell to support App Control. The security features of App Control and AppLocker improve with each new release of PowerShell. The following sections describe how this support changed in each version of PowerShell. The changes are cumulative, so the features described in the later versions include changes from earlier versions.

Changes in PowerShell 7.7

PowerShell 7.7-preview.4 added support for the `FileOnlyEntry` **AppSetting** for PowerShell. This is an extra setting to run PowerShell in a more restricted mode. For more information, see How to use AppControl to secure PowerShell.

Changes in PowerShell 7.4

On Windows, when PowerShell runs under an App Control policy, its behavior changes based on the defined security policy. Under an App Control policy, PowerShell runs trusted scripts and modules allowed by the policy in `FullLanguage` mode. All other scripts and script blocks are untrusted and run in `ConstrainedLanguage` mode. PowerShell throws errors when the untrusted scripts attempt to perform disallowed actions. It's difficult to know why a script fails to run correctly in `ConstrainedLanguage` mode.

PowerShell 7.4 now supports App Control policies in **Audit** mode. In audit mode, PowerShell runs the untrusted scripts in `ConstrainedLanguage` mode but logs messages to the event log instead of throwing errors. The log messages describe what restrictions would apply if the policy were in **Enforce** mode.

Changes in PowerShell 7.3

- PowerShell 7.3 now supports the ability to block or allow PowerShell script files via the App Control API.

Changes in PowerShell 7.2

- There was a corner-case scenario in AppLocker where you only have **Deny** rules and constrained mode isn't used to enforce the policy that allows you to bypass the execution policy. Beginning in PowerShell 7.2, a change was made to ensure AppLocker rules take precedence over a `Set-ExecutionPolicy -ExecutionPolicy Bypass` command.
- PowerShell 7.2 now disallows the use of the `Add-Type` cmdlet in a `NoLanguage` mode PowerShell session on a locked down machine.
- PowerShell 7.2 now disallows scripts from using COM objects in AppLocker system lockdown conditions. Cmdlets that use COM or DCOM internally aren't affected.

Further reading

- For more information about how App Control works and what restrictions it enforces, see How App Control works with PowerShell.
- For more information about securing PowerShell with App Control, see How to use App Control.

Last updated on 08/17/2026

How App Control works with PowerShell

This article explains how **App Control for Business** secures PowerShell and the restrictions it imposes. The secure behavior of PowerShell varies based on the version of Windows and PowerShell you're using.

How PowerShell detects a system lockdown policy

PowerShell detects both **AppLocker** and **App Control for Business** system-wide polices. Microsoft is no longer investing in AppLocker. AppLocker will only receive security fixes. App Control is the preferred application control system for Windows.

Legacy App Control policy enforcement detection

PowerShell uses the legacy App Control `WldpGetLockdownPolicy` API to discover two things:

- System-wide policy enforcement: `None`, `Audit`, `Enforce`
- Individual file policy: `None`, `Audit` (allowed by policy), `Enforce` (not allowed by policy)

All versions of PowerShell (v5.1 - v7.x) support this App Control policy detection.

Latest App Control policy enforcement detection

App Control introduced new APIs in recent versions of Windows. Beginning with version 7.3, PowerShell uses the new `WldpCanExecuteFile` API to decide how a file should be handled. Windows PowerShell 5.1 doesn't support this new API. The new API takes precedence over the legacy API for individual files. However, PowerShell continues to use the legacy API to get the system-wide policy configuration. If the new API isn't available, PowerShell falls back to the old API behavior.

The new API provides the following information for each file:

- `WLDP_CAN_EXECUTE_ALLOWED`
- `WLDP_CAN_EXECUTE_BLOCKED`
- `WLDP_CAN_EXECUTE_REQUIRE_SANDBOX`

PowerShell behavior under lockdown policy

PowerShell can run in both interactive and non-interactive modes.

- In interactive mode, PowerShell is a command-line application that takes users command-line input as commands or scripts to run. Results are displayed back to the user.
- In non-interactive mode, PowerShell loads modules and runs script files without user input. Result data streams are either ignored or redirected to a file.

Interactive mode running under policy enforcement

PowerShell runs commands in `ConstrainedLanguage` mode. This mode prevents interactive users from running certain commands or executing arbitrary code. For more information about the restrictions in this mode, see the PowerShell restrictions under lockdown policy section of this article.

Noninteractive mode running under policy enforcement

When PowerShell runs a script or loads a module, it uses the App Control API to get the policy enforcement for the file.

PowerShell version 7.3 or higher uses the `WldpCanExecuteFile` API if available. This API returns one of the following results:

- `WLDP_CAN_EXECUTE_ALLOWED`: The policy allows the file for use in `FullLanguage` mode with a few restrictions.
- `WLDP_CAN_EXECUTE_BLOCKED`: The policy disallows the file. PowerShell throws an error when the file is run or loaded.
- `WLDP_CAN_EXECUTE_REQUIRE_SANDBOX`: The policy doesn't approve the file, but it can be run or loaded in `ConstrainedLanguage` mode.

In Windows PowerShell 5.1 or if `WldpCanExecuteFile` API isn't available, PowerShell's per file behavior is:

- `None`: The file is run loaded in `FullLanguage` mode with a few restrictions.
- `Audit`: The file is run or loaded in `FullLanguage` mode with no restrictions. In PowerShell 7.4 or higher, the policy logs restriction information to the Windows event logs.
- `Enforce`: The file is run or loaded in `ConstrainedLanguage` mode.

PowerShell restrictions under lockdown policy

When PowerShell detects the system is under an App Control lockdown policy, it applies restrictions even if the script is trusted and running in `FullLanguage` mode. These restrictions

prevent known behaviors of PowerShell that could result in arbitrary code execution on a locked-down system. The lockdown policy enforces the following restrictions:

- Module dot-sourcing with wildcard function export restriction

  Any module that uses script dot-sourcing and exports functions using wildcard names results in an error. Blocking wildcard exports prevents script injection from a malicious user who can plant an untrusted script that gets dot-sourced into a trusted module. The malicious script could then gain access to the trusted module's private functions.

  Security recommendation: Never use script dot-sourcing in a module and always export module functions with explicit names (no wildcard characters).

- Nested module with wildcard function export restriction

  If a parent module exports functions using function name wildcard characters, PowerShell removes any function name in a nested module from the function export list. Blocking wildcard exports from nested modules prevents accidental exporting of dangerous nested functions through wildcard name matching.

  Security recommendation: Always export module functions with explicit names (no wildcard characters).

- Interactive shell parameter type conversion

  When the system is locked down, interactive PowerShell sessions run in `ConstrainedLanguage` mode to prevent arbitrary code execution. Trusted modules loaded into the session run in `FullLanguage` mode. If a trusted module cmdlet uses complex types for its parameters, type conversion during parameter binding can fail if the conversion isn't allowed across trust boundaries. The failure occurs when PowerShell tries to convert a value by running a type constructor. Type constructors aren't allowed to run in `ConstrainedLanguage` mode.

  In this example, parameter binding type conversion is normally allowed, but fails when run in `ConstrainedLanguage` mode. The `ConnectionPort` type constructor isn't allowed:

  PowerShell

  ```
  PS> Create-ConnectionOnPort -Connection 22
  Create-ConnectionOnPort: Cannot bind parameter 'Connection'. Cannot convert the
  "22"
  value of type "System.Int32" to type "ConnectionPort".
  ```

- `Enter-PSHostProcess` cmdlet disallowed

  The `Enter-PSHostProcess` cmdlet is disabled and throws an error if used. This command is used for attach-and-debug sessions. It allows you to connect to any other PowerShell session on the local machine. The cmdlet is disabled to prevent information disclosure and arbitrary code execution.

PowerShell restrictions under constrained language mode

A script or function not approved by the App Control policy is untrusted. When you run an untrusted command, PowerShell either blocks the command from running (new behavior) or runs the command in `ConstrainedLanguage` mode. The following restrictions apply to `ConstrainedLanguage` mode:

- `Add-Type` cmdlet disallowed

  Blocking `Add-Type` prevents the execution of arbitrary .NET code.

- `Import-LocalizedData` cmdlet restricted

  Blocking the **SupportedCommand** parameter of `Import-LocalizedData` prevents the execution of arbitrary code.

- `Invoke-Expression` cmdlet restricted

  All script blocks passed to the `Invoke-Expression` cmdlet are run in `ConstrainedLanguage` mode to prevent arbitrary code execution.

- `New-Object` cmdlet restricted

  The `New-Object` cmdlet is restricted to use only allowed .NET and COM types, to prevent access to untrusted types.

- `ForEach-Object` cmdlet restriction

  Type method invocation for variables passed to the `ForeEach-Object` is disallowed for any .NET type not in the approved list. In general, `ConstrainedLanguage` mode disallows any object method invocation except for approved .NET types to prevent access to untrusted .NET types.

- `Export-ModuleMember` cmdlet restriction

Using `Export-ModuleMember` cmdlet to export functions in a nested module script file where the child module isn't trusted and the parent module is trusted, results in an error. Blocking this scenario prevents a malicious untrusted module from exporting dangerous functions from a trusted module.

- `New-Module` cmdlet restriction

  When you run `New-Module` in a trusted script, any script block provided by the `ScriptBlock` parameter is marked to run in `ConstrainedLanguage` mode to prevent the injection of arbitrary code into a trusted execution context.

- `Configuration` keyword not allowed

  The `Configuration` language keyword isn't allowed in `ConstrainedLanguage` mode to prevent code injection attacks.

- `class` keyword not allowed

  The `class` language keyword isn't allowed in `ConstrainedLanguage` mode to prevent the injection of arbitrary code.

- Script Block processing scope restrictions

  Child script blocks aren't allowed to run in parent script block scopes if the script blocks have different trust levels. For example, you create a child relationship when you dot-source one script into another. Blocking this scenario prevents an untrusted script from getting access to dangerous functions in the trusted script scope.

- Prevent command discovery of untrusted script functions

  PowerShell command discovery doesn't return functions from an untrusted source, such as an untrusted script or module, to a trusted function. Blocking discovery of untrusted commands prevents code injection through command planting.

- Hashtable to object conversion not allowed

  `ConstrainedLanguage` mode blocks hashtable to object conversions in the `Data` sections of PowerShell data (`.psd1`) files to prevent potential code injection attacks.

- Automatic type conversion restricted

  `ConstrainedLanguage` mode blocks automatic type conversion except for a small set of approved safe types to prevent potential code injection attacks.

- Implicit module function export restriction

  If a module doesn't explicitly export functions, PowerShell implicitly exports all defined module functions automatically as a convenience feature. In `ConstrainedLanguage` mode, implicit exports no longer occur when a module is loaded across trust boundaries. Blocking implicit exports prevents unintended exposure of dangerous module functions not meant for public use.

- Script files can't be imported as modules

  PowerShell allows you to import script files (`.ps1`) as a module. All defined functions become publicly accessible. `ConstrainedLanguage` mode blocks importation of script file to prevent unintended exposure of dangerous script functions.

- Setting variables `AllScope` restriction

  `ConstrainedLanguage` mode disables the ability to set `AllScope` on variables. Limiting the scope of variables prevents the variables from interfering with the session state of trusted commands.

- Type method invocation not allowed

  `ConstrainedLanguage` mode doesn't allow method invocation on unapproved types. Blocking methods on unapproved types prevents invocation of .NET type methods that might be dangerous or allow code injection.

- Type property setters not allowed

  `ConstrainedLanguage` mode restricts invocation of property setters on unapproved types. Blocking property setters on unapproved types prevents code injection attacks.

- Type creation not allowed

  `ConstrainedLanguage` mode blocks type creation on unapproved types to block untrusted constructors that could allow code injection.

- Module scope operator not allowed

  `ConstrainedLanguage` mode doesn't allow the use of the module scope operator. For example: `& (Get-Module MyModule) MyFunction`. Blocking the module scope operator prevents access to module private functions and variables.

Further reading

- For more information about PowerShell language modes, see about_Language_Modes.
- For information about how to configure and use App Control, see How to use App Control for PowerShell.

Last updated on 08/17/2026

How to use App Control to secure PowerShell

This article describes how to set up a **App Control for Business** policy. You can configure the policy to enforce or audit the policy's rule. In audit mode, PowerShell behavior doesn't change but it logs Event ID 16387 messages to the `PowerShellCore/Analytic` event log. In enforcement mode, PowerShell applies the policy's restrictions.

This article assumes you're using a test machine so that you can test PowerShell behavior under a machine wide App Control policy before you deploy the policy in your environment.

App Control policies

An App Control policy is described in an XML file that contains information about policy options, files allowed, and signing certificates recognized by the policy. When the policy is applied, only approved files are allowed to load and run. PowerShell either blocks unapproved script files from running or runs them in `ConstrainedLanguage` mode, depending on policy options.

You create and manipulate App Control policy using the **ConfigCI** module, which is available on all supported Windows versions. You can use this Windows PowerShell module in Windows PowerShell 5.1 or in PowerShell 7 through the **Windows Compatibility** layer. It's easier to use this module in Windows PowerShell. The policy you create can be applied to any version of PowerShell.

Steps to create an App Control policy

For testing, you just need to create a default policy and a self signed code signing certificate.

1. Create a default policy

   PowerShell

   ```powershell
   New-CIPolicy -Level PcaCertificate -FilePath .\SystemCIPolicy.xml -UserPEs
   ```

   This command creates a default policy file called `SystemCIPolicy.xml` that allows all Microsoft code-signed files to run.

> ⓘ Note
>
> Running this command can take up to two hours because it must scan the entire test machine.

2. Disable Audit Mode in default policy

   A new policy is always created in `Audit` mode. To test policy enforcement, you need to disable Audit mode when you apply the policy. Edit the `SystemCIPolicy.xml` file using a text editor like `notepad.exe` or Visual Studio Code (VS Code). Comment out the `Audit mode` option.

   XML

   ```
   <!--
   <Rule>
     <Option>Enabled:Audit Mode</Option>
   </Rule>
   -->
   ```

3. Create a self-signed code signing certificate

   You need a code signing certificate to sign any test binaries or script files that you want to run on your test machine. The `New-SelfSignedCertificate` is provided by the **PKI** module. For best results, you should run this command in Windows PowerShell 5.1.

   PowerShell

   ```
   $newSelfSignedCertificateSplat = @{
       DnsName = $Env:COMPUTERNAME
       CertStoreLocation = "Cert:\CurrentUser\My\"
       Type = 'CodeSigningCert'
   }
   $cert = New-SelfSignedCertificate @newSelfSignedCertificateSplat
   Export-Certificate -Cert $cert -FilePath C:\certs\signing.cer
   Import-Certificate -FilePath C:\certs\signing.cer -CertStoreLocation
   "Cert:\CurrentUser\Root\"
   $cert = Get-ChildItem Cert:\CurrentUser\My\ -CodeSigningCert

   dir C:\bin\PowerShell\pwsh.exe | Set-AuthenticodeSignature -Certificate $cert
   ```

4. Add the code signing certificate to the policy

   Use the following command to add the new code signing certificate to the policy.

PowerShell

```
Add-SignerRule -FilePath .\SystemCIPolicy.xml -CertificatePath
C:\certs\signing.cer -User
```

5. Convert the XML policy file to a policy enforcement binary file

   Finally, you need to convert the XML file to a binary file used by App Control to apply a policy.

   PowerShell

   ```
   ConvertFrom-CIPolicy -XmlFilePath .\SystemCIPolicy.xml -BinaryFilePath
   .\SIPolicy.p7b
   ```

6. Apply the App Control policy

   To apply the policy to your test machine, copy the `SIPolicy.p7b` file to the required system location, `C:\Windows\System32\CodeIntegrity`.

   > ⓘ **Note**
   >
   > You must copy some policy definitions to a subfolder such as `C:\Windows\System32\CodeIntegrity\CiPolicies`. For more information, see [App Control Admin Tips & Known Issues][01].

7. Disable the App Control policy

   To disable the policy, rename the `SIPolicy.p7b` file. If you need to do more testing, you can change the name back to reenable the policy.

   PowerShell

   ```
   Rename-Item -Path .\SIPolicy.p7b -NewName .\SIPolicy.p7b.off
   ```

Test using App Control policy auditing

PowerShell 7.4 added a new feature to support App Control policies in **Audit** mode. In audit mode, PowerShell runs the untrusted scripts in `ConstrainedLanguage` mode without errors, but logs messages to the event log instead. The log messages describe what restrictions would apply if the policy were in **Enforce** mode.

Viewing audit events

PowerShell logs audit events to the **PowerShellCore/Analytic** event log. The log isn't enabled by default. To enable the log, open the **Windows Event Viewer**, right-click on the **PowerShellCore/Analytic** log and select **Enable Log**.

Alternatively, you can run the following command from an elevated PowerShell session.

PowerShell

```
wevtutil.exe sl PowerShellCore/Analytic /enabled:true /quiet
```

You can view the events in the Windows Event Viewer or use the `Get-WinEvent` cmdlet to retrieve the events.

PowerShell

```
Get-WinEvent -LogName PowerShellCore/Analytic -Oldest |
    Where-Object Id -EQ 16387 | Format-List
```

Output

```
TimeCreated  : 4/19/2023 10:11:07 AM
ProviderName : PowerShellCore
Id           : 16387
Message      : App Control Audit.

    Title: Method or Property Invocation
    Message: Method or Property 'WriteLine' on type 'System.Console' invocation will
not
        be allowed in ConstrainedLanguage mode.
        At C:\scripts\Test1.ps1:3 char:1
        + [System.Console]::WriteLine("pwnd!")
        + ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    FullyQualifiedId: MethodOrPropertyInvocationNotAllowed
```

The event message includes the script position where the restriction would be applied. This information helps you understand where you need to change your script so that it runs under the App Control policy.

> **ⓘ Important**
>
> After you review the audit events, you should disable the Analytic log. Analytic logs grow quickly and consume large amounts of disk space.

Viewing audit events in the PowerShell debugger

If you set the `$DebugPreference` variable to `Break` for an interactive PowerShell session, PowerShell breaks into the command-line script debugger at the current location in the script where the audit event occurred. The breakpoint allows you to debug your code and inspect the current state of the script in real time.

Additional configuration settings for PowerShell

App Control supports additional configuration settings for PowerShell.

- `BlockScriptOnPolicyFailure`: This setting blocks the execution of scripts that fail to meet the App Control policy requirements. When enabled, App Control blocks any script that's not explicitly allowed by the policy rather than running it in **ConstrainedLanguage** mode.

  > ⓘ **Note**
  >
  > This setting is enforced by App Control and applies to all versions of PowerShell (`powershell.exe` and `pwsh.exe`).

- `FileOnlyEntry`: This setting prevents PowerShell from running with arguments that don't result in the execution of a specified file. When enabled, App Control blocks the execution of PowerShell commands that are passed via the pipeline, to the `-Command` or `-EncodedCommand` parameters, or interactive sessions that allow direct command execution. When enabled, PowerShell only runs scripts that are specified with the `-File` parameter.

  Examples of blocked executions:

  PowerShell

  ```
  pwsh
  pwsh -NoExit -File ./file.ps1
  pwsh -Command Get-ChildItem
  "Get-ChildItem" | pwsh -File -
  pwsh -EncodedCommand <base64-string>
  ```

  Examples of allowed executions:

  PowerShell

  ```
  pwsh -File ./file.ps1
  ```

```
pwsh -NoProfile -NonInteractive -File ./file.ps1 arg1 arg2
```

> **Note**
>
> This setting only applies to PowerShell 7.6.6 and newer versions. If you configure this setting for earlier versions, the **AppSetting** is silently ignored.

To use these settings, you must add them to the App Control policy XML file (`SystemCIPolicy.xml`). Open the XML file in a text editor and add the `<AppSettings>` section of the policy. The following example shows the `<AppSettings>` XML with both settings enabled.

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<SiPolicy xmlns="urn:schemas-microsoft-com:sipolicy">

    <!-- Existing policy content -->

    <AppSettings>
        <AppSetting
            AppManifest="Powershell"
            Name="BlockScriptOnPolicyFailure"
            Value="true" />

        <AppSetting
            AppManifest="Powershell"
            Name="FileOnlyEntry"
            Value="true" />
    </AppSettings>

</SiPolicy>
```

Further reading

- Create an App Control policy for fully managed devices
- Understand App Control script enforcement
- App Control Admin Tips & Known Issues

Last updated on 09/24/2026

Preventing script injection attacks

PowerShell scripts, like other programming languages, can be vulnerable to injection attacks. An injection attack occurs when a user provides input to a vulnerable function that includes extra commands. The vulnerable function runs the extra commands, which can be a serious security vulnerability. For example, a malicious user could abuse the vulnerable function to run arbitrary code on a remote computer, possibly compromising that computer and gaining access to other machines on the network.

Once you're aware of the issue, there are several ways to protect against injection attacks.

Example of vulnerable code

PowerShell code injection vulnerabilities involve user input that contains script code. The user input is added to vulnerable script where PowerShell parses and runs it.

PowerShell

```powershell
function Get-ProcessById
{
    param ($ProcId)

    Invoke-Expression -Command "Get-Process -Id $ProcId"
}
```

The `Get-ProcessById` function looks up a local process by its Id value. It takes a `$ProcId` parameter argument of any type. The `$ProcId` is then converted to a string and inserted into another script. `Invoke-Expression` cmdlet parses and runs the provided string. This function works fine when a valid process Id integer is passed in.

PowerShell

```powershell
Get-ProcessById $PID
```

```
 NPM(K)    PM(M)      WS(M)     CPU(s)      Id  SI ProcessName
 ------    -----      -----     ------      --  -- -----------
     97    50.09     132.72       1.20   12528   3 pwsh
```

However, the `$ProcId` parameter doesn't specify a type. It accepts any arbitrary string value that can include other commands.

PowerShell

```powershell
Get-ProcessById "$PID; Write-Host 'pwnd!'"
```

In this example, the function correctly retrieved the process identified by `$PID`, but also ran the injected script `Write-Host 'pwnd!'`.

Output

```
 NPM(K)    PM(M)      WS(M)     CPU(s)      Id  SI ProcessName
 ------    -----      -----     ------      --  -- -----------
     92    45.66     122.52       1.06   21736   3 pwsh
pwnd!
```

Ways to guard against injection attacks

The are several ways to guard against an injection attack.

Use typed input

You can specify a type for the `$ProcId` argument.

PowerShell

```powershell
function Get-ProcessById
{
    param ([int] $ProcId)

    Invoke-Expression -Command "Get-Process -Id $ProcId"
}
Get-ProcessById "$PID; Write-Host 'pwnd!'"
```

Output

```
Get-ProcessById:
Line |
   7 |  Get-ProcessById "$PID; Write-Host 'pwnd!'"
     |                  ~~~~~~~~~~~~~~~~~~~~~~~~~~
     | Cannot process argument transformation on parameter 'ProcId'. Cannot convert
value
"8064; Write-Host 'pwnd!'" to type "System.Int32". Error: "The input string '8064;
Write-Host 'pwnd!'
was not in a correct format."
```

Here, the `$ProcId` input parameter is restricted to an integer type, so an error occurs when a string is passed in that can't be converted to an integer.

Don't use `Invoke-Expression`

Instead of using `Invoke-Expression`, directly call `Get-Process`, and let PowerShell's parameter binder validate the input.

PowerShell

```powershell
function Get-ProcessById
{
    param ($ProcId)

    Get-Process -Id $ProcId
}
Get-ProcessById "$PID; Write-Host 'pwnd!'"
```

Output

```
Get-Process:
Line |
   5 |      Get-Process -Id $ProcId
     |                      ~~~~~~~
     | Cannot bind parameter 'Id'. Cannot convert value "8064; Write-Host 'pwnd!'"
to type
"System.Int32". Error: "The input string '8064; Write-Host 'pwnd!' was not in a
correct
format."
```

As a best practice, you should avoid using `Invoke-Expression`, especially when handling user input. `Invoke-Expression` is dangerous because it parses and runs whatever string content you provide, making it vulnerable to injection attacks. It's better to rely on PowerShell parameter binding.

Wrap strings in single quotes

However, there are times when using `Invoke-Expression` is unavoidable and you also need to handle user string input. You can safely handle user input using single quotes around each string input variable. The single quote ensures that PowerShell's parser treats the user input as a single string literal.

PowerShell

```powershell
function Get-ProcessById
{
    param ($ProcId)

    Invoke-Expression -Command "Get-Process -Id '$ProcId'"
}

Get-ProcessById "$PID; Write-Host 'pwnd!'"
```

Output

```
Get-Process: Cannot bind parameter 'Id'. Cannot convert value "8064; Write-Host " to
type
"System.Int32". Error: "The input string '8064; Write-Host' was not in a correct
format."
```

However, this version of the function isn't safe from injection attacks. A malicious user can still use single quotes in their input to inject code.

PowerShell

```powershell
Get-ProcessById "$PID'; Write-Host 'pwnd!';'"
```

This example uses single quotes in the user input to force the function to run three separate statements, one of which is arbitrary code injected by the user.

Output

```
 NPM(K)    PM(M)      WS(M)     CPU(s)      Id  SI ProcessName
 ------    -----      -----     ------      --  -- -----------
     97    46.08     183.10       1.08    2524   3 pwsh
pwnd!
```

Use the `EscapeSingleQuotedStringContent()` method

To protect against the user inserting their own single quote characters to exploit the function, you must use the `EscapeSingleQuotedStringContent()` API. `EscapeSingleQuotedStringContent()` is a public static method of the PowerShell **System.Management.Automation.Language.CodeGeneration** class. This method makes the user-provided input safe by escaping any single quotes included in the user input.

PowerShell

```powershell
function Get-ProcessById
{
    param ($ProcId)

    $ProcIdClean = [System.Management.Automation.Language.CodeGeneration]::
        EscapeSingleQuotedStringContent("$ProcId")
    Invoke-Expression -Command "Get-Process -Id '$ProcIdClean'"
}
Get-ProcessById "$PID'; Write-Host 'pwnd!';'"
```

Output

```
Get-Process: Cannot bind parameter 'Id'. Cannot convert value "8064'; Write-Host
'pwnd!';'" to type
"System.Int32". Error: "The input string '8064'; Write-Host 'pwnd!';'' was not in a
correct format."
```

For more information, see EscapeSingleQuotedStringContent().

Detecting vulnerable code with Injection Hunter

Injection Hunter is a module written by Lee Holmes that contains PowerShell Script Analyzer rules for detecting code injection vulnerabilities. Use one of the following commands to install the module from the PowerShell Gallery:

PowerShell

```powershell
# Use PowerShellGet v2.x
Install-Module InjectionHunter

# Use PowerShellGet v3.x
Install-PSResource InjectionHunter
```

You can use this module to automate security analysis during builds, continuous integration processes, deployments, and other scenarios.

PowerShell

```powershell
$RulePath = (Get-Module -List InjectionHunter).Path
Invoke-ScriptAnalyzer -CustomRulePath $RulePath -Path .\Invoke-Dangerous.ps1
```

Output

```
RuleName                            Severity     ScriptName Line  Message
--------                            --------     ---------- ----  -------
InjectionRisk.InvokeExpression      Warning      Invoke-Dan 3     Possible script
injection risk via the
                                                 gerous.ps1       Invoke-Expression
cmdlet. Untrusted input can cause
                                                                  arbitrary
PowerShell expressions to be run.
                                                                  Variables may be
used directly for dynamic parameter
                                                                  arguments,
splatting can be used for dynamic
                                                                  parameter names,
and the invocation operator can be
                                                                  used for dynamic
command names. If content escaping
                                                                  is truly needed,
```

```
PowerShell has several valid quote
                                                                characters, so
[System.Management.Automation.Languag

e.CodeGeneration]::Escape* should be used.
```

For more information, see PSScriptAnalyzer.

Related links

- Lee Holmes' blog post about Injection Hunter
- Injection Hunter

Last updated on 12/09/2025

Just Enough Administration

Article • 04/01/2024

Just Enough Administration (JEA) is a security technology that enables delegated administration for anything managed by PowerShell. With JEA, you can:

- **Reduce the number of administrators on your machines** using virtual accounts or group-managed service accounts to perform privileged actions on behalf of regular users.
- **Limit what users can do** by specifying which cmdlets, functions, and external commands they can run.
- **Better understand what your users are doing** with transcripts and logs that show you exactly which commands a user executed during their session.

Why is JEA important?

Highly privileged accounts used to administer your servers pose a serious security risk. Should an attacker compromise one of these accounts, they could launch lateral attacks across your organization. Each compromised account gives an attacker access to even more accounts and resources, and puts them one step closer to stealing company secrets, launching a denial-of-service attack, and more.

It's not always easy to remove administrative privileges, either. Consider the common scenario where the DNS role is installed on the same machine as your Active Directory Domain Controller. Your DNS administrators require local administrator privileges to fix issues with the DNS server. But to do so, you must make them members of the highly privileged **Domain Admins** security group. This approach effectively gives DNS Administrators control over your whole domain and access to all resources on that machine.

JEA addresses this problem through the principle of **Least Privilege**. With JEA, you can configure a management endpoint for DNS administrators that gives them access only to the PowerShell commands they need to get their job done. This means you can provide the appropriate access to repair a poisoned DNS cache or restart the DNS server without unintentionally giving them rights to Active Directory, or to browse the file system, or run potentially dangerous scripts. Better yet, when the JEA session is configured to use temporary privileged virtual accounts, your DNS administrators can connect to the server using **non-admin** credentials and still run commands that typically require admin privileges. JEA enables you to remove users from widely privileged local/domain administrator roles and carefully control what they can do on each machine.

Next steps

To learn more about the requirements to use JEA, see the Prerequisites article.

Samples and DSC resource

Sample JEA configurations and the JEA DSC resource can be found in the JEA GitHub repository.

JEA Prerequisites

Article • 04/01/2024

Just Enough Administration is a feature included in PowerShell 5.0 and higher. This article describes the prerequisites that must be satisfied to start using JEA.

Check which version of PowerShell is installed

To check which version of PowerShell is installed on your system, check the `$PSVersionTable` variable in a Windows PowerShell prompt.

PowerShell

```
$PSVersionTable.PSVersion
```

Output

```
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      14393  1000
```

JEA is available with PowerShell 5.0 and higher. For full functionality, it's recommended that you install the latest version of PowerShell available for your system. The following table describes JEA's availability on Windows Server:

Expand table

Server Operating System	JEA Availability
Windows Server 2016+	Preinstalled
Windows Server 2012 R2	Full functionality with WMF 5.1
Windows Server 2012	Full functionality with WMF 5.1
Windows Server 2008 R2	Reduced functionality[1] with WMF 5.1

You can also use JEA on your home or work computer:

Expand table

Client Operating System	JEA Availability
Windows 10 1607+	Preinstalled
Windows 10 1603, 1511	Preinstalled, with reduced functionality[2]
Windows 10 1507	Not available
Windows 8, 8.1	Full functionality with WMF 5.1
Windows 7	Reduced functionality[1] with WMF 5.1

- [1] JEA can't be configured to use group-managed service accounts on Windows Server 2008 R2 or Windows 7. Virtual accounts and other JEA features *are* supported.
- [2] The following JEA features aren't supported on Windows 10 versions 1511 and 1603:
  - Running as a group-managed service account
  - Conditional access rules in session configurations
  - The user drive
  - Granting access to local user accounts

  To get support for these features, update Windows to version 1607 (Anniversary Update) or higher.

Install Windows Management Framework

If you're running an older version of PowerShell, you may need to update your system with the latest Windows Management Framework (WMF) update. For more information, see the WMF documentation.

It's recommended that you test your workload's compatibility with WMF before upgrading all of your servers.

Windows 10 users should install the latest feature updates to obtain the current version of Windows PowerShell.

Enable PowerShell Remoting

PowerShell Remoting provides the foundation on which JEA is built. It's necessary to ensure PowerShell Remoting is enabled and properly secured before you can use JEA. For more information, see WinRM Security.

PowerShell Remoting is enabled by default on Windows Server 2012 and higher. You can enable PowerShell Remoting by running the following command in an elevated PowerShell window.

```PowerShell
Enable-PSRemoting
```

Enable PowerShell module and script block logging (optional)

The following steps enable logging for all PowerShell actions on your system. PowerShell Module Logging isn't required for JEA, however it's recommended you turn on logging to ensure the commands users run are logged in a central location.

You can configure the PowerShell Module Logging policy using Group Policy.

1. Open the Local Group Policy Editor on a workstation or a Group Policy Object in the Group Policy Management Console on an Active Directory Domain Controller
2. Navigate to **Computer Configuration\Administrative Templates\Windows Components\Windows PowerShell**
3. Double-click on **Turn on Module Logging**
4. Click **Enabled**
5. In the Options section, click on **Show** next to Module Names
6. Type `*` in the pop-up window to log commands from all modules.
7. Click **OK** to set the policy
8. Double-click on **Turn on PowerShell Script Block Logging**
9. Click **Enabled**
10. Click **OK** to set the policy
11. (On domain-joined machines only) Run `gpupdate` or wait for Group Policy to process the updated policy and apply the settings

You can also enable system-wide PowerShell transcription through Group Policy.

Next steps

- Create a role capability file
- Create a session configuration file

See also

- WinRM Security
- PowerShell ♥ the Blue Team

JEA Role Capabilities

When creating a JEA endpoint, you need to define one or more role capabilities that describe what someone can do in a JEA session. A role capability is a PowerShell data file with the `.psrc` extension that lists all the cmdlets, functions, providers, and external programs that are made available to connecting users.

Determine which commands to allow

The first step in creating a role capability file is to consider what the users need access to. The requirements gathering process can take a while, but it's important. Giving users access to too few cmdlets and functions can prevent them from getting their job done. Allowing access to too many cmdlets and functions can allow users to do more than you intended and weaken your security stance.

How you go about this process depends on your organization and goals. The following tips can help ensure you're on the right path.

1. **Identify** the commands users are using to get their jobs done. This may involve surveying IT staff, checking automation scripts, or analyzing PowerShell session transcripts and logs.
2. **Update** use of command-line tools to PowerShell equivalents, where possible, for the best auditing and JEA customization experience. External programs can't be constrained as granularly as native PowerShell cmdlets and functions in JEA.
3. **Restrict** the scope of the cmdlets to only allow specific parameters or parameter values. This is especially important if users should manage only part of a system.
4. **Create** custom functions to replace complex commands or commands that are difficult to constrain in JEA. A simple function that wraps a complex command or applies additional validation logic can offer additional control for admins and end-user simplicity.
5. **Test** the scoped list of allowable commands with your users or automation services, and adjust as necessary.

Examples of potentially dangerous commands

Careful selection of commands is important to ensure the JEA endpoint doesn't allow the user to elevate their permissions.

> **Important**

Essential information required for user successCommands in a JEA session are often run with elevated privileges.

The following list contains examples of commands that can be used maliciously if allowed in an unconstrained state. This isn't an exhaustive list and should only be used as a cautionary starting point.

- **Risk:** Granting the connecting user admin privileges to bypass JEA

  Example:

  ```PowerShell
  Add-LocalGroupMember -Member 'CONTOSO\jdoe' -Group 'Administrators'
  ```

  Related commands:

  - `Add-ADGroupMember`
  - `Add-LocalGroupMember`
  - `net.exe`
  - `dsadd.exe`

- **Risk:** Running arbitrary code, such as malware, exploits, or custom scripts to bypass protections

  Example:

  ```PowerShell
  Start-Process -FilePath '\\san\share\malware.exe'
  ```

  Related commands:

  - `Start-Process`
  - `New-Service`
  - `Invoke-Item`
  - `Invoke-WmiMethod`
  - `Invoke-CimMethod`
  - `Invoke-Expression`
  - `Invoke-Command`
  - `New-ScheduledTask`
  - `Register-ScheduledJob`

Create a role capability file

You can create a new PowerShell role capability file with the New-PSRoleCapabilityFile cmdlet.

PowerShell

```
New-PSRoleCapabilityFile -Path .\MyFirstJEARole.psrc
```

You should edit the created role capability file to allow only the commands required for the role. The PowerShell help documentation contains several examples of how you can configure the file.

Allowing PowerShell cmdlets and functions

To authorize users to run PowerShell cmdlets or functions, add the cmdlet or function name to the **VisibleCmdlets** or **VisibleFunctions** fields. If you aren't sure whether a command is a cmdlet or function, you can run `Get-Command <name>` and check the **CommandType** property in the output.

PowerShell

```
VisibleCmdlets = @('Restart-Computer', 'Get-NetIPAddress')
```

Sometimes the scope of a specific cmdlet or function is too broad for your users' needs. A DNS admin, for example, may only need access to restart the DNS service. In multi-tenant environments, tenants have access to self-service management tools. Tenants should be limited to managing their own resources. For these cases, you can restrict which parameters are exposed from the cmdlet or function.

PowerShell

```
VisibleCmdlets = @{
    Name       = 'Restart-Computer'
    Parameters = @{ Name = 'Name' }
}
```

In more advanced scenarios, you may also need to restrict the values a user may use with these parameters. Role capabilities let you define a set of values or a regular expression pattern that determine what input is allowed.

PowerShell

```
VisibleCmdlets = @(
    @{
        Name       = 'Restart-Service'
        Parameters = @{ Name = 'Name'; ValidateSet = @('Dns', 'Spooler') }
    }
    @{
        Name       = 'Start-Website'
        Parameters = @{ Name = 'Name'; ValidatePattern = 'HR_*' }
    }
)
```

> **Note**
>
> The common PowerShell parameters are always allowed, even if you restrict the available parameters. You shouldn't explicitly list them in the Parameters field.

The list below describes the various ways you can customize a visible cmdlet or function. You can mix and match any of the below in the **VisibleCmdlets** field.

- **Use case:** Allow the user to run `My-Func` without any restrictions on the parameters.

  PowerShell
  ```
  @{ Name = 'My-Func' }
  ```

- **Use case:** Allow the user to run `My-Func` from the module **MyModule** without any restrictions on the parameters.

  PowerShell
  ```
  @{ Name = 'MyModule\My-Func' }
  ```

- **Use case:** Allow the user to run any cmdlet or function with the verb `My`.

  PowerShell
  ```
  @{ Name = 'My-*' }
  ```

- **Use case:** Allow the user to run any cmdlet or function with the noun `Func`.

  PowerShell
  ```
  @{ Name = '*-Func' }
  ```

- **Use case:** Allow the user to run `My-Func` with the `Param1` and `Param2` parameters. Any value can be supplied to the parameters.

  ```PowerShell
  @{ Name = 'My-Func'; Parameters = @{ Name = 'Param1'}, @{ Name = 'Param2' }}
  ```

- **Use case:** Allow the user to run `My-Func` with the `Param1` parameter. Only `Value1` and `Value2` can be supplied to the parameter.

  ```PowerShell
  @{
      Name       = 'My-Func'
      Parameters = @{ Name = 'Param1'; ValidateSet = @('Value1', 'Value2') }
  }
  ```

- **Use case:** Allow the user to run `My-Func` with the `Param1` parameter. Any value starting with `contoso` can be supplied to the parameter.

  ```PowerShell
  @{
      Name       = 'My-Func'
      Parameters = @{ Name = 'Param1'; ValidatePattern = 'contoso.*' }
  }
  ```

> ⚠ **Warning**
>
> For best security practices, it isn't recommended to use wildcards when defining visible cmdlets or functions. Instead, you should explicitly list each trusted command to ensure no other commands that share the same naming scheme are unintentionally authorized.

You can't apply both a **ValidatePattern** and **ValidateSet** to the same cmdlet or function.

If you do, the **ValidatePattern** overrides the **ValidateSet**.

For more information about **ValidatePattern**, check out this *Hey, Scripting Guy!* post and the PowerShell Regular Expressions reference content.

Allowing external commands and PowerShell scripts

To allow users to run executables and PowerShell scripts (`.ps1`) in a JEA session, you have to add the full path to each program in the **VisibleExternalCommands** field.

PowerShell

```
VisibleExternalCommands = @(
    'C:\Windows\System32\whoami.exe'
    'C:\Program Files\Contoso\Scripts\UpdateITSoftware.ps1'
)
```

Where possible, use PowerShell cmdlet or function equivalents for any external executables you authorize since you have control over the parameters allowed with PowerShell cmdlets and functions.

Many executables allow you to read the current state and then change it by providing different parameters.

For example, consider the role of a file server admin that manages network shares hosted on a system. One way of managing shares is to use `net share`. However, allowing `net.exe` is dangerous because the user could use the command to gain admin privileges with the command `net group Administrators unprivilegedjeauser /add`. A more secure option is to allow the Get-SmbShare cmdlet, which achieves the same result but has a much more limited scope.

When making external commands available to users in a JEA session, always specify the complete path to the executable. This prevents the execution of similarly named and potentially malicious programs located elsewhere on the system.

Allowing access to PowerShell providers

By default, no PowerShell providers are available in JEA sessions. This reduces the risk of sensitive information and configuration settings being disclosed to the connecting user.

When necessary, you can allow access to the PowerShell providers using the `VisibleProviders` command. For a full list of providers, run `Get-PSProvider`.

PowerShell

```
VisibleProviders = 'Registry'
```

For simple tasks that require access to the file system, registry, certificate store, or other sensitive providers, consider writing a custom function that works with the provider on the

user's behalf. The functions, cmdlets, and external programs available in a JEA session aren't subject to the same constraints as JEA. They can access any provider by default. Also consider using the user drive when users need to copy files to or from a JEA endpoint.

Creating custom functions

You can author custom functions in a role capability file to simplify complex tasks for your end users. Custom functions are also useful when you require advanced validation logic for cmdlet parameter values. You can write simple functions in the **FunctionDefinitions** field:

PowerShell

```
VisibleFunctions = 'Get-TopProcess'

FunctionDefinitions = @{
    Name        = 'Get-TopProcess'
    ScriptBlock = {
        param($Count = 10)

        Get-Process |
            Sort-Object -Property CPU -Descending |
            Microsoft.PowerShell.Utility\Select-Object -First $Count
    }
}
```

> **Important**
>
> Don't forget to add the name of your custom functions to the **VisibleFunctions** field so they can be run by the JEA users.

The body (script block) of custom functions runs in the default language mode for the system and isn't subject to JEA's language constraints. This means that functions can access the file system and registry, and run commands that weren't made visible in the role capability file. Take care to avoid running arbitrary code when using parameters. Avoid piping user input directly into cmdlets like `Invoke-Expression`.

In the above example, notice that the fully qualified module name (FQMN) `Microsoft.PowerShell.Utility\Select-Object` was used instead of the shorthand `Select-Object`. Functions defined in role capability files are still subject to the scope of JEA sessions, which includes the proxy functions JEA creates to constrain existing commands.

By default, `Select-Object` is a constrained cmdlet in all JEA sessions that doesn't allow the selection of arbitrary properties on objects. To use the unconstrained `Select-Object` in functions, you must explicitly request the full implementation using the FQMN. Any constrained cmdlet in a JEA session has the same constraints when invoked from a function. For more information, see about_Command_Precedence.

If you're writing several custom functions, it's more convenient to put them in a PowerShell script module. You make those functions visible in the JEA session using the **VisibleFunctions** field like you would with built-in and third-party modules.

For tab completion to work properly in JEA sessions you must include the built-in function `TabExpansion2` in the **VisibleFunctions** list.

Make the role capabilities available to a configuration

Prior to PowerShell 6, for PowerShell to find a role capability file it must be stored in a `RoleCapabilities` folder in a PowerShell module. The module can be stored in any folder included in the `$Env:PSModulePath` environment variable, however you shouldn't place it in `$Env:SystemRoot\System32` or a folder where untrusted users could modify the files.

The following example creates a PowerShell script module called **ContosoJEA** in the `$Env:ProgramFiles` path to host the role capabilities file.

PowerShell

```
# Create a folder for the module
$modulePath = Join-Path $Env:ProgramFiles "WindowsPowerShell\Modules\ContosoJEA"
New-Item -ItemType Directory -Path $modulePath

# Create an empty script module and module manifest.
# At least one file in the module folder must have the same name as the folder
itself.
$rootModulePath = Join-Path $modulePath "ContosoJEAFunctions.psm1"
$moduleManifestPath = Join-Path $modulePath "ContosoJEA.psd1"
New-Item -ItemType File -Path $RootModulePath
New-ModuleManifest -Path $moduleManifestPath -RootModule "ContosoJEAFunctions.psm1"

# Create the RoleCapabilities folder and copy in the PSRC file
$rcFolder = Join-Path $modulePath "RoleCapabilities"
New-Item -ItemType Directory $rcFolder
Copy-Item -Path .\MyFirstJEARole.psrc -Destination $rcFolder
```

For more information about PowerShell modules, see Understanding a PowerShell Module.

Starting in PowerShell 6, the **RoleDefinitions** property was added to the session configuration file. This property lets you specify the location of a role configuration file for your role definition. See the examples in New-PSSessionConfigurationFile.

Updating role capabilities

You can edit a role capability file to update the settings at any time. Any new JEA sessions started after the role capability has been updated will reflect the revised capabilities.

This is why controlling access to the role capabilities folder is so important. Only highly trusted administrators should be allowed to change role capability files. If an untrusted user can change role capability files, they can easily give themselves access to cmdlets that allow them to elevate their privileges.

For administrators looking to lock down access to the role capabilities, ensure Local System has read-only access to the role capability files and containing modules.

How role capabilities are merged

Users are granted access to all matching role capabilities in the session configuration file when they enter a JEA session. JEA tries to give the user the most permissive set of commands allowed by any of the roles.

VisibleCmdlets and VisibleFunctions

The most complex merge logic affects cmdlets and functions, which can have their parameters and parameter values limited in JEA.

The rules are as follows:

1. If a cmdlet is only made visible in one role, it's visible to the user with any applicable parameter constraints.
2. If a cmdlet is made visible in more than one role, and each role has the same constraints on the cmdlet, the cmdlet is visible to the user with those constraints.
3. If a cmdlet is made visible in more than one role, and each role allows a different set of parameters, the cmdlet and all the parameters defined across every role are visible to the user. If one role doesn't have constraints on the parameters, all parameters are allowed.
4. If one role defines a validate set or validate pattern for a cmdlet parameter, and the other role allows the parameter but doesn't constrain the parameter values, the validate set or pattern is ignored.

5. If a validate set is defined for the same cmdlet parameter in more than one role, all values from all validate sets are allowed.
6. If a validate pattern is defined for the same cmdlet parameter in more than one role, any values that match any of the patterns are allowed.
7. If a validate set is defined in one or more roles, and a validate pattern is defined in another role for the same cmdlet parameter, the validate set is ignored and rule (6) applies to the remaining validate patterns.

Below is an example of how roles are merged according to these rules:

PowerShell

```
# Role A Visible Cmdlets
$roleA = @{
    VisibleCmdlets = @(
        'Get-Service'
         @{
            Name       = 'Restart-Service'
            Parameters = @{ Name = 'DisplayName'; ValidateSet = 'DNS Client' }
        }
    )
}

# Role B Visible Cmdlets
$roleB = @{
    VisibleCmdlets = @(
        @{
            Name       = 'Get-Service';
            Parameters = @{ Name = 'DisplayName'; ValidatePattern = 'DNS.*' }
        }
        @{
            Name       = 'Restart-Service'
            Parameters = @{ Name = 'DisplayName'; ValidateSet = 'DNS Server' }
        }
    )
}

# Resulting permissions for a user who belongs to both role A and B
# - The constraint in role B for the DisplayName parameter on Get-Service
#   is ignored because of rule #4
# - The ValidateSets for Restart-Service are merged because both roles use
#   ValidateSet on the same parameter per rule #5
$mergedAandB = @{
    VisibleCmdlets = @(
        'Get-Service'
        @{
            Name = 'Restart-Service';
            Parameters = @{
                Name = 'DisplayName'
                ValidateSet = 'DNS Client', 'DNS Server'
            }
```

```
        }
    )
}
```

VisibleExternalCommands, VisibleAliases, VisibleProviders, ScriptsToProcess

All other fields in the role capability file are added to a cumulative set of allowable external commands, aliases, providers, and startup scripts. Any command, alias, provider, or script available in one role capability is available to the JEA user.

Be careful to ensure that the combined set of providers from one role capability and cmdlets/functions/commands from another don't allow users unintentional access to system resources. For example, if one role allows the `Remove-Item` cmdlet and another allows the `FileSystem` provider, you are at risk of a JEA user deleting arbitrary files on your computer. Additional information about identifying users' effective permissions can be found in the auditing JEA article.

Next steps

Create a session configuration file

Last updated on 03/24/2025

JEA Session Configurations

A JEA endpoint is registered on a system by creating and registering a PowerShell session configuration file. Session configurations define who can use the JEA endpoint and which roles they have access to. They also define global settings that apply to all users of the JEA session.

Create a session configuration file

To register a JEA endpoint, you must specify how that endpoint is configured. There are many options to consider. The most important options are:

- Who has access to the JEA endpoint
- Which roles they may be assigned
- Which identity JEA uses under the covers
- The name of the JEA endpoint

These options are defined in a PowerShell data file with a `.pssc` extension known as a PowerShell session configuration file. The session configuration file can be edited using any text editor.

Run the following command to create a blank template configuration file.

PowerShell

```
New-PSSessionConfigurationFile -SessionType RestrictedRemoteServer -Path
.\MyJEAEndpoint.pssc
```

> **Tip**
>
> Only the most common configuration options are included in the template file by default. Use the `-Full` switch to include all applicable settings in the generated PSSC.

The `-SessionType RestrictedRemoteServer` field indicates that the session configuration is used by JEA for secure management. Sessions of this type operate in **NoLanguage** mode and only have access to the following default commands (and aliases):

- `Clear-Host` (`cls`, `clear`)
- `Exit-PSSession` (`exsn`, `exit`)

- `Get-Command` (`gcm`)
- `Get-FormatData`
- `Get-Help`
- `Measure-Object` (`measure`)
- `Out-Default`
- `Select-Object` (`select`)

No PowerShell providers are available, nor are any external programs (executables or scripts).

For more information about language modes, see about_Language_Modes.

Choose the JEA identity

Behind the scenes, JEA needs an identity (account) to use when running a connected user's commands. You define which identity JEA uses in the session configuration file.

Local Virtual Account

Local virtual accounts are useful when all roles defined for the JEA endpoint are used to manage the local machine and a local administrator account is sufficient to run the commands successfully. Virtual accounts are temporary accounts that are unique to a specific user and only last for the duration of their PowerShell session. On a member server or workstation, virtual accounts belong to the local computer's **Administrators** group. On an Active Directory Domain Controller, virtual accounts belong to the domain's **Domain Admins** group.

PowerShell

```
# Setting the session to use a virtual account
RunAsVirtualAccount = $true
```

If the roles defined by the session configuration don't require full administrative privilege, you can specify the security groups to which the virtual account will belong. On a member server or workstation, the specified security groups must be local groups, not groups from a domain.

When one or more security groups are specified, the virtual account isn't assigned to the local or domain administrators group.

PowerShell

```
# Setting the session to use a virtual account that only belongs to the
NetworkOperator and NetworkAuditor local groups
```

```
RunAsVirtualAccount = $true
RunAsVirtualAccountGroups = 'NetworkOperator', 'NetworkAuditor'
```

> ⓘ Note
>
> Virtual accounts are temporarily granted the Logon as a service right in the local server security policy. If one of the VirtualAccountGroups specified has already been granted this right in the policy, the individual virtual account will no longer be added and removed from the policy. This can be useful in scenarios such as domain controllers where revisions to the domain controller security policy are closely audited. This is only available in Windows Server 2016 with the November 2018 or later rollup and Windows Server 2019 with the January 2019 or later rollup.

Group-managed service account

A group-managed service account (GMSA) is the appropriate identity to use when JEA users need to access network resources such as file shares and web services. GMSAs give you a domain identity that's used to authenticate with resources on any machine within the domain. The rights that a GMSA provides are determined by the resources you're accessing. You don't have admin rights on any machines or services unless the machine or service administrator has explicitly granted those rights to the GMSA.

PowerShell

```
# Configure JEA sessions to use the GMSA in the local computer's domain
# with the sAMAccountName of 'MyJEAGMSA'
GroupManagedServiceAccount = 'Domain\MyJEAGMSA'
```

GMSAs should only be used when necessary:

- It's difficult to trace back actions to a user when using a GMSA. Every user shares the same run-as identity. You must review PowerShell session transcripts and logs to correlate individual users with their actions.
- The GMSA may have access to many network resources that the connecting user doesn't need access to. Always try to limit effective permissions in a JEA session to follow the principle of least privilege.

> ⓘ Note

Group managed service accounts are only available on domain-joined machines using PowerShell 5.1 or newer.

For more information about securing a JEA session, see the security considerations article.

Session transcripts

It's recommended that you configure a JEA endpoint to automatically record transcripts of users' sessions. PowerShell session transcripts contain information about the connecting user, the run as identity assigned to them, and the commands run by the user. They can be useful to an auditing team who needs to understand who made a specific change to a system.

To configure automatic transcription in the session configuration file, provide a path to a folder where the transcripts should be stored.

PowerShell

```
TranscriptDirectory = 'C:\ProgramData\JEAConfiguration\Transcripts'
```

Transcripts are written to the folder by the **Local System** account, which requires read and write access to the directory. Standard users should have no access to the folder. Limit the number of security administrators that have access to audit the transcripts.

User drive

If your connecting users need to copy files to or from the JEA endpoint, you can enable the user drive in the session configuration file. The user drive is a **PSDrive** that's mapped to a unique folder for each connecting user. This folder allows users to copy files to or from the system without giving them access to the full file system or exposing the FileSystem provider. The user drive contents are persistent across sessions to accommodate situations where network connectivity may be interrupted.

PowerShell

```
MountUserDrive = $true
```

By default, the user drive allows you to store a maximum of 50MB of data per user. You can limit the amount of data a user can consume with the *UserDriveMaximumSize* field.

PowerShell

```
# Enables the user drive with a per-user limit of 500MB (524288000 bytes)
MountUserDrive = $true
UserDriveMaximumSize = 524288000
```

If you don't want data in the user drive to be persistent, you can configure a scheduled task on the system to automatically clean up the folder every night.

> ⓘ **Note**
>
> The user drive is only available in PowerShell 5.1 or newer.

For more information about PSDrives, see Managing PowerShell drives.

Role definitions

Role definitions in a session configuration file define the mapping of **users** to **roles**. Every user or group included in this field is granted permission to the JEA endpoint when it's registered. Each user or group can be included as a key in the hashtable only once, but can be assigned multiple roles. The name of the role capability should be the name of the role capability file, without the `.psrc` extension.

PowerShell

```
RoleDefinitions = @{
    'CONTOSO\JEA_DNS_ADMINS'     = @{ RoleCapabilities = 'DnsAdmin', 'DnsOperator',
'DnsAuditor' }
    'CONTOSO\JEA_DNS_OPERATORS' = @{ RoleCapabilities = 'DnsOperator', 'DnsAuditor'
}
    'CONTOSO\JEA_DNS_AUDITORS'   = @{ RoleCapabilities = 'DnsAuditor' }
}
```

If a user belongs to more than one group in the role definition, they get access to the roles of each. When two roles grant access to the same cmdlets, the most permissive parameter set is granted to the user.

When specifying local users or groups in the role definitions field, be sure to use the computer name, not **localhost** or wildcards. You can check the computer name by inspecting the `$Env:COMPUTERNAME` variable.

PowerShell

```
RoleDefinitions = @{
    'MyComputerName\MyLocalGroup' = @{ RoleCapabilities = 'DnsAuditor' }
}
```

Role capability search order

As shown in the example above, role capabilities are referenced by the base name of the role capability file. The base name of a file is the filename without the extension. If multiple role capabilities are available on the system with the same name, PowerShell uses its implicit search order to select the effective role capability file. JEA does **not** give access to all role capability files with the same name.

JEA uses the `$Env:PSModulePath` environment variable to determine which paths to scan for role capability files. Within each of those paths, JEA looks for valid PowerShell modules that contain a "RoleCapabilities" subfolder. As with importing modules, JEA prefers role capabilities that are shipped with Windows to custom role capabilities with the same name.

For all other naming conflicts, precedence is determined by the order in which Windows enumerates the files in the directory. The order isn't guaranteed to be alphabetical. The first role capability file found that matches the specified name is used for the connecting user. Since the role capability search order isn't deterministic, it's **strongly recommended** that role capabilities have unique filenames.

Conditional access rules

All users and groups included in the **RoleDefinitions** field are automatically granted access to JEA endpoints. Conditional access rules allow you to refine this access and require users to belong to additional security groups that don't impact the roles to which they're assigned. This is useful when you want to integrate a just-in-time privileged access management solution, smartcard authentication, or other multifactor authentication solution with JEA.

Conditional access rules are defined in the RequiredGroups field in a session configuration file. There, you can provide a hashtable (optionally nested) that uses 'And' and 'Or' keys to construct your rules. Here are some examples of how to use this field:

PowerShell

```
# Example 1: Connecting users must belong to a security group called "elevated-jea"
RequiredGroups = @{ And = 'elevated-jea' }

# Example 2: Connecting users must have signed on with 2 factor authentication or a
```

```
smart card
# The 2 factor authentication group name is "2FA-logon" and the smart card group
# name is "smartcard-logon"
RequiredGroups = @{ Or = '2FA-logon', 'smartcard-logon' }

# Example 3: Connecting users must elevate into "elevated-jea" with their JIT
system and
# have logged on with 2FA or a smart card
RequiredGroups = @{ And = 'elevated-jea', @{ Or = '2FA-logon', 'smartcard-logon' }}
```

> **Note**
>
> Conditional access rules are only available in PowerShell 5.1 or newer.

Other properties

Session configuration files can also do everything a role capability file can do, just without the ability to give connecting users access to different commands. If you want to allow all users access to specific cmdlets, functions, or providers, you can do so right in the session configuration file. For a full list of supported properties in the session configuration file, run `Get-Help New-PSSessionConfigurationFile -Full`.

Testing a session configuration file

You can test a session configuration using the Test-PSSessionConfigurationFile cmdlet. It's recommended that you test your session configuration file if you've manually edited the `.pssc` file. Testing ensures the syntax is correct. If a session configuration file fails this test, it can't be registered on the system.

Sample session configuration file

The following example shows how to create and validate a session configuration for JEA. The role definitions are created and stored in the `$roles` variable for convenience and readability. it isn't a requirement to do so.

PowerShell

```
$roles = @{
    'CONTOSO\JEA_DNS_ADMINS'     = @{ RoleCapabilities = 'DnsAdmin', 'DnsOperator',
'DnsAuditor' }
    'CONTOSO\JEA_DNS_OPERATORS' = @{ RoleCapabilities = 'DnsOperator', 'DnsAuditor'
}
    'CONTOSO\JEA_DNS_AUDITORS'   = @{ RoleCapabilities = 'DnsAuditor' }
```

```
}

$parameters = @{
    SessionType = 'RestrictedRemoteServer'
    Path = '.\JEAConfig.pssc'
    RunAsVirtualAccount = $true
    TranscriptDirectory = 'C:\ProgramData\JEAConfiguration\Transcripts'
    RoleDefinitions = $roles
    RequiredGroups = @{ Or = '2FA-logon', 'smartcard-logon' }
}
New-PSSessionConfigurationFile @parameters
Test-PSSessionConfigurationFile -Path .\JEAConfig.pssc # should yield True
```

Updating session configuration files

To change the properties of a JEA session configuration, including the mapping of users to roles, you must unregister. Then, re-register the JEA session configuration using an updated session configuration file.

Next steps

- Register a JEA configuration
- Author JEA roles

Last updated on 03/24/2025

Registering JEA Configurations

Article • 04/01/2024

Once you have your role capabilities and session configuration file created, the last step is to register the JEA endpoint. Registering the JEA endpoint with the system makes the endpoint available for use by users and automation engines.

Single machine configuration

For small environments, you can deploy JEA by registering the session configuration file using the Register-PSSessionConfiguration cmdlet.

Before you begin, ensure that the following prerequisites have been met:

- One or more roles has been created and placed in the **RoleCapabilities** folder of a PowerShell module.
- A session configuration file has been created and tested.
- The user registering the JEA configuration has administrator rights on the system.
- You've selected a name for your JEA endpoint.

The name of the JEA endpoint is required when users connect to the system using JEA. The Get-PSSessionConfiguration cmdlet lists the names of the endpoints on a system. Endpoints that start with `microsoft` are typically shipped with Windows. The `microsoft.powershell` endpoint is the default endpoint used when connecting to a remote PowerShell endpoint.

PowerShell

```
Get-PSSessionConfiguration | Select-Object Name
```

Output

```
Name
----
microsoft.powershell
microsoft.powershell.workflow
microsoft.powershell32
```

Run the following command to register the endpoint.

PowerShell

```
Register-PSSessionConfiguration -Path .\MyJEAConfig.pssc -Name
'JEAMaintenance' -Force
```

> ⚠ **Warning**
>
> The previous command restarts the WinRM service on the system. This terminates all PowerShell remoting sessions and any ongoing DSC configurations. We recommended you take production machines offline before running the command to avoid disrupting business operations.

After registration, you're ready to use JEA. You may delete the session configuration file at any time. The configuration file isn't used after registration of the endpoint.

Multi-machine configuration with DSC

When deploying JEA on multiple machines, the simplest deployment model uses the JEA Desired State Configuration (DSC) resource to quickly and consistently deploy JEA on each machine.

To deploy JEA with DSC, ensure the following prerequisites are met:

- One or more role capabilities have been authored and added to a PowerShell module.
- The PowerShell module containing the roles is stored on a (read-only) file share accessible by each machine.
- Settings for the session configuration have been determined. You don't need to create a session configuration file when using the JEA DSC resource.
- You have credentials that allow administrative actions on each machine or access to the DSC pull server used to manage the machines.
- You've downloaded the JEA DSC resource.

Create a DSC configuration for your JEA endpoint on a target machine or pull server. In this configuration, the **JustEnoughAdministration** DSC resource defines the session configuration file and the **File** resource copies the role capabilities from the file share.

The following properties are configurable using the DSC resource:

- Role Definitions
- Virtual account groups
- Group-managed service account name
- Transcript directory

- User drive
- Conditional access rules
- Startup scripts for the JEA session

The syntax for each of these properties in a DSC configuration is consistent with the PowerShell session configuration file.

Below is a sample DSC configuration for a general server maintenance module. It assumes that a valid PowerShell module containing role capabilities is located on the `\\myfileshare\JEA` file share.

PowerShell

```
Configuration JEAMaintenance
{
    Import-DscResource -Module JustEnoughAdministration,
PSDesiredStateConfiguration

    File MaintenanceModule
    {
        SourcePath = "\\myfileshare\JEA\ContosoMaintenance"
        DestinationPath = "C:\Program
Files\WindowsPowerShell\Modules\ContosoMaintenance"
        Checksum = "SHA-256"
        Ensure = "Present"
        Type = "Directory"
        Recurse = $true
    }

    JeaEndpoint JEAMaintenanceEndpoint
    {
        EndpointName = "JEAMaintenance"
        RoleDefinitions = "@{ 'CONTOSO\JEAMaintenanceAuditors' = @{
RoleCapabilities = 'GeneralServerMaintenance-Audit' };
'CONTOSO\JEAMaintenanceAdmins' = @{ RoleCapabilities =
'GeneralServerMaintenance-Audit', 'GeneralServerMaintenance-Admin' } }"
        TranscriptDirectory = 'C:\ProgramData\JEAConfiguration\Transcripts'
        DependsOn = '[File]MaintenanceModule'
    }
}
```

Next, the configuration is applied on a system by directly invoking the Local Configuration Manager or updating the pull server configuration.

The DSC resource also allows you to replace the default **Microsoft.PowerShell** endpoint. When replaced, the resource automatically registers a backup endpoint named **Microsoft.PowerShell.Restricted**. The backup endpoint has the default WinRM ACL that allows Remote Management Users and local Administrators group members to access it.

Unregistering JEA configurations

The Unregister-PSSessionConfiguration cmdlet removes a JEA endpoint. Unregistering a JEA endpoint prevents new users from creating new JEA sessions on the system. It also allows you to update a JEA configuration by re-registering an updated session configuration file using the same endpoint name.

PowerShell

```
# Unregister the JEA endpoint called "ContosoMaintenance"
Unregister-PSSessionConfiguration -Name 'ContosoMaintenance' -Force
```

> ⚠ **Warning**
>
> Unregistering a JEA endpoint causes the WinRM service to restart. This interrupts most remote management operations in progress, including other PowerShell sessions, WMI invocations, and some management tools. Only unregister PowerShell endpoints during planned maintenance windows.

Next steps

Test the JEA endpoint

Using JEA

Article • 04/01/2024

This article describes the various ways you can connect to and use a JEA endpoint.

Using JEA interactively

If you're testing your JEA configuration or have simple tasks for users, you can use JEA the same way you would a regular PowerShell remoting session. For complex remoting tasks, it's recommended to use implicit remoting. Implicit remoting allows users to operate with the data objects locally.

To use JEA interactively, you need:

- The name of the computer you're connecting to (can be the local machine)
- The name of the JEA endpoint registered on that computer
- Credentials that have access to the JEA endpoint on that computer

Given that information, you can start a JEA session using the New-PSSession or Enter-PSSession cmdlets.

PowerShell

```
$sessionParams = @{
    ComputerName      = 'localhost'
    ConfigurationName = 'JEAMaintenance'
    Credential        = Get-Credential
}
Enter-PSSession @sessionParams
```

If the current user account has access to the JEA endpoint, you can omit the **Credential** parameter.

When the PowerShell prompt changes to `[localhost]: PS>` you know that you're now interacting with the remote JEA session. You can run `Get-Command` to check which commands are available. Consult with your administrator to learn if there are any restrictions on the available parameters or allowed parameter values.

Remember, JEA sessions operate in `NoLanguage` mode. Some of the ways you typically use PowerShell may not be available. For instance, you can't use variables to store data or inspect the properties on objects returned from cmdlets. The following example shows two approaches to get the same commands to work in `NoLanguage` mode.

PowerShell

```powershell
# Using variables is prohibited in NoLanguage mode. The following will not
work:
# $vm = Get-VM -Name 'SQL01'
# Start-VM -VM $vm

# You can use pipes to pass data through to commands that accept input from
the pipeline
Get-VM -Name 'SQL01' | Start-VM

# You can also wrap subcommands in parentheses and enter them inline as
arguments
Start-VM -VM (Get-VM -Name 'SQL01')

# You can also use parameter sets that don't require extra data to be passed
in
Start-VM -VMName 'SQL01'
```

For more complex command invocations that make this approach difficult, consider using implicit remoting or creating custom functions that wrap the functionality you require. For more information on `NoLanguageMode`, see about_Language_Modes.

Using JEA with implicit remoting

PowerShell has an implicit remoting model that lets you import proxy cmdlets from a remote machine and interact with them as if they were local commands. Implicit remoting is explained in this *Hey, Scripting Guy!* blog post. Implicit remoting is useful when working with JEA because it allows you to work with JEA cmdlets in a full language mode. You can use tab completion, variables, manipulate objects, and even use local scripts to automate tasks against a JEA endpoint. Anytime you invoke a proxy command, the data is sent to the JEA endpoint on the remote machine and executed there.

Implicit remoting works by importing cmdlets from an existing PowerShell session. You can optionally choose to prefix the nouns of each proxy cmdlet with a string of your choosing. The prefix allows you to distinguish the commands that are for the remote system. A temporary script module containing all the proxy commands is created and imported for the duration of your local PowerShell session.

PowerShell

```powershell
# Create a new PSSession to your JEA endpoint
$jeaSession = New-PSSession -ComputerName 'SERVER01' -ConfigurationName
'JEAMaintenance'

# Import the entire PSSession and prefix each imported cmdlet with "JEA"
Import-PSSession -Session $jeaSession -Prefix 'JEA'
```

```
# Invoke "Get-Command" on the remote JEA endpoint using the proxy cmdlet
Get-JEACommand
```

> **ⓘ Important**
>
> Some systems may not be able to import an entire JEA session due to constraints in the default JEA cmdlets. To get around this, only import the commands you need from the JEA session by explicitly providing their names to the `-CommandName` parameter. A future update will address the issue with importing entire JEA sessions on affected systems.

If you're unable to import a JEA session because of JEA constraints on the default parameters, follow the steps below to filter out the default commands from the imported set. You can continue use commands like `Select-Object`, but you'll just use the local version installed on your computer instead of the one imported from the remote JEA session.

PowerShell

```
# Create a new PSSession to your JEA endpoint
$jeaSession = New-PSSession -ComputerName 'SERVER01' -ConfigurationName
'JEAMaintenance'

# Get a list of all the commands on the JEA endpoint
$commands = Invoke-Command -Session $jeaSession -ScriptBlock { Get-Command }

# Filter out the default cmdlets
$jeaDefaultCmdlets = @(
    'Clear-Host'
    'Exit-PSSession'
    'Get-Command'
    'Get-FormatData'
    'Get-Help'
    'Measure-Object'
    'Out-Default'
    'Select-Object'
)
$filteredCommands = $commands.Name | Where-Object { $jeaDefaultCmdlets -
notcontains $_ }

# Import only commands explicitly added in role capabilities and prefix each
# imported cmdlet with "JEA"
Import-PSSession -Session $jeaSession -Prefix 'JEA' -CommandName
$filteredCommands
```

You can also persist the proxied cmdlets from implicit remoting using Export-PSSession. For more information about implicit remoting, see the documentation for Import-PSSession and Import-Module.

Using JEA programmatically

JEA can also be used in automation systems and in user applications, such as in-house helpdesk apps and websites. The approach is the same as that for building apps that talk to unconstrained PowerShell endpoints. Ensure the program is designed to work with limitation imposed by JEA.

For simple, one-off tasks, you can use Invoke-Command to run commands in a JEA session.

PowerShell

```
Invoke-Command -ComputerName 'SERVER01' -ConfigurationName 'JEAMaintenance'
-ScriptBlock {
    Get-Process
    Get-Service
}
```

To check which commands are available for use when you connect to a JEA session, run `Get-Command` and iterate through the results to check for the allowed parameters.

PowerShell

```
$commandParameters = @{
    ComputerName      = 'SERVER01'
    ConfigurationName = 'JEAMaintenance'
    ScriptBlock       = { Get-Command }
}
Invoke-Command @commandParameters |
    Where-Object { $_.CommandType -in @('Function', 'Cmdlet') } |
    Format-Table Name, Parameters
```

If you're building a C# app, you can create a PowerShell runspace that connects to a JEA session by specifying the configuration name in a WSManConnectionInfo object.

C#

```
// using System.Management.Automation;
var computerName = "SERVER01";
var configName   = "JEAMaintenance";
// See
https://learn.microsoft.com/dotnet/api/system.management.automation.pscreden
```

```
tial
var creds        = // create a PSCredential object here

WSManConnectionInfo connectionInfo = new WSManConnectionInfo(
    false,                  // Use SSL
    computerName,           // Computer name
    5985,                   // WSMan Port
    "/wsman",               // WSMan Path
                            // Connection URI with config name
    string.Format(
        CultureInfo.InvariantCulture,
        "http://schemas.microsoft.com/powershell/{0}",
        configName
    ),
    creds                   // Credentials
);

// Now, use the connection info to create a runspace where you can run the
commands
using (Runspace runspace = RunspaceFactory.CreateRunspace(connectionInfo))
{
    // Open the runspace
    runspace.Open();

    using (PowerShell ps = PowerShell.Create())
    {
        // Set the PowerShell object to use the JEA runspace
        ps.Runspace = runspace;

        // Now you can add and invoke commands
        ps.AddCommand("Get-Command");
        foreach (var result in ps.Invoke())
        {
            Console.WriteLine(result);
        }
    }

    // Close the runspace
    runspace.Close();
}
```

Using JEA with PowerShell Direct

Hyper-V in Windows 10 and Windows Server 2016 offers PowerShell Direct, a feature that allows Hyper-V administrators to manage virtual machines with PowerShell regardless of the network configuration or remote management settings on the virtual machine.

You can use PowerShell Direct with JEA to give a Hyper-V administrator limited access to your VM. This can be useful if you lose network connectivity to your VM and need a

datacenter admin to fix the network settings.

No additional configuration is required to use JEA over PowerShell Direct. However, the guest operating system running inside the virtual machine must be Windows 10, Windows Server 2016, or higher. The Hyper-V admin can connect to the JEA endpoint by using the `-VMName` or `-VMId` parameters on PSRemoting cmdlets:

PowerShell

```
$sharedParams = @{
    ConfigurationName = 'NICMaintenance'
    Credential        = Get-Credential -UserName 'localhost\JEAformyHoster'
}
# Entering a JEA session using PowerShell Direct when the VM name is unique
Enter-PSSession -VMName 'SQL01' @sharedParams

# Entering a JEA session using PowerShell Direct using VM ids
$vm = Get-VM -VMName 'MyVM' | Select-Object -First 1
Enter-PSSession -VMId $vm.VMId @sharedParams
```

It's recommended you create a dedicated user account with the minimum rights needed to manage the system for use by a Hyper-V administrator. Remember, even an unprivileged user can sign into a Windows machine by default, including using unconstrained PowerShell. That allows them to browse the file system and learn more about your OS environment. To lock down a Hyper-V administrator and limit them to only access a VM using PowerShell Direct with JEA, you must deny local logon rights to the Hyper-V admin's JEA account.

JEA Security Considerations

JEA helps you improve your security posture by reducing the number of permanent administrators on your machines. JEA uses a PowerShell session configuration to create a new entry point for users to manage the system. Users who need elevated, but not unlimited, access to the machine to do administrative tasks can be granted access to the JEA endpoint. Since JEA allows these users to run administrative commands without having full administrator access, you can then remove those users from highly privileged security groups.

Run-As account

Each JEA endpoint has a designated **run-as** account under which the connecting user's actions are executed. This account is configurable in the session configuration file, and the account you choose has a significant bearing on the security of your endpoint.

Virtual accounts are the recommended way of configuring the **run-as** account. Virtual accounts are one-time, temporary local accounts that are created for the connecting user to use during the duration of their JEA session. As soon as their session is terminated, the virtual account is destroyed and can't be used anymore. The connecting user doesn't know the credentials for the virtual account. The virtual account can't be used to access the system via other means like Remote Desktop or an unconstrained PowerShell endpoint.

By default, virtual accounts are members of the local **Administrators** group on the machine. This membership gives them full rights to manage anything on the system, but no rights to manage resources on the network. When the user connects to other machines from the JEA session, the user context is that of the local computer account, not the virtual account.

Domain controllers are a special case since there isn't a local **Administrators** group. Instead, virtual accounts belong to **Domain Admins** and can manage the directory services on the domain controller. The domain identity is still restricted for use on the domain controller where the JEA session was instantiated. Any network access appears to come from the domain controller computer object instead.

In both cases, you may assign the virtual account to specific security groups, especially when the task can be done without local or domain administrator privileges. If you already have a security group defined for your administrators, grant the virtual account membership to that group. Group membership for virtual accounts is limited to local security groups on workstation and

member servers. On domain controllers, virtual accounts must be members of domain security groups. Once the virtual account has been added to one or more security groups, it no longer belongs to the default groups (local or domain administrators).

The following table summarizes the possible configuration options and resulting permissions for virtual accounts:

Expand table

Computer type	Virtual account group configuration	Local user context	Network user context
Domain controller	Default	Domain user, member of `<DOMAIN>\Domain Admins`	Computer account
Domain controller	Domain groups A and B	Domain user, member of `<DOMAIN>\A`, `<DOMAIN>\B`	Computer account
Member server or workstation	Default	Local user, member of `BUILTIN\Administrators`	Computer account
Member server or workstation	Local groups C and D	Local user, member of `<COMPUTER>\C` and `<COMPUTER>\D`	Computer account

When you look at Security audit and Application event logs, you see that each JEA user session has a unique virtual account. This unique account helps you track user actions in a JEA endpoint back to the original user who ran the command. Virtual account names follow the format `WinRM Virtual Users\WinRM_VA_<ACCOUNTNUMBER>_<DOMAIN>_<sAMAccountName>` For example, if user **Alice** in domain **Contoso** restarts a service in a JEA endpoint, the username associated with any service control manager events would be `WinRM Virtual Users\WinRM_VA_1_contoso_alice`.

Group-managed service accounts (gMSAs) are useful when a member server needs to have access to network resources in the JEA session. For example, when a JEA endpoint is used to control access to a REST API service hosted on a different machine. It's easy to write functions to invoke the REST APIs, but you need a network identity to authenticate with the API. Using a group-managed service account makes the second hop possible while maintaining control over which computers can use the account. The security group (local or domain) memberships of the gMSA defined the effective permissions for the gMSA account.

When a JEA endpoint is configured to use a gMSA, the actions of all JEA users appear to come from the same gMSA. The only way to trace actions back to a specific user is to identify the set of commands run in a PowerShell session transcript.

Pass-through credentials are used when you don't specify a **run-as** account. PowerShell uses the connecting user's credential to run commands on the remote server. To use pass-through credentials, you must grant the connecting user direct access to privileged management groups. This configuration is **NOT** recommended for JEA. If the connecting user already has administrator privileges, they can bypass JEA and manage the system using other access methods.

Standard run-as accounts allow you to specify any user account under which the entire PowerShell session runs. Session configurations using fixed **run-as** accounts (with the `-RunAsCredential` parameter) aren't JEA-aware. Role definitions no longer function as expected. Every user authorized to access the endpoint is assigned the same role.

You shouldn't use a **RunAsCredential** on a JEA endpoint because it's difficult to trace actions back to specific users and lacks support for mapping users to roles.

WinRM Endpoint ACL

As with regular PowerShell remoting endpoints, each JEA endpoint has a separate access control list (ACL) that controls who can authenticate with the JEA endpoint. If improperly configured, trusted users may not be able to access the JEA endpoint, and untrusted users may have access. The WinRM ACL doesn't affect the mapping of users to JEA roles. Mapping is controlled by the **RoleDefinitions** field in the session configuration file used to register the endpoint.

By default, when a JEA endpoint has multiple role capabilities, the WinRM ACL is configured to allow access to all mapped users. For example, a JEA session configured using the following commands grants full access to `CONTOSO\JEA_Lev1` and `CONTOSO\JEA_Lev2`.

PowerShell

```
$newPSSessionConfigurationFileSplat = @{
    Path = '.\jea.pssc'
    SessionType = 'RestrictedRemoteServer'
    RoleDefinitions = @{
        'CONTOSO\JEA_Lev1' = 'Lev1Role'
        'CONTOSO\JEA_Lev2' = 'Lev2Role'
    }
    RunAsVirtualAccount = $true
}
New-PSSessionConfigurationFile @newPSSessionConfigurationFileSplat
Register-PSSessionConfiguration -Path '.\jea.pssc' -Name 'MyJEAEndpoint'
```

You can audit user permissions with the Get-PSSessionConfiguration cmdlet.

PowerShell

```
Get-PSSessionConfiguration -Name 'MyJEAEndpoint' | Select-Object Permission
```

Output

```
Permission
----------
CONTOSO\JEA_Lev1 AccessAllowed
CONTOSO\JEA_Lev2 AccessAllowed
```

To change which users have access, run either `Set-PSSessionConfiguration -Name 'MyJEAEndpoint' -ShowSecurityDescriptorUI` for an interactive prompt or `Set-PSSessionConfiguration -Name 'MyJEAEndpoint' -SecurityDescriptorSddl <SDDL string>` to update the permissions. Users need at least *Invoke* rights to access the JEA endpoint.

It's possible to create a JEA endpoint that doesn't map a defined role to every user that has access. These users can start a JEA session, but only have access to the default cmdlets. You can audit user permissions in a JEA endpoint by running `Get-PSSessionCapability`. For more information, see Auditing and Reporting on JEA.

Least privilege roles

When designing JEA roles, it's important to remember that the virtual and group-managed service accounts running behind the scenes can have unrestricted access to the local machine. JEA role capabilities help limit the commands and applications that can be run in that privileged context. Improperly designed roles can allow dangerous commands that may permit a user to break out of the JEA boundaries or obtain access to sensitive information.

For example, consider the following role capability entry:

PowerShell

```
@{
    VisibleCmdlets = 'Microsoft.PowerShell.Management\*-Process'
}
```

This role capability allows users to run any PowerShell cmdlet with the noun **Process** from the **Microsoft.PowerShell.Management** module. Users may need to access cmdlets like `Get-Process` to see what applications are running on the system and `Stop-Process` to kill applications that aren't responding. However, this entry also allows `Start-Process`, which can be used to start up an arbitrary program with full administrator permissions. The program doesn't need to be

installed locally on the system. A connected user could start a program from a file share that gives the user local administrator privileges, runs malware, and more.

A more secure version of this same role capability would look like:

PowerShell

```
@{
    VisibleCmdlets = 'Microsoft.PowerShell.Management\Get-Process',
                     'Microsoft.PowerShell.Management\Stop-Process'
}
```

Avoid using wildcards in role capabilities. Be sure to regularly audit effective user permissions to see which commands are accessible to a user. For more information, see the *Check effective rights* section of the Auditing and Reporting on JEA article.

Best practice recommendations

The following are best practice recommendations to ensure the security of your JEA endpoints:

Limit the use and capabilities of PowerShell providers

Review how the allowed providers are used to ensure that you don't create vulnerabilities in your configured session.

> ⚠ **Warning**
>
> Don't allow the **FileSystem** provider. If users can write to any part of the file system, it's possible to completely bypass security.
>
> Don't allow the **Certificate** provider. With the provider enabled, a user could gain access to stored private keys.

Don't allow commands that can create new runspaces.

> ⚠ **Warning**
>
> The Windows Compatibility feature in PowerShell 7 creates a new runspace to host Windows PowerShell. Don't allow any commands that would run via the Windows Compatibility feature. The `*-Job` cmdlets can create new runspaces without the restrictions.

Don't allow commands that add TypeData or FormatData

Restricted endpoints must not expose the `Update-TypeData`, `Remove-TypeData`, or `Update-FormatData` commands. The `*-TypeData` commands allow you to add **ScriptProperty** members to types. The `Update-FormatData` command allows you to add **ScriptBlock** definitions to create custom formatting. The **ScriptProperty** and **ScriptBlock** members might be evaluated in `FullLanguage` mode, even when the session is configured to use a more restrictive language mode.

Don't allow the `Trace-Command` cmdlet.

> ⚠ **Warning**
>
> Using `Trace-Command` brings all traced commands into the session.

Don't create your own proxy implementations for the *restricted commands*.

PowerShell has a set of proxy commands for restricted command scenarios. These proxy commands ensure that input parameters can't compromise the security of the session. The following commands have restricted proxies:

- `Exit-PSSession`
- `Get-Command`
- `Get-FormatData`
- `Get-Help`
- `Measure-Object`
- `Out-Default`
- `Select-Object`

If you create your own implementation of these commands, you may inadvertently allow users to run code prohibited by the JEA proxy commands.

JEA doesn't protect against admins

One of the core principles of JEA is that it allows nonadministrators to do some administrative tasks. JEA doesn't protect against users who already have administrator privileges. Users who belong **Domain Admins**, local **Administrators**, or other highly privileged groups can circumvent

JEA's protections in other ways. For example, they could sign in with RDP, use remote MMC consoles, or connect to unconstrained PowerShell endpoints. Also, local administrator on a system can modify JEA configurations to add more users or change a role capability to extend the scope of what a user can do in their JEA session. It's important to evaluate your JEA users' extended permissions to see if there are other ways to gain privileged access to the system.

In addition to using JEA for regular day-to-day maintenance, it's common to have a just-in-time privileged access management system. These systems allow designated users to temporarily become a local administrator only after they complete a workflow that documents their use of those permissions.

Last updated on 06/17/2026

Auditing and Reporting on JEA

Article • 04/01/2024

After you've deployed JEA, you need to regularly audit the JEA configuration. Auditing helps you assess that the correct people have access to the JEA endpoint and their assigned roles are still appropriate.

Find registered JEA sessions on a machine

To check which JEA sessions are registered on a machine, use the Get-PSSessionConfiguration cmdlet.

PowerShell

```
# Filter for sessions that are configured as 'RestrictedRemoteServer' to
# find JEA-like session configurations
Get-PSSessionConfiguration | Where-Object { $_.SessionType -eq
'RestrictedRemoteServer' }
```

Output

```
Name          : JEAMaintenance
PSVersion     : 5.1
StartupScript :
RunAsUser     :
Permission    : CONTOSO\JEA_DNS_ADMINS AccessAllowed,
CONTOSO\JEA_DNS_OPERATORS AccessAllowed,
                CONTOSO\JEA_DNS_AUDITORS AccessAllowed
```

The effective rights for the endpoint are listed in the **Permission** property. These users have the right to connect to the JEA endpoint. However, the roles and commands they have access to is determined by the **RoleDefinitions** property in the session configuration file that was used to register the endpoint. Expand the **RoleDefinitions** property to evaluate the role mappings in a registered JEA endpoint.

PowerShell

```
# Get the desired session configuration
$jea = Get-PSSessionConfiguration -Name 'JEAMaintenance'

# Enumerate users/groups and which roles they have access to
$jea.RoleDefinitions.GetEnumerator() | Select-Object Name, @{
  Name = 'Role Capabilities'
```

```
  Expression = { $_.Value.RoleCapabilities }
}
```

Find available role capabilities on the machine

JEA gets role capabilities from the `.psrc` files stored in the **RoleCapabilities** folder inside a PowerShell module. The following function finds all role capabilities available on a computer.

PowerShell

```
function Find-LocalRoleCapability {
    $results = @()

    # Find modules with a "RoleCapabilities" subfolder and add any PSRC
files to the result set
    Get-Module -ListAvailable | ForEach-Object {
        $psrcpath = Join-Path -Path $_.ModuleBase -ChildPath
'RoleCapabilities'
        if (Test-Path $psrcpath) {
            $results += Get-ChildItem -Path $psrcpath -Filter *.psrc
        }
    }

    # Format the results nicely to make it easier to read
    $results | Select-Object @{ Name = 'Name'; Expression = {
$_.Name.TrimEnd('.psrc') }}, @{
        Name = 'Path'; Expression = { $_.FullName }
    } | Sort-Object Name
}
```

> **Note**
>
> The order of results from this function isn't necessarily the order in which the role capabilities will be selected if multiple role capabilities share the same name.

Check effective rights for a specific user

The Get-PSSessionCapability cmdlet enumerates all the commands available on a JEA endpoint based on a user's group memberships. The output of `Get-PSSessionCapability` is identical to that of the specified user running `Get-Command -CommandType All` in a JEA session.

PowerShell

```
Get-PSSessionCapability -ConfigurationName 'JEAMaintenance' -Username
'CONTOSO\Alice'
```

If your users aren't permanent members of groups that would grant them additional JEA rights, this cmdlet may not reflect those extra permissions. This happens when using just-in-time privileged access management systems to allow users to temporarily belong to a security group. Carefully evaluate the mapping of users to roles and capabilities to ensure that users only get the level of access needed to do their jobs successfully.

PowerShell event logs

If you enabled module or script block logging on the system, you can see events in the Windows event logs for each command a user runs in a JEA session. To find these events, open **Microsoft-Windows-PowerShell/Operational** event log and look for events with event ID **4104**.

Each event log entry includes information about the session in which the command was run. For JEA sessions, the event includes information about the **ConnectedUser** and the **RunAsUser**. The **ConnectedUser** is the actual user who created the JEA session. The **RunAsUser** is the account JEA used to execute the command.

Application event logs show changes being made by the **RunAsUser**. So having module and script logging enabled is required to trace a specific command invocation back to the **ConnectedUser**.

Application event logs

Commands run in a JEA session that interact with external applications or services may log events to their own event logs. Unlike PowerShell logs and transcripts, other logging mechanisms don't capture the connected user of the JEA session. Instead, those applications only log the virtual run-as user. To determine who ran the command, you need to consult a session transcript or correlate PowerShell event logs with the time and user shown in the application event log.

The WinRM log can also help you correlate run-as users to the connecting user in an application event log. Event ID **193** in the **Microsoft-Windows-Windows Remote Management/Operational** log records the security identifier (SID) and account name for both the connecting user and run as user for every new JEA session.

Session transcripts

If you configured JEA to create a transcript for each user session, a text copy of every user's actions are stored in the specified folder.

The following command (as an administrator) finds all transcript directories.

PowerShell

```
Get-PSSessionConfiguration |
  Where-Object { $_.TranscriptDirectory -ne $null } |
    Format-Table Name, TranscriptDirectory
```

Each transcript starts with information about the time the session started, which user connected to the session, and which JEA identity was assigned to them.

```
**********************
Windows PowerShell transcript start
Start time: 20160710144736
Username: CONTOSO\Alice
RunAs User: WinRM Virtual Users\WinRM VA_1_CONTOSO_Alice
Machine: SERVER01 (Microsoft Windows NT 10.0.14393.0)
[...]
```

The body of the transcript contains information about each command the user invoked. The exact syntax of the command used is unavailable in JEA sessions because of the way commands are transformed for PowerShell remoting. However, you can still determine the effective command that was executed. Below is an example transcript snippet from a user running `Get-Service Dns` in a JEA session:

```
PS>CommandInvocation(Get-Service): "Get-Service"
>> ParameterBinding(Get-Service): name="Name"; value="Dns"
>> CommandInvocation(Out-Default): "Out-Default"
>> ParameterBinding(Out-Default): name="InputObject"; value="Dns"

Running  Dns                DNS Server
```

A **CommandInvocation** line is written for each command a user runs. **ParameterBindings** record each parameter and value supplied with the command. In the previous example, you can see that the parameter **Name** was supplied the with value **Dns** for the `Get-Service` cmdlet.

The output of each command also triggers a **CommandInvocation**, usually to `Out-Default`. The **InputObject** of `Out-Default` is the PowerShell object returned from the command. The details of that object are printed a few lines below, closely mimicking what the user would have seen.

See also

PowerShell ♥ the Blue Team blog post on security

Running Remote Commands

You can run commands on one or hundreds of computers with a single PowerShell command. Windows PowerShell supports remote computing using various technologies, including WMI, RPC, and WS-Management.

PowerShell supports WMI, WS-Management, and SSH remoting. In PowerShell 7 and higher, RPC is supported only on Windows.

For more information about remoting in PowerShell, see the following articles:

- SSH Remoting in PowerShell
- WSMan Remoting in PowerShell

Windows PowerShell remoting without configuration

Many Windows PowerShell cmdlets have the **ComputerName** parameter that enables you to collect data and change settings on one or more remote computers. These cmdlets use varying communication protocols and work on all Windows operating systems without any special configuration.

These cmdlets include:

- Restart-Computer
- Test-Connection
- Clear-EventLog
- Get-EventLog
- Get-HotFix
- Get-Process
- Get-Service
- Set-Service
- Get-WinEvent
- Get-WmiObject

Typically, cmdlets that support remoting without special configuration have the **ComputerName** parameter and don't have the **Session** parameter. To find these cmdlets in your session, type:

PowerShell

```
Get-Command | Where-Object {
    $_.Parameters.Keys -contains "ComputerName" -and
```

```
    $_.Parameters.Keys -notcontains "Session"
}
```

Windows PowerShell remoting

By using the WS-Management protocol, Windows PowerShell remoting lets you run any Windows PowerShell command on one or more remote computers. You can establish persistent connections, start interactive sessions, and run scripts on remote computers.

To use Windows PowerShell remoting, the remote computer must be configured for remote management. For more information, including instructions, see About Remote Requirements.

Once you configure Windows PowerShell remoting, many remoting strategies are available to you. This article lists just a few of them. For more information, see About Remote.

Start an interactive session

To start an interactive session with a single remote computer, use the Enter-PSSession cmdlet. For example, to start an interactive session with the Server01 remote computer, type:

PowerShell

```
Enter-PSSession Server01
```

The command prompt changes to display the name of the remote computer. Any commands that you type at the prompt run on the remote computer and the results are displayed on the local computer.

To end the interactive session, type:

PowerShell

```
Exit-PSSession
```

For more information about the `Enter-PSSession` and `Exit-PSSession` cmdlets, see:

- Enter-PSSession
- Exit-PSSession

Run a Remote Command

To run a command on one or more computers, use the Invoke-Command cmdlet. For example, to run a Get-UICulture command on the Server01 and Server02 remote computers, type:

PowerShell

```
Invoke-Command -ComputerName Server01, Server02 -ScriptBlock {Get-UICulture}
```

The output is returned to your computer.

Output

```
LCID    Name     DisplayName               PSComputerName
----    ----     -----------               --------------
1033    en-US    English (United States)   server01.corp.fabrikam.com
1033    en-US    English (United States)   server02.corp.fabrikam.com
```

Run a Script

To run a script on one or many remote computers, use the **FilePath** parameter of the `Invoke-Command` cmdlet. The script must be on or accessible to your local computer. The results are returned to your local computer.

For example, the following command runs the `DiskCollect.ps1` script on the remote computers, Server01 and Server02.

PowerShell

```
Invoke-Command -ComputerName Server01, Server02 -FilePath C:\Scripts\DiskCollect.ps1
```

Establish a Persistent Connection

Use the `New-PSSession` cmdlet to create a persistent session on a remote computer. The following example creates remote sessions on Server01 and Server02. The session objects are stored in the `$s` variable.

PowerShell

```
$s = New-PSSession -ComputerName Server01, Server02
```

Now that the sessions are established, you can run any command in them. And because the sessions are persistent, you can collect data from one command and use it in another command.

For example, the following command runs a `Get-HotFix` command in the sessions in the `$s` variable and it saves the results in the `$h` variable. The `$h` variable is created in each of the sessions in `$s`, but it doesn't exist in the local session.

PowerShell

```
Invoke-Command -Session $s {$h = Get-HotFix}
```

Now you can use the data in the `$h` variable with other commands in the same session. The results are displayed on the local computer. For example:

PowerShell

```
Invoke-Command -Session $s {$h | where {$_.InstalledBy -ne "NT AUTHORITY\SYSTEM"}}
```

Advanced Remoting

PowerShell includes cmdlets that allow you to:

- Configure and create remote sessions both from the local and remote ends
- Create customized and restricted sessions
- Import commands from a remote session that actually run implicitly on the remote session
- Configure the security of a remote session

PowerShell on Windows includes a WSMan provider. The provider creates a `WSMan:` drive that lets you navigate through a hierarchy of configuration settings on the local computer and remote computers.

For more information about the WSMan provider, see WSMan Provider and About WS-Management Cmdlets, or in the Windows PowerShell console, type `Get-Help WSMan`.

For more information, see:

- PowerShell Remoting FAQ
- Register-PSSessionConfiguration
- Import-PSSession

For help with remoting errors, see about_Remote_Troubleshooting.

See Also

- about_Remote
- about_Remote_Requirements
- about_Remote_Troubleshooting
- about_PSSessions
- about_WS-Management_Cmdlets

- Invoke-Command
- Import-PSSession
- New-PSSession
- Register-PSSessionConfiguration
- WSMan Provider

Last updated on 12/09/2025

PowerShell remoting over SSH

09/08/2025

Overview

PowerShell remoting normally uses WinRM for connection negotiation and data transport. SSH is now available for Linux and Windows platforms and allows true multiplatform PowerShell remoting.

WinRM provides a robust hosting model for PowerShell remote sessions. SSH-based remoting doesn't currently support remote endpoint configuration and Just Enough Administration (JEA).

SSH remoting lets you do basic PowerShell session remoting between Windows and Linux computers. SSH remoting creates a PowerShell host process on the target computer as an SSH subsystem. Eventually we'll implement a general hosting model, similar to WinRM, to support endpoint configuration and JEA.

The `New-PSSession`, `Enter-PSSession`, and `Invoke-Command` cmdlets now have a new parameter set to support this new remoting connection.

```
[-HostName <string>]  [-UserName <string>]  [-KeyFilePath <string>]
```

To create a remote session, you specify the target computer with the **HostName** parameter and provide the user name with **UserName**. When running the cmdlets interactively, you're prompted for a password. You can also use SSH key authentication using a private key file with the **KeyFilePath** parameter. Creating keys for SSH authentication varies by platform.

General setup information

PowerShell 6 or higher, and SSH must be installed on all computers. Install both the SSH client (`ssh.exe`) and server (`sshd.exe`) so that you can remote to and from the computers. OpenSSH for Windows is now available in Windows 10 build 1809 and Windows Server 2019. For more information, see Manage Windows with OpenSSH. For Linux, install SSH, including sshd server, that's appropriate for your platform. You also need to install PowerShell from GitHub to get the SSH remoting feature. The SSH server must be configured to create an SSH subsystem to host a PowerShell process on the remote computer. And, you must enable **password** or **key-based** authentication.

Install the SSH service on a Windows computer

1. Install the latest version of PowerShell. For more information, see Installing PowerShell on Windows.

   You can confirm that PowerShell has SSH remoting support by listing the `New-PSSession` parameter sets. You'll notice there are parameter set names that begin with **SSH**. Those parameter sets include **SSH** parameters.

   PowerShell

   ```
   (Get-Command New-PSSession).ParameterSets.Name
   ```

   Output

   ```
   Name
   ----
   SSHHost
   SSHHostHashParam
   ```

2. Install the latest Win32 OpenSSH. For installation instructions, see Getting started with OpenSSH.

   > **Note**
   >
   > If you want to set PowerShell as the default shell for OpenSSH, see Configuring Windows for OpenSSH.

3. Edit the `sshd_config` file located at `$Env:ProgramData\ssh`.

   - Make sure password authentication is enabled:

     ```
     PasswordAuthentication yes
     ```

   - Create the SSH subsystem that hosts a PowerShell process on the remote computer:

     ```
     Subsystem powershell C:/progra~1/powershell/7/pwsh.exe -sshs
     ```

> **Note**
>
> There is a bug in OpenSSH for Windows that prevents you from using a path with spaces for the subsystem executable. There are two ways to work around this issue:
>
> - Use the Windows *8.3-style* short name for the PowerShell executable path
> - Create a symbolic link to the PowerShell executable that results in a path without spaces
>
> For more information, see issue #784 in the PowerShell/Win32-OpenSSH repository.

You only need to get the 8.3-style name for the segment of the path that contains the space. By default PowerShell 7 is installed in `C:\Program Files\PowerShell\7\`. The 8.3-style name for `Program Files` should be `progra~1`. You can use the following command to verify the name:

PowerShell

```powershell
Get-CimInstance Win32_Directory -Filter 'Name="C:\\Program Files"' |
    Select-Object EightDotThreeFileName
```

The 8.3 name is a legacy feature of the NTFS file system that can be disabled. This feature must be enabled for the volume on which PowerShell is installed.

Alternatively, you can create a symbolic link to the PowerShell executable that results in a path without spaces. This method is preferred because it allows you to update the link if the path to the PowerShell executable ever changes, without also needing to update your `sshd_config` file.

Use the following command to create a symbolic link to the executable:

PowerShell

```powershell
$newItemSplat = @{
     ItemType = 'SymbolicLink'
     Path = 'C:\ProgramData\ssh\'
     Name = 'pwsh.exe'
     Value = (Get-Command pwsh.exe).Source
}
New-Item @newItemSplat
```

This command creates the symbolic link in the same directory used by the OpenSSH server to store the host keys and other configuration.

- Optionally, enable key authentication:

```
PubkeyAuthentication yes
```

For more information, see Managing OpenSSH Keys.

4. Restart the **sshd** service.

```PowerShell
Restart-Service sshd
```

5. Add the path where OpenSSH is installed to your PATH environment variable. For example, `C:\Program Files\OpenSSH\`. This entry allows for the `ssh.exe` to be found.

Install the SSH service on an Ubuntu Linux computer

1. Install the latest version of PowerShell, see Installing PowerShell on Ubuntu.
2. Install Ubuntu OpenSSH Server.

```Bash
sudo apt install openssh-client
sudo apt install openssh-server
```

3. Edit the `sshd_config` file at location `/etc/ssh`.

- Make sure password authentication is enabled:

```
PasswordAuthentication yes
```

- Optionally, enable key authentication:

```
```

```
PubkeyAuthentication yes
```

For more information about creating SSH keys on Ubuntu, see the manpage for ssh-keygen.

- Add a PowerShell subsystem entry:

```
Subsystem powershell /usr/bin/pwsh -sshs -NoLogo
```

> **Note**
>
> The default location of the PowerShell executable is `/usr/bin/pwsh`. The location can vary depending on how you installed PowerShell.

4. Restart the **ssh** service.

```
Bash

sudo systemctl restart sshd.service
```

Install the SSH service on a macOS computer

1. Install the latest version of PowerShell. For more information, Installing PowerShell on macOS.

   Make sure SSH Remoting is enabled by following these steps:
   a. Open `System Settings`.
   b. Click on `General`
   c. Click on `Sharing`.
   d. Check `Remote Login` to set `Remote Login: On`.
   e. Allow access to the appropriate users.

2. Edit the `sshd_config` file at location `/private/etc/ssh/sshd_config`.

   Use a text editor such as **nano**:

```
Bash
```

```
sudo nano /private/etc/ssh/sshd_config
```

- Make sure password authentication is enabled:

```

PasswordAuthentication yes
```

- Add a PowerShell subsystem entry:

```

Subsystem powershell /usr/local/bin/pwsh -sshs -NoLogo
```

> ⓘ **Note**
>
> The default location of the PowerShell executable is `/usr/local/bin/pwsh`. The location can vary depending on how you installed PowerShell.

- Optionally, enable key authentication:

```

PubkeyAuthentication yes
```

3. Restart the **sshd** service.

```Bash
sudo launchctl stop com.openssh.sshd
sudo launchctl start com.openssh.sshd
```

> ⓘ **Note**
>
> When you upgrade your operating system, the SSH configuration file might be overwritten. Make sure you check the configuration file after an upgrade.

Authentication

PowerShell remoting over SSH relies on the authentication exchange between the SSH client and SSH service and doesn't implement any authentication schemes itself. The result is that any configured authentication schemes including multi-factor authentication are handled by SSH and independent of PowerShell. For example, you can configure the SSH service to require public key authentication and a one-time password for added security. Configuration of multi-factor authentication is outside the scope of this documentation. Refer to documentation for SSH on how to correctly configure multi-factor authentication and validate it works outside of PowerShell before attempting to use it with PowerShell remoting.

> ⓘ **Note**
>
> Users retain the same privileges in remote sessions. Meaning, Administrators have access to an elevated shell, and normal users do not.

PowerShell remoting example

The easiest way to test remoting is to try it on a single computer. In this example, we create a remote session back to the same Linux computer. We're using PowerShell cmdlets interactively so we see prompts from SSH asking to verify the host computer and prompting for a password. You can do the same thing on a Windows computer to ensure remoting is working. Then, remote between computers by changing the host name.

Linux to Linux

PowerShell

```
$session = New-PSSession -HostName UbuntuVM1 -UserName TestUser
```

Output

```
The authenticity of host 'UbuntuVM1 (9.129.17.107)' can't be established.
ECDSA key fingerprint is SHA256:2kCbnhT2dUE6WCGgVJ8Hyfu1z2wE4lifaJXLO7QJy0Y.
Are you sure you want to continue connecting (yes/no)?
TestUser@UbuntuVM1s password:
```

PowerShell

```
$session
```

Output

```
 Id Name    ComputerName    ComputerType    State    ConfigurationName
Availability
 -- ----    ------------    ------------    -----    -----------------     --------
----
  1 SSH1    UbuntuVM1       RemoteMachine   Opened   DefaultShell
Available
```

PowerShell

```
Enter-PSSession $session
```

Output

```
[UbuntuVM1]: PS /home/TestUser> uname -a
Linux TestUser-UbuntuVM1 4.2.0-42-generic 49~16.04.1-Ubuntu SMP Wed Jun 29
20:22:11 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux

[UbuntuVM1]: PS /home/TestUser> Exit-PSSession
```

PowerShell

```
Invoke-Command $session -ScriptBlock { Get-Process pwsh }
```

Output

```
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
PSComputerName
-------  ------    -----      -----     ------     --  -- -----------    ---------
-----
      0       0        0         19       3.23  10635 635 pwsh           UbuntuVM1
      0       0        0         21       4.92  11033 017 pwsh           UbuntuVM1
      0       0        0         20       3.07  11076 076 pwsh           UbuntuVM1
```

Linux to Windows

PowerShell

```
Enter-PSSession -HostName WinVM1 -UserName PTestName
```

```

PTestName@WinVM1s password:
```

PowerShell

```
[WinVM1]: PS C:\Users\PTestName\Documents> cmd /c ver
```

Output

```
Microsoft Windows [Version 10.0.10586]
```

Windows to Windows

PowerShell

```
C:\Users\PSUser\Documents>pwsh.exe
```

Output

```
PowerShell
Copyright (c) Microsoft Corporation. All rights reserved.
```

PowerShell

```
$session = New-PSSession -HostName WinVM2 -UserName PSRemoteUser
```

Output

```
The authenticity of host 'WinVM2 (10.13.37.3)' can't be established.
ECDSA key fingerprint is SHA256:kSU6slAROyQVMEynVIXAdxSiZpwDBigpAF/TXjjWjmw.
Are you sure you want to continue connecting (yes/no)?
Warning: Permanently added 'WinVM2,10.13.37.3' (ECDSA) to the list of known hosts.
PSRemoteUser@WinVM2's password:
```

PowerShell

```
$session
```

Output

```
 Id Name            ComputerName    ComputerType    State
ConfigurationName     Availability
 -- ----            ------------    ------------    -----         ----------------
-     ------------
  1 SSH1            WinVM2          RemoteMachine   Opened        DefaultShell
Available
```

PowerShell

```
Enter-PSSession -Session $session
```

Output

```
[WinVM2]: PS C:\Users\PSRemoteUser\Documents> $PSVersionTable

Name                           Value
----                           -----
PSEdition                      Core
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
SerializationVersion           1.1.0.1
BuildVersion                   3.0.0.0
CLRVersion
PSVersion                      6.0.0-alpha
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
GitCommitId                    v6.0.0-alpha.17


[WinVM2]: PS C:\Users\PSRemoteUser\Documents>
```

Limitations

- The **sudo** command doesn't work in a remote session to a Linux computer.
- PSRemoting over SSH doesn't support Profiles and doesn't have access to `$PROFILE`. Once in a session, you can load a profile by dot sourcing the profile with the full filepath. This isn't related to SSH profiles. You can configure the SSH server to use PowerShell as the default shell and to load a profile through SSH. See the SSH documentation for more information.
- Prior to PowerShell 7.1, remoting over SSH didn't support second-hop remote sessions. This capability was limited to sessions using WinRM. PowerShell 7.1 allows `Enter-PSSession` and `Enter-PSHostProcess` to work from within any interactive remote session.

See also

- Installing PowerShell on Linux
- Installing PowerShell on macOS
- Installing PowerShell on Windows
- Manage Windows with OpenSSH
- Managing OpenSSH Keys

- Ubuntu SSH

Using WS-Management (WSMan) Remoting in PowerShell

Enabling PowerShell remoting

To enable PowerShell remoting, run the `Enable-PSRemoting` cmdlet in an elevated PowerShell session. Running `Enable-PSRemoting` configures a remoting endpoint for the specific installation version that you're running the cmdlet in. For example, when you run `Enable-PSRemoting` while running PowerShell 7.4, PowerShell creates a remoting endpoint runs PowerShell 7.4. If you run `Enable-PSRemoting` while running PowerShell 7-preview, PowerShell creates a remoting endpoint that runs PowerShell 7-preview. You can create multiple remoting endpoints for different versions of that run side-by-side.

Running `Enable-PSRemoting` creates two endpoints for that version.

- One has a simple name corresponding to the PowerShell major version. that hosts the session. For example, **PowerShell.7.4**.
- The other configuration name contains the full version number. For example, **PowerShell.7.4.7**.

You can connect to the latest version of PowerShell 7 host version using the simple name, **PowerShell.7.4**. You can connect to a specific version of PowerShell using the longer, version-specific name.

Use the **ConfigurationName** parameter with the `New-PSSession` and `Enter-PSSession` cmdlets to connect to a named configuration.

Remoting to older versions of Windows

The following prerequisites must be met to enable PowerShell remoting over WSMan on older versions of Windows.

- Install the Windows Management Framework (WMF) 5.1 (as necessary). For more information about WMF, see WMF Overview.
- Install the Universal C Runtime on Windows versions predating Windows 10. It's available via direct download or Windows Update. Fully patched systems already have this package installed.

WSMan remoting isn't supported on non-Windows platforms

Since the release of PowerShell 6, support for remoting over WS-Management (WSMan) on non-Windows platforms is only available to a limited set of Linux distributions. On non-Windows, WSMan relied on the Open Management Infrastructure (OMI) project. The OMI WSMan client depends on **OpenSSL 1.0**. All Linux distributions use **OpenSSL 2.0**, which isn't backward-compatible. There are no supported distributions that have the dependencies needed for the OMI WSMan client to work.

WSMan-based remoting is still supported between Windows systems. Remoting over SSH is supported for all platforms. For more information, see PowerShell remoting over SSH.

Further reading

- Enable-PSRemoting
- Enter-PSSession
- New-PSSession

Last updated on 12/09/2025

Security Considerations for PowerShell Remoting using WinRM

PowerShell Remoting is the recommended way to manage Windows systems. PowerShell Remoting is enabled by default in Windows Server 2012 R2 and higher. This document covers security concerns, recommendations, and best practices when using PowerShell Remoting.

What is PowerShell Remoting?

PowerShell Remoting uses Windows Remote Management (WinRM) to allow users to run PowerShell commands on remote computers. WinRM is the Microsoft implementation of the Web Services for Management (WS-Management) protocol. You can find more information about using PowerShell Remoting at Running Remote Commands.

PowerShell Remoting isn't the same as using the **ComputerName** parameter of a cmdlet to run it on a remote computer, which uses Remote Procedure Call (RPC) as its underlying protocol.

PowerShell Remoting default settings

PowerShell Remoting with WinRM listens on the following ports:

- HTTP: 5985
- HTTPS: 5986

By default, PowerShell Remoting only allows connections from members of the Administrators group. Sessions are launched under the user's context, so all operating system access controls applied to individual users and groups continue to apply to them while connected over PowerShell Remoting.

On private networks, the default Windows Firewall rule for PowerShell Remoting accepts all connections. On public networks, the default Windows Firewall rule allows PowerShell Remoting connections only from within the same subnet. You have to explicitly change that rule to open PowerShell Remoting to all connections on a public network.

> ⚠ **Warning**
>
> The firewall rule for public networks is meant to protect the computer from potentially malicious external connection attempts. Use caution when removing this rule. For more information about configuring WinRM, see Installation and configuration for Windows Remote Management.

Process isolation

PowerShell Remoting uses WinRM for communication between computers. WinRM runs as a service under the Network Service account, and spawns isolated processes running as user accounts to host PowerShell instances. An instance of PowerShell running as one user has no access to a process running an instance of PowerShell as another user.

Event logs generated by PowerShell Remoting

Researchers from Mandiant presented a session at the BlackHat conference that provides a good summary of the event logs and other security evidence generated by PowerShell Remoting sessions. For more information, see Investigating PowerShell Attacks.

Encryption and transport protocols

It's helpful to consider the security of a PowerShell Remoting connection from two perspectives: initial authentication, and ongoing communication.

Regardless of the transport protocol used (HTTP or HTTPS), WinRM always encrypts all PowerShell remoting communication after initial authentication.

Initial authentication

Authentication confirms the identity of the client to the server - and ideally - the server to the client.

When a client connects to a domain server using its computer name, the default authentication protocol is Kerberos. Kerberos guarantees both the user identity and server identity without sending any sort of reusable credential.

When a client connects to a domain server using its IP address, or connects to a workgroup server, Kerberos authentication isn't possible. In that case, PowerShell Remoting relies on the NTLM authentication protocol. The NTLM authentication protocol guarantees the user identity without sending any sort of delegable credential. To prove user identity, the NTLM protocol requires that both the client and server compute a session key from the user's password without ever exchanging the password itself. The server typically doesn't know the user's password, so it communicates with the domain controller, which does know the user's password and calculates the session key for the server.

The NTLM protocol doesn't, however, guarantee server identity. As with all protocols that use NTLM for authentication, an attacker with access to a domain-joined computer's machine

account could invoke the domain controller to compute an NTLM session-key and impersonate the server.

NTLM-based authentication is disabled by default. You can enable NTLM by either configuring SSL on the target server, or by configuring the WinRM TrustedHosts setting on the client.

Using SSL certificates to validate server identity during NTLM-based connections

Since the NTLM authentication protocol can't ensure the identity of the target server (only that it already knows your password), you can configure target servers to use SSL for PowerShell Remoting. Assigning an SSL certificate to the target server (if issued by a Certificate Authority that the client also trusts) enables NTLM-based authentication that guarantees both the user identity and server identity.

Ignoring NTLM-based server identity errors

If deploying an SSL certificate to a server for NTLM connections is infeasible, you can suppress the resulting identity errors by adding the server to the WinRM **TrustedHosts** list. Adding a server name to the **TrustedHosts** list shouldn't be considered as any form of statement of the trustworthiness of the hosts themselves - as the NTLM authentication protocol can't guarantee that you are in fact connecting to the host you're intending to connect to. Instead, you should consider the TrustedHosts setting to be the list of hosts for which you wish to suppress the error generated by being unable to verify the server's identity.

Ongoing Communication

Once initial authentication is complete, the WinRM encrypts the ongoing communication. When you connect over HTTPS, WinRM uses the TLS protocol to negotiate the encryption used to transport data. When you connect over HTTP, WinRM uses the message-level encryption negotiated by the initial authentication protocol.

- Basic authentication provides no encryption.
- NTLM authentication uses an RC4 cipher with a 128-bit key.
- The `etype` in the TGS ticket determines Kerberos authentication encryption. Modern systems use the AES-256 algorithm.
- CredSSP encryption uses the TLS cipher suite negotiated in the handshake.

Making the second hop

By default, PowerShell Remoting uses Kerberos (if available) or NTLM for authentication. Both of these protocols authenticate to the remote machine without sending credentials to it. This is the most secure way to authenticate, but because the remote machine doesn't have the user's credentials, it can't access other computers and services on the user's behalf. This is known as the *second hop problem*.

There are several ways to avoid this problem. For descriptions of these methods, and the pros and cons of each, see Making the second hop in PowerShell Remoting.

References

- Windows Remote Management (WinRM)
- Web Services for Management (WS-Management)
- 2.2.9.1 Encrypted Message Types
- Kerberos
- NTLM authentication protocol
- Investigating PowerShell Attacks

Last updated on 12/09/2025

Making the second hop in PowerShell Remoting

The following scenario outlines the *second hop problem*:

1. You're logged in to *ServerA*.
2. From *ServerA*, you start a remote PowerShell session to connect to *ServerB*.
3. A command you run on *ServerB* via your PowerShell Remoting session attempts to access a resource on *ServerC*.
4. Access to the resource on *ServerC* is denied, because the credentials you used to create the PowerShell Remoting session aren't passed from *ServerB* to *ServerC*.

There are several ways to address this problem. The following table lists the methods in order of preference.

Expand table

Configuration	Note
CredSSP	Balances ease of use and security
Resource-based Kerberos constrained delegation	Higher security with simpler configuration
Kerberos constrained delegation	High security but requires Domain Administrator
Kerberos delegation (unconstrained)	Not recommended
Just Enough Administration (JEA)	Can provide the best security but requires more detailed configuration
PSSessionConfiguration using RunAs	Simpler to configure but requires credential management
Pass credentials inside an `Invoke-Command` script block	Simplest to use but you must provide credentials

CredSSP

You can use the Credential Security Support Provider (CredSSP) for authentication. CredSSP caches credentials on the remote server (*ServerB*), so using it opens you up to credential theft attacks. If the remote computer is compromised, the attacker has access to the user's credentials. CredSSP is disabled by default on both client and server computers. You should enable CredSSP only in the most trusted environments. For example, a domain administrator connecting to a domain controller because the domain controller is highly trusted.

For more information about security concerns when using CredSSP for PowerShell Remoting, see Accidental Sabotage: Beware of CredSSP.

For more information about credential theft attacks, see Mitigating Pass-the-Hash (PtH) Attacks and Other Credential Theft.

For an example of how to enable and use CredSSP for PowerShell remoting, see Enable PowerShell "Second-Hop" Functionality with CredSSP.

Pros

- It works for all servers with Windows Server 2008 or later.

Cons

- Has security vulnerabilities.
- Requires configuration of both client and server roles.
- doesn't work with the Protected Users group. For more information, see Protected Users Security Group.

Kerberos constrained delegation

You can use legacy constrained delegation (not resource-based) to make the second hop. Configure Kerberos constrained delegation with the option "Use any authentication protocol" to allow protocol transition.

Pros

- Requires no special coding
- Credentials aren't stored.

Cons

- Doesn't support the second hop for WinRM.
- Requires Domain Administrator access to configure.
- Must be configured on the Active Directory object of the remote server (*ServerB*).
- Limited to one domain. Can't cross domains or forests.
- Requires rights to update objects and Service Principal Names (SPNs).
- *ServerB* can acquire a Kerberos ticket to *ServerC* on behalf of the user without user intervention.

> ⓘ Note
>
> Active Directory accounts that have the **Account is sensitive and can't be delegated** property set can't be delegated. For more information, see Security Focus: Analysing 'Account is sensitive and can't be delegated' for Privileged Accounts and Kerberos Authentication Tools and Settings.

Resource-based Kerberos constrained delegation

Windows Server 2012 introduced resource-based Kerberos constrained delegation. You configure credential delegation on the server object where resources reside. In the second hop scenario described previously, you configure *ServerC* to specify from where it accepts delegated credentials.

Pros

- Credentials aren't stored.
- Configured using PowerShell cmdlets. No special coding required.
- Doesn't require Domain Administrator access to configure.
- Works across domains and forests.

Cons

- Requires Windows Server 2012 or later.
- Doesn't support the second hop for WinRM.
- Requires rights to update objects and Service Principal Names (SPNs).

> ⓘ Note
>
> Active Directory accounts that have the **Account is sensitive and can't be delegated** property set can't be delegated. For more information, see Security Focus: Analysing 'Account is sensitive and can't be delegated' for Privileged Accounts and Kerberos Authentication Tools and Settings.

Example

Let's look at a PowerShell example that configures resource-based constrained delegation on *ServerC* that allow delegated credentials from a *ServerB*. This example assumes that all servers

are running supported versions of Windows Server, and that there is at least one Windows domain controller for each trusted domain.

Before you can configure constrained delegation, you must add the `RSAT-AD-PowerShell` feature to install the Active Directory PowerShell module, and then import that module into your session:

PowerShell

```
Add-WindowsFeature RSAT-AD-PowerShell
Import-Module ActiveDirectory
Get-Command -ParameterName PrincipalsAllowedToDelegateToAccount
```

Several available cmdlets now have a **PrincipalsAllowedToDelegateToAccount** parameter:

Output

```
CommandType Name                  ModuleName
----------- ----                  ----------
Cmdlet      New-ADComputer        ActiveDirectory
Cmdlet      New-ADServiceAccount  ActiveDirectory
Cmdlet      New-ADUser            ActiveDirectory
Cmdlet      Set-ADComputer        ActiveDirectory
Cmdlet      Set-ADServiceAccount  ActiveDirectory
Cmdlet      Set-ADUser            ActiveDirectory
```

The **PrincipalsAllowedToDelegateToAccount** parameter sets the Active Directory object attribute **msDS-AllowedToActOnBehalfOfOtherIdentity**. This attribute contains an access control list (ACL) that specifies the accounts that have permission to delegate credentials to the associated account. For this example, it's the machine account for *ServerA*.

Now let's create the variables that represent the servers:

PowerShell

```
# Set up variables for reuse
$ServerA = $Env:COMPUTERNAME
$ServerB = Get-ADComputer -Identity ServerB
$ServerC = Get-ADComputer -Identity ServerC
```

WinRM (and therefore PowerShell remoting) runs as the computer account by default. You can see this by looking at the **StartName** property of the `winrm` service:

PowerShell

```
Get-CimInstance Win32_Service -Filter 'Name="winrm"' | Select-Object StartName
```

Output

```
StartName
---------
NT AUTHORITY\NetworkService
```

For *ServerC* to allow delegation from a PowerShell remoting session on *ServerB*, we must set the **PrincipalsAllowedToDelegateToAccount** parameter on *ServerC* to the computer object of *ServerB*:

PowerShell

```powershell
# Grant resource-based Kerberos constrained delegation
Set-ADComputer -Identity $ServerC -PrincipalsAllowedToDelegateToAccount $ServerB

# Check the value of the attribute directly
$x = Get-ADComputer -Identity $ServerC -Properties msDS-
AllowedToActOnBehalfOfOtherIdentity
$x.'msDS-AllowedToActOnBehalfOfOtherIdentity'.Access

# Check the value of the attribute indirectly
Get-ADComputer -Identity $ServerC -Properties PrincipalsAllowedToDelegateToAccount
```

The Kerberos Key Distribution Center (KDC) caches denied-access attempts (negative cache) for 15 minutes. If *ServerB* previously attempted to access *ServerC*, you need to clear the cache on *ServerB* by invoking the following command:

PowerShell

```powershell
Invoke-Command -ComputerName $ServerB.Name -Credential $cred -ScriptBlock {
    klist purge -li 0x3e7
}
```

You could also restart the computer, or wait at least 15 minutes to clear the cache.

After clearing the cache, you can successfully run code from *ServerA* through *ServerB* to *ServerC*:

PowerShell

```powershell
# Capture a credential
$cred = Get-Credential Contoso\Alice

# Test kerberos double hop
Invoke-Command -ComputerName $ServerB.Name -Credential $cred -ScriptBlock {
    Test-Path \\$($Using:ServerC.Name)\C$
    Get-Process lsass -ComputerName $($Using:ServerC.Name)
    Get-EventLog -LogName System -Newest 3 -ComputerName $($Using:ServerC.Name)
}
```

In this example, the `Using:` scope modifier is used to make the `$ServerC` variable visible to *ServerB*. For more information about the `Using:` scope modifier, see about_Remote_Variables.

To allow multiple servers to delegate credentials to *ServerC*, set the value of the **PrincipalsAllowedToDelegateToAccount** parameter on *ServerC* to an array:

PowerShell

```
# Set up variables for each server
$ServerB1 = Get-ADComputer -Identity ServerB1
$ServerB2 = Get-ADComputer -Identity ServerB2
$ServerB3 = Get-ADComputer -Identity ServerB3
$ServerC  = Get-ADComputer -Identity ServerC

$servers = @(
    $ServerB1,
    $ServerB2,
    $ServerB3
)

# Grant resource-based Kerberos constrained delegation
Set-ADComputer -Identity $ServerC -PrincipalsAllowedToDelegateToAccount $servers
```

If you want to make the second hop across domains, use the **Server** parameter to specify fully qualified domain name (FQDN) of the domain controller of the domain to which *ServerB* belongs:

PowerShell

```
# For ServerC in Contoso domain and ServerB in other domain
$ServerB = Get-ADComputer -Identity ServerB -Server dc1.alpineskihouse.com
$ServerC = Get-ADComputer -Identity ServerC
Set-ADComputer -Identity $ServerC -PrincipalsAllowedToDelegateToAccount $ServerB
```

To remove the ability to delegate credentials to ServerC, set the value of the **PrincipalsAllowedToDelegateToAccount** parameter on *ServerC* to `$null`:

PowerShell

```
Set-ADComputer -Identity $ServerC -PrincipalsAllowedToDelegateToAccount $null
```

Information on resource-based Kerberos constrained delegation

- What's New in Kerberos Authentication
- How Windows Server 2012 Eases the Pain of Kerberos Constrained Delegation, Part 1

- How Windows Server 2012 Eases the Pain of Kerberos Constrained Delegation, Part 2
- Understanding Kerberos Constrained Delegation for Microsoft Entra application proxy deployments with Integrated Windows Authentication
- [MS-ADA2 Active Directory Schema Attributes M2.210 Attribute msDS-AllowedToActOnBehalfOfOtherIdentity]MS-ADA2
- [MS-SFU Kerberos Protocol Extensions: Service for User and Constrained Delegation Protocol 1.3.2 S4U2proxy]MS-SFU
- Remote Administration Without Constrained Delegation Using PrincipalsAllowedToDelegateToAccount

Kerberos delegation (unconstrained)

You can also use Kerberos unconstrained delegation to make the second hop. Like all Kerberos scenarios, credentials aren't stored. This method doesn't support the second hop for WinRM.

> ⚠ **Warning**
>
> This method provides no control of where delegated credentials are used. It's less secure than CredSSP. This method should only be used for testing scenarios.

Just Enough Administration (JEA)

JEA allows you to restrict what commands an administrator can run during a PowerShell session. It can be used to solve the second hop problem.

For information about JEA, see Just Enough Administration.

Pros

- No password maintenance when using a virtual account.

Cons

- Requires WMF 5.0 or later.
- Requires configuration on every intermediate server (*ServerB*).

PSSessionConfiguration using RunAs

You can create a session configuration on *ServerB* and set its **RunAsCredential** parameter.

For information about using **PSSessionConfiguration** and **RunAs** to solve the second hop problem, see Another solution to multi-hop PowerShell remoting.

Pros

- Works with any server with WMF 3.0 or later.

Cons

- Requires configuration of **PSSessionConfiguration** and **RunAs** on every intermediate server (*ServerB*).
- Requires password maintenance when using a domain **RunAs** account

Pass credentials inside an Invoke-Command script block

You can pass credentials inside the **ScriptBlock** parameter of a call to the Invoke-Command cmdlet.

Pros

- Doesn't require special server configuration.
- Works on any server running WMF 2.0 or later.

Cons

- Requires an awkward code technique.
- If running WMF 2.0, requires different syntax for passing arguments to a remote session.

Example

The following example shows how to pass credentials in a script block:

PowerShell

```
# This works without delegation, passing fresh creds
# Note $Using:Cred in nested request
$cred = Get-Credential Contoso\Administrator
Invoke-Command -ComputerName ServerB -Credential $cred -ScriptBlock {
    hostname
    Invoke-Command -ComputerName ServerC -Credential $Using:cred -ScriptBlock
```

```
{hostname}
}
```

See also

PowerShell Remoting Security Considerations

Last updated on 12/09/2025

Securing a restricted PowerShell remoting session

There are scenarios where you want to host a PowerShell session that, for security reasons, has been limited to a subset of PowerShell commands.

By definition, a restricted session is one where `Import-Module` isn't allowed to be used. There may be other limitations, but this is the primary requirement. If the user can import a module, then they can run anything they want.

Examples of restricted sessions include:

- Just-Enough-Administration (JEA)
- Custom restricted remoting implementations such as the Exchange and Teams modules

For most system administrators, JEA provides the best experience for creating restricted sessions and should be your first choice. For more information about JEA, see the JEA Overview.

Recommendations for custom session implementations

If your scenario requires a custom implementation, then you should follow these recommendations.

Limit the use and capabilities of PowerShell providers

Review how the allowed providers are used to ensure that you don't create vulnerabilities in your restricted session implementation.

> ⚠ **Warning**
>
> Don't allow the **FileSystem** provider. If users can write to any part of the file system, it's possible to completely bypass security.
>
> Don't allow the **Certificate** provider. With the provider enabled, a user could gain access to stored private keys.

Don't allow commands that can create new runspaces

> ⚠ **Warning**
>
> The Windows Compatibility feature in PowerShell 7 creates a new runspace to host Windows PowerShell. Don't allow any commands that would run via the Windows Compatibility feature. The `*-Job` cmdlets can create new runspaces without the restrictions.

Don't allow commands that add TypeData or FormatData

Restricted endpoints must not expose the `Update-TypeData`, `Remove-TypeData`, or `Update-FormatData` commands. The `*-TypeData` commands allow you to add **ScriptProperty** members to types. The `Update-FormatData` command allows you to add **ScriptBlock** definitions to create custom formatting. The **ScriptProperty** and **ScriptBlock** members might be evaluated in `FullLanguage` mode, even when the session is configured to use a more restrictive language mode.

Don't allow the `Trace-Command` cmdlet.

> ⚠ **Warning**
>
> Using `Trace-Command` brings all traced commands into the session.

Don't create your own proxy implementations for the restricted commands

PowerShell has a set of proxy commands for restricted command scenarios. These proxy commands ensure that input parameters can't compromise the security of the session. The following commands have restricted proxies:

- `Exit-PSSession`
- `Get-Command`
- `Get-FormatData`
- `Get-Help`
- `Measure-Object`
- `Out-Default`
- `Select-Object`

If you create your own implementation of these commands, you may inadvertently allow users to run code prohibited by the JEA proxy commands.

You can run the following command to get a list of restricted commands:

PowerShell

```
$commands = [System.Management.Automation.CommandMetadata]::GetRestrictedCommands(
    [System.Management.Automation.SessionCapabilities]::RemoteServer
)
```

You can examine the restricted proxy commands by using the following command:

PowerShell

```
$commands = [System.Management.Automation.CommandMetadata]::GetRestrictedCommands(
    [System.Management.Automation.SessionCapabilities]::RemoteServer
)
$getHelpProxyBlock =
[System.Management.Automation.ProxyCommand]::Create($commands['Get-Help'])
```

Configure the session to use NoLanguage mode

PowerShell `NoLanguage` mode disables the PowerShell scripting language completely. You can't run scripts or use variables. You can only run native commands and cmdlets.

For more information about language modes, see about_Language_Modes.

Don't allow the debugger to be used in the session

By default, the PowerShell debugger runs code in `FullLanguage` mode. Set the **UseFullLanguageModeInDebugger** property in the **SessionState** to false.

For more information, see UseFullLanguageModeInDebugger.

Last updated on 07/01/2026

PowerShell Remoting FAQ

When you work remotely, you type commands in PowerShell on one computer (known as the "local computer"), but the commands run on another computer (known as the "remote computer"). The experience of working remotely should be as much like working directly at the remote computer as possible.

> **Note**
>
> To use PowerShell remoting, the remote computer must be configured for remoting. For more information, see about_Remote_Requirements.

Must both computers have PowerShell installed?

Yes. To work remotely, the local and remote computers must have PowerShell, the Microsoft .NET Framework, and the Web Services for Management (WS-Management) protocol. Any files and other resources that are needed to execute a particular command must be on the remote computer.

Computers running Windows PowerShell 3.0 and computers running Windows PowerShell 2.0 can connect to each other remotely and run remote commands. However, some features, such as the ability to disconnect from a session and reconnect to it, work only when both computers are running Windows PowerShell 3.0.

You must have permission to connect to the remote computer, permission to run PowerShell, and permission to access data stores (such as files and folders), and the registry on the remote computer.

For more information, see about_Remote_Requirements.

How does remoting work?

When you submit a remote command, the command is transmitted across the network to the PowerShell engine on the remote computer, and it runs in the PowerShell client on the remote

computer. The command results are sent back to the local computer and appear in the PowerShell session on the local computer.

To transmit the commands and receive the output, PowerShell uses the WS-Management protocol. For information about the WS-Management protocol, see WS-Management Protocol in the Windows documentation.

Beginning in Windows PowerShell 3.0, PowerShell store remote session information on the remote computer. This enables you to disconnect from the session and reconnect from a different session or a different computer without interrupting the commands or losing state.

Is PowerShell remoting secure?

When you connect to a remote computer, the system uses the username and password credentials on the local computer or the credentials that you supply in the command to log you in to the remote computer. The credentials and the rest of the transmission are encrypted.

To increase protection, you can configure the remote computer to use Secure Sockets Layer (SSL) instead of HTTP for Windows Remote Management (WinRM) requests. Then, users can use the **UseSSL** parameter of the `Invoke-Command`, `New-PSSession`, and `Enter-PSSession` cmdlets when establishing a connection. This option uses the more secure HTTPS channel instead of HTTP.

Do all remote commands require PowerShell remoting?

No. Some cmdlets have a **ComputerName** parameter that lets you get objects from the remote computer.

These cmdlets don't use PowerShell remoting. You can use them on any computer running PowerShell. The following cmdlets don't require PowerShell remoting:

- `Get-HotFix`
- `Rename-Computer`
- `Restart-Computer`
- `Stop-Computer`

To find all the cmdlets with a **ComputerName** parameter, type:

PowerShell

```
Get-Help * -Parameter ComputerName
# or
Get-Command -ParameterName ComputerName
```

To determine whether the **ComputerName** parameter of a particular cmdlet requires PowerShell remoting, see the parameter description. To display the parameter description, type:

PowerShell

```
Get-Help <cmdlet-name> -Parameter ComputerName
```

For example:

PowerShell

```
Get-Help Get-HotFix -Parameter ComputerName
```

For all other commands, use the `Invoke-Command` cmdlet.

How do I run a command on a remote computer?

To run a command on a remote computer, use the `Invoke-Command` cmdlet. Enclose your command in braces (`{}`). Use the **ScriptBlock** parameter of `Invoke-Command` to specify the command.

Use the **ComputerName** parameter of `Invoke-Command` to specify a remote computer. Or, you can create a persistent connection to a remote computer (a session) and then use the **Session** parameter of `Invoke-Command` to run the command in the session.

For example, the following commands run a `Get-Process` command remotely.

PowerShell

```
Invoke-Command -ComputerName Server01, Server02 -ScriptBlock {Get-Process}

#  - OR -

Invoke-Command -Session $s -ScriptBlock {Get-Process}
```

To interrupt a remote command, type `Ctrl` + `C`. The interruption request is passed to the remote computer, where it terminates the remote command.

For more information about remote commands, see about_Remote and the Help for the cmdlets that support remoting.

Can I just telnet into a remote computer?

You can use the `Enter-PSSession` cmdlet to start an interactive session with a remote computer.

At the PowerShell prompt, type:

PowerShell

```
Enter-PSSession <ComputerName>
```

The command prompt changes to show that you are connected to the remote computer.

```
<ComputerName>\C:>
```

The commands you enter run on the remote computer as though you entered them directly on the remote computer.

To end the interactive session, type:

PowerShell

```
Exit-PSSession
```

An interactive session is a persistent session that uses the WS-Management protocol. It isn't the same as using Telnet, but it provides a similar experience.

For more information, see `Enter-PSSession`.

Can I create a persistent connection?

Yes. You can run remote commands by specifying the name of the remote computer, its NetBIOS name, or its IP address. Or, you can run remote commands by specifying a PowerShell session (PSSession) that's connected to the remote computer.

When you use the **ComputerName** parameter of `Invoke-Command` or `Enter-PSSession`, PowerShell establishes a temporary connection. PowerShell uses the connection to run only the current

command, and then it closes the connection. This behavior works well for running a single command or several unrelated commands, even on many remote computers.

When you use the `New-PSSession` cmdlet to create a PSSession, PowerShell establishes a persistent connection for the PSSession. Then, you can run multiple commands in the PSSession, including commands that share data.

Typically, you create a PSSession to run a series of related commands that share data. Otherwise, the temporary connection created by the **ComputerName** parameter is sufficient for most commands.

For more information about sessions, see about_PSSessions.

Can I run commands on more than one computer at a time?

Yes. The **ComputerName** parameter of the `Invoke-Command` cmdlet accepts multiple computer names, and the **Session** parameter accepts multiple PSSessions.

When you run an `Invoke-Command` command, PowerShell runs the commands on all of the specified computers or in all of the specified PSSessions.

PowerShell can manage hundreds of concurrent remote connections. However, the number of remote commands that you can send can be limited by the resources of your computer and its capacity to establish and maintain multiple network connections.

For more information, see the example in the `Invoke-Command` Help article.

Where are my profiles?

PowerShell profiles aren't run automatically in remote sessions, so the commands that the profile adds aren't present in the session. In addition, the `$PROFILE` automatic variable isn't populated in remote sessions.

To run a profile in a session, use the `Invoke-Command` cmdlet.

For example, the following command runs the **CurrentUserCurrentHost** profile from the local computer in the session in `$s`.

```
```

```
Invoke-Command -Session $s -FilePath $PROFILE
```

The following command runs the **CurrentUserCurrentHost** profile from the remote computer in the session in `$s`.

PowerShell

```
Invoke-Command -Session $s {
  . "$HOME\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1"
}
```

After you run this command, the commands that the profile adds to the session are available in `$s`.

You can also use a startup script in a session configuration to run a profile in every remote session that uses the session configuration.

For more information about PowerShell profiles, see about_Profiles. For more information about session configurations, see `Register-PSSessionConfiguration`.

How does throttling work on remote commands?

To help you manage the resources on your local computer, PowerShell includes a per-command throttling feature that lets you limit the number of concurrent remote connections that are established for each command.

The default is 32 concurrent connections, but you can use the **ThrottleLimit** parameter of the cmdlets to set a custom throttle limit for particular commands.

The throttling feature applies to each command, not to the entire session or to the computer. If you're running commands concurrently in several sessions or PSSessions, the number of concurrent connections is the sum of the concurrent connections in all the sessions.

To find cmdlets with a **ThrottleLimit** parameter, type:

```
Get-Help * -Parameter ThrottleLimit
-or-
Get-Command -ParameterName ThrottleLimit
```

Is the output of remote commands different from local output?

When you use PowerShell locally, you send and receive "live" .NET Framework objects; "live" objects are objects that are associated with actual programs or system components. When you invoke the methods or change the properties of live objects, the changes affect the actual program or component. And, when the properties of a program or component change, the properties of the object that represent them also change.

However, because most live objects can't be transmitted over the network, PowerShell *serializes* the objects returned by remote commands. Serialization converts each object into Common Language Infrastructure XML (CLIXML) data elements for transmission.

When PowerShell receives a serialized object, it converts the CLIXML into a deserialized object type. The deserialized object is an accurate record of the properties of the object, but it's no longer a *live* object. The deserialized object has no methods because they're no longer effective.

Typically, you can use deserialized objects just as you would use live objects, but you must be aware of their limitations. Also, the objects returned by `Invoke-Command` have additional properties that help you to determine the origin of the command.

Some object types, such as DirectoryInfo objects and GUIDs, are converted back into live objects when they're received. These objects don't need any special handling or formatting.

For information about interpreting and formatting remote output, see about_Remote_Output.

Can I run background jobs remotely?

Yes. A PowerShell background job is a PowerShell command that runs asynchronously without interacting with the session. When you start a background job, the command prompt returns immediately. You can continue to work in the session while the job runs.

You can start a background job even while other commands are running because background jobs always run asynchronously in a temporary session.

You can run background jobs on a local or remote computer. By default, a background job runs on the local computer. However, you can use the **AsJob** parameter of the `Invoke-Command` cmdlet to run any remote command as a background job. And, you can use `Invoke-Command` to run a `Start-Job` command remotely.

For more information about background jobs in PowerShell, see about_Jobs and about_Remote_Jobs.

Can I run Windows programs on a remote computer?

You can use PowerShell remote commands to run Windows-based programs on remote computers. For example, you can run `Shutdown.exe` or `Ipconfig.exe` on a remote computer.

However, you can't use PowerShell commands to open the user interface for any program on a remote computer.

When you start a Windows program on a remote computer, the PowerShell command prompt doesn't return until the program is finished or until you press `Ctrl` + `C` to interrupt the command. For example, if you run the `ipconfig.exe` program on a remote computer, the command prompt doesn't return until `ipconfig.exe` is completed.

If you use remote commands to start a program that has a user interface, the program process starts, but the user interface doesn't appear. The program is still running. The command prompt doesn't return until you stop the program or you press `Ctrl` + `C`, which interrupts the command and stops the process.

For example, if you use a PowerShell command to run `Notepad` on a remote computer, the Notepad process starts on the remote computer, but the Notepad user interface doesn't appear. To interrupt the command and restore the command prompt, press `Ctrl` + `C`.

Can I limit the commands that users can run remotely on my computer?

Yes. Every remote session must use one of the session configurations on the remote computer. You can manage the session configurations on your computer. You can set permissions on those session configurations to limit who can run commands remotely on your computer. You can also limit which commands they can run.

A session configuration configures the environment for the session. You can define the configuration by using an assembly that implements a new configuration class or by using a script that runs in the session. The configuration can determine the commands that are available in the session. And, the configuration can include settings that protect the computer, such as

settings that limit the amount of data that the session can receive remotely in a single object or command. You can also specify a security descriptor that determines the permissions that are required to use the configuration.

The `Enable-PSRemoting` cmdlet creates the default session configurations on your computer: Microsoft.PowerShell, Microsoft.PowerShell.Workflow, and Microsoft.PowerShell32 (64-bit operating systems only). `Enable-PSRemoting` sets the security descriptor for the configuration to allow only members of the Administrators group on your computer to use them.

You can use the session configuration cmdlets to edit the default session configurations, to create new session configurations, and to change the security descriptors of all the session configurations.

Beginning in Windows PowerShell 3.0, the `New-PSSessionConfigurationFile` cmdlet lets you create custom session configurations by using a text file. The file includes options for setting the language mode and for specifying the cmdlets and modules that are available in sessions that use the session configuration.

When users use the `Invoke-Command`, `New-PSSession`, or `Enter-PSSession` cmdlets, they can use the **ConfigurationName** parameter to indicate the session configuration that's used for the session. And, they can change the default configuration that their sessions use by changing the value of the `$PSSessionConfigurationName` preference variable in the session.

For more information about session configurations, see the Help for the session configuration cmdlets. To find the session configuration cmdlets, type:

PowerShell

```
Get-Command *PSSessionConfiguration
```

What are fan in and fan out configurations?

The most common PowerShell remoting scenario involving multiple computers is the one-to-many configuration, in which one local computer (the administrator's computer) runs PowerShell commands on numerous remote computers. This is known as the "fan-out" scenario.

However, in some enterprises, the configuration is many-to-one, where many client computers connect to a single remote computer that's running PowerShell, such as a file server or a kiosk. This is known as the "fan-in" configuration.

PowerShell remoting supports both fan-out and fan-in configurations.

For the fan-out configuration, PowerShell uses the Web Services for Management (WS-Management) protocol and the WinRM service that supports the Microsoft implementation of WS-Management. When a local computer connects to a remote computer, WS-Management establishes a connection and uses a plug-in for PowerShell to start the PowerShell host process (Wsmprovhost.exe) on the remote computer. The user can specify an alternate port, an alternate session configuration, and other features to customize the remote connection.

To support the "fan-in" configuration, PowerShell uses internet Information Services (IIS) to host WS-Management, to load the PowerShell plug-in, and to start PowerShell. In this scenario, instead of starting each PowerShell session in a separate process, all PowerShell sessions run in the same host process.

IIS hosting and fan-in remote management aren't supported in Windows XP or in Windows Server 2003.

In a fan-in configuration, the user can specify a connection URI and an HTTP endpoint, including the transport, computer name, port, and application name. IIS forwards all the requests with a specified application name to the application. The default is WS-Management, which can host PowerShell.

You can also specify an authentication mechanism and prohibit or allow redirection from HTTP and HTTPS endpoints.

Can I test remoting on a single computer not in a domain?

Yes. PowerShell remoting is available even when the local computer isn't in a domain. You can use the remoting features to connect to sessions and to create sessions on the same computer. The features work the same as they do when you connect to a remote computer.

To run remote commands on a computer in a workgroup, change the following Windows settings on the computer.

Caution: These settings affect all users on the system and they can make the system more vulnerable to a malicious attack. Use caution when making these changes.

- Windows Vista, Windows 7, Windows 8:

  Create the following registry entry, and then set its value to 1: LocalAccountTokenFilterPolicy in `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`

You can use the following PowerShell command to add this entry:

```powershell
$parameters = @{
  Path='HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System'
  Name='LocalAccountTokenFilterPolicy'
  propertyType='DWord'
  Value=1
}
New-ItemProperty @parameters
```

- Windows Server 2003, Windows Server 2008, Windows Server 2012, Windows Server 2012 R2:

  The default setting of the **Network Access: Sharing and security model for local accounts** policy is `Classic`. Verify the setting is set to `Classic`.

Can I run remote commands on a computer in another domain?

Yes. Typically, the commands run without error. However, you might need to use the **Credential** parameter with `Invoke-Command`, `New-PSSession`, or `Enter-PSSession` cmdlets. The credentials can be required even when the current user is a member of the Administrators group on the local and remote computers.

However, if the remote computer isn't in a domain that the local computer trusts, the remote computer might not be able to authenticate the user's credentials.

To enable authentication, use the following command to add the remote computer to the list of trusted hosts for the local computer in WinRM. Type the command at the PowerShell prompt.

```powershell
Set-Item WSMan:\localhost\Client\TrustedHosts -Value <Remote-computer-name>
```

For example, to add the Server01 computer to the list of trusted hosts on the local computer, type the following command at the PowerShell prompt:

```powershell
Set-Item WSMan:\localhost\Client\TrustedHosts -Value Server01
```

Does PowerShell support remoting over SSH?

Yes. For more information, see PowerShell remoting over SSH.

See also

about_Remote

about_Profiles

about_PSSessions

about_Remote_Jobs

about_Remote_Variables

Invoke-Command

New-PSSession

Last updated on 12/09/2025

Desired State Configuration (DSC) Overview

DSC is a management platform that enables you to manage your IT and development infrastructure with configuration as code.

There are four versions of DSC available:

- **Microsoft DSC 3.0** is the new version of DSC. This version provides true cross-platform support. It is a standalone product that's not dependent on PowerShell, however, you can still use your existing PowerShell DSC resources.
- **PowerShell DSC 3.0 (preview)** is the version of DSC supported by the Azure Machine Configuration on Linux.
- **PowerShell DSC 2.0** is the version of DSC that shipped in PowerShell 7.

  With the release of PowerShell 7.2, the **PSDesiredStateConfiguration** module is no longer included in the PowerShell package. Separating DSC into its own module allows us to invest and develop DSC independent of PowerShell and reduces the size of the PowerShell package. Users of DSC will enjoy the benefit of upgrading DSC without the need to upgrade PowerShell, accelerating the time to deployment of new DSC features. Users that want to continue using DSC v2 can download **PSDesiredStateConfiguration** 2.0.5 from the PowerShell Gallery.
- **PowerShell DSC 1.1** is the legacy version of DSC that originally shipped in Windows PowerShell 5.1.

For more information, see the Desired State Configuration overview article.

Last updated on 01/29/2026

The PowerShell Gallery

Article • 04/13/2023

The PowerShell Gallery is the central repository for PowerShell content. In it, you can find PowerShell scripts, modules containing PowerShell cmdlets and Desired State Configuration (DSC) resources. Some of these packages are authored by Microsoft, and others are authored by the PowerShell community.

The **PowerShellGet** module contains cmdlets for discovering, installing, updating, and publishing PowerShell packages from the PowerShell Gallery. These packages can contain artifacts such as Modules, DSC Resources, Role Capabilities, and Scripts. Make sure you have the latest version of **PowerShellGet** installed.

The documentation for **PowerShellGet** and the PowerShell Gallery has been moved to a new location so that we can manage the version-specific information separate from the versions of PowerShell.

See the new documentation in PowerShellGet and the PowerShell Gallery.

Community Update

A list of resources and a summary of new articles and community contributions.

What's new in Docs?

WHAT'S NEW

2026 Updates

2025 Updates

2024 Updates

2023 Updates

2022 Updates

2021 Updates

2020 Updates

Contributor Hall of Fame

Learning resources

TRAINING

PowerShell 101

Deep Dives

PowerShell Learn modules

VIDEO

Microsoft Virtual Academy videos

Jason Helmick's - The Show

The PSConfEU Channel

The PowerShell.org Channel

Community resources

OVERVIEW

Community support

DOWNLOAD

Digital art

REFERENCE

PowerShell 7 usage stats

Top contributors to PowerShell

Getting support from the community

The PowerShell Community is a vibrant and active group of users. This article can help you get connected with other member of the community.

The PowerShell community can file issues, bugs, or feature requests in our GitHub repository. If you have questions, you may find help from other members of the community in one of these public forums:

- User Groups
- PowerShell Tech Community
- DSC Community
- PowerShell.org
- Stack Overflow
- r/PowerShell subreddit
- PowerShell Virtual User Group - join via:
  - Slack
  - Discord

For information about our support policy, see the PowerShell Support Lifecycle.

Other community resources

- PowerShell Podcast
- PowerShell Summit videos
- PowerShell Conference EU videos

Last updated on 08/19/2026

Community Contributor Hall of Fame

The PowerShell Community is a vibrant and collaborative group. We greatly appreciate all the help and support we get from the community. You can be a contributor too. To learn how, read our Contributor's Guide.

These GitHub users are the All-Time Top Community Contributors. These tables list the top contributors and the total contributions from the community for each year.

Pull Requests merged

Pull Requests help us fix those issues and make the documentation better for everyone.

Expand table

PRs Merged	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	To
Community	9	198	455	470	322	168	101	135	117	94	150	34	22
matt9ucci			157	80	30	1	6						2
nschonni				14	138	10							1
kiazhi		25	79	12									1
alexandair		57	7	24	2	1				1			
sethvs			1	42		20	1	10		6	2	1	
doctordns		5	32	20	7	9	5		1				
surfingoldelephant											58	5	
ArieHein					1					8	25	3	
ehmiiz								22	14				
yecril71pl						21	3	3					
changeworld								3			22		
skycommand			1	3	3	6		1	4	1	4		
Dan1el42		20											
NReilingh		2		13	3								
it-praktyk				16	1								
vors		15	1										

PRs Merged	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	To
kvprasoon		2	1	6	2	2	2						
k-takai				5	1	7							
purdo17				13									
bergmeister			1	3	3	1	1	2	1	1			
markekraus			11	1									
exchange12rocks			7	3			1					1	
alexbuckgit								9	2				
hrxn							2	2	2	5			
baardhermansen					2	1		1	2			5	

GitHub issues opened

GitHub issues help us identify errors and gaps in our documentation.

Expand table

Issues Opened	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	To
Community	6	52	101	216	580	568	368	236	279	238	166	92	29
mklement0			19	60	56	61	28	8	20	24	2		2
ehmiiz								20	14				
iRon7						2	2	2	10	8	8		
iSazonov			1	4	10	8	4	3		1			
jszabo98				2	15	6	1		1	2		2	
surfingoldelephant										6	19	2	
kilasuit					3	2	1	4	1	4	4	4	
peetrike				1		4	2	6	4	3	1	1	
juvtib						15	7						
doctordns			5	3	5	7	1						
JustinGrote				1	3	6	1	1	3	2	3		
vexx32				3	11			3					
rkeithhill			1	2	2	2	3	1	2		1	1	

Issues Opened	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	To
KirkMunro				7	7	1							
alexandair		9	4	2									
clamb123							14						
tabad									11	2			
trollyanov							11	1					
LaurentDardenne			3	2				5	2				
Liturgist					1	1	1	2	5	2			
jsilverm						8			4				
CarloToso									11				
vors	1	6	2	1									
ThomasNieto						3		2	4	1			
UberKluger						1	7	2					
matt9ucci			2	5			2		1				
o-l-a-v					1		1		4	2	2		
ArmaanMcleod									4	6			

Last updated on 09/03/2026

What's new in PowerShell Docs for 2026

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2026-August

Updated content

- Added note about mismatched types in ValidateRange
- Improved the description of settings precedence in PowerShell configuration settings
- Updated release notes for the Aug-2026 releases
- Lots of small updates based on anonymous user feedback

GitHub stats

- 38 PRs merged (7 from Community)
- 25 issues opened (10 from Community, 15 Spam)
- 25 issues closed (10 from Community, 15 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Special thanks to @ArieHein for another large PR to clean up 160 articles.

Expand table

GitHub Id	PRs merged	Issues opened
Meterel	3	
ArieHein	2	
michael-hollingsworth	1	
Sylvain13bdr	1	

2026-July

Updated content

- Release notes for monthly maintenance releases of PowerShell
- More updates to PSScriptAnalyzer rules
- `New-Guid` changed in PowerShell 7.7 to create v7 format UUIDs

GitHub stats

- 23 PRs merged (2 from Community)
- 38 issues opened (7 from Community, 30 Spam)
- 38 issues closed (8 from Community, 30 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Special thanks to @ArieHein for another large PR to clean up 160 articles.

Expand table

GitHub Id	PRs merged	Issues opened
janv8000	1	
SufficientDaikon	1	

2026-June

Updated content

- Release notes for monthly maintenance releases of PowerShell
- Major cleanup of PSScriptAnalyzer rules documentation, including refactoring the rules table. The rules table now shows which rules are always enabled, and which rules can be configured or disabled.

GitHub stats

- 46 PRs merged (8 from Community)
- 42 issues opened (11 from Community, 31 Spam)
- 43 issues closed (12 from Community, 31 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Special thanks to @ArieHein for another large PR to clean up 160 articles.

Expand table

GitHub Id	PRs merged	Issues opened
ArieHein	1	
baardhermansen	4	
michael-hollingsworth	1	
naffee	1	
PtJade-Ceramic	1	1
trashCode		2

2026-May

Updated content

- Updated release notes for minor releases of 7.4, 7.5, 7.6, and 7.7-preview.
- Update release notes for Microsoft.PowerShell.PSResourceGet v1.3.0-preview1
- Start work on PSScriptAnalyzer rule documentation refresh
  - Standardizing the structure and formatting
  - Add links to related documentation
  - Grammar and style edits

GitHub stats

- 30 PRs merged (4 from Community)
- 44 issues opened (10 from Community, 33 Spam)
- 46 issues closed (10 from Community, 33 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Anyplace7803	1	
exchange12rocks	1	
Gijsreyn	1	
jsuther1974	1	

2026-April

New content

- about_Error_Handling
  - Special thanks to @SufficientDaikon for their hard work creating this new article and updating all related content.
- PowerShell 7.7-preview.1 release
  - What's New in PowerShell 7.7 - PowerShell
  - Now, you can select PowerShell 7.7 from the version selector to view documentation PowerShell 7.7 cmdlet reference.

Updated content

- Updated release notes for minor releases of 7.4, 7.5, and 7.6.

GitHub stats

- 26 PRs merged (4 from Community)
- 40 issues opened (8 from Community, 29 Spam)
- 40 issues closed (9 from Community, 29 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Anyplace7803	1	

GitHub Id	PRs merged	Issues opened
Gijsreyn	1	
MukundaKatta	1	
SufficientDaikon	1	1

2026-March

New and updated content

- Updated release notes for PowerShell 7.6.0 GA release
- Major rework of setup articles to simplify the structure
  - Moved Homebrew installation instructions for macOS to Alternate ways to install PowerShell
- Updated PSScriptAnalyzer docs for v1.25 release including 6 new or updated rules
  - AlignAssignmentStatement
  - AvoidLongLines
  - UseConsistentParameterSetName
  - UseConsistentParametersKind
  - UseConstrainedLanguageMode
  - UseSingleValueFromPipelineParameter

GitHub stats

- 40 PRs merged (3 from Community)
- 56 issues opened (18 from Community, 33 Spam)
- 47 issues closed (12 from Community, 33 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
baardhermansen	1	
derkallevombau	1	

GitHub Id	PRs merged	Issues opened
dodexahedron	1	
SufficientDaikon		3
kilasuit		2

2026-February

New and updated content

- Update PSResourceGet release notes for RC3
- Other general maintenance updates

GitHub stats

- 20 PRs merged (4 from Community)
- 46 issues opened (13 from Community, 32 Spam)
- 45 issues closed (13 from Community, 32 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
manpritxelentor	1	
putku45	1	
sethvs	1	
umutKaracelebi	1	

2026-January

New and updated content

- New article - Windows PowerShell update message
- Updated How to use the PowerShell documentation
- Updated AI Shell docs to reflect the deprecated status

- Updated PowerShellGet docs
  - Install a package manager for PowerShell
  - Bootstrapping NuGet
- Retired PowerShellGet 1.x docs - 41 articles removed

GitHub stats

- 49 PRs merged (7 from Community)
- 48 issues opened (17 from Community, 31 Spam)
- 56 issues closed (24 from Community, 32 Spam)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
surfingoldelephant	5	2
taku-nm	1	
alexvy86	1	

Last updated on 09/03/2026

What's new in PowerShell Docs for 2025

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2025-December

New content

- Troubleshoot PowerShell startup issues

Updated content

- Updated PSResourceGet docs for 1.2.0-preview5 release
  - Use the Azure Artifacts Credential Provider with Azure Artifacts feeds
  - Retired 26 obsolete reference articles for PowerShellGet proxy module
- Updated Invoke-WebRequest in PowerShell 5.1 for a breaking change introduced by a CVE security patch.

GitHub stats

- 22 PRs merged (3 from Community)
- 31 issues opened (16 from Community)
- 30 issues closed (15 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
surfingoldelephant	3	4
kborowinski	1	
aberus	1	
iRon7		3

2025-November

Content updates

- Retired the Windows PowerShell Programmer's Guide
  - 82 out-of-date articles moved to the archive site
- Retired the Windows PowerShell ISE Scripting Object Model
  - 15 articles moved to the archive site
- Refactored the instructions for installing PowerShell on Windows
  - Split out instructions for Install PowerShell on Windows IoT and Nano Server

GitHub stats

- 20 PRs merged (2 from Community)
- 34 issues opened (11 from Community)
- 32 issues closed (9 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
alirobe	1	
sleggy	1	
iRon7		2

2025-October

Content updates

- What's New in PowerShell 7.6
- New PlatyPS documentation
  - Types of help in PowerShell
  - Create new command help Markdown using PlatyPS
  - Update Markdown help files using Microsoft.PowerShell.PlatyPS
  - Edit Markdown help files
  - Test Markdown help files
  - Convert and publish the help files

GitHub stats

- 18 PRs merged (3 from Community)
- 32 issues opened (29 from Community)
- 34 issues closed (29 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
OscarFalck17	1	
StuartMosquera	1	
kMitty		2
MartinGC94		2

2025-September

Content updates

- Update release notes for PowerShell v7.5.3 and v7.4.12 releases
- Update release notes for PSReadLine v2.4.4-beta4
- Improved the documentation for comment-based help and creating `about_*` topics

GitHub stats

- 21 PRs merged (6 from Community)
- 28 issues opened (25 from Community)
- 31 issues closed (28 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
skycommand	2	
lukebarone	1	
PraveenGururaj	1	
Blake-Madden	1	
Gijsreyn	1	
TMA-2	1	1

2025-August

Content updates

- AI Shell updates for preview 6 release
- MCP support
- DSC updates for v3.1.1 release

GitHub stats

- 16 PRs merged (3 from Community)
- 34 issues opened (31 from Community)
- 36 issues closed (32 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
changeworld	2	
Dickson-Mwendia	1	
chwarr	1	

2025-July

New content

- The Monad Manifesto - No new but certainly a classic.

Content updates

- In July, we completed the deployment of a new build pipeline for PowerShell reference content. Now all reference content for Microsoft products and services is using the new build system. The webpage has a new right-hand navigation menu and we greatly improved the documentation of parameters by including full details about each parameter set.

GitHub stats

- 19 PRs merged (8 from Community)
- 31 issues opened (31 from Community)
- 30 issues closed (30 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
changeworld	3	
sethvs	2	
yashanshh	2	
Gijsreyn	1	
TNGBBK		2

2025-June

No new content this month. Only minor edits and bug fixes.

GitHub stats

- 18 PRs merged (5 from Community)
- 22 issues opened (22 from Community)
- 22 issues closed (22 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
skycommand	2	
JustinGrote	1	
OranguTech	1	
changeworld	1	
ThomasNieto	1	
SamErde	1	
vlad1300		2

2025-May

Content updates

- Add new article - Optimize performance using parallel execution
- Updated release notes for the PowerShell 7.4.10 release
- Updated release notes for AIShell 1.0.0-preview.4
- Updated docs for PSResoureGet 1.1.1 release - now supporting MAR
- Retired PowerShell content for Windows Server 20212/2012r2/MDOP (5338 articles)

GitHub stats

- 27 PRs merged (10 from Community)
- 32 issues opened (31 from Community)
- 31 issues closed (30 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
changeworld	3	
cflanderscpc	1	
HeyItsGilbert	1	
dodexahedron	1	
MaratMussabekov	1	
geeksbsmrt	1	
SP3269	1	
ajansveld	1	
robinmalik	1	
andreshungbz	1	
mklement0		2

2025-April

Content updates

- Updated release notes for the 7.5.1 and 7.6-preview.4 releases
- Refactored documentation of filter functions
- Documented bugs with CIM/CDXML bugs and PipelineVariable parameter

GitHub stats

- 24 PRs merged (6 from Community)
- 23 issues opened (22 from Community)
- 23 issues closed (23 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
surfingoldelephant	3	1

GitHub Id	PRs merged	Issues opened
IISResetMe	1	
changeworld	1	
indented-automation	1	

2025-March

Content updates

- DSC v3 GA release - Complete reorg of the documentation as well as updating for the GA release
- PSScriptAnalyzer v1.24.0 release - updated docs for release
- Tons for quality improvements from surfingoldelephant
  - 38 PRs cleaning up 1505 files

GitHub stats

- 68 PRs merged (40 from Community)
- 30 issues opened (28 from Community)
- 30 issues closed (28 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
surfingoldelephant	38	2
Allyyyyy	1	
jborean93	1	
JustinGrote		2

2025-February

Content updates

- Updated the Microsoft Update FAQ
- Tons of quality improvements from the community
  - surfingoldelephant - 13 PRs on 497 files
  - ArieHein - 10 PRs on 140 files

GitHub stats

- 52 PRs merged (29 from Community)
- 26 issues opened (25 from Community)
- 28 issues closed (26 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
surfingoldelephant	13	3
ArieHein	10	
changeworld	4	
deadlydog	2	
piedquance		4

2025-January

New content

- about_Comments - thanks to @surfingoldelephant
- Created reference content for PowerShell 7.6-preview
- What's New in PowerShell 7.6

Updates

- Updates for PowerShell 7.5.0 GA release
  - What's New in PowerShell 7.5
  - Release history of modules and cmdlets
- Special thanks to @ArieHein for his contributions (15 PRs on 234 files) to fix typos and adherence to style guidelines.

GitHub stats

- 74 PRs merged (30 from Community)
- 32 issues opened (32 from Community)
- 37 issues closed (37 Community issues closed)

Top Community Contributors

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ArieHein	15	1
changeworld	8	
surfingoldelephant	4	5
cnotin	1	
daniel-brandenburg	1	
o-l-a-v	2	

Last updated on 01/05/2026

What's new in PowerShell Docs for 2024

Article • 03/30/2025

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2024-December

- Added about_Type_Conversion
- Updated and improved documentation for several PSScriptAnalyzer rules
  - AvoidDefaultValueSwitchParameter
  - AvoidGlobalFunctions
  - AvoidOverwritingBuiltInCmdlets
  - AvoidUsingCmdletAliases
  - AvoidUsingWriteHost
  - PlaceOpenBrace
  - PossibleIncorrectComparisonWithNull
  - PossibleIncorrectUsageOfAssignmentOperator
  - ProvideCommentHelp
  - UseApprovedVerbs
  - UseCompatibleCmdlets
  - UseCompatibleCommands
  - UseCompatibleTypes
  - UseShouldProcessForStateChangingFunctions
  - UseSupportsShouldProcess

Top Community Contributors

GitHub stats

- 22 PRs merged (7 from Community)
- 30 issues opened (30 from Community)
- 24 issues closed (24 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
sethvs	3	
ArieHein	1	
bharathalleni	1	
jhribal	1	
Saibamen	1	
skycommand	1	
surfingoldelephant		6
ArmaanMcleod		2
dpareit		2

2024-November

New and updated content

- 15 new article about AI Shell - What is AI Shell?
- Updated What's New in PowerShell 7.5 for the RC1 release

Top Community Contributors

GitHub stats

- 21 PRs merged (9 from Community)
- 14 issues opened (14 from Community)
- 14 issues closed (14 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ArieHein	5	
abeu1	1	

GitHub Id	PRs merged	Issues opened
alexandair	1	
igoragoli	1	
jmillerca	1	
SamB	1	
uiolee	1	1

2024-October

New content

- Improve the accessibility of DSC output in PowerShell
- Added cmdlet reference for the Microsoft.PowerShell.PlatyPS Module v1.0.0-preview1 release

Top Community Contributors

GitHub stats

- 30 PRs merged (11 from Community)
- 22 issues opened (22 from Community)
- 22 issues closed (22 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
jhribal	4	
ArieHein	2	
Bergbok	1	1
colinwebster-hc	1	
HotCakeX	1	
pmsjt	1	

GitHub Id	PRs merged	Issues opened
CameronSWilliamson	1	
FARICJH59		2

2024-September

New content

- Improve the accessibility of output in PowerShell
- What's new in PSResourceGet?
- about_PSReadLine_Release_Notes

Top Community Contributors

GitHub stats

- 29 PRs merged (1 from Community)
- 19 issues opened (19 from Community)
- 20 issues closed (20 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
iRon7		5
jikuja		2

2024-August

- Updates for new releases of PowerShell 7.2.23, 7.4.5 and 7.5-preview.4
- Two new cmdlets in 7.5-preview.4
  - ConvertFrom-CliXml
  - ConvertTo-CliXml

Top Community Contributors

GitHub stats

- 25 PRs merged (4 from Community)
- 20 issues opened (18 from Community)
- 20 issues closed (17 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
RokeJulianLockhart	1	
mohitNarang	1	
aberus	1	

2024-July

- Updates for new releases of PowerShell 7.2.22 and 7.4.4
- Improved registry examples in:
  - Working with registry entries
  - about_Registry_Provider

Top Community Contributors

GitHub stats

- 38 PRs merged (11 from Community)
- 20 issues opened (20 from Community)
- 24 issues closed (24 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
guillermooo	4	
darthwalsh	2	

GitHub Id	PRs merged	Issues opened
shekeriev	1	
Blake-Madden	1	
paaspaas00	1	
ninmonkey	1	1
o-l-a-v	1	

2024-June

New content

- TabExpansion2

Top Community Contributors

GitHub stats

- 35 PRs merged (5 from Community)
- 16 issues opened (13 from Community)
- 16 issues closed (13 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
mklement0		2
sethvs	2	
InvalidAccountNameEntered	1	
OS3DrNick	1	
trackd	1	
landon-lengyel	1	
msklv	1	

2024-May

Retired PowerShell 7.3 content

- Moved 467 articles to *Previous Versions* PowerShell content archive

Updated content for PowerShell 7.5-preview.3 release

- What's New in PowerShell 7.5

New content for DSC v3 alpha release

- See Desired State Configuration changelog

Top Community Contributors

GitHub stats

- 31 PRs merged (11 from Community)
- 22 issues opened (21 from Community)
- 19 issues closed (18 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
berlintay	1	
dcarpenter31	1	
i5513		2
MatejKafka	1	
real-guanyuming-he	1	
santisq	2	
stephanadler1	1	
stevenjudd	2	

2024-Apr

New content for PSResourceGet

- Supported repository configurations
  - Added Azure Container Registry information
- Use ACR repositories with PSResourceGet

Refreshed and reorganized Security content to be more discoverable

- PowerShell Security

New content for DSC v3 alpha.7 release

- See Desired State Configuration changelog

Top Community Contributors

GitHub stats

- 31 PRs merged (11 from Community)
- 22 issues opened (21 from Community)
- 19 issues closed (18 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Blake-Madden	3	
Hrxn	2	
aberus	1	
nickcox	1	
kilasuit	1	
glenn-slayden	1	
darthwalsh	1	
landon-l8	1	
joshooaj	1	
mklement0		5

GitHub Id	PRs merged	Issues opened
RokeJulianLockhart		2

2024-Mar

- DSC v3.0-alpha.5 release
  - See the updated Changelog
- PSScriptAnalyzer 1.22.0 release - 3 new rules
  - AvoidMultipleTypeAttributes
  - AvoidSemicolonsAsLineTerminators
  - AvoidUsingBrokenHashAlgorithms
  - See the CHANGELOG for a complete list of updates

Top Community Contributors

GitHub stats

- 30 PRs merged (1 from Community)
- 31 issues opened (28 from Community)
- 30 issues closed (26 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Hrxn	3	
muzimuzhi	1	
bergmeister	1	

2024-Feb

- PowerShell 7.5.0-preview.2 release
  - See What's New in PowerShell 7.5
- Lots of updates for the DSC v3-alpha.5 release including 6 new articles
  - See Desired State Configuration changelog

- Worked with the Windows team to publish preview content for Windows Server 2025
  - This work included changes to the reference content and to the module source code to resolve errors reported by `Update-Help`

Top Community Contributors

GitHub stats

- 31 PRs merged (6 from Community)
- 26 issues opened (23 from Community)
- 27 issues closed (25 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Certezalito	1	
sethvs	1	
chrisdent-de	1	
jborean93	1	
brucificus	1	
XPlantefeve	1	
mklement0		4

2024-Jan

- PowerShell 7.5.0-preview.1 release
  - See What's New in PowerShell 7.5
- Lots of updates for the DSC v3-alpha.4 release including 6 new articles
  - See Desired State Configuration changelog

Top Community Contributors

GitHub stats

- 37 PRs merged (6 from Community)
- 36 issues opened (35 from Community)
- 36 issues closed (34 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
b-rad15	1	
flcdrg	1	
holtkampjs	1	
JamesDBartlett3	1	
MuzaffarNurillaew	1	
ybeltrikov	1	
ArmaanMcleod		3
mklement0		3
deraeler		2
J0F3		2
tabad		2
wwilliams69		2

What's new in PowerShell Docs for 2023

Article • 03/30/2025

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2023-Dec

Lots of minor updates but no new content. The Docs team is taking a break for the holidays.

Top Community Contributors

GitHub stats

- 24 PRs merged (4 from Community)
- 21 issues opened (18 from Community)
- 18 issues closed (15 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Blake-Madden	2	
shyguyCreate	1	
ryanperrymba	1	
jp2images	1	
mklement0		2

2023-Nov

Updated content

- Updated release notes for PowerShell 7.4.0 GA
  - What's New in PowerShell 7.4 - PowerShell
  - PowerShell Gallery - Microsoft.PowerShell.WhatsNew 0.5.4
  - PowerShell SDK 7.4 reference - .NET API browser
- Retired Windows PowerShell content to archive site
  - Added What is Windows PowerShell?
- Major updates to: about_Classes and about_Enum

New content

- New Class articles added
  - about Classes Constructors
  - about Classes Inheritance
  - about Classes Methods
  - about Classes Properties

Top Community Contributors

GitHub stats

- 37 PRs merged (7 from Community)
- 32 issues opened (31 from Community)
- 33 issues closed (31 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
jmiller76	2	
VShrubowich	1	
TEBandCo	1	
skycommand	1	
darthwalsh	1	
diecknet	1	
tabad		7
mklement0		2

GitHub Id	PRs merged	Issues opened
radkedan		2

2023-Oct

New content

- Securing a restricted PowerShell remoting session

Updated content

- Updated release notes for PowerShell 7.4-rc.1
- Updated release notes for PSReadLine GA 2.3.4
- Updated release notes for PSResourceGet 1.0.0

Docs platform changes

- Released new feedback experience at the bottom of each page

Top Community Contributors

GitHub stats

- 55 PRs merged (11 from Community)
- 41 issues opened (37 from Community)
- 37 issues closed (33 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
baardhermansen	1	
diecknet	1	
dipstar	1	
ehmiiz	1	1
G2-Games	1	
indented-automation	1	

GitHub Id	PRs merged	Issues opened
jhribal	1	
Joeselorm	1	
matziq	1	
RAJU2529	1	
ThomasNieto	1	2
iRon7		2
JustinWebDev		2
o-l-a-v		2
tabad		4

2023-Sep

New content

- How to create a feedback provider

Updated Content

- Updated What's new in PowerShell 7.4 for PowerShell 7.4-preview.6
- Documented the changes to search scope in How to use the documentation
- Updated What's new in Crescendo 1.1 for the GA release
- Updated the setup scripts for supported Linux distributions
- Updated DSC v3 content for the alpha.3 release

Quality improvement project contributions

- @ehmiiz contributed 5 PRs to update 35 files

New Learn platform features

- Deployed the new feedback experience at that bottom of each page

Top Community Contributors

GitHub stats

- 52 PRs merged (9 from Community)

- 24 issues opened (22 from Community)
- 24 issues closed (23 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ehmiiz	5	5
bubbletroubles	1	
JamesDBartlett3	1	
not-not-kevin	1	
skycommand	1	
jsilverm		3
mklement0		2

2023-Aug

New content

66 New articles for DSC v3 (alpha)

- See Microsoft Desired State Configuration v3 overview to get started with the new documentation

Updated Content

- Crescendo 1.1-RC1 release updates
  - What's new in Crescendo 1.1
- New PSResourceGet beta24 content updates
  - See Supported repository configurations
- Lots updates for the PowerShell 7.4-preview.5 release
  - What's New in PowerShell 7.4 (preview)
  - New features for 10 cmdlets
- Updated support status and installation notes for Raspberry Pi
  - Community support for PowerShell on Linux - PowerShell

Top Community Contributors

GitHub stats

- 29 PRs merged (3 from Community)
- 16 issues opened (12 from Community)
- 21 issues closed (17 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ehmiiz	1	1
crisman	1	
deadlydog	1	
JamesDBartlett3	1	

2023-Jul

Updated content

- 7.4-preview.4 release notes
- Add publish information to Supported repository configurations
- Updated the release notes shipped with Get-WhatsNew
- Fixed invalid ///-comments in SDK API reference
- Updated the man page that ships in PowerShell for Linux and macOS

Top Community Contributors

GitHub stats

- 31 PRs merged (10 from Community)
- 21 issues opened (17 from Community)
- 17 issues closed (13 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
crisman	2	3
TSanzo-BLE	2	
MilekJakub	1	
Atman-Shastri	1	
BraveJhawk	1	
coolhome	1	
johndward01	1	
lor3k	1	

2023-Jun

New Azure Cloud Shell content

- Using Cloud Shell in an Azure virtual network
- Deploy Azure Cloud Shell in a VNET with quickstart templates

New PowerShellGet v3 content

- Supported repository configurations
- Cmdlet reference for Microsoft.PowerShell.PSResourceGet
- Cmdlet reference for the PowerShellGet compatibility module

Lot of updates for the PowerShell 7.4-preview.4 release

- What's New in PowerShell 7.4 (preview)
- Using Experimental Features in PowerShell
- New features for 12 cmdlets

Quality Project contributions

- @XXLMandalorian013 contributed 3 PRs to update 2 files in the MicrosoftDocs/windows-docs-powershell repository

Top Community Contributors

GitHub stats

- 59 PRs merged (15 from Community)
- 44 issues opened (30 from Community)
- 47 issues closed (31 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
thegraffix	4	
crisman	4	3
RAJU2529	1	
dstreefkerk	1	
vontompers	1	
mubed	1	
Frederisk	1	
khaffner	1	
noamper	1	
aksarben		4

2023-May

New content

- New cmdlet in 7.4 - Get-SecureRandom
- Using Windows Defender Application Control

Updated content

- How to use the PowerShell documentation
  - Added descriptions for various navigation elements on the site
  - Added information about how to use the new **Download PDF** feature
- Exploring the Windows PowerShell ISE
  - Added screenshots for the ISE user interface elements
- Update documentation for Add-Member
  - Corrected some parameter values

    - Improved the description and added examples for the **SecondValue** parameter

Quality Project contributions

- @robderickson contributed 1 PR to update 10 files in the MicrosoftDocs/windows-docs-powershell repository

Top Community Contributors

GitHub stats

- 38 PRs merged (7 from Community)
- 24 issues opened (18 from Community)
- 20 issues closed (17 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
rwp0	1	
pronichkin	1	
Ooggle	1	
mavaddat	1	
IanKemp	1	
mcdonaldjc	1	
Brizio	1	
dotnvo	1	
r0bfr	1	
mklement0		3
aksarben		2
crisman		2

2023-April

New content

- Handling errors in Crescendo
- Transforming arguments in Crescendo

Updated content

- Updated release notes for PowerShell 7.4-preview.3
- Migrated the PowerShell Gallery and PowerShellGet docs to new location to enable version selectors for PowerShellGet

PowerShell Summit 2023 - Hack-a-Doc event

- We hosted a Hack-a-Doc event on April 27th. Special thanks to the following 19 people. They contributed 62 PRs to update 204 files in the MicrosoftDocs/windows-powershell-docs repository.

Expand table

GitHub Id	name	Count of PRs	Count of file
RobBiddle	Robert Biddle	28	48
pbossman	Phil Bossman	1	27
ThomasNieto	Thomas Nieto	1	24
kevinCefalu	Kevin Cefalu	1	24
robderickson	Rob Derickson	4	17
Snozzberries	Michael Soule	13	16
Spoonsk	Joseph Gast	1	12
thedavecarroll	Dave Carroll	1	11
raynbowbrite	Vanda Paladino	2	7
majst32	Melissa Januszko	1	6
XXLMandalorian013	Drew McClellan	1	4
ThePoShWolf	Anthony Howell	1	1
mdowst	Matthew Dowst	1	1
thepowerstring		1	1
KevinMarquette	Kevin Marquette	1	1

GitHub Id	name	Count of PRs	Count of file
53883		1	1
zockan	Michael Svegmar	1	1
lanwench	Paula Kingsley	1	1
stevenjudd	Steven Judd	1	1
Grand Total		62	204

Top Community Contributors

GitHub stats

- 23 PRs merged (2 from Community)
- 16 issues opened (14 from Community)
- 17 issues closed (15 Community issues closed)

The following people contributed to PowerShell docs this month by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
NLZ	1	
Jonathan-Quilter	1	

2023-March

New content

- Create a class-based DSC Resource for Machine Configuration
- Using PSReadLine key handlers

Updated content

- Release notes for PowerShell 7.4-preview.2

Quality project updates from the community

- One of our top contributors @ehmiiz blogged about contributing to Docs

- How to Learn Git, Markdown and PowerShell by Contributing to the PowerShell-Docs Repository

Top Community Contributors

GitHub stats

- 60 PRs merged (13 from Community)
- 44 issues opened (31 from Community)
- 50 issues closed (36 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
skycommand	2	
martincostello	1	
iRon7	1	4
chrullrich	1	
FlintyLemming	1	
ehmiiz	1	
vvavrychuk	1	
bb-froggy	1	
BenjamimCS	1	
kirillkrylov	1	
bergmeister	1	
lizy14	1	
CarloToso		5
MartinGC94		2
rgl		2

2023-February

New Content

- Preventing script injection attacks (Thanks @PaulHigin)

Content updates

- Major update to about_PowerShell_Config
- Update to about_Logging_Non-Windows for macOS instructions
- Major update to Class-based DSC Resources and other related articles for DSC v2

Quality project updates from the community

- Added alias information to 4 cmdlet articles (Thanks @ehmiiz!)

Top Community Contributors

GitHub stats

- 35 PRs merged (8 from Community)
- 20 issues opened (14 from Community)
- 17 issues closed (10 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
joshua-russell	4	1
1NF053C	1	
doctordns	1	
Hrxn	1	
KyleMit	1	
VertigoRay	1	
ehmiiz	1	
ArmaanMcleod		2
mklement0		2

2023-January

New Content

- What's new in PowerShell 7.4 (preview)
- about_Data_Files

Content updates

- Updated docs for 7.4-preview.1 release
- Major update to about_Language_Modes
- Major update to about_Logging_Non-Windows

Quality project updates from the community

- Added alias information to 40 cmdlet articles (Thanks @ehmiiz !)
- Added alias information to 52 cmdlet articles (Thanks @szabolevo !)

Top Community Contributors

GitHub stats

- 71 PRs merged (21 from Community)
- 53 issues opened (33 from Community)
- 62 issues closed (40 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ehmiiz	5	5
szabolevo	5	7
anderjef	2	
turbedi	1	
desk7	1	
cobrabr	1	
ZYinMD	1	

GitHub Id	PRs merged	Issues opened
tompazourek	1	
kenyon	1	
cjvandyk	1	
JTBrinkmann	1	
mklement0		4
CarloToso		3
KyleMit		2
iRon7		2

What's new in PowerShell Docs for 2022

Article • 03/30/2025

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2022-December

New Content

- about_PSItem
- Configuring a light colored theme
- What's new in Crescendo 1.1
- Export-CrescendoCommand
- PowerShell 7.4 (preview) cmdlet reference - a direct copy of the 7.3 content in preparation for the preview release of PowerShell 7.4

More Quality project updates

- Added alias information to 83 cmdlet articles (Thanks @ehmiiz!)
- Added alias information to 8 cmdlet articles (Thanks @szabolevo!)

GitHub stats

- 51 PRs merged (14 from Community)
- 50 issues opened (28 from Community)
- 46 issues closed (23 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ehmiiz	8	7
changeworld	3	
szabolevo	1	

GitHub Id	PRs merged	Issues opened
amkhrjee	1	
xtqqczze	1	3
ALiwoto	1	2
mklement0		3

2022-November

New Content

- Contributing quality improvements
  - See examples under Quality project updates
- Product terminology and branding guidelines
- Labelling in GitHub

Content updates

- Updated release notes for the PowerShell 7.3 GA release
- Updated about_Telemetry
- Improved the description of delay-binding in about_Script_Blocks
- Added a best practice recommendation to about_Functions_Advanced_Parameters

Quality project updates

- Added alias information to 129 cmdlet articles (Thanks @ehmiiz!)
- Added links to PRs in the PowerShell 7.3 release notes (Thanks @skycommand!)
- Converted hyperlinks to link references in 5 articles (Thanks @chadmando!)

GitHub stats

- 52 PRs merged (12 from Community)
- 41 issues opened (27 from Community)
- 42 issues closed (28 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ehmiiz	9	8
chadmando	1	
baardhermansen	1	
skycommand	1	
mklement0		3
peetrike		2

2022-October

New Content

- Create a class-based DSC Resource
- Hacktoberfest and other hack-a-thon events

Content updates

- Hacktoberfest 2022 cleanup efforts
  - Thank you to @ehmiiz, @TSanzo-BLE, and @chadmando for their Hacktoberfest PRs! Their 11 PRs touched 114 articles.
- Published PowerShell SDK .NET API content for PowerShell 7.2 and 7.3-preview
  - The first updates since PowerShell 7.1 released in November 2020
  - Removed the unsupported versions 6.0 and 7.1
- Added a list of aliases not available on Linux and macOS to PowerShell differences on non-Windows platforms

GitHub stats

- 65 PRs merged (21 from Community)
- 42 issues opened (28 from Community)
- 34 issues closed (23 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ehmiiz	5	5

GitHub Id	PRs merged	Issues opened
TSanzo-BLE	4	
yecril71pl	2	
chadmando	2	
GigaScratch	1	1
rbleattler	1	
spjeff	1	
adamdriscoll	1	
manuelcarriernunes	1	
michelangelobottura	1	
dmpe	1	
KamilPacanek	1	
SetTrend		2

2022-September

No new content this month.

Content updates

- PSScriptAnalyzer 1.21 update release
- Release notes for 7.3-preview.8

GitHub stats

- 31 PRs merged (8 from Community)
- 18 issues opened (9 from Community)
- 16 issues closed (8 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
DonaldDWebster	1	
emerconghaile	1	
floojah	1	
imere	1	
mcdonaldjc	1	
rayden84	1	
sirsql	1	
tig	1	
b-long		2

2022-August

New content

- New landing page for What's new content
- Shell experience docs
  - Running commands in the shell
- DSC 2.0 docs
  - Conceptual content - 15 new articles
  - PSDscResources module reference - 58 new articles

Content updates

- PowerShell VS Code docs
- Release notes for 7.3-preview.7
- Cleaned up markdown tables in About topics for better accessibility and localization

Other Projects

- Get-WhatsNew cmdlet released - This cmdlet displays release notes for all versions of PowerShell so you can see what's new for a particular version.

GitHub stats

- 57 PRs merged (16 from Community)

- 24 issues opened (12 from Community)
- 26 issues closed (15 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
sethvs	4	
BEEDELLROKEJULIANLOCKHART	1	
Chemerevsky	1	
ClaudioESSilva	1	
davidhaymond	1	
DavidMetcalfe	1	
dharmatech	1	
kozhemyak	1	
mcawai	1	
NaridaL	1	
Nicicalu	1	
sdarwin	1	
seansaleh	1	

2022-July

New content

- Optimizing your shell experience
- Using tab completion
- Using command predictors
- Getting dynamic help
- Using aliases
- Customizing your shell environment

Content updates

- Updated PowerShell 7.3-preview.6 release notes
- Started reviewing and testing PowerShellGet v3 cmdlet reference (currently in beta) to ensure accuracy and release readiness.
- Refresh of our Community Contributor Guide

Top Community Contributors

GitHub stats

- 50 PRs merged (6 from Community)
- 22 issues opened (14 from Community)
- 29 issues closed (19 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Alvynskio	1	
bergmeister	1	
BusHero	1	
lewis-yeung	1	
sethvs	1	
tommymaynard	1	

2022-June

New content migrated from GitHub wiki

- Limitations of PowerShell transcripts
- Avoid using Invoke-Expression
- Avoid assigning variables in expressions
- about_Case-Sensitivity
- Updated about_Arrays

New SecretManagement content

- Understanding the security features of SecretManagement and SecretStore

- Using the SecretStore in automation
- Using Azure Key Vault in automation

Content updates

- Updated release notes for 7.3-preview.5 and PSReadLine 2.2.6

Top Community Contributors

GitHub stats

- 44 PRs merged (8 from Community)
- 23 issues opened (14 from Community)
- 23 issues closed (13 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
mcdonaldjc	2	1
radrow	1	
yecril71pl	1	
muhahaaa	1	
windin7cc	1	
fabiod89	1	
NaridaL	1	

2022-May

New content

- Create a Crescendo configuration using the Crescendo cmdlets
- Overview of the SecretManagement and SecretStore modules
- Get started with the SecretStore module
- Understanding the SecretManagement module
- Managing a SecretStore vault

Content updates

- Renamed the `staging` branch to `main`
- Updated the Table of Contents for easier discovery
  - Moved Support Lifecycle to the top level
  - Moved Contributor Guide to the top level
- 7.3-preview.4 release notes
- Bulk formatting cleanup for many docs
  - PowerShell-Docs content - 272 files
  - Secrets management - 17 files
- Updated the PSScriptAnalyzer README and deleted docs that were migrated to Microsoft Learn
- Removed CentOS and Fedora from docs - no longer supported
- Retired 7.1 content - no longer supported
  - Collapse release notes into diff article
  - Delete or move content to archive repo

Top Community Contributors

GitHub stats

- 53 PRs merged (12 from Community)
- 38 issues opened (21 from Community)
- 39 issues closed (26 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
tommymaynard	5	
naveensrinivasan	2	
rikurauhala	1	
joshua6point0	1	
rhorber	1	
Raton-Laveur	1	
StephenRoille	1	

GitHub Id	PRs merged	Issues opened
krlinus		2

2022-April

New content

- No new content this month

Content updates

- Rewrote the install instructions for PowerShellGet
- Created separate article for Installing PowerShellGet on older Windows systems

Other projects

- PowerShell + DevOps Summit April 25-28
  - Gave presentation about contributing to Docs
  - Lightning demo about argument completers
  - Interview for the PowerShell Podcast

Top Community Contributors

GitHub stats

- 24 PRs merged (3 from Community)
- 22 issues opened (17 from Community)
- 21 issues closed (15 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
Hrxn	1	
kevinholtkamp	1	
MikeyBronowski	1	
tommymaynard		4

2022-March

New Content

- New PowerShell docs
  - about_Member-Access_Enumeration
  - about_Module_Manifests
  - How to create a command-line predictor
- Utility modules updates
  - New docs for Crescendo release
    - Install Crescendo
    - Choose a command-line tool
    - Decide which features to amplify
    - Create a Crescendo cmdlet
    - Generate and test a Crescendo module
  - Moved PlatyPS article from PowerShell docs to the PlatyPS documentation
    - Moved PlatyPS article
  - Migrated more PSScriptAnalyzer documentation from the source code repository
    - Using PSScriptAnalyzer
    - Rules and recommendations
    - Creating custom rules

Content updates

- Bulk cleanup of related links in About_ topics
- Added issue and PR templates to all docs repos
- Updates for 7.3 preview content
  - New tab completions
  - Support for SSH options on remoting cmdlets
  - New experimental feature `PSAMSIMethodInvocationLogging`

Other projects

- Created a prototype cmdlet `Get-WhatsNew` based on the draft RFC
  - Check out the RFC and provide feedback

New team member

- Welcome Mikey Lombardi to the docs team

Top Community Contributors

GitHub stats

- 49 PRs merged (8 from Community)
- 26 issues opened (14 from Community)
- 33 issues closed (18 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
AspenForester	1	
codaamok	1	
DianaKuzmenko	1	
MikeyBronowski	1	
poshdude	1	
robcmo	1	
sertdfyguhi	1	
stampycode	1	

2022-February

New Content

- about_Calling_Generic_Methods

Content updates

- Catching up on issues
- Updates for 7.3 preview content

Top Community Contributors

GitHub stats

- 22 PRs merged (3 from Community)
- 24 issues opened (19 from Community)

- 18 issues closed (16 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
sethvs	2	
guilhermgonzaga	1	

2022-January

New Content

- No new content. We're down to one writer for PowerShell. I was out of the office for half of December for vacation then half of January for COVID.

Content updates

- Catching up on issues
- Updates for 7.3 preview content

Top Community Contributors

GitHub stats

- 51 PRs merged (10 from Community)
- 29 issues opened (26 from Community)
- 46 issues closed (39 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
sethvs	3	
UberKluger	1	
MiguelDomingues	1	

GitHub Id	PRs merged	Issues opened
reZach	1	
Hertz-Hu	1	
julian-hansen	1	
Hrxn	1	
peteraritchie	1	

What's new in PowerShell Docs for 2021

Article • 03/30/2025

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2021-December

New Content

- Added PowerShell 7.3-preview.1 [preview content]
- New DSC 3.0 content
  - PowerShell Desired State Configuration overview
  - Manage configuration using PowerShell DSC
  - DSC Configurations
  - DSC Resources

Content updates

- Moved Desired State Configuration content to new docset and repository
  - DSC is now being developed outside of the PowerShell product.
  - The move allows for better versioning of documentation for DSC.

Top Community Contributors

GitHub stats

- 24 PRs merged (4 from Community)
- 30 issues opened (26 from Community)
- 12 issues closed (7 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
dAu6jARL	1	

GitHub Id	PRs merged	Issues opened
shriharshmishra	1	
a-sync	1	
bogdangrigg	1	

2021-November

New Content

- about_Built-in_Functions

Content updates

- PowerShell 7.2 GA documentation updates
- Update GitHub Issue and PR templates - piloting the new YAML-based forms for issues
- Updated Crescendo reference for Preview 4 release

Top Community Contributors

GitHub stats

- 48 PRs merged (13 from Community)
- 31 issues opened (24 from Community)
- 34 issues closed (28 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
matt9ucci	4	
yecril71pl	3	
tholabrk	1	
lukejjh	1	
Oechiih	1	

GitHub Id	PRs merged	Issues opened
bergmeister	1	
Hrxn	1	
jebeckham	1	

2021-October

New Content

- PSScriptAnalyzer documentation
  - Overview
  - Rules documentation

Content updates

- Lots of general editorial and freshness updates across 450 files
- PowerShell 7.2-rc.1 documentation updates

Top Community Contributors

GitHub stats

- 49 PRs merged (12 from Community)
- 33 issues opened (30 from Community)
- 33 issues closed (32 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
doctordns	4	
diecknet	1	
Kagre	1	
KexyBiscuit	1	
JohnRoos	1	

GitHub Id	PRs merged	Issues opened
Zhu-Panda	1	
philanderson888	1	
BlackFalcons	1	
milaandahiya	1	
mklement0		2

2021-September

New Content

- SDK documentation
  - How to validate an argument using a script
  - ValidateScript Attribute Declaration
- Learning content
  - PowerShell security features

Content updates

- Install documentation - Did a complete rewrite of the setup documentation. There is now a separate article for each supported OS.
  - Install on Windows
  - Install on macOS
  - Install on Linux
    - Alpine
    - CentOS
    - Debian
    - Fedora
    - Raspberry Pi OS
    - Red Hat Enterprise Linux
    - Ubuntu
    - Alternate install methods
    - Community supported Linux
- Supporting documentation
  - Using PowerShell in Docker
  - Arm Processor support
  - Microsoft Update for PowerShell FAQ
  - PowerShell Support Lifecycle

- Reformatted and updated the PSScriptAnalyzer rules documentation. Next month we plan to publish these docs to Microsoft Learn.
  - Rules documentation on GitHub
- Lots of general editorial updates across 4500 files
- PowerShell 7.2-preview.10 documentation updates

Top Community Contributors

GitHub stats

- 68 PRs merged (6 from Community)
- 32 issues opened (29 from Community)
- 49 issues closed (41 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
RaghuRocks3	1	
Zhu-Panda	1	
Jaykul	1	
juvtib	1	
przmv	1	
mklement0		2

2021-August

New content

- about_ANSI_Terminals
- about_PSCustomObject

Content updates

- PowerShell 7.2-preview.9 documentation updates

Top Community Contributors

GitHub stats

- 66 PRs merged (14 from Community)
- 42 issues opened (30 from Community)
- 53 issues closed (38 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
AvrumFeldman	1	
benmccallum	1	
bitdeft		2
BraINstinct0	1	
diddledani	1	
doctordns	1	
gravitional	1	
homotechsual	1	
imba-tjd	1	
juvtib	1	
kozhemyak	1	
omarys	1	
ryandasilva2992	1	
sethvs	1	
sneakernuts	1	

2021-July

New content

- about_Functions_Argument_Completion
- about_Tab_Expansion

Content updates

- about_Functions_Advanced_Parameters - major updates and links to new articles
- PowerShell 7.2-preview.8 documentation updates

Top Community Contributors

GitHub stats

- 51 PRs merged (9 from Community)
- 56 issues opened (50 from Community)
- 59 issues closed (52 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
rkeithhill	2	
juvtib	2	
SetTrend	1	
Rob-S	1	
xtqqczze	1	
akashdabhi03	1	
kurtmckee	1	
clamb123		13
mklement0		2
jerryKahnPerl		2

2021-June

New content

- about_Intrinsic_Members
- about_Booleans

Content updates

- Converted about_Remote_FAQ to new YAML format and moved to conceptual TOC
- Moved **PSDesiredStateConfiguration** out of 7.2 docs and into PowerShell-Docs-Modules
  - DSC is being removed from PowerShell to become an optional module that is loaded from the PowerShell Gallery
  - Long-term plan is to move all DSC documentation out of PowerShell-Docs into a new repository for DSC content
- Totally rewrote the PowerShell release notes to summarize the current state, making it easier for users to find the information without having to read every release note.
  - Differences between Windows PowerShell 5.1 and PowerShell (core) 7.x
  - PowerShell differences on non-Windows platforms
- PowerShell 7.2-preview.7 documentation updates

Top Community Contributors

GitHub stats

- 43 PRs merged (1 from Community)
- 36 issues opened (32 from Community)
- 49 issues closed (41 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
rkeithhill	1	
frenchiveruti		2
paulaustin-automutatio		2
ringerc		2
trollyanov		2
UberKluger		2

2021-May

New Content

- Migrated two articles from the Windows Server content to the PowerShell docset
  - PowerShell scripting performance considerations
  - PowerShell module authoring considerations
- Added PowerShell Language Specification 3.0
  - The specification document is available from the Microsoft Download Center as a Microsoft Word document.
- Updated content for PowerShell 7.2-Preview6 release
- Moved Samples under the Learn node in the Table of Contents

Top Community Contributors

GitHub stats

- 53 PRs merged (6 from Community)
- 37 issues opened (35 from Community)
- 39 issues closed (36 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
kvprasoon	2	
jErprog	1	
aamnah	1	
BetaLyte	1	
TheNCuber	1	
trollyanov		6
Tarjei-stavanger		3
aungminko93750		3
SetTrend		2
cdichter		2
reuvygroovy		2

2021-April

New Content

- Published new Learn content Write your first PowerShell code
- Updated docs for PowerShell 7.2-preview.5
- Updated metadata on ~3300 articles in the Windows module documentation
  - Preparing for Windows Server 2022 release and fixing updateable help
  - This is still a work in progress

Top Community Contributors

GitHub stats

- 45 PRs merged (5 from Community)
- 42 issues opened (33 from Community)
- 55 issues closed (32 Community issues closed)

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ealmonte32	1	
hananyajacobson	1	
MarcChasse	1	
Melvin-Abraham	1	
robertkruk	1	
MikeMM70		2

2021-March

New content

- Working on simplify and expanding the Overview content
  - Added What is a PowerShell command?
- Started a new tutorial series - PowerShell Bits
  - Discover PowerShell

- Help content for PowerShell utility modules
  - Microsoft.PowerShell.Crescendo
  - Microsoft.PowerShell.SecretManagement
  - Microsoft.PowerShell.SecretStore
  - PlatyPS
  - PSScriptAnalyzer
- Working with the Windows team to update help for Windows management modules
  - Added content for Windows Server 2019 and Windows Server 2022 (preview)
  - Continuing to work on improving updateable help for these modules
- PowerShell 7.2-preview documentation updates

Top Community Contributors

GitHub stats

- 42 PRs merged (5 from Community)
- 63 issues opened (42 from Community)
- 47 issues closed

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
bl-ue	1	
brianary	1	
Lx	1	
matt9ucci	1	
kfasick	1	
mklement0		10
juvtib		6
BoJackem23		2

2021-February

New content

- PowerShell 7.2-preview documentation updates

Top Community Contributors

GitHub stats

- 40 PRs merged (12 from Community)
- 40 issues opened (30 from Community)
- 35 issues closed

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
bbodenmiller	1	
brianary	1	
exchange12rocks	1	
IvenBach	1	
jamiepinheiro	1	
jdoubleu	1	
LogicalToolkit	1	
matt9ucci	1	
mihir-ajmera	1	
revolter	1	
secretGeek	1	
springcomp	1	
Ayanmullick		2

2021-January

New content

- PowerShell 7.2-preview documentation updates

Top Community Contributors

GitHub stats

- 44 PRs merged (14 from Community)
- 46 issues opened (38 from Community)
- 35 issues closed

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
AndreyMatrosov	4	
revolter	2	
cconrad	1	
Hrxn	1	
kilasuit	1	
NN---	1	
snickler	1	
vinian	1	
zeekbrown	1	
brianary		2
mklement0		2
plastikfan		2

What's new in PowerShell Docs for 2020

Article • 03/30/2025

This article lists notable changes made to docs each month and celebrates the contributions from the community.

Help us make the documentation better for you. Read the Contributor's Guide to learn how to get started.

2020-December

- Updated contributor guide
  - documented the `&preserve_view=true` query parameter for hyperlinks
  - documented cross-reference syntax for hyperlinks
  - added information about localization

- GitHub stats
  - 44 PRs merged (11 from Community)
  - 50 issues opened (43 from Community)
  - 50 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
doctordns	3	2
chadbaldwin	1	
dawedawe	1	
dumpvn	1	
kvprasoon	1	
lbearl	1	
petershen0307	1	

GitHub Id	PRs merged	Issues opened
skycommand	1	
springcomp	1	
Cwilson-dataselfcom		5
bobbybatatina		2

2020-November

- PowerShell 7.1 GA Release
  - What's New in PowerShell 7.1
  - Converted 7.1 docs to release status
  - Added 7.2 (preview) docs
  - Retired v6 docs to archive repository

- Blog posts
  - You've got Help!
  - Updating help for the PSReadLine module

- Documentation maintenance
  - Updated 137 articles to remove MSDN and TechNet references
  - Updated 171 articles to indicate Windows-only compatibility
  - Updated 38 articles to address build warnings and suggestions
  - Added include to 24 DSC articles
  - Major rewrite of the PowerShell Jobs articles
    - about_Jobs
    - about_Remote_Jobs
    - about_Thread_Jobs

- GitHub stats
  - 50 PRs merged (8 from Community)
  - 55 issues opened (45 from Community)
  - 51 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
AlanFlorance	1	
gilbertbw	1	
ianwalkeruk	1	
JeremyTBradshaw	1	
matt9ucci	1	
Rob-S	1	
ShaydeNofziger	1	
skycommand	1	
juvtib		8
iRon7		2
l-ip		2
stephenrgentry		2
Vixb1122		2

2020-October

- New articles
  - about_Character_Encoding
  - about_Output_Streams
  - Using Visual Studio Code to debug compiled cmdlets (thanks @fsackur)
  - Add Credential support to PowerShell functions (thanks @joshduffney)

- Documentation maintenance
  - Updates for 7.1-rc content
  - Updated all article descriptions to improve SEO

- GitHub stats
  - 61 PRs merged (7 from Community)
  - 49 issues opened (42 from Community)
  - 61 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
doctordns	1	
escape208	1	
nickdevore	1	
fsackur	1	
Duffney	1	
skycommand	1	
yecril71pl	1	
mklement0		3
Abdullah0820		2

2020-September

- Documentation maintenance
  - Updates for 7.1-preview content
- Community presentation
  - How to contribute to Docs for RTPUG - https://www.youtube.com/watch?v=0_DEB61YOMc
- GitHub stats
  - 41 PRs merged (9 from Community)
  - 52 issues opened (47 from Community)
  - 51 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
doctordns	1	
fatherjack	1	
goforgold	1	
jonathanweinberg	1	
kvprasoon	1	
skycommand	1	
springcomp	1	
themichaelbender	1	
toddryan	1	
mklement0		13
setpeetrike		2

2020-August

- New PowerShell documentation
  - About_Calculated_Properties
  - Writing Progress across multiple threads with ForEach-Object -Parallel
- Documentation maintenance
  - Updates for 7.1-preview content
- GitHub stats
  - 69 PRs merged (26 from Community)
  - 68 issues opened (49 from Community)
  - 58 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
sethvs	10	2
yecril71pl	10	
mklement0	1	7
springcomp	1	2
SquirrelAssassin		2
thorstenbutz		2
aetos382	1	
crumdev	1	
joshSi	1	
kmoad	1	

2020-July

- New PowerShell documentation
  - Resurrected old ETS docs - 7 articles added
  - Added article about creating updateable help using PlatyPS
- Documentation maintenance
  - Updates for 7.1-preview content
  - Updated page header - simplified menu choices and added a download button
  - Fixed several `Update-Help` issues
    - Help for PSDesiredStateConfiguration and ThreadJob modules now downloads
    - Published updateable help for PowerShell 7.1 preview
    - Updateable help for PowerShell 5.1 now includes About topics
- GitHub stats
  - 99 PRs merged (29 from Community)
  - 51 issues opened (44 from Community)
  - 71 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filing issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
yecril71pl	10	3
sethvs	10	
springcomp	3	2
txtor	2	1
baardhermansen	1	
skycommand	1	
srjennings	1	
xtqqczze	1	
mklement0		3
Allexxann		2
sharpninja		2
XuHeng1021		2

2020-June

- New PowerShell documentation
  - Published new PowerShell 101 content contributed by Mike F. Robbins
  - Added two recent blog posts from Rob Holt to the **Scripting and development** docs
    - Choosing the right PowerShell NuGet package for your .NET project
    - Resolving PowerShell module assembly dependency conflicts

- Documentation maintenance
  - Archived older content to https://aka.ms/PSLegacyDocs
    - PowerShell Web Access SDK content
    - PowerShell Workflows SDK content
  - Updates for 7.1-preview content

- GitHub stats

- 83 PRs merged (15 from Community)
- 68 issues opened (52 from Community)
- 74 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filling issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
doctordns	3	
gabrielmccoll	2	
adrianhale	1	
aisbergde	1	
beatcracker	1	
bergmeister	1	
DarioArzaba	1	
gforceg	1	
jpomfret	1	
Karl-WE	1	
signalwarrant	1	
skycommand	1	
tkhadimullin	1	
johnkberry		2
juvtib		2
mklement0		2
Sagatboy33		4

2020-May

- New PowerShell documentation
  - Created a new Deep dives section containing content from community contributor Kevin Marquette
    - Everything you want to know about arrays
    - Everything you want to know about hashtables
    - Everything you want to know about PSCustomObject
    - Everything you want to know about string substitution
    - Everything you want to know about if/then/else
    - Everything you want to know about switch
    - Everything you want to know about exceptions
    - Everything you want to know about $null
    - Everything you want to know about ShouldProcess
    - How to create a Standard Library binary module
  - Published the PowerShell 7.0 .NET API reference
  - `Update-Help -Force` for PowerShell 5.1 now downloads updated content for the core PowerShell modules

- Documentation maintenance
  - Major reorganization of the Table of Contents
    - New content under **Learning PowerShell**
    - Windows PowerShell 5.1 content collected in one location
  - Archived older content to https://aka.ms/PSLegacyDocs
    - PowerShell Web Access
    - PowerShell Workflows Guide
  - Updates for 7.1-preview content

- GitHub stats
  - 81 PRs merged (21 from Community)
  - 61 issues opened (53 from Community)
  - 64 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filling issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
nschonni	10	